PROYECTO CLS · EDICIÓN 5 · REVISIÓN 5.4 (AMPLIADA) · TOMO I DE II

Curso de reversing

Tomo I · Tronco Principal (Etapas 0 a 5) y Material de Apoyo

De los fundamentos al análisis guiado en user-mode: arquitectura x86/x64, formato PE y DLLs (ciclo de vida y sideloading), IDA, Ghidra, x64dbg (flujo táctico y extracción masiva de strings), reversing de Go y Rust, evasión en user-mode (API hooking, direct/indirect syscalls y call stack spoofing), inyección de procesos en memoria, criptografía híbrida y APIs nativas (CryptoAPI/CNG), .NET, C++ profundo, WinDbg/TTD, malware y vulnerabilidades.

Contenido de este volumen (Tomo I): Apertura y 3 Rutas de Estudio · Etapas 0, 1, 2 (2A, 2B, 2C y optativas 2.22–2.24), 3, 4 y 5 · Hilo conductor multi-nivel y Kit de 5 plantillas C/C++ · 3 Hojas de referencia rápida (*Cheat Sheets*) · Glosario · Preguntas y respuestas · Fuentes y límites.

En el Tomo II y Separata: La **Etapa 6 (Kernel y drivers KMDF)** y la **Rama 2D (Windows internals y Hyper-V)** se encuentran encuadernadas en el **Tomo II**. Para consulta rápida junto al teclado, usá la **Separata de Escritorio**.

> **Material educativo y defensivo.** Las prácticas usan programas propios, software abierto o crackmes diseñados para aprender. No se enseña a eludir licencias, DRM ni controles de acceso.

Contenido — Tomo I

Apertura

Cómo estudiar y qué se revisó p. 3-9 · 7 págs.

Mapa del curso y rutas de estudio p. 10-13 · 4 págs.

Etapa 0 **p. 14 · 1 pág.**

0.1 Laboratorio p. 15-20 · 6 págs.

Etapa 1 **p. 21-22 · 2 págs.**

1.1 Números y hexadecimal p. 23-28 · 6 págs.

1.2 Memoria y stack p. 29-35 · 7 págs.

1.3 Procesos e hilos p. 36-41 · 6 págs.

1.4 Formato PE p. 42-47 · 6 págs.

1.5 C y Python p. 48-54 · 7 págs.

Etapa 2 **p. 55-58 · 4 págs.**

2.1 Registros y flags p. 59-63 · 5 págs.

2.2 Instrucciones p. 64-69 · 6 págs.

2.3 Punto flotante y SSE p. 70-74 · 5 págs.

2.4 Flujo p. 75-81 · 7 págs.

2.5 Llamadas p. 82-87 · 6 págs.

2.6 Debugger p. 88-99 · 12 págs.

2.7 IDA p. 100-110 · 11 págs.

2.8 Optimizaciones p. 111-114 · 4 págs.

2.9 Herramientas p. 115-120 · 6 págs.

2.10 Ofuscación nativa p. 121-127 · 7 págs.

2.11 TLS callbacks p. 128-131 · 4 págs.

2.12 MFC p. 132-135 · 4 págs.

2.13 Huella de compilador p. 136-143 · 8 págs.

2.14 PE fino p. 144-151 · 8 págs.

2.15 Taxonomía anti-análisis p. 152-160 · 9 págs.

2.16 Ghidra/Cutter p. 161-165 · 5 págs.

2.17 Packers didácticos p. 166-172 · 7 págs.

2.18 C++ profundo p. 173-180 · 8 págs.

2.19 WinDbg, TTD y heap p. 181-184 · 4 págs.

2.20 Patch diffing y CVE p. 185-189 · 5 págs.

2.21 Scripting de debugger p. 190-195 · 6 págs.

2.22 (opt) Malware dinámico p. 196-199 · 4 págs.

2.23 (opt) Shellcode, red y C2 p. 200-202 · 3 págs.

2.24 (opt) Fuzzing y simbólico p. 203-207 · 5 págs.

2.25 Ejecución simbólica y concólica p. 208-212 · 5 págs.

2.26 Ofuscación por máquina virtual p. 213-216 · 4 págs.

2.27 Protectores y packers multicapa p. 217-220 · 4 págs.

2.28 Neutralizar anti-análisis p. 221-224 · 4 págs.

2.29 DBI: DynamoRIO y Pin p. 225-228 · 4 págs.

2.30 Frida Stalker p. 229-231 · 3 págs.

2.31 Extensiones de Ghidra y SLEIGH p. 232-234 · 3 págs.

2.32 Go y Rust en profundidad p. 235-238 · 4 págs.

2.33 VB6 y p-code p. 239-242 · 4 págs.

Etapa 3 **p. 243-244 · 2 págs.**

3.1 .NET runtime p. 245-250 · 6 págs.

3.2 Leer IL p. 251-256 · 6 págs.

Etapa 4 **p. 257-258 · 2 págs.**

4.1 Método de análisis p. 259-269 · 11 págs.

4.2 Reconocer criptografía p. 270-277 · 8 págs.

4.3 Informe p. 278-282 · 5 págs.

4.4 Dinámico seguro p. 283-287 · 5 págs.

4.5 Reconstruir formatos y protocolos p. 288-292 · 5 págs.

4.6 Protocolos de terceros p. 293-296 · 4 págs.

Etapa 5 **p. 297-298 · 2 págs.**

5.1 Malware p. 299-303 · 5 págs.

5.2 Vulnerabilidades p. 304-310 · 7 págs.

5.3 Automatización p. 311-319 · 9 págs.

5.4 Fuera de alcance p. 320-323 · 4 págs.

Material de apoyo (en este Tomo I)

Hilo conductor y plantillas C/C++ p. 324-332 · 9 págs.

Hojas de referencia rápida (Cheat Sheets) p. 333-339 · 7 págs.

Glosario p. 340-350 · 11 págs.

Preguntas y respuestas p. 351-359 · 9 págs.

Fuentes y límites p. 360-368 · 9 págs.

En el Tomo II (Ramas Especializadas)

Etapa 6 · Kernel y drivers en Windows (6.1 a 6.6) ver Tomo II

Rama 2D · Windows internals y Hyper-V (2D.1 a 2D.13) ver Tomo II

De miles de mensajes a un curso que se puede estudiar

Este paquete convierte el material útil del proyecto **CLS** en un curso de reversing en español, progresivo y verificable. No es un índice ni una lista de enlaces: cada lección explica un concepto, lo conecta con lo anterior, muestra un ejemplo paso a paso, propone una práctica y te permite comprobar si aprendiste.

Está pensado para alguien que empieza, con un marco **educativo, defensivo y autorizado**. Las prácticas usan programas propios, software abierto o crackmes diseñados para aprender.

> **Empezá por acá**
>
> 1. Leé esta portada completa (10 minutos).
> 2. Abrí el Mapa del curso y elegí la Etapa 0.
> 3. Armá el laboratorio de la lección 0.1.
> 4. Estudiá una lección por vez y hacé la práctica antes de avanzar.
> 5. Anotá tu avance en seguimiento.csv.

Qué vas a poder hacer

Al terminar el recorrido base vas a poder:

- Explicar cómo se representa un programa en memoria y en disco.
- Leer assembly x86/x64 y reconstruir condiciones, bucles y llamadas.
- Usar x64dbg e IDA (o Ghidra) con un método, no a los botones.
- Analizar un programa .NET propio y comparar fuente, IL y decompilación.
- Escribir un informe reproducible con evidencia, límites y hashes.
- Decidir cuándo un programa se ejecuta y cuándo no.
- Reconocer código ofuscado, TLS callbacks, binarios MFC, la huella del compilador y técnicas anti-análisis, y leer el mismo binario en Ghidra (bloque **2B · Windows intermedio**).

El curso: siete etapas numeradas y una rama 2D

Las **Etapas 0–4** forman el recorrido obligatorio. La **Etapa 5**, la **Etapa 6** y la rama optativa **2D · Windows internals** (asociada a la Etapa 2, después de 2C) no agregan requisitos para completar el curso base.

Etapa	Foco	Horas estimadas	Puerta de salida
0	Laboratorio y método seguro	4–6 h	VM aislada, snapshot, hash y bitácora
1	Fundamentos	21–25 h	Números, memoria, PE y un programa propio en C
2	Introducción al reversing	32–40 h (2A) + ~25 h (2B) + ~30–40 h (2C)	Assembly, debugger e IDA; bloques **2B intermedio** y **2C avanzado** (+ 3 rutas optativas)
3	.NET como primera victoria	7–10 h	Leer IL y comparar tres vistas
4	Práctica guiada	19–25 h	Método, formatos/protocolos, cripto, informe y dinámico seguro (5 lecciones)
5	Ramas opcionales	variable	Una rama con práctica propia y límites claros
6	Kernel y drivers (rama optativa)	45–65 h estimadas	Revisar un driver propio y sus controles en VM aislada

La profundización **Etapa 2 · Rama optativa 2D — Windows internals** tiene 13 lecciones (estimación editorial 55–70 h). Se muestra junto a la Etapa 2, pero queda fuera de sus requisitos obligatorios. La extensión Hyper-V también es optativa y separada.

Las horas son **estimaciones editoriales**, no mediciones de alumnos ni duraciones declaradas por CLS. Las de 2B/2C se estiman por bloque; 2D/Etapa 6 pueden requerir más tiempo según las prácticas y experiencia previa.

> **Cómo se cuentan las lecciones y en qué orden cursar 2.22–2.24**
>
> La **Etapa 2** reúne 2A (2.1–2.9), 2B (2.10–2.16) y 2C (2.17–2.21), además de 3 rutas optativas (2.22, 2.23 y 2.24) que no son obligatorias. Como esas tres optativas declaran como prerrequisitos las lecciones `4.1`, `4.2`, `4.4` y `5.1`, se recomienda cursarlas **después de completar la Etapa 4 y la lección 5.1**, manteniendo intacta la numeración del curso. La Etapa 4 comprende **5 lecciones** (`4.1`, `4.5`, `4.2`, `4.3` y `4.4`; 19–25 h). La rama optativa 2D aporta 13 lecciones fuera de las horas obligatorias y la Etapa 6 suma 6 lecciones optativas. El apartado 5.4 «Fuera de alcance» es una página de límites, no una lección a cursar.

Cómo estudiar

Cada lección tiene la misma estructura:

1. **Objetivo** y prerrequisitos.
2. **Explicación** con lenguaje sencillo y conexión con lo anterior.
3. **Ejemplo paso a paso**, con valores y direcciones cuando el material los aporta.
4. **Práctica** en el laboratorio.
5. **Preguntas de repaso** con respuestas razonadas.
6. **Puerta de salida**: qué tenés que poder hacer antes de avanzar.

7. **Fuentes**: rutas locales, capítulos y timestamps.

La regla es simple: **no avances si no podés hacer la práctica**. Leer sin ejecutar deja la sensación de entender y no la comprensión.

Qué se revisó de verdad (y qué no)

Esta edición no leyó "todo CLS". Sería falso decirlo: el corpus tiene más de 8.000 archivos y 144,9 horas de transcripciones. Lo que se hizo:

Lote	Qué contiene	Qué se hizo
Reversing desde cero	47 capítulos, 28,6 h	42 con citas (38 en 2A + ofuscación 26/30, TLS 38 y MFC 39 en 2B); 5 excluidos por criterio (13, 17, 27, 29, 31)
Videos y material adicional	40 videos, 22,5 h (MASM32 + OllyDbg)	Encabezados y pasajes de los 40; varios completos
Curso IDA desde cero	69 .docx + 47 PDF	69 páginas-tutorial ilustradas (texto + 3.278 capturas); tutes 1–28 integrados en las lecciones
Python ISO video tutes	55 transcripciones, 12,7 h	20 completas + 2 parciales; **no enseña Python**
Introducción al reversing con IDA Pro	1 .docx de 2016	Texto extraído y leído
Teorias_Numeradas	2.023 contenedores (2.017 con título legible)	Clasificados por título: 426 educativos / 1.591 cracking comercial
Transcripciones de Telegram	179 videos (76,5 h) + 357 audios (2,7 h)	Tamizadas por tema (heurística); 5 juntadas ilustradas + 99 audios en hilos con reproductor
Chat unido	100.791 mensajes	Metadatos y muestras; **no mensaje por mensaje**
Comprimidos	2.299	Listado generado; no se extrajeron
Imágenes y documentos	3.235 imágenes, 567 documentos	Sin OCR; documentos leídos: los 69 `.docx` del Curso IDA (texto + capturas) y 2 `.docx` sueltos
Inventario limpio	8.154 archivos	LEEME, resumen y consultas agregadas

El detalle fila por fila está en registro_de_cobertura.csv y mapa_de_cobertura.csv. Los criterios y las exclusiones están en Fuentes y límites.

Lo que no se puede afirmar

- No se ejecutó ningún binario del corpus.
- No se extrajo ningún comprimido.
- No se abrió ningún adjunto del chat.
- No se leyeron **de punta a punta** los 47 capítulos del curso principal (sí 38, con cita) ni las 795 transcripciones completas (se tamizaron por tema y se procesaron las técnicas).
- El tamizado de Telegram y la clasificación de Teorias_Numeradas son **por título/palabras clave**, no lectura completa de cada archivo.
- Un título, un nombre de archivo o una mención **no** demuestran lo que enseña un recurso.

Cómo se leen las fuentes

En todo el curso vas a encontrar cuatro tipos de afirmación:

Etiqueta	Significado
Contenido observado en CLS	Figura en un archivo leído; se cita ruta, capítulo y timestamp cuando existe
Síntesis del material CLS	Resumen con palabras propias de varios fragmentos observados
Conocimiento externo	Teoría general que no depende de un archivo de CLS
Recomendación editorial	Sugerencia de método o seguridad agregada por esta guía

Cuando el audio y la teoría discrepan, se citan ambos y se explica la diferencia. Ejemplos: la explicación de little-endian del capítulo 01 invierte la palabra "significativo"; el capítulo 07 dice "cuatro breakpoints por proceso" cuando son por hilo; el capítulo 03 se enreda con los rangos de WORD. Las lecciones lo marcan.

Qué hay en el paquete

Archivo o carpeta	Para qué
guia/	Las lecciones HTML, etapa por etapa
Materiales	Hub: un solo lugar con enlaces a todo
Buscar	Buscador offline por palabra en todo el curso
Banco de prácticas	Crackmes del corpus para practicar en VM, con hash y mapeo al curso
Índice de videotutoriales	Dónde está en Windows cada video mencionado, con enlace y ruta
Curso IDA · índice por tema	Los 69 tutes de IDA, con tema y cantidad de palabras
Teorias_Numeradas · educativo vs. cracking	Los 2.023 contenedores (2.017 con título legible) clasificados por título
Telegram por tema	523 transcripciones tamizadas; dónde están las sesiones de práctica
fichas/	Qué es cada recurso, qué se revisó y qué falta
hilo-conductor-y-plantillas.html	Evolución de un mismo binario en los 3 niveles (`check.c` → `BlocLab`) y kit de 5 plantillas C/C++ en `plantillas_codigo/`
hojas-referencia-rapida.html	Hojas de referencia rápida (*Cheat Sheets*) para Principiante, Intermedio y Avanzado
glosario.html	Términos en lenguaje de principiante
preguntas_y_respuestas.html	Repaso por etapa, con respuestas
mapa_de_cobertura.csv	Tema ↔ recurso ↔ archivo/capítulo ↔ estado
registro_de_cobertura.csv	Lotes, archivos, revisados, pendientes y motivo
seguimiento.csv	Tu planilla de avance
Fuentes y límites	Criterios, alcance y limitaciones
CLS_Curso_Reversing.pdf	Edición de estudio (solo teoría) para leer sin pantalla; las prácticas y los materiales están acá en el HTML
plantilla_informe.md	Base para tus informes

Todo funciona **offline**. Las fuentes locales se enlazan con rutas relativas; los enlaces a transcripciones, videos y documentos abren el archivo original. El paquete trae HTML, CSV, el PDF y las **imágenes de los tutes ilustrados y los fotogramas de las juntadas** (~130 MB). Los **videos y audios** (decenas de GB) **se vinculan, no se copian**: para que abran, mantené las carpetas del proyecto una al lado de la otra (o usá los ZIP de compartir).

Compartir el curso

Los enlaces locales son **relativos** y funcionan si se conserva la estructura del paquete. Para compartir, prepará una copia curada con los archivos que tengas derecho a redistribuir; no compartas la carpeta de proyecto completa ni exportaciones de chats, corpus sin revisar o medios de terceros. La edición pública de presentación contiene el PDF de teoría; el paquete local conserva el HTML y los tutoriales de apoyo. Más detalle en el Índice de videotutoriales.

Requisitos

- Una computadora con **Windows**. Esta edición cubre **solo reversing de Windows** (binarios PE, x86/x64); Linux, macOS, Android y ARM quedan fuera, aunque los fundamentos se trasladen.
- VirtualBox, VMware o Hyper-V para el laboratorio.
- x64dbg, un editor hexadecimal y un visor de PE (gratuitos).
- IDA Free o Ghidra. **No** hace falta IDA Pro.
- Ganas de escribir programas propios: son el mejor material de práctica.

Seguridad y legalidad

Leé esto antes de ejecutar cualquier cosa

- No ejecutes adjuntos del chat, cracks, keygens, instaladores desconocidos ni archivos en cuarentena.
- No analices malware en tu máquina personal. La rama de malware de esta guía es **estática y defensiva**.
- No uses las técnicas para eludir licencias, DRM, autenticación o controles de acceso.
- No analices software ajeno sin permiso. Los ejercicios de vulnerabilidades se hacen en programas propios o laboratorios diseñados para eso.
- Si encontrás una vulnerabilidad, reportala en privado y no publiques el exploit.

Qué falta en esta edición (y por qué)

Esta edición es, a propósito, una **cobertura parcial y honesta**: lee y cita un núcleo representativo del material y documenta el resto como pendiente, en lugar de aparentar que revisó todo. Lo que queda afuera cae en dos grupos.

Excluido por criterio (ética y alcance), no por falta de tiempo:

- Cinco capítulos de «Reversing desde cero» (13, 17, 27, 29, 31): desempaquetado de protectores comerciales (AcProtect). Enseñarlos cruzaría a vulnerar software comercial, fuera del marco educativo y defensivo de esta guía. (La **ofuscación** de los caps. 26 y 30 ya **no** está excluida: es la lección 2.10; TLS y MFC son 2.11 y 2.12.)

- La mayor parte de `Teorias_Numeradas`: muchos títulos apuntan a cracking de software comercial, y además son contenedores sin abrir (varios con contraseña).
- Android, Linux/ELF y macOS: esta edición cubre **solo Windows**.

Ya procesado en esta tanda (índices nuevos, antes pendientes):

- Los 69 documentos del «Curso IDA desde cero»: se extrajo **texto + 3.278 capturas** y se armaron **69 páginas-tutorial ilustradas** (índice en Curso IDA · índice por tema). Además, los **tutes 1–28 quedaron integrados en las lecciones** (Números, Instrucciones, IDA, Herramientas, Debugger, Llamadas, Flujo y Vulnerabilidades), citados uno por uno.
- Los 2.023 contenedores de `Teorias_Numeradas`: se clasificaron por título en educativos vs. práctica sobre software comercial (426 / 1.591 de 2.017 legibles). Ver Teorias_Numeradas · qué es educativo y qué no. Siguen sin abrir (varios con contraseña); la clasificación es por título, no por contenido.
- Las transcripciones de Telegram: se tamizaron 523 por palabras clave y, con lo técnico, se armaron **5 juntadas ilustradas** (fotogramas + transcripción + reproducir) y los **99 audios técnicos en hilos por sesión** con reproductor. Índice general en Transcripciones de Telegram por tema.

Pendiente por volumen o formato (se podría sumar en una próxima tanda):

- Leer de verdad (no por título/heurística) las transcripciones y tutes de IDA ya indexados: el índice dice dónde mirar, pero citar con precisión exige abrir cada uno.
- Los contenedores de **exploiting de Windows**: material avanzado de la Etapa 5, sin extraer; la guía cubre el defecto y las mitigaciones de forma conceptual y defensiva, no la construcción de exploits.

El detalle fila por fila está en registro_de_cobertura.csv. Regla de fondo: una mención o un título **no** demuestran lo que enseña un recurso, así que nada de esto se presenta como leído: los tres índices nuevos clasifican **títulos y texto extraído**, no una lectura completa.

Atribución

Este curso es una obra derivada del proyecto CLS. El material didáctico principal es de **Ricardo Narvaja y Solid** («Reversing desde cero»), **Cayetano de Juan Úbeda** (MASM32 y OllyDbg) y **Daniel Bermúdez** (lógica de programación). Las fichas indican autoría, ruta y límites de cada recurso. Esta guía no está afiliada a CLS ni a los autores; los errores de interpretación son de esta edición.

> **Empezá ahora**
>
> Etapa 0 · Laboratorio · Mapa del curso · Fichas · Glosario

Mapa del curso

El recorrido obligatorio son las Etapas 0–4. Las Etapas 5–6 y la rama 2D son optativas y no agregan requisitos para completar el curso base. Cada rama tiene prerrequisitos y una puerta de salida propios.

Etapa	Foco	Horas estimadas	Lecciones
0	Laboratorio y método seguro	4 a 6 horas	1
1	Fundamentos	21 a 25 horas	5
2	Introducción al reversing	32 a 40 h (2A) + ~25 h (2B) + ~30–40 h (2C avanzado)	33
3	.NET como primera victoria	7 a 10 horas	2
4	Práctica guiada	19 a 25 horas	6
5	Ramas opcionales	Variable: 5 a 10 horas por rama	4
6	Rama optativa · Kernel y drivers	45–65 horas como estimación inicial; puede requerir más tiempo si C, WinDbg o WDK son nuevos.	6

Cómo estudiar

1. Leé la lección completa, sin saltear los ejemplos.
2. Hacé la práctica en el laboratorio; no solo leas.
3. Respondé las preguntas **antes** de abrir las respuestas.
4. Verificá la puerta de salida. Si no la pasás, repetí la práctica.
5. Anotá en `seguimiento.csv` qué hiciste, cuánto tardaste y qué duda quedó.

Tres perfiles o rutas de estudio (Principiante, Intermedio y Avanzado)

Aunque el curso completo supera las 400 páginas al sumar el tronco obligatorio (Etapas 0–4), los bloques intermedio y avanzado de la Etapa 2 (2B y 2C), la rama de Windows internals (2D) y la de Kernel y drivers (Etapa 6), **no hace falta leerlo de corrido de principio a fin en una sola sentada**. Podés recorrerlo en tres hitos progresivos claramente delimitados según tu nivel actual y tu objetivo:

Perfil y nivel	Carga estimada	Secuencia exacta de etapas y lecciones	Herramientas centrales	Hito práctico al cerrar el nivel
Ruta 1 · Fundamentos Firmes *(Nivel Principiante)*	~65 a 75 h	Etapa 0 (0.1) → Etapa 1 (1.1–1.5) → Etapa 2A (2.1–2.9) → Etapa 3 (3.1–3.2) → Etapa 4 base (4.1, 4.2, 4.3 y 4.4)	Visor PE (PE-bear / DIE), editor hex, IDA Free, x64dbg, ILSpy / dnSpyEx, compilador C y Python	Analizar un binario propio en C y uno en .NET de punta a punta sin ver el fuente, confirmar hipótesis con breakpoints y redactar un informe reproducible.
Ruta 2 · Analista de Binarios y Malware Defensivo *(Nivel Intermedio)*	+45 a 55 h *(acumulado ~115–130 h)*	Bloque **2B** (2.10–2.16) → 2.17 (Packers) → 2.21 (Scripting y Frida) → 4.5 (Formatos y protocolos) → 5.1 (Malware estático) y 5.3 (Automatización) → Optativas 2.22, 2.23 y 2D.1–2D.5	Ghidra, Scylla, trazas condicionales de x64dbg, Frida, IDAPython (`ida_typeinf`), `pefile`, `capstone`, YARA y Sysinternals / WinDbg básico	Desempaquetar un binario UPX propio (x86/x64) hasta el OEP, recuperar strings ocultas y APIs por PEB/hash, reconstruir un formato binario propio (`CLSF`) y escribir reglas YARA e informes de persistencia.
Ruta 3 · Investigador de Vulnerabilidades, Internals y Kernel *(Nivel Avanzado)*	+80 a 110 h *(curso completo)*	Núcleo duro de **2C** (2.18 C++ profundo, 2.19 WinDbg/TTD/Heap, 2.20 Patch diffing) → 5.2 (Vulnerabilidades) → 2.24 (Fuzzing y simbólico) → 2D.6–2D.13 (COM/RPC/WMI/IPC/Excepciones/ETW) → Etapa 6 (6.1–6.6, Kernel y drivers)	WinDbg Preview + Time-Travel Debugging (TTD), GFlags / Page Heap, BinDiff / Diaphora / Version Tracking, OleViewDotNet, WDK / KMDF y Driver Verifier en VM descartable	Reconstruir clases C++ (vtable, RTTI, SSO, excepciones x64), diagnosticar corrupciones de heap con Page Heap/TTD, aislar un parche de seguridad con diffing, auditar canales IPC/COM y revisar un driver `.sys` estática y dinámicamente.

Consejo para elegir tu punto de entrada

Si empezás desde cero, completá primero la **Ruta 1** sin desviarte a 2B ni a 2C cuando llegues al final de 2.9: saltar de 2.9 directo a la Etapa 3 (.NET) y a la Etapa 4 base te da una victoria real y consolida el método en la mitad de tiempo. Una vez aprobada la puerta de salida de la Etapa 4, volvé para cursar la **Ruta 2** y luego la **Ruta 3**. En cualquier nivel, acompañate con el Hilo conductor multi-nivel y kit de plantillas C/C++ y las Hojas de referencia rápida (Cheat Sheets).

Qué es cada etapa

Etapa 0 · Laboratorio y método seguro

Armar un entorno aislado, tomar snapshots, verificar hashes y registrar cada práctica con su alcance.

Prerrequisitos: Ninguno.

Etapa 1 · Fundamentos

Leer un binario sencillo: números, memoria, procesos, formato PE y programas propios en C y Python.

Prerrequisitos: Etapa 0.

Etapa 2 · Introducción al reversing

Leer assembly x86/x64, seguir el flujo y usar debugger y desensamblador con criterio. Se divide en 2A (base, 2.1–2.9), 2B (Windows intermedio, 2.10–2.16: ofuscación, TLS, MFC, huella, PE fino, anti-análisis, Ghidra) y 2C (Windows avanzado, 2.17+: packers didácticos, C++ profundo, WinDbg/heap, patch diffing, scripting) y 2E (desofuscación y análisis dinámico avanzado, 2.25+: ejecución simbólica, ofuscación por VM, protectores multicapa, neutralizar anti-análisis, DBI con DynamoRIO/Pin y Frida Stalker, extensiones de Ghidra/SLEIGH, y ampliaciones de Go/Rust y VB6). Las rutas de malware dinámico, shellcode/C2 y fuzzing quedan como optativas, fuera de la puerta obligatoria.

Prerrequisitos: Etapa 1.

Etapa 3 · .NET como primera victoria

Entender el runtime .NET, leer IL y comparar fuente, IL y decompilación en un programa propio.

Prerrequisitos: Etapa 2.

Etapa 4 · Práctica guiada

Analizar programas propios con método, reconstruir un formato o protocolo desde muestras, contener la ejecución y escribir un informe reproducible.

Prerrequisitos: Etapas 0 a 3.

Etapa 5 · Ramas opcionales

Elegir una especialización dentro de Windows, con prerrequisitos y práctica propia, sin convertir el corpus en una lista de tareas peligrosas.

Prerrequisitos: Etapas 0 a 4 completas.

Etapa 6 · Rama optativa · Kernel y drivers

Comprender, construir y revisar un driver KMDF didáctico con una interfaz inocua, depurarlo solo en VM descartable y explicar mitigaciones actuales.

Prerrequisitos: Etapa 0 (laboratorio aislado), C, assembly, memoria y sincronización, WinDbg y 2D.1 (objetos/tokens), 2D.2 (memoria/loader) y 2D.11 (excepciones/debugging). Etapa 2C es recomendada, no requisito; no hace falta completar toda 2D.

Material de apoyo

- Índice de videotutoriales: dónde está en Windows cada video que menciona el curso, con enlace y ruta absoluta.
- Curso IDA desde cero (69 tutoriales): índice por tema, con el texto extraído de cada uno.
- Teorias_Numeradas clasificadas: los 2.017 títulos separados en educativos vs cracking de software comercial.
- Transcripciones de Telegram por tema: las 523 transcripciones tamizadas; dónde están las sesiones de práctica.
- Fichas de recursos: qué se revisó, qué falta y qué límites tiene cada fuente.
- Glosario: términos en lenguaje de principiante.
- Preguntas y respuestas: repaso por etapa.
- Fuentes y límites: criterios, cobertura y exclusiones.

Dónde seguir practicando (legal y seguro)

Esta guía practica con **programas propios**, pero conviene seguir con retos pensados para aprender. Estos recursos son **recomendación externa** (no salen del corpus CLS) y permiten practicar reversing de Windows sin problemas legales:

- crackmes.one: miles de crackmes subidos para aprender; filtrá por plataforma **Windows** y por dificultad. Analizalos en tu VM.
- Flare-On Challenge: competencia anual y gratuita de reversing, centrada en Windows; los retos de años anteriores quedan publicados con soluciones.
- **CTFs (categoría rev)**: CTFtime lista competencias; muchos retos de reversing son binarios de Windows. Buenísimos para practicar con límite de tiempo.
- **Talleres abiertos**: materiales públicos de análisis de malware y reversing (por ejemplo, OALabs o Malware Unicorn) con binarios de práctica.
- **Tus propios programas**: seguí compilando ejemplos con condiciones cada vez más complejas; es la práctica más segura y la que más enseña.

Regla que no cambia: todo en la VM aislada de la Etapa 0, y nada de software ajeno sin permiso.

Etapa 0 · Laboratorio y método seguro

> **Objetivo medible**
>
> Armar un entorno aislado, tomar snapshots, verificar hashes y registrar cada práctica con su alcance.

Prerrequisitos: Ninguno. **Duración estimada:** 4 a 6 horas. Es una estimación de esta guía, derivada de la suma de las lecciones; no es una duración declarada por CLS.

Lecciones

#	Lección	Qué aprendés
0.1	Un laboratorio que podés romper sin miedo	VM, snapshots, red aislada, transferencia por ISO y hashes.

Recursos de CLS para esta etapa

- Inventario limpio de CLS
- Transcripciones de Telegram

Evaluación de salida

- Mostrar una VM con snapshot limpia y red aislada.
- Transferir un archivo sin carpeta compartida y verificar su SHA-256 en host y guest.
- Escribir una bitácora con snapshot, programa, hash y observaciones.
- Explicar qué harías si el programa deja el sistema en un estado raro.

> **Entregable de la etapa**
>
> Un archivo en tu bitácora con: lecciones completadas, ejercicios resueltos, dudas abiertas y evidencia (capturas, valores, hashes). Sin evidencia no se considera aprobada.

Errores frecuentes

- Usar carpeta compartida "solo para pasar el archivo".
- Tomar la snapshot después de copiar la muestra.
- Confundir una snapshot con un backup de los archivos del host.

Qué no entra en esta etapa

Los temas que no tienen material revisado o que exceden el marco se listan en Kernel y otros territorios. No los uses como atajo: cada rama tiene prerrequisitos.

Un laboratorio que podés romper sin miedo

> **Objetivo de la lección**
>
> Al terminar vas a poder armar una máquina virtual aislada, tomar una snapshot, transferir un programa de práctica sin compartir carpetas y registrar su hash antes y después de analizarlo. Ese es el piso de todo lo que sigue.

Prerrequisitos: ninguno. **Duración sugerida:** 4 a 6 horas, casi todas de trabajo práctico. Es una estimación aproximada de esta guía, no una duración declarada por el material de CLS.

Por qué el laboratorio va primero

Reversing significa ejecutar y observar programas que no escribiste. A veces son programas de práctica inofensivos; otras veces son binarios de origen desconocido. La diferencia entre aprender y arruinar tu máquina está en **dónde** los ejecutás.

Un laboratorio de reversing tiene una regla simple: **lo que se rompe, se descarta; lo que importa, vive afuera**. El programa sospechoso corre en una máquina virtual (VM) que podés borrar y recrear. Tus documentos, tus cuentas, tus claves y tus carpetas compartidas se quedan en el host, fuera del alcance.

El curso de CLS usa esta idea desde el principio. En el capítulo 01, Solid abre un ejecutable de práctica en x64dbg y explica que el debugger sirve para ver el comportamiento de un programa **mientras está corriendo en memoria** (`Reversing desde cero, cap. 01`, 00:00:26). Para observar hay que ejecutar. Por eso el entorno importa.

> **Lo que este curso no hace**
>
> No vas a ejecutar adjuntos del chat, cracks, keygens, instaladores desconocidos ni muestras en cuarentena. Tampoco vas a analizar malware dinámicamente en tu máquina personal. Las prácticas usan **programas propios**, software abierto o crackmes diseñados para aprender, y siempre dentro de la VM.

Las tres capas: host, hipervisor e invitado

Conviene fijar tres palabras que vas a ver todo el tiempo:

Capa	Qué es	Ejemplos	Qué va adentro
Host (anfitrión)	Tu computadora real	Windows 11, tu notebook	Tus documentos, cuentas, claves
Hipervisor	El programa que crea y administra máquinas virtuales	VirtualBox, VMware Workstation Player, Hyper-V	La configuración de las VMs
Guest (invitado)	El sistema operativo que corre dentro de la VM	Windows 10/11, Linux	Las herramientas de análisis y las muestras

El guest es descartable. El host no. Todo lo que hagas para "compartir" entre ambos —carpetas compartidas, portapapeles bidireccional, unidades de red, USB— es un puente, y los puentes se cruzan en las dos direcciones.

> **El error más común**
>
> Crear la VM, activar "carpeta compartida" para pasar el programa, ejecutarlo y descubrir que el malware cifró la carpeta del host. Si necesitás pasar un archivo, usá una imagen ISO de solo lectura o una unidad extraíble virtual que montás, copiás y desmontás. Nunca una carpeta compartida permanente.

Reglas de diseño del laboratorio

Estas reglas son **recomendación editorial** de la guía, basadas en prácticas estándar de análisis seguro. No salen de un archivo de CLS; se apoyan en el principio de aislamiento que el propio material usa cuando analiza binarios.

1. **Una VM por objetivo.** Si analizás dos programas distintos, usá dos snapshots o dos VMs. Así sabés qué cambió cada uno.
2. **Snapshot limpia antes de empezar.** La snapshot se toma con el sistema actualizado, sin la muestra y sin conexión a tu red personal. Ese es tu "punto cero".
3. **Red host-only o NAT sin acceso a la LAN.** La VM no debe ver tus otras computadoras ni tus unidades de red. Si el programa necesita internet, evaluá si de verdad lo necesita; para un crackme, no.
4. **Sin credenciales reales.** Dentro de la VM usá una cuenta local descartable, sin tus contraseñas, sin tu correo, sin tus tokens.
5. **Sin carpetas compartidas ni portapapeles bidireccional.** Si el hipervisor lo permite, desactivalos explícitamente.
6. **Herramientas instaladas desde el inicio.** x64dbg, un editor hexadecimal, Python, un desensamblador. Instalarlas después de la muestra contamina la línea base.
7. **Registro desde el minuto cero.** Anotá qué VM, qué snapshot, qué archivo, qué hash y qué hiciste.

Ejemplo paso a paso: laboratorio mínimo con VirtualBox

Voy a mostrar el procedimiento con VirtualBox porque es gratuito y multiplataforma. Los conceptos se trasladan a VMware o Hyper-V; los nombres de los botones cambian.

Paso 1. Crear la VM

1. Descargá VirtualBox desde el sitio oficial y una imagen ISO de Windows 10/11 desde Microsoft. **No** uses imágenes "preactivadas" ni ISOs de origen dudoso: son un riesgo y además complican la trazabilidad.
2. Creá una VM nueva: 4 GB de RAM, 2 CPUs y un disco virtual de 60 GB. Para reversing de programas de práctica alcanza.
3. Instalá el sistema operativo. Cuando termine, aplicá las actualizaciones y reiniciá.

Por qué: una VM con recursos suficientes evita que el debugger se trabe y que confundas un problema de rendimiento con un comportamiento del programa.

Paso 2. Aislar la red

En la configuración de la VM, revisá la pestaña **Red**:

- Adaptador 1: **NAT** o **Host-only**. Nat permite salir a internet; host-only solo habla con el host.
- Desactivá el adaptador puente (bridged): ese conecta la VM directamente a tu red local.
- En **General > Avanzado**, desactivá el portapapeles compartido y el arrastrar y soltar.
- En **Carpetas compartidas**, no agregues ninguna.

Por qué: el adaptador puente es el que convierte una VM en otro dispositivo de tu red. Para analizar un crackme no lo necesitás.

Paso 3. Tomar la snapshot "limpia"

1. Apagá la VM correctamente (no la suspendas).
2. En VirtualBox, seleccioná la VM y usá **Instantáneas > Tomar instantánea**.
3. Nombrala `00-limpia-sin-muestra` y escribí una descripción con la fecha.

Por qué: una snapshot guarda el estado del disco. Si algo sale mal, restaurás y volvés exactamente al punto anterior. No es un backup de tus archivos del host; es el estado de la VM.

Paso 4. Preparar la transferencia sin puentes

Para pasar un programa de práctica:

1. Creá una imagen ISO con el archivo. En Windows podés usar `oscdimg` o una herramienta gráfica; en Linux, `genisoimage`.
2. Montá la ISO en la VM como unidad de CD/DVD, copiá el archivo al disco del guest y **desmontá** la ISO.
3. Verificá el hash dentro del guest y comparalo con el del host.

En PowerShell, el comando es:

```
Get-FileHash -Algorithm SHA256 .\programa.exe
```

Salida típica (el hash es inventado a modo de ejemplo):

```
Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          A1B2C3D4E5F60718293A4B5C6D7E8F90123456789ABCDEF0123456789ABCDEF0
C:\lab\programa.exe
```

Por qué el hash: dos archivos con el mismo SHA-256 son, en la práctica, el mismo archivo. Si el hash del host y el del guest coinciden, sabés que la transferencia no alteró nada. Si no coinciden, no analices: primero entendé por qué cambió.

Paso 5. Ejecutar y volver atrás

1. Confirmá que estás en la snapshot `00-limpia-sin-muestra`.
2. Anotá en tu bitácora: fecha, nombre del archivo, hash, herramienta y objetivo.
3. Ejecutá el programa **solo si es propio, abierto o un crackme de práctica**.
4. Al terminar, apagá la VM y restaurá la snapshot. No reutilices el estado "sucio" para la próxima muestra.

Por qué: restaurar no es opcional. Un programa puede dejar archivos, claves de registro o servicios corriendo. La snapshot te devuelve al punto conocido.

Qué muestra CLS y qué corregimos

En CLS hay un ejemplo real de laboratorio con dos VMs: el video 37 de «Videos y material adicional» analiza el malware Shelter dentro de una VM con Windows 7 de 32 bits, y usa otra VM con Kali Linux para simular el ataque (`Video 37, introducción`, 00:00:00–00:00:40).

Ese video **sí ejecuta el malware** y monta un reverse shell dentro de las VMs. La separación en máquinas virtuales es correcta como idea, pero el curso no convierte eso en una práctica recomendada: para esta guía, Shelter es un **caso de estudio observado**, no una tarea. Si querés analizar malware, la rama defensiva de la Etapa 5 te muestra cómo hacer triage estático y por qué no se empieza ejecutando.

También conviene registrar una diferencia de criterio: el inventario del proyecto aclara que, durante su confección, **no se movió, extrajo ni ejecutó ningún archivo** (`Inventario limpio, LEEME`). Esa es la conducta que esta guía adopta para el corpus: leer, listar y citar; no ejecutar.

Práctica: tu laboratorio en una página

Hacé esto y guardá el resultado. Es el primer entregable del curso.

1. Creá la VM con las reglas de arriba.
2. Tomá la snapshot limpia.
3. Escribí tu propio programa de prueba. El más simple: un `.exe` que pida una palabra y muestre "hola" si es `clase`. Compilalo en el guest (o en el host y pasalo por ISO).
4. Calculá el SHA-256 en el host y en el guest. Pegá ambos en la bitácora.
5. Ejecutalo en x64dbg si ya lo instalaste; si no, ejecutalo suelto y anotá qué pasa.
6. Restaurá la snapshot y verificá que el archivo que copiaste ya no está.

Entregable mínimo

Un archivo `bitacora.md` con: nombre de la VM, snapshot usada, programa, hash del host, hash del guest, red configurada, qué hiciste y qué observaste. Sin ese registro, la práctica no cuenta como terminada.

Preguntas de repaso

1. ¿Qué diferencia hay entre una snapshot y un backup? Una snapshot guarda el estado de la VM en un momento; sirve para volver atrás después de un experimento. Un backup es una copia de tus datos para recuperarlos si se pierden. La snapshot no protege tus archivos del host y no reemplaza un backup.

2. El inventario marca un archivo como `util`. ¿Alcanza para ejecutarlo? No. `util` es una clasificación de catálogo, no un certificado de seguridad, calidad ni vigencia. La decisión de ejecutar depende de qué es el archivo, de dónde salió y de si tenés permiso para analizarlo.

3. ¿Por qué se desactiva la carpeta compartida? Porque es un canal bidireccional entre guest y host. Un programa con comportamiento malicioso puede leer, cifrar o borrar archivos del host a través de ese canal. La ISO de solo lectura evita esa vía.

4. ¿Para qué sirve comparar el hash antes y después de copiar? Para detectar alteraciones en la transferencia y para identificar el archivo sin ambigüedad. Si los hashes no coinciden, el archivo que analizás no es el que creías.

5. ¿La VM alcanza para analizar cualquier muestra? No. Una VM mal configurada puede tener acceso a la red local, al portapapeles o a carpetas compartidas. Además, algunas muestras detectan la VM y cambian de comportamiento. Para empezar, alcanza con programas propios y crackmes; para malware real hace falta un laboratorio dedicado y procedimientos que esta guía no cubre en la Etapa 0.

Antes de avanzar

Tenés que poder:

- Explicar con tus palabras la diferencia entre host, hipervisor y guest.
- Crear una VM, aislarla de la red local y tomar una snapshot.
- Transferir un archivo sin carpeta compartida y verificar su SHA-256 en ambos lados.
- Describir qué harías si un programa deja el sistema raro: apagar, restaurar, revisar la bitácora.

Puerta de salida de la Etapa 0

Si no podés mostrar tu `bitacora.md` con los dos hashes y la snapshot, no pases a la lección siguiente. El resto del curso asume que tenés un entorno donde equivocarte no cuesta caro.

Fuentes de esta lección

- `Reversing desde cero, cap. 01 (x64dbg primeros pasos)` — qué hace un debugger y por qué se ejecuta el programa.
- `Videos y material adicional, Video 37` — ejemplo observado de laboratorio con dos VMs y ejecución de malware; se cita como caso, no como modelo.
- `Inventario limpio, LEEME` — criterio de no ejecutar ni extraer archivos durante el inventario.
- `Fuentes y límites de esta edición` — criterios de alcance y seguridad.

> **Sobre las capas de esta lección**
>
> **Contenido observado en CLS:** el uso de VMs en el video 37 y el criterio de no ejecución del inventario. **Conocimiento externo:** funcionamiento de hipervisores, snapshots y hashes. **Recomendación editorial:** las siete reglas de diseño y el procedimiento paso a paso.

Etapa 1 · Fundamentos

> **Objetivo medible**
>
> Leer un binario sencillo: números, memoria, procesos, formato PE y programas propios en C y Python.

Prerrequisitos: Etapa 0. **Duración estimada:** 21 a 25 horas. Es una estimación de esta guía, derivada de la suma de las lecciones; no es una duración declarada por CLS.

Lecciones

#	Lección	Qué aprendés
1.1	Bits, bytes, hexadecimal y números con signo	Binario, hex, complemento a dos, little-endian y VA→offset.
1.2	Memoria: stack, heap y marcos de función	Stack, prólogo/epílogo, argumentos y shadow space en x64.
1.3	Procesos, hilos y mapa de memoria	Procesos, hilos, handles, mapa de memoria y permisos.
1.4	El formato PE: cabeceras, secciones e imports	Cabecera PE, secciones, IAT y ASLR.
1.5	C y Python mínimos para analizar	Programas propios en C y herramientas en Python.

Recursos de CLS para esta etapa

- Reversing desde cero (caps. 01–05)
- Python ISO video tutes (lógica, no Python)
- Curso de MASM32 (Videos 1–22)

Evaluación de salida

- Convertir valores entre binario, hex y decimal sin calculadora.
- Dibujar el stack de una función de 32 bits y explicar el de 64 bits.
- Calcular un file offset a partir de una VA y verificarlo en el archivo.
- Compilar un programa propio en C y leer su hash y cabecera PE.

> **Entregable de la etapa**
>
> Un archivo en tu bitácora con: lecciones completadas, ejercicios resueltos, dudas abiertas y evidencia (capturas, valores, hashes). Sin evidencia no se considera aprobada.

Errores frecuentes

- Memorizar la tabla hexadecimal sin entender el valor posicional.

- Confundir dirección virtual con offset del archivo.
- Creer que el curso de Python de CLS enseña Python (enseña lógica).

Qué no entra en esta etapa

Los temas que no tienen material revisado o que exceden el marco se listan en Kernel y otros territorios. No los uses como atajo: cada rama tiene prerrequisitos.

Bits, bytes, hexadecimal y números con signo

> **Objetivo de la lección**
>
> Al terminar vas a poder convertir entre binario, hexadecimal y decimal sin calculadora, interpretar un valor de 32 bits como positivo o negativo según el contexto, y leer los bytes de un número en memoria sin confundir el orden. También vas a poder verificar una dirección virtual contra el archivo en disco.

Prerrequisitos: Etapa 0 (laboratorio y bitácora). **Duración sugerida:** 4 a 5 horas, con ejercicios. Estimación aproximada de esta guía.

Por qué esto no es un trámite

Cuando abrís un programa en un debugger ves tres columnas: direcciones, bytes y una versión legible de esos bytes. Todo está en hexadecimal. Si no podés pasar de `FB` a `251` ni entender por qué `FFFFFFFF` a veces es `-1`, cada pantalla se vuelve un jeroglífico y terminás copiando pasos sin entender.

La buena noticia: el reversing cotidiano usa muy pocos conceptos numéricos, y son los mismos siempre. Esta lección los junta.

Bit, byte y nibble

Un **bit** es la unidad mínima: vale 0 o 1. Un **byte** son 8 bits. Un **nibble** son 4 bits, que es exactamente lo que representa un dígito hexadecimal.

Por eso hexadecimal y binario se llevan tan bien: cada dígito hexadecimal equivale a cuatro bits.

Hex	Binario	Decimal
0	0000	0
1	0001	1
2	0010	2
4	0100	4
8	1000	8
A	1010	10
F	1111	15

Para leer un byte en hexadecimal, separá los dos dígitos y convertí cada uno por separado:

```
0x8F  ->  8 = 1000     F = 1111   ->  1000 1111  ->  128 + 8 + 4 + 2 + 1 = 143
0xFB  ->  F = 1111     B = 1011   ->  1111 1011  ->  128+64+32+16+8+2+1 = 251
```

El método mental que usa Solid en el capítulo 01 es asignar a cada bit su potencia de dos, de derecha a izquierda: 1, 2, 4, 8, 16, 32, 64, 128. Para representar 10, prendés 8 y 2:

```
bit:     7   6   5   4   3   2   1   0
valor: 128  64  32  16   8   4   2   1
10  ->   0   0   0   0   1   0   1   0     = 0x0A
```

Esto aparece explicado en el capítulo 01 (`Reversing desde cero, cap. 01`, 00:19:00–00:22:00).

> **El audio se equivoca en una palabra**
>
> En el capítulo 01, alrededor de 00:22:33, la transcripción dice que "el byte más significativo es el de la derecha". La idea que quiere transmitir es correcta —en memoria, el byte **menos** significativo va primero— pero la palabra está invertida. Quedate con la regla de abajo, no con la frase del audio.

> **En el Curso IDA: convertir bases sin calculadora**
>
> El Curso IDA aprovecha la **barra de Python** que IDA trae en la parte de abajo para pasar de una base a otra al instante: `hex(143)` devuelve `0x8f`, `bin(10)` te da `0b1010`, `0x8f` tipeado solo te lo muestra en decimal. Si esa barra no aparece, es que falta el soporte de Python de IDA (reinstalarlo lo soluciona) (Curso IDA · tute 2, Sistemas de numeración).

Little-endian: cómo se guarda un número en memoria

x86 y x64 son **little-endian**: el byte menos significativo (el de menor peso) se guarda en la dirección más baja. Si en memoria ves estos cuatro bytes:

```
dirección:  1000  1001  1002  1003
contenido:    0D    0C    0B    0A
```

el valor de 32 bits es `0x0A0B0C0D`, no `0x0D0C0B0A`. Para leerlo, das vuelta los bytes.

La confusión típica es Mirar la columna de bytes del debugger y creer que ya es el número. No: es la representación **en memoria**. Los registros, en cambio, te muestran el valor ya interpretado.

El capítulo 01 lo explica con un ejemplo de cuatro bytes `41 42 43 44` y aclara que en memoria se leen al revés (00:22:33–00:23:30). El capítulo 02 lo vuelve a mostrar al mover un valor desde una dirección a `EAX`: los bytes `36 00 00 38` en memoria se convierten en `0x38000036` en el registro (`Reversing desde cero, cap. 02`, 00:07:12–00:07:50).

Números con signo: complemento a dos

Un mismo patrón de bits puede interpretarse de dos maneras. `0xFB` es 251 si el dato es **sin signo** y −5 si es **con signo**. El procesador no guarda el signo en un bit aparte: usa una convención llamada **complemento a**

dos.

La regla práctica:

- Si el bit más alto es 0, el número es positivo y vale lo mismo que en sin signo.
- Si el bit más alto es 1, el número es negativo. Para saber cuánto vale, invertí todos los bits y sumá 1 (o restá el valor a 2^n).

Ejemplo con 8 bits:

```
 5  = 0000 0101
-5  = 1111 1010 + 1 = 1111 1011 = 0xFB
```

Comprobación: `0xFB` = 251. Y 256 − 251 = 5, así que representa −5. La relación es `valor_con_signo = valor_sin_signo - 2^8`.

Los rangos que hay que saber de memoria:

Tamaño	Nombre	Bits	Sin signo	Con signo
1 byte	BYTE	8	0 a 255	−128 a 127
2 bytes	WORD	16	0 a 65.535	−32.768 a 32.767
4 bytes	DWORD	32	0 a 4.294.967.295	−2.147.483.648 a 2.147.483.647
8 bytes	QWORD	64	0 a 18.446.744.073.709.551.615	−9.223.372.036.854.775.808 a 9.223.372.036.854.775.807

Por eso `0xFFFFFFFF` es 4.294.967.295 sin signo y −1 con signo. Y `0x7FFFFFFF` es el máximo positivo de 32 bits: 2.147.483.647.

El capítulo 03 explica la idea de dividir el rango en dos mitades, una positiva y otra negativa (`Reversing desde cero, cap. 03`, 00:14:46–00:17:10). El audio se enreda al pasar de DWORD a WORD y dice un rango que no corresponde; la tabla de arriba es la que vale.

> **El tamaño importa: una suma que "se da vuelta"**
>
> Si un programa acumula valores en un registro de **un byte**, al pasar de `0xFF` el resultado se envuelve: `0xFF + 2 = 0x01`, porque solo se guarda la parte baja. Lo mismo vale para `WORD` o `DWORD` en su tope. Es un detalle que cambia por completo lo que compara una función. El capítulo 34 lo usa: un programa suma los caracteres que ingresás en un byte, y una suma que pasa de `0xFF` deja un valor chico (por ejemplo `0x149` guarda `0x49`). Cuando leas una comparación, fijate **qué tamaño** tiene el acumulador (`Reversing desde cero, cap. 34`).

Ejemplo paso a paso: de una dirección virtual al offset del archivo

Este es el cálculo que más vas a repetir cuando pases de un debugger a un editor hexadecimal o a un desensamblador. El capítulo 05 lo hace con un ejecutable concreto (`Reversing desde cero, cap. 05`,

00:00:40–00:04:20).

Datos del ejemplo:

- Image Base: `0x6D0000`
- Dirección virtual (VA) de la instrucción: `0x6D1372`
- Tamaño reservado para el header en memoria: `0x1000`
- `PointerToRawData` de la sección `.text`: `0x400`

Paso 1. Calculá la RVA (Relative Virtual Address), que es la distancia desde la base:

```
RVA = VA - Image Base
    = 0x6D1372 - 0x6D0000
    = 0x1372
```

Paso 2. Restá el espacio que el header ocupa en memoria (`0x1000`) y sumá el offset real donde empieza `.text` en el archivo (`0x400`):

```
File Offset = RVA - 0x1000 + 0x400
            = 0x1372 - 0x1000 + 0x400
            = 0x372 + 0x400
            = 0x772
```

Paso 3. Verificá en el archivo. En el offset `0x772` del `.exe` deberían estar los bytes `E8 C4 03 00 00`, los mismos que muestra el debugger en esa dirección. El capítulo 05 lo comprueba con un editor hexadecimal (00:03:41–00:04:30).

> **Por qué se resta 0x1000 y no el tamaño real del header**
>
> El cargador de Windows alinea el header a `0x1000` en memoria, aunque el header real sea más chico (en el ejemplo, `.text` empieza en el archivo en `0x400`). Por eso la fórmula usa la diferencia entre la alineación de memoria y el offset real de la sección. Cada sección tiene su propio `PointerToRawData`; no uses `0x400` para `.rdata` ni para `.data`.

Ejemplo paso a paso: a dónde salta un `call` directo

En un `call` directo, el opcode `E8` va seguido de un desplazamiento de 4 bytes. La dirección destino se calcula sumando al desplazamiento la dirección de la **instrucción siguiente** (no la del `call`).

En el capítulo 05, el `call` está en `0x6D1372`, mide 5 bytes y su desplazamiento es `0x3C4`:

```
destino = VA_del_call + tamaño_del_call + desplazamiento
        = 0x6D1372 + 5 + 0x3C4
        = 0x6D1377 + 0x3C4
        = 0x6D173B
```

El debugger muestra `0x6D173B` como destino. Si el salto fuera hacia atrás, el desplazamiento sería negativo y se rellenaría con `FF` a la izquierda antes de sumar (cap. 05, 00:09:00–00:12:00).

Práctica

1. Convertí a hexadecimal: 42, 200, 255, 1024, 4096.
2. Convertí a decimal: `0x2A`, `0xC8`, `0xFF`, `0x400`, `0x1000`.
3. ¿Qué valor con signo representa `0xFFFFFFFB` en 32 bits? ¿Y sin signo?
4. Escribí los bytes que ocupa `0x11223344` en memoria little-endian.
5. Con Image Base `0x400000`, VA `0x401040`, header `0x1000` y `.text` en `0x400`, calculá el file offset.
6. Un `call` en `0x401000` tiene desplazamiento `0x20`. ¿A qué dirección salta? (Ojo con el tamaño de la instrucción.)
7. Abrí PowerShell y comprobá tus conversiones:

```
'{0:X}' -f 42          # 2A
'{0:X8}' -f 4294967295 # FFFFFFFF
[Convert]::ToInt32('FFFFFFFB',16)  # -5
```

Preguntas de repaso

1. ¿Cuántos bits hay en un byte y cuántos dígitos hexadecimales lo representan? Ocho bits y dos dígitos hexadecimales, porque cada dígito representa cuatro bits.

2. ¿Por qué `0xFB` puede ser 251 o −5? Porque el patrón de bits es el mismo. El tipo del dato y la instrucción que lo usa deciden si se interpreta con signo o sin signo.

3. ¿Qué significa little-endian? Que el byte menos significativo se almacena en la dirección más baja. Para leer un valor de memoria hay que invertir el orden de los bytes.

4. ¿La dirección que muestra el debugger es la misma que la del archivo en disco? No. La del debugger es una dirección virtual; la del archivo es un offset. Se relacionan con la fórmula `offset = RVA − alineación_del_header + PointerToRawData`.

5. ¿Por qué el destino de un `call` se calcula desde la instrucción siguiente? Porque el desplazamiento se mide desde el final de la instrucción actual, que es donde el procesador ya actualizó `EIP` / `RIP`. Sumar el tamaño de la instrucción es parte del cálculo.

Antes de avanzar

Tenés que poder, sin mirar la tabla:

- Pasar un byte de hexadecimal a binario y a decimal.
- Explicar complemento a dos con el ejemplo de `0xFB`.
- Escribir `0x0A0B0C0D` como bytes en memoria.
- Calcular un file offset a partir de una VA y un `call` directo a partir de su desplazamiento.

Puerta de salida de la Etapa 1 · bloque 1

Si los ejercicios 3 a 6 te salen con seguridad, seguí con memoria y procesos. Si no, repetilos con lápiz y papel antes de abrir un debugger: la pantalla no perdona la aritmética floja.

Fuentes de esta lección

- `Reversing desde cero, cap. 01` — bits, bytes, little-endian, tamaños (00:18:00–00:24:00).
- `Reversing desde cero, cap. 02` — lectura de un valor desde memoria (00:07:00–00:08:00).
- `Reversing desde cero, cap. 03` — números con signo (00:14:46–00:17:10).
- `Reversing desde cero, cap. 05` — VA, RVA, file offset y desplazamientos (00:00:40–00:12:00).
- Curso IDA · tute 2, Sistemas de numeración — binario/decimal/hexadecimal y la barra de Python de IDA para convertir bases.

Capas de esta lección

Contenido observado en CLS: el ejemplo numérico de la VA `0x6D1372`, la fórmula del offset y el cálculo del `call` a `0x6D173B`. **Conocimiento externo:** complemento a dos, rangos y aritmética hexadecimal. **Recomendación editorial:** el orden de los ejercicios y la advertencia sobre las imprecisiones del audio.

Memoria: stack, heap y marcos de función

> **Objetivo de la lección**
>
> Al terminar vas a poder dibujar el stack de una función antes y después de un `call`, ubicar los argumentos, el return address, el `EBP` guardado y las variables locales, y explicar por qué en 64 bits los primeros argumentos no están en el stack.

Prerrequisitos: números y little-endian (lección anterior). **Duración sugerida:** 4 horas. Estimación aproximada.

Las cuatro zonas que importan

Un proceso ve un espacio de direcciones dividido en regiones. Para empezar alcanza con cuatro:

Zona	Qué guarda	Crece hacia	Vida
Código (`.text`)	Instrucciones	—	Todo el proceso
Datos (`.data`, `.rdata`)	Variables globales, constantes, strings	—	Todo el proceso
Heap	Memoria pedida en tiempo de ejecución (`malloc`, `new`)	Direcciones altas	Hasta que la liberás
Stack	Argumentos, direcciones de retorno, variables locales	Direcciones bajas	Hasta que la función retorna

El stack y el heap son distintos no solo por su ubicación: cambian por motivos distintos. El stack lo administra el procesador con `push` / `pop` y llamadas; el heap lo administra el runtime (la librería C, el recolector de basura, etc.). Esta lección se concentra en el stack porque es el que vas a ver todo el tiempo en un debugger.

El stack es una pila LIFO

LIFO significa *last in, first out*: lo último que apilás es lo primero que sacás. En x86, el stack crece hacia direcciones **más bajas**. `ESP` (en 32 bits) o `RSP` (en 64) apunta siempre a la cima actual.

```
push eax      ; ESP = ESP - 4 ; [ESP] = EAX
pop  ebx      ; EBX = [ESP] ; ESP = ESP + 4
```

En 64 bits el decremento es de 8 bytes, salvo que empujes algo más chico.

Ejemplo paso a paso: el stack de una función de 32 bits

Este ejemplo sale del capítulo 18, donde Ricardo compara una función basada en `EBP` con una basada en `ESP` usando un ejecutable de práctica (`Reversing desde cero, cap. 18`, 00:02:40–00:08:30). La función `main`

recibe tres argumentos: `argc`, `argv` y `envp`.

Antes de llamar

La función que llama apila los argumentos **de derecha a izquierda**:

```
push envp
push argv
push argc
call main
```

El stack queda así (cada celda son 4 bytes; las direcciones bajas abajo):

```
        +------------------+
esp --> | argc             |
        | argv             |
        | envp             |
        +------------------+
```

El `call`

`call` hace dos cosas: empuja la dirección de retorno y salta. La dirección de retorno es la de la instrucción **siguiente** al `call`, para que `ret` sepa dónde volver.

```
        +------------------+
        | argc             |
        | argv             |
        | envp             |
        | return address   |  <-- ESP
        +------------------+
```

En el ejemplo del video, la dirección de retorno es `0x011912EA` (cap. 18, 00:07:16).

El prólogo

Casi todas las funciones de 32 bits empiezan igual:

```
push ebp          ; guarda el EBP de la función que llamó
mov  ebp, esp     ; fija un punto de referencia estable
sub  esp, 0x28    ; reserva espacio para variables locales
```

Después del prólogo, el stack tiene una forma reconocible:

```
        +------------------+  <-- EBP + 0x10
        | envp (3.er arg)  |
        | argv (2.° arg)   |  <-- EBP + 0x0C
        | argc (1.er arg)  |  <-- EBP + 0x08
        | return address   |  <-- EBP + 0x04
        | EBP guardado     |  <-- [EBP]
        +------------------+
        | local [EBP-4]    |
        | local [EBP-8]    |  <-- espacio reservado por sub esp, 0x28
esp --> | ...              |
        +------------------+
```

Cuidado con el orden: los argumentos se apilan de derecha a izquierda, así que el primero de la función queda en la dirección más baja, justo arriba del return address. Por eso `[EBP+8]` es `argc`, `[EBP+0x0C]` es `argv` y `[EBP+0x10]` es `envp`. Lo importante no es memorizar los números de este ejemplo, sino el método: **contá desde `EBP` y verificá en el panel de stack**.

En el video, Ricardo muestra la correspondencia entre la vista estática del stack de IDA y la realidad del debugger: los tres `push`, el return address, el `push ebp`, el `mov ebp, esp` y el `sub esp, 0x28` aparecen en ese orden (cap. 18, 00:02:34–00:10:00).

El epílogo

Al final, la función deshace lo que hizo:

```
mov esp, ebp      ; libera las variables locales
pop  ebp          ; recupera el EBP de la función que llamó
ret               ; saca el return address y vuelve
```

`leave` es una instrucción que hace las dos primeras operaciones juntas. Si ves `leave; ret`, es el mismo patrón.

> **Por qué `EBP` y no `ESP`**
>
> `ESP` cambia cada vez que la función empuja algo para llamar a otra función. Si las variables locales se referenciaran con `ESP`, cada `push` cambiaría sus direcciones. Por eso el prólogo copia `ESP` a `EBP` y usa `EBP` como referencia estable: `[EBP-4]`, `[EBP-8]`, etc. (cap. 18, 00:10:30–00:12:10).

64 bits: los argumentos viajan en registros

En x64 cambia la convención. Los primeros cuatro argumentos enteros van en registros:

Argumento	Registro
1.º	`RCX`
2.º	`RDX`
3.º	`R8`
4.º	`R9`

Los demás van en el stack, y el llamador debe reservar **32 bytes de shadow space** (`0x20`) antes del `call`, aunque no use todos los registros. Por eso en el capítulo 18 se ve `sub rsp, 0x138` para las variables locales y referencias del tipo `[rsp+0x20]`, `[rsp+0x28]`, etc. (00:14:00–00:20:00).

En 64 bits es común que las funciones sean **basadas en RSP**: no hay `push rbp; mov rbp, rsp` porque `RSP` no cambia tanto (los argumentos no se apilan uno por uno). Las variables locales se referencian como `[rsp+0xNN]`, y el desensamblador suele mostrar el offset relativo al valor inicial de `RSP` para que no tengas que recalcular.

> **El error de contar mal el shadow space**
>
> Cuando el quinto argumento va al stack, no va inmediatamente después del return address: va después de los 32 bytes reservados para los cuatro registros. Si ves `[rsp+0x28]` como primer argumento apilado, es porque `0x20` son los shadow slots y `0x28` el primer argumento real (cap. 18, 00:17:00–00:18:30).

Heap: la memoria que pedís vos

El heap se usa cuando el tamaño no se conoce en tiempo de compilación o cuando el dato tiene que sobrevivir a la función que lo creó. En C:

```
char *buf = malloc(64);
if (buf == NULL) { /* error */ }
strcpy(buf, "hola");
free(buf);
```

En Python, `bytearray(64)` o una lista cumplen un rol parecido, pero el manejo lo hace el intérprete.

Tres diferencias prácticas con el stack:

1. **Vos administrás la vida** (en C). Si te olvidás de `free`, hay fuga de memoria; si liberás dos veces, corrupción.
2. **No tiene un orden LIFO.** Podés liberar en cualquier orden.
3. **Los errores son más silenciosos.** Un desborde de stack suele terminar en crash; un desborde de heap puede pasar desapercibido y explotar después.

Estructuras y objetos en memoria

Una **estructura** (`struct`) es un dato compuesto: agrupa varios campos, que pueden ser de **distinto tipo**, uno detrás del otro. Es la diferencia con un array, que repite el mismo tipo. En memoria, una estructura es un bloque donde cada campo vive en un **offset fijo** desde el inicio.

Por eso, a bajo nivel, trabajar con una estructura casi siempre se ve igual: un registro tiene la **dirección base** de la estructura y las instrucciones le suman un offset para llegar a cada campo.

```
mov eax, [ebp-4]      ; EAX = dirección base de la estructura
mov ecx, [eax+0x58]   ; lee el campo que está en el offset 0x58
lea edx, [eax+4]      ; NO lee: deja en EDX un puntero al campo en +4
```

La distinción entre `mov ecx, [eax+0x58]` (leer el **valor** del campo) y `lea edx, [eax+4]` (calcular la **dirección** del campo, para pasarla como puntero) es la que más confunde al principio. Si no hay corchetes alrededor del resultado, es una dirección.

> **El padding cambia los offsets**
>
> El tamaño de una estructura **no** es siempre la suma de sus campos. Por alineación, el compilador inserta bytes de relleno (*padding*). En 32 bits la alineación por defecto es 4, así que una estructura con un `DWORD` (4) y un `BYTE` (1) no mide 5 sino **8**: se agregan 3 bytes para que el tamaño sea múltiplo de 4. Si al leer un listado los offsets no cierran, sospechá del padding (cap. 20b).

Un **objeto** de C++ es una estructura con un par de agregados. Cuando ves `new`, en el fondo se llama a `malloc` para reservar el tamaño de la clase (por ejemplo `0x3C` bytes) y después se llama al **constructor** para inicializar los campos. A los métodos, la instancia (el puntero `this`) se pasa normalmente en **ECX**; dentro del método, `[ecx+offset]` accede a cada campo.

Si la clase tiene **métodos virtuales**, el primer campo del objeto (offset 0) es un puntero a la **vtable**: una tabla de punteros a esos métodos. Por eso, al reversear, "levanta el primer campo y llama a `[campo+N]`" es la firma de una llamada a un método virtual. El cap. 33 lo muestra con una clase propia cuyo objeto empieza con el puntero a la vtable y cuyos cuatro métodos virtuales están en la tabla, en orden.

Buffers, strings y el centinela

Un string en C es una secuencia de bytes terminada en `0x00`. `"Hola"` ocupa cinco bytes: `48 6F 6C 61 00`. Ese cero es el que permite a `strlen` saber dónde termina.

Cuando una función copia datos a un buffer sin verificar el tamaño, puede escribir más allá del final. Ese es el punto de partida de los desbordes de buffer, que en esta guía se estudian solo en programas propios y en la rama de vulnerabilidades. El capítulo 15 muestra un detalle defensivo relacionado: el **security cookie** que el compilador coloca en el stack y verifica antes de retornar, calculado con un XOR contra `RSP`, para detectar que alguien pisó el return address (`Reversing desde cero, cap. 15`, 00:07:34–00:08:00).

Práctica

1. Dibujá el stack de una función `int suma(int a, int b)` en 32 bits, desde el `push b` hasta el `ret`. Marcá `EBP`, `ESP`, argumentos, return address y variable local.
2. Escribí en C una función que reserve un buffer de 16 bytes en el stack y otra que use `malloc(16)`. Compilalas y mirá en x64dbg dónde queda cada buffer.
3. En el programa anterior, poné un breakpoint en la primera instrucción de la función y anotá `ESP` y `EBP`. Después del prólogo, volvé a anotarlos. ¿Cuánto cambió `ESP`?
4. Escribí el mismo programa para x64 y verificá en qué registros llegan `a` y `b`.
5. Explicá por qué `[ebp+8]` suele ser el primer argumento en 32 bits y por qué en 64 bits no lo es.

Preguntas de repaso

1. ¿Quién empuja el return address? La instrucción `call`, no la función llamada. `ret` lo consume.

2. ¿Por qué el stack crece hacia abajo? Es una convención de la arquitectura x86: `push` decrementa el puntero y `pop` lo incrementa. Lo importante es que el modelo mental sea consistente, no la dirección física.

3. ¿Qué guarda `push ebp` en el prólogo? El valor de `EBP` de la función que llamó, para poder restaurarlo al final y no romper las referencias de esa función.

4. ¿Dónde están las variables locales? Entre `EBP` y `ESP`, en direcciones menores que `EBP` (`[EBP-4]`, `[EBP-8]`, etc.). El `sub esp, N` reserva ese espacio.

5. ¿Los argumentos de una función de 64 bits siempre están en el stack? No. Los primeros cuatro enteros van en `RCX`, `RDX`, `R8` y `R9`; el resto va al stack después de 32 bytes de shadow space.

6. ¿Qué diferencia hay entre un desborde de stack y uno de heap? El de stack suele pisar el return address y provocar un crash inmediato; el de heap corrompe estructuras internas y puede no manifestarse hasta mucho después.

Antes de avanzar

Tenés que poder:

- Dibujar el stack de un `call` de 32 bits con argumentos, return address, `EBP` guardado y locales.
- Nombrar los registros de argumentos de x64 y explicar el shadow space.
- Explicar la diferencia entre stack y heap con un ejemplo de cada uno.
- Reconocer un prólogo y un epílogo en un listado de assembly.

> **Puerta de salida**
>
> Si podés explicar el stack sin mirar el dibujo, estás listo para verlo en vivo en el debugger. Si no, volvé al ejemplo del capítulo 18 y reconstruilo en papel.

Fuentes de esta lección

- `Reversing desde cero, cap. 18` — stack de 32 y 64 bits, prólogo, epílogo, shadow space.
- `Reversing desde cero, cap. 15` — security cookie y protección del return address.
- `Reversing desde cero, cap. 01` — ventana de stack y mapa de memoria.
- `Reversing desde cero, cap. 20b` — estructuras, acceso base+offset, stack vs heap y padding/alineación (transcripción .txt, sin marcas).
- `Reversing desde cero, cap. 33` — objetos C++: `new` / `malloc`, constructor, `this` en ECX y puntero a la vtable en el offset 0 (transcripción .txt, sin marcas).

> **Capas de esta lección**
>
> **Contenido observado en CLS:** el ejemplo del capítulo 18 (argumentos, return address `0x011912EA`, `sub esp, 0x28`, shadow space), la cookie del capítulo 15, el padding del 20b y el objeto con vtable del 33. **Conocimiento externo:** layout general del espacio de direcciones, heap y strings en C. **Recomendación editorial:** los diagramas y ejercicios.

Procesos, hilos y mapa de memoria

> **Objetivo de la lección**
>
> Al terminar vas a poder distinguir programa, proceso e hilo; explicar qué comparte y qué no comparte un hilo con los demás; crear un programa propio con dos hilos y encontrarlos en el debugger; y leer las regiones del mapa de memoria de un proceso.

Prerrequisitos: memoria y stack. **Duración sugerida:** 3 horas. Estimación aproximada.

Programa, proceso, hilo

Un **programa** es un archivo en disco: bytes, cabeceras, código. Un **proceso** es ese programa cargado y en ejecución, con su propio espacio de direcciones, sus handles y sus hilos. Un **hilo** (*thread*) es una secuencia de ejecución dentro del proceso.

La diferencia importa porque el debugger te muestra las tres cosas a la vez: el archivo que abriste, el proceso que creó y los hilos que están corriendo.

Concepto	Qué es	Qué comparte
Programa	Archivo `.exe` en disco	Nada: es estático
Proceso	Instancia en ejecución	Espacio de direcciones, handles, token
Hilo	Flujo de ejecución	Código, datos globales, heap, handles
Hilo (propio)	Contexto de CPU, stack, estado	Nada de eso con otros hilos

Cada hilo tiene **su propio stack** y **su propio contexto de CPU** (registros). Si dos hilos comparten una variable global, los dos ven la misma dirección; si cada uno declara una variable local, cada uno tiene su copia en su stack.

El capítulo 15b lo define así: un hilo es "una función o una parte del programa que se ejecuta" al mismo tiempo que otra (`Reversing desde cero, cap. 15b`, 00:00:55).

Ejemplo paso a paso: dos hilos en un programa propio

El capítulo 15b usa un programa de ejemplo con un hilo que incrementa un contador y un `main` que espera una tecla. La estructura es esta:

```c
#include <windows.h>
#include <stdio.h>

volatile int MyCounter = 0;   // 'volatile': impide que el compilador optimice/elimine el bucle

DWORD WINAPI MyThread(LPVOID lpParam) {
    volatile int *contador = (volatile int *)lpParam;
    while (*contador < 0xFFF) {
        (*contador)++;
    }
    return 0;
}

int main(void) {
    HANDLE h = CreateThread(NULL, 0, MyThread, &MyCounter, 0, NULL);
    if (h == NULL) return 1;
    printf("Presione Enter para terminar...\n");
    getchar();
    printf("Contador: %d\n", MyCounter);
    CloseHandle(h);
    return 0;
}
```

> **Compilá con `volatile` o el bucle desaparece en Release**
>
> Sin `volatile`, en Release (`/O2` en MSVC, `-O2` en GCC) el compilador asume que nadie más toca `MyCounter` y **elimina el bucle** (lo reduce a `*contador = 0xFFF;`) o lee el valor una sola vez en un registro. Con `volatile`, cada iteración lee/escribe memoria de verdad y el ejemplo es reproducible en cualquier flag de compilación.

Los argumentos de `CreateThread` que importan para el análisis son:

Posición	Nombre	En el ejemplo
3.º	`lpStartAddress`	`MyThread`: dónde empieza el hilo
4.º	`lpParameter`	`&MyCounter`: puntero que recibe el hilo
5.º	`dwCreationFlags`	`0` = arranca ya; `4` (`CREATE_SUSPENDED`) = creado detenido
Retorno	`HANDLE`	Se usa con `ResumeThread` o `CloseHandle`

El capítulo muestra exactamente esto: el tercer argumento es la dirección de inicio, el cuarto es el puntero a la variable, la bandera `0` hace que arranque al instante y `4` lo deja suspendido hasta un `ResumeThread` (cap. 15b, 00:02:00–00:03:00).

Qué ves en el debugger

1. Abrí el programa en x64dbg y dejalo en el entry point.
2. Abrí **Debugger** → **Thread List**. Vas a ver el hilo principal y el que creaste.

3. Poné un breakpoint en `MyThread` y dejá correr. Cuando el hilo llegue, el debugger se detiene **en ese hilo**.
4. En la lista de hilos, el que está detenido aparece en negrita.
5. Cambiá de hilo y mirá el stack: cada uno tiene el suyo.

En el ejemplo del curso aparecen dos hilos en la versión de 32 bits y cuatro en la de 64 bits (el principal más tres que crea el programa) (cap. 15b, 00:10:02 y 00:15:12).

> **El debugger no puede mostrar dos hilos a la vez**
>
> Cuando un hilo se detiene en un breakpoint, los demás siguen o quedan suspendidos según cómo esté configurado el debugger, pero la pantalla te muestra **un contexto**. Si "de repente salta a otro lado", puede ser un cambio de hilo, no un salto del programa. Revisá la columna de hilo antes de asustarte (cap. 15b, 00:09:50 y 00:14:05).

Handles: los números que identifican recursos

Un **handle** es un identificador opaco que el sistema operativo te da para referenciar un recurso: una ventana, un archivo, un hilo, un evento. No es una dirección de memoria ni un puntero; es un índice a una tabla interna del proceso.

El capítulo 01 muestra la pestaña **Handles** de x64dbg y explica que son "identificadores de ventanas y archivos y demás cosas" (`Reversing desde cero, cap. 01`, 00:06:20).

Para el análisis: cuando una API devuelve un handle y el programa lo guarda en una variable, esa variable no contiene "el archivo", contiene un número que el sistema traduce. Si ves un valor pequeño como `0x000000A4`, es un handle, no una dirección.

El mapa de memoria

La pestaña **Memory Map** de x64dbg muestra las regiones del espacio de direcciones del proceso: imagen del ejecutable, DLLs cargadas, heap, stack y regiones reservadas. Cada fila tiene base, tamaño, permisos y tipo.

En el capítulo 05 se lee así: el ejecutable está mapeado desde `0x6D0000`; los primeros `0x1000` bytes son el header; después vienen `.text`, `.rdata`, `.data`, `.rsrc` y `.reloc` (`Reversing desde cero, cap. 05`, 00:00:10–00:01:00).

Los permisos se leen en tres letras:

Permiso	Significado	Ejemplo
`R`	Lectura	`.rdata`
`W`	Escritura	`.data`, heap, stack
`X`	Ejecución	`.text`
`RWX`	Todo	Sospechoso en un binario normal; típico de unpacking o shellcode

Una región `RWX` no es prueba de malware, pero es una pregunta que vale la pena hacerse: ¿por qué este programa necesita escribir y ejecutar en la misma página?

Hilos, excepciones y TLS: el puente a temas avanzados

Cada hilo tiene su propio estado de excepciones. En 32 bits, el sistema encadena los manejadores en el stack y el primer DWORD del bloque de información del hilo (`FS:[0]`) apunta al primero de la cadena (cap. 16b, 00:02:29). En 64 bits, esa información se guarda en tablas `.pdata` / `.rdata` y no en el stack (cap. 36, 00:00:27).

También cada hilo puede tener **TLS** (*Thread Local Storage*): variables globales que en realidad tienen una copia por hilo. El capítulo 38 lo trata en detalle; acá alcanza con saber que existe y que explica por qué una variable "global" puede tener valores distintos según el hilo.

No necesitás dominar esto para seguir. Lo importante es que cuando veas `FS:[0]`, `GS:[0]`, `.pdata` o `TlsAlloc`, sepas que estás frente a infraestructura del sistema, no a código del programa.

> **Qué son FS y GS: apuntan al TEB**
>
> Los registros de segmento `FS` (x86, 32 bits) y `GS` (x64) no hacen segmentación clásica: apuntan siempre al **TEB** (*Thread Environment Block*) del hilo actual. En x86, `FS:[0]` es el primer campo del TEB y guarda el puntero a la cadena de manejadores de excepciones SEH en el stack, y `FS:[0x30]` lleva al **PEB**. En x64, `GS:[0x30]` apunta al TEB y `GS:[0x60]` al PEB. Lo vas a ver constantemente en la resolución de APIs por PEB (2.14) y en los chequeos anti-debug (2.15).

Práctica

1. Compilá el programa de dos hilos de arriba (en el guest, con Visual Studio o MinGW).
2. Abrilo en x64dbg y completá esta tabla:

Pregunta	Tu respuesta
¿Cuántos hilos hay al inicio?	
¿En qué dirección empieza `MyThread`?	
¿Qué valor tiene el argumento que recibe?	
¿Qué pasa si usás `CREATE_SUSPENDED` (`4`)?	
¿Qué región de memoria es el stack? ¿Qué permisos tiene?	

1. Agregá un `Sleep(5000)` dentro del hilo y observá la lista de hilos mientras duerme.
2. Poné un breakpoint en el `inc`/`add` del bucle y logueá el **valor del contador** leyendo la memoria apuntada, no `{EAX}` a secas: en x64, `MyCounter` llega por `RCX`, así que `{EAX}` loguearía los 32 bits bajos del *puntero*, no el dato. En x64dbg usá `Contador = {[&MyCounter]}` (o `{d:rcx}` si `RCX` tiene el puntero).

Preguntas de repaso

1. ¿Qué comparte un hilo con los demás hilos del mismo proceso? El espacio de direcciones (código, datos globales, heap) y los handles. No comparte el stack ni los registros.

2. ¿Un proceso puede tener varios hilos? ¿Y varios procesos el mismo código? Sí a las dos. Varios procesos pueden ejecutar la misma imagen (por ejemplo, dos instancias del mismo `.exe`), cada uno con su propio espacio de direcciones.

3. ¿Qué hace `CREATE_SUSPENDED`? Crea el hilo detenido. No empieza a ejecutar hasta que otro hilo llame a `ResumeThread`.

4. ¿Por qué el debugger muestra un solo hilo a la vez? Porque muestra el contexto de CPU de un hilo; los demás existen pero no se pueden "mirar" simultáneamente en la misma ventana.

5. ¿Qué indica una región `RWX`? Que se puede leer, escribir y ejecutar. Es inusual en un binario compilado normal y aparece en etapas de desempaquetado, generación de código o shellcode. No prueba nada por sí sola, pero orienta la investigación.

Antes de avanzar

Tenés que poder:

- Explicar la diferencia entre programa, proceso e hilo con un ejemplo.
- Nombrar los argumentos de `CreateThread` que importan y qué hace cada uno.
- Encontrar la lista de hilos y el mapa de memoria en x64dbg.
- Leer los permisos de una región y decir si es normal o sospechosa.

Puerta de salida

Si podés crear tu propio programa de dos hilos y encontrarlo en el debugger, estás listo para el formato PE. Si no, hacelo antes: es la mejor forma de entender qué significa "contexto de hilo".

Fuentes de esta lección

- `Reversing desde cero, cap. 15b` — hilos, `CreateThread`, lista de hilos, cambio de contexto.
- `Reversing desde cero, cap. 01` — handles y mapa de memoria.
- `Reversing desde cero, cap. 05` — image base y secciones en el mapa.
- `Reversing desde cero, cap. 16b` y `cap. 36` — estado de excepciones por hilo.

Capas de esta lección

Contenido observado en CLS: el ejemplo de `CreateThread` con `MyCounter`, la lista de hilos y la lectura del mapa del capítulo 05. **Conocimiento externo:** definición de proceso, handles como tabla interna, permisos de página. **Recomendación editorial:** el programa de práctica y la tabla de ejercicios.

El formato PE: cabeceras, secciones e imports

> **Objetivo de la lección**
>
> Al terminar vas a poder abrir un `.exe` en un visor de PE, ubicar el entry point, la image base y las secciones; convertir una VA a un offset del archivo; y explicar qué es la IAT y por qué un `call` indirecto pasa por ella.

Prerrequisitos: números, memoria y procesos. **Duración sugerida:** 4 horas. Estimación aproximada.

Qué es un PE y por qué te importa

PE (*Portable Executable*) es el formato de los ejecutables y DLLs de Windows, tanto de 32 como de 64 bits. El cargador de Windows lo lee para saber qué código ejecutar, qué DLLs cargar, qué direcciones ajustar y qué recursos mostrar.

Cuando abrís un programa en un debugger, el sistema ya hizo ese trabajo: la imagen está mapeada en memoria. Cuando lo abrís en un editor hexadecimal, ves el archivo tal cual. **La diferencia entre ambos mundos es el formato PE**, y por eso el capítulo 05 insiste en la relación entre dirección virtual, RVA y offset.

El video 33 del curso de OllyDbg lo dice sin vueltas: el PE es "como un gran diccionario o una enciclopedia" en la cabecera, que le dice a Windows de qué se compone el archivo, dónde están los datos, en qué sección está el código y qué librerías puede usar (`Videos y material adicional, Video 33`, 00:00:00–00:02:00).

El recorrido de una cabecera

No hace falta memorizar cada campo. Alcanza con saber el recorrido y dónde mirar.

1. Cabecera DOS

Todo PE empieza con `MZ` (los bytes `4D 5A`). En el offset `0x3C` hay un DWORD llamado `e_lfanew` que indica dónde empieza la cabecera PE. Ese salto existe por compatibilidad histórica con MS-DOS.

2. Firma PE y cabecera de archivo

En `e_lfanew` están los bytes `50 45 00 00` ("PE\0\0") y después la **IMAGE_FILE_HEADER**: arquitectura (`Machine`), cantidad de secciones (`NumberOfSections`), tamaño de la cabecera opcional y características.

3. Cabecera opcional

Acá están los datos que más vas a usar:

Campo	Qué te dice
`Magic`	`0x10B` = PE32; `0x20B` = PE32+ (64 bits)
`AddressOfEntryPoint`	RVA del entry point
`ImageBase`	Dirección preferida de carga
`SectionAlignment`	Alineación en memoria (normalmente `0x1000`)
`FileAlignment`	Alineación en el archivo (normalmente `0x200` o `0x400`)
`DataDirectory[]`	Tablas de imports, exports, recursos, relocalizaciones, TLS, excepciones

4. Tabla de secciones

Cada sección tiene nombre, tamaño virtual, RVA, tamaño en archivo, offset en archivo y características. Las clásicas:

Sección	Contenido	Permisos típicos
`.text`	Código	R-X
`.rdata`	Constantes, strings, imports	R--
`.data`	Variables globales inicializadas	RW-
`.rsrc`	Recursos (iconos, diálogos, strings)	R--
`.reloc`	Relocalizaciones	R--
`.pdata`	Información de excepciones en x64	R--

Ejemplo paso a paso: de la VA al offset

Volvemos al ejecutable del capítulo 05, que ya usamos en la lección de números. El debugger muestra el entry point en `0x6D1372` y el mapa de memoria dice que la imagen empieza en `0x6D0000`.

Paso 1. RVA.

```
RVA = 0x6D1372 − 0x6D0000 = 0x1372
```

Paso 2. Offset en el archivo. El header ocupa `0x1000` en memoria y `.text` empieza en el archivo en `0x400`:

```
offset = 0x1372 − 0x1000 + 0x400 = 0x772
```

Paso 3. Verificación. En el offset `0x772` del archivo están los bytes `E8 C4 03 00 00`: el `call` que el debugger muestra en `0x6D1372`. La comprobación está en el capítulo 05, entre 00:02:30 y 00:03:10

(`Reversing desde cero, cap. 05`).

> **La fórmula general**
>
> `offset = VA - ImageBase - VirtualAddress_de_la_sección + PointerToRawData_de_la_sección`
>
> Para `.text` del ejemplo: `VA - 0x6D0000 - 0x1000 + 0x400`. Cada sección tiene su propio `VirtualAddress` y su propio `PointerToRawData`. Si usás los de `.text` para una dirección de `.rdata`, el cálculo da cualquier cosa.

Imports e IAT

Un programa de Windows casi nunca hace todo solo: llama a funciones de DLLs (`kernel32.dll`, `user32.dll`, etc.). Esas funciones no están en el `.exe`; están en la DLL que se carga en tiempo de ejecución. El PE guarda una **tabla de imports** con los nombres (o números) de las funciones que necesita.

Cuando el cargador carga el programa, completa la **IAT** (*Import Address Table*): una tabla de punteros donde cada entrada contiene la dirección real de la función en memoria.

Por eso un `call` a una API se ve así:

```
call dword ptr [0x6E300C]
```

No es un `call` directo a una dirección fija: es un `call` **indirecto** que lee el puntero guardado en `0x6E300C`. Si esa entrada de la IAT apunta a `kernel32!SetUnhandledExceptionFilter`, el `call` termina ahí.

El capítulo 05 lo muestra en vivo: el opcode es `FF15`, la dirección entre corchetes es `0x6E300C`, y adentro está el puntero a la función de `kernel32`. Cuando Ricardo cambia manualmente el contenido de esa entrada por `0x6D13B8`, el mismo `call` pasa a llamar a una dirección interna del programa. Eso demuestra que **la instrucción no cambió: cambió el puntero de la IAT** (cap. 05, 00:13:00–00:18:00).

> **IAT redirection: tema avanzado, no tarea inicial**
>
> Cambiar entradas de la IAT es una técnica que usan los empacadores para ocultar llamadas. El capítulo 17 la estudia en el marco del desempaquetado de protectores comerciales. En esta guía se explica **qué es** y cómo reconocerla, pero no se propone como ejercicio: no vas a desempaquetar protectores comerciales ni a eludir sus protecciones.

Imports por nombre y por ordinal

En la tabla de imports, cada función puede identificarse por **nombre** (`MessageBoxA`) o por **ordinal** (un número). Los ordinales se usan cuando el nombre no se exporta o para complicar el análisis. Si ves `ORDINAL 1234` en un import, no es un error: es una referencia por número.

Exports

Una DLL puede **exportar** funciones para que otros programas las usen. La tabla de exports tiene nombre, ordinal y RVA de cada función. En un `.exe` normalmente no hay exports; en una DLL sí, y son el punto de entrada para analizarla.

Relocalizaciones y ASLR

Si el cargador no puede cargar la imagen en su `ImageBase` preferida, tiene que **reubicarla**: sumar una diferencia constante a todas las direcciones absolutas. Para eso sirve la sección `.reloc`.

ASLR (*Address Space Layout Randomization*) es la mitigación que hace que la imagen se cargue en una dirección distinta cada vez. Por eso el debugger muestra `0x6D0000` hoy y quizá otra base mañana, y por eso IDA (que analiza el archivo estático) muestra la base preferida. El capítulo 26b lo explica con un ejemplo concreto (`Reversing desde cero, cap. 26b`).

Consecuencia práctica: **no compares direcciones absolutas entre el archivo y la ejecución**. Compará RVA u offsets, o rebaseá el desensamblador a la base que ves en el debugger. El capítulo 14 muestra cómo hacerlo en IDA (`Reversing desde cero, cap. 14`, 00:05:30–00:08:00).

Cómo reconocer un binario empacado

Un empacador comprime o cifra el código original y lo descomprime en tiempo de ejecución. Señales típicas en el PE:

- Nombres de sección como `UPX0`, `UPX1`, `.aspack`, `.packed`.
- Muy pocos imports, o imports raros para un programa de interfaz gráfica.
- Entry point en una sección que no es `.text`.
- Secciones con tamaños virtuales enormes y datos crudos pequeños (el espacio se llena al descomprimir).
- `RWX` en la sección de destino.

El capítulo 10 trabaja con UPX como ejemplo didáctico de desempaquetado. Esta guía **no** incluye el desempaquetado de protectores comerciales como práctica; si querés entender el concepto, estudiá el caso UPX sobre un binario propio y quedate con la idea de "código que se materializa en memoria", no con la receta para saltar protecciones.

Práctica

1. Compilá tu propio `.exe` (el de los dos hilos sirve) y abrilo con un visor de PE: PE-bear, CFF Explorer o Detect It Easy. Anotá:

- `Machine` y `Magic` (¿32 o 64 bits?).
- `ImageBase` y `AddressOfEntryPoint`.
- Secciones con sus RVA, `PointerToRawData` y permisos.
- Tres imports con nombre y la DLL de la que vienen.

1. Calculá el offset del entry point y verificalo en un editor hexadecimal: en ese offset deberían estar los primeros bytes de la función de entrada.
2. Cambiá la vista del visor a "RVA" y a "offset" y comprobá que los números cambian. Explicá por qué.
3. Escribí un script en Python que lea la cabecera y muestre `ImageBase`, `AddressOfEntryPoint` y las secciones. Podés usar `struct` y `os.path.getsize`; no necesitás librerías externas.

```
import struct

with open("programa.exe", "rb") as f:
    data = f.read()

pe = struct.unpack_from("<I", data, 0x3C)[0]
assert data[pe:pe+4] == b"PE\0\0"
machine, nsec = struct.unpack_from("<HH", data, pe+4)
opt_size = struct.unpack_from("<H", data, pe+20)[0]
opt = pe + 24
magic = struct.unpack_from("<H", data, opt)[0]
entry = struct.unpack_from("<I", data, opt+16)[0]
base = struct.unpack_from("<I", data, opt+28)[0] if magic == 0x10B else struct.unpack_from("<Q",
data, opt+24)[0]
print(f"magic={magic:#x} machine={machine:#x} secciones={nsec}")
print(f"entry RVA={entry:#x} image base={base:#x}")
sec = pe + 24 + opt_size
for i in range(nsec):
    off = sec + i * 40
    name = data[off:off+8].rstrip(b"\0").decode(errors="replace")
    vsize, vaddr, rsize, raw = struct.unpack_from("<IIII", data, off+8)
    print(f"{name:8} RVA={vaddr:#08x} raw={raw:#08x} vsize={vsize:#x}")
```

Preguntas de repaso

1. ¿Qué diferencia hay entre RVA y offset? La RVA es una dirección relativa a la image base, en el espacio de memoria. El offset es una posición dentro del archivo en disco. Se relacionan con la alineación del header y el `PointerToRawData` de la sección.

2. ¿Qué es la IAT? La tabla de direcciones de imports. El cargador la completa con las direcciones reales de las funciones de las DLLs; las llamadas indirectas leen de ahí.

3. ¿Por qué un `call` a `MessageBoxA` no es un `call` a una dirección fija? Porque `MessageBoxA` vive en `user32.dll`, que se carga en una dirección que puede variar. El `.exe` llama a través de la IAT, que el cargador completa.

4. ¿Qué indica `Magic = 0x20B`? Que la cabecera opcional es PE32+, es decir, un binario de 64 bits.

5. ¿Por qué ASLR complica comparar IDA con el debugger? Porque la imagen se carga en una base distinta de la preferida. Las direcciones absolutas cambian; las RVA no. Hay que rebasar o comparar offsets.

6. ¿Un binario con secciones `UPX0` / `UPX1` es malware? No necesariamente: es un binario empacado con UPX. Puede ser un programa legítimo comprimido o un malware que usa el mismo empacador. La señal orienta, no concluye.

Antes de avanzar

Tenés que poder:

- Recorrer la cabecera PE y ubicar entry point, image base y secciones.
- Convertir una VA en offset y verificarla en el archivo.
- Explicar qué es la IAT y leer un `call dword ptr [dirección]`.
- Reconocer las señales de un binario empacado y decir por qué no lo vas a desempaquetar en este curso.

> **Puerta de salida de la Etapa 1**
>
> Si podés hacer el recorrido completo con tu propio `.exe` y explicar cada paso, terminaste los fundamentos. La Etapa 2 empieza con el lenguaje del procesador: assembly.

Fuentes de esta lección

- `Reversing desde cero, cap. 05` — VA, RVA, offset, secciones, call directo e indirecto, IAT.
- `Videos y material adicional, Video 33` — formato PE, secciones, imports, relocalizaciones.
- `Reversing desde cero, cap. 14` — rebase y file offset entre IDA y el debugger.
- `Reversing desde cero, cap. 26b` — ASLR.
- `Reversing desde cero, cap. 10` — UPX como caso de binario empacado.

> **Capas de esta lección**
>
> **Contenido observado en CLS:** el ejemplo `0x6D1372 → 0x772`, la IAT en `0x6E300C`, la lectura del PE del video 33 y la explicación de ASLR del capítulo 26b. **Conocimiento externo:** estructura de la cabecera PE y del directorio de imports. **Recomendación editorial:** el script de Python y el límite explícito sobre desempaquetado.

C y Python mínimos para analizar

> **Objetivo de la lección**
>
> Al terminar vas a poder leer y escribir programas chicos en C (variables, condiciones, bucles, funciones, punteros) y en Python (tipos, listas, funciones, archivos binarios), compilarlos, ejecutarlos en el laboratorio y usarlos como conejillos de indias para el debugger.

Prerrequisitos: Etapa 0 y las lecciones anteriores de la Etapa 1. **Duración sugerida:** 6 a 8 horas. Estimación aproximada.

Una advertencia honesta sobre el material de CLS

La carpeta `Python ISO video tutes` parece un curso de Python, pero **no lo es**. Las 55 transcripciones son un curso de **lógica y fundamentos de programación** dictado por Daniel Bermúdez, basado en pseudocódigo y con ejemplos en Basic, Pascal y C. En las transcripciones no hay una sola línea de Python ejecutable: ni `import`, ni `print(...)`, ni `raw_input`. El propio autor anuncia una segunda parte con Python "puro" que **no está en la carpeta**. El material cita Python 2.5.1 como versión vigente en abril de 2007 y usa `print hola` como sentencia, sintaxis que Python 3 ya no acepta.

Eso no lo vuelve inútil. Lo que sí enseña —algoritmos, secuencia, decisión, ciclos, arreglos, matrices y funciones— es exactamente el prerequisito mental para programar. Pero si venías a aprender Python con estos videos, no lo vas a lograr. Para Python moderno, esta guía usa conocimiento externo y prácticas propias, y lo marca como tal.

> **Estado en el catálogo**
>
> El material está listado en `Python ISO video tutes` y en `mapa_carpetas_etapas.csv` como apoyo a la Etapa 1. La ficha completa, con lo que se leyó y lo que falta, está en `ficha de Python ISO`.

Cómo aprender Python de verdad (recursos externos)

Como el corpus CLS no enseña Python moderno, esta guía completa el camino con **cursos y tutoriales externos**. No son material del grupo; son **recomendación editorial**, gratuitos o con parte gratuita, y actuales. Verificá siempre el enlace antes de usarlo.

Recurso	Qué es	Para quién
El Tutorial de Python (oficial) — docs.python.org/3/tutorial	La referencia gratuita y al día; Python 3	Base sólida, consulta permanente
Automate the Boring Stuff (Al Sweigart) — automatetheboringstuff.com	Libro gratuito online, muy práctico, pensado para Windows	Quien nunca programó y quiere resultados rápidos
Real Python — realpython.com	Tutoriales por tema (archivos, bytes, struct, módulos)	Afianzar temas puntuales
Python Crash Course (Eric Matthes)	Libro de pago, proyecto guiado	Quien prefiere un recorrido ordenado
freeCodeCamp — Python (YouTube)	Curso en video gratuito	Quien aprende mejor mirando

La ruta mínima que necesitás para esta guía (no hace falta dominar todo Python): tipos y variables → `str` vs `bytes` → listas y diccionarios → `if` / `for` / `while` → funciones y módulos (`import`) → leer y escribir archivos **binarios** → el módulo `struct` para desempaquetar enteros. Con eso ya podés escribir las herramientas que usa el resto del curso.

> **El puente con el curso de lógica de CLS**
>
> Lo que el "Python ISO" de CLS sí te deja —secuencia, decisión, ciclos, arreglos, funciones— es el prerrequisito mental. Si lo mirás, usalo para **la lógica**, y aprendé la **sintaxis de Python 3** con los recursos de arriba. Es la combinación que te lleva al siguiente tema: scripts que leen binarios.

El subconjunto de C que vas a leer en un debugger

No necesitás ser experto en C. Necesitás reconocer cinco construcciones:

1. **Variables y tipos:** `int`, `char`, `unsigned`, punteros (`int *p`).
2. **Condiciones:** `if`, `else`, `switch`.
3. **Bucles:** `for`, `while`.
4. **Funciones:** parámetros, valor de retorno, variables locales.
5. **Punteros y arrays:** `&x` (dirección de), `*p` (contenido de), `a[i]`.

Ejemplo paso a paso: el programa del capítulo 15

El capítulo 15 analiza un crackme de práctica que pide dos números. La lógica que Ricardo deduce del listado es:

- el primer número tiene que ser **menor o igual** que `0x32` (50 decimal);
- el segundo tiene que ser **mayor o igual** que `0x32`;
- la suma de los dos, multiplicada por 4, tiene que dar `0x1E0` (480 decimal).

Un programa equivalente, escrito por vos, es este:

```c
#include <stdio.h>

int main(void) {
    int a, b;
    printf("Primer valor: ");
    if (scanf("%d", &a) != 1) return 1;
    printf("Segundo valor: ");
    if (scanf("%d", &b) != 1) return 1;

    if (a > 50 || b < 50) {
        puts("No cumple las condiciones");
        return 0;
    }
    if ((a + b) * 4 == 480) {
        puts("Correcto");
    } else {
        puts("Incorrecto");
    }
    return 0;
}
```

Cómo se lee, línea por línea:

- `int a, b;` reserva dos enteros en el stack.
- `scanf("%d", &a)` lee un decimal y lo guarda en la dirección de `a`. El `&` es la dirección; sin él, la función no podría modificar la variable.
- `a > 50 || b < 50` usa `||` (OR lógico): si cualquiera de las dos condiciones se cumple, rechaza.
- `(a + b) * 4 == 480` reproduce la comparación del binario. Con `a = 40` y `b = 80`, da `(120) * 4 = 480`, así que imprime `Correcto`.

Compilación (en el guest, con MinGW o Visual Studio):

```
gcc -O0 -g -o check.exe check.c
```

`-O0` desactiva optimizaciones para que el assembly sea fácil de seguir; `-g` agrega información de depuración. En Visual Studio, compilá en modo Debug.

Qué mirar en el debugger: el `if (a > 50 || b < 50)` se convierte en dos comparaciones y saltos; el `(a+b)*4` se convierte en un `add` seguido de un `shl eax, 2` (desplazar dos bits a la izquierda es multiplicar por 4). Eso conecta directo con la lección de instrucciones.

El subconjunto de Python que vas a usar como herramienta

Python no compila a un `.exe` nativo (en el sentido clásico), así que no es el lenguaje que vas a ver en un debugger de Windows. Es la **navaja suiza** para procesar archivos, calcular hashes, parsear cabeceras y automatizar tareas.

Ejemplo: leer los primeros bytes de un ejecutable

```
from pathlib import Path

data = Path("check.exe").read_bytes()
print("Tamaño:", len(data))
print("Primeros dos bytes:", data[:2])          # b'MZ'
print("Primeros 16 bytes:", data[:16].hex(" "))
```

Salida esperada (los valores exactos dependen del compilador):

```
Tamaño: 53248
Primeros dos bytes: b'MZ'
Primeros 16 bytes: 4d 5a 90 00 03 00 00 00 04 00 00 00 ff ff 00 00
```

`read_bytes()` devuelve un objeto `bytes`: una secuencia inmutable de valores entre 0 y 255. `data[:2]` toma los dos primeros; `.hex(" ")` los muestra en hexadecimal separados por espacios. No hace falta instalar nada.

Ejemplo: calcular un SHA-256

```
import hashlib
from pathlib import Path

datos = Path("check.exe").read_bytes()
print(hashlib.sha256(datos).hexdigest())
```

Ese hash es el mismo que devuelve `Get-FileHash` en PowerShell. Sirve para la bitácora de la Etapa 0.

Ejemplo: desempaquetar un entero con `struct`

Los bytes sueltos no alcanzan: para leer una cabecera hay que interpretar grupos de bytes como números. Eso lo hace el módulo `struct`. En un PE, en el offset `0x3C` hay un entero de 4 bytes (little-endian) que apunta a dónde empieza la firma `PE\0\0`:

```
import struct
from pathlib import Path

data = Path("check.exe").read_bytes()
(pe_off,) = struct.unpack_from("<I", data, 0x3C)   # "<I" = entero de 4 bytes, little-endian
print("La cabecera PE empieza en:", hex(pe_off))
print("Firma en ese offset:", data[pe_off:pe_off+4])  # b'PE\x00\x00'
```

`struct.unpack_from("<I", data, 0x3C)` lee 4 bytes desde el offset `0x3C` y los arma como un entero sin signo (`I`) en little-endian (`<`). Esto conecta directo con la lección de PE: lo que ahí leés a mano, acá lo automatizás.

Ejemplo: el mismo cálculo del crackme, en Python

```
a = int(input("Primer valor: "))
b = int(input("Segundo valor: "))
if a <= 50 and b >= 50 and (a + b) * 4 == 480:
    print("Correcto")
else:
    print("Incorrecto")
```

Compará las dos versiones: en C el `scanf` recibe una dirección; en Python `input()` devuelve un string y `int()` lo convierte. En C la condición se escribe con `||` y `&&`; en Python, con `or` y `and`. La lógica es la misma.

> **Python 2 vs Python 3**
>
> Si encontrás material viejo con `print "hola"` o `raw_input`, es Python 2. Hoy se usa Python 3: `print("hola")`, `input()`, divisiones con `/` que devuelven float y `//` para entero. El curso de CLS es de esa época; no copies esa sintaxis.

Hacia dónde sigue este camino

No hace falta parsear cada cabecera a mano con `struct`: hay librerías que ya lo hacen. El camino natural desde acá es:

1. `pefile` — lee cabeceras PE, secciones e imports de un ejecutable de Windows con unas pocas líneas.
2. `capstone` — desensambla bytes a instrucciones x86/x64 desde Python.
3. **IDAPython / Ghidra scripting** — automatizar el análisis dentro del desensamblador.

Esos tres se desarrollan, con ejemplos, en la rama de automatización de la Etapa 5. Lo de esta lección — `bytes`, `hex`, `struct`, archivos— es la base que te deja usarlos sin que sean magia.

Práctica

1. Compilá el programa `check.c` y probalo con `40 80`, `50 50`, `51 80` y `40 79`. Anotá qué imprime cada caso.
2. Abrilo en x64dbg, poné un breakpoint en `main` y seguí con F8 hasta la comparación. Anotá en qué instrucción se convierte `(a+b)*4`.
3. Escribí un programa en C que pida una palabra y la compare con `"clase"` usando `strcmp`. Compilalo y encontralo en el debugger.
4. Escribí un script en Python que recorra una carpeta, calcule el SHA-256 de cada archivo y escriba un CSV con `ruta,hash`. Usalo sobre tu carpeta de prácticas.
5. Modificá el script para que lea la cabecera PE y muestre `ImageBase` y `AddressOfEntryPoint` (podés reutilizar el de la lección de PE). Usá `struct.unpack_from` sobre los offsets correctos.

6. Instalá `pefile` (`pip install pefile`) y obtené los mismos valores con la librería (`pe.OPTIONAL_HEADER.ImageBase`, `pe.OPTIONAL_HEADER.AddressOfEntryPoint`). Compará con tu versión a mano: ¿coinciden? Así ves qué te ahorra la librería.
7. Hacé el Tutorial oficial de Python (secciones 3 a 7) o los primeros capítulos de *Automate the Boring Stuff*, y rehacé el script del item 4 sin mirar el ejemplo.

Preguntas de repaso

1. ¿Por qué `scanf` necesita `&a` y `printf` no? Porque `scanf` tiene que **escribir** en la variable, y para eso necesita su dirección. `printf` solo lee el valor.

2. ¿Qué diferencia hay entre `int` y `unsigned int`? El rango y la interpretación. `int` usa complemento a dos y incluye negativos; `unsigned int` solo representa valores no negativos y puede llegar al doble del máximo positivo.

3. ¿Por qué `x << 2` equivale a multiplicar por 4? Porque cada desplazamiento a la izquierda multiplica por 2. Dos desplazamientos multiplican por 4. Es la operación que usa el compilador para optimizar multiplicaciones por potencias de dos.

4. ¿Python sirve para analizar un `.exe`? Sí, como herramienta: leer bytes, parsear cabeceras, calcular hashes, automatizar. No es el lenguaje del binario que analizás, salvo que el programa esté hecho en Python y empaquetado con un generador de ejecutables.

5. ¿El curso de Python de CLS enseña Python? No. Enseña lógica de programación con pseudocódigo y ejemplos en Basic, Pascal y C. Es útil como base, pero no reemplaza un curso de Python.

Antes de avanzar

Tenés que poder:

- Leer un programa en C de 20 líneas y decir qué hace.
- Escribir, compilar y ejecutar un programa propio en C.
- Leer y escribir un archivo binario en Python y calcular su hash.
- Explicar por qué el material de Python de CLS no es un curso de Python.

> **Puerta de salida de la Etapa 1**
>
> Con esto terminás los fundamentos. Si podés escribir tus propios binarios, ya tenés con qué practicar todo lo que sigue sin depender de software ajeno.

Fuentes de esta lección

- `Python ISO video tutes, Fundamentos de programación` — qué es programar, intérprete vs compilador, `print` en Python 2.
- `Python ISO video tutes, Qué es Python` — presentación del curso y versión de la época.
- `Python ISO video tutes, Arreglos` — ejemplo del tipo de contenido real de la colección.

- `Reversing desde cero, cap. 15` — lógica del programa de práctica y comparación con `0x32` / `0x1E0`.
- `C Y REVERSING (contenedor)` — material de C del proyecto, **pendiente de revisar**: solo se verificó su existencia y sus dos archivos `.rar`.

> **Capas de esta lección**
>
> **Contenido observado en CLS:** la lógica del programa del capítulo 15 y el contenido real de las transcripciones de Python ISO. **Conocimiento externo:** sintaxis de C y Python 3, compilación, hash, `struct`, y los recursos de aprendizaje externos (Tutorial oficial, Automate the Boring Stuff, Real Python, etc.). **Recomendación editorial:** los programas de práctica, la corrección sobre el curso de Python de CLS y la ruta de Python hacia `pefile` / `capstone`.

Etapa 2 · Introducción al reversing

Objetivo medible

Leer assembly x86/x64, seguir el flujo y usar debugger y desensamblador con criterio. Se divide en 2A (base, 2.1–2.9), 2B (Windows intermedio, 2.10–2.16: ofuscación, TLS, MFC, huella, PE fino, anti-análisis, Ghidra) y 2C (Windows avanzado, 2.17+: packers didácticos, C++ profundo, WinDbg/heap, patch diffing, scripting) y 2E (desofuscación y análisis dinámico avanzado, 2.25+: ejecución simbólica, ofuscación por VM, protectores multicapa, neutralizar anti-análisis, DBI con DynamoRIO/Pin y Frida Stalker, extensiones de Ghidra/SLEIGH, y ampliaciones de Go/Rust y VB6). Las rutas de malware dinámico, shellcode/C2 y fuzzing quedan como optativas, fuera de la puerta obligatoria.

Prerrequisitos: Etapa 1. **Duración estimada:** 32 a 40 h (2A) + ~25 h (2B) + ~30–40 h (2C avanzado). Es una estimación de esta guía, derivada de la suma de las lecciones; no es una duración declarada por CLS.

Lecciones

#	Lección	Qué aprendés
2.1	Registros, subregistros y flags	Registros, solapamiento y flags ZF/SF/CF/OF.
2.2	Instrucciones: mover, calcular y manipular bits	MOV, LEA, aritmética, lógica, shifts y cadenas.
2.3	Punto flotante y SSE: los registros xmm	x87, SSE escalar (movsd/addsd/ucomisd) e IEEE 754.
2.4	Flujo: comparar, saltar y repetir	CMP/TEST, Jcc, CMOVcc, bucles y tablas de salto.
2.5	Llamadas, convenciones y marcos de función	CALL/RET, cdecl/stdcall/fastcall y x64.
2.6	El debugger: x64dbg de punta a punta	Interfaz, F7/F8/F9, breakpoints y parcheo ético.
2.7	IDA y el análisis estático	Strings, xrefs, grafo, rebase y decompilador.
2.8	Cómo cambia el código con optimizaciones	Inlining, strength reduction, división por constante y tail calls.
2.9	Herramientas: del OllyDbg histórico al taller actual	Qué usar para cada tarea y qué está discontinuado.
2.10	Ofuscación nativa: leer lo que no quiere ser leído	Secciones invertidas, basura, saltos al medio y strings ocultas; hallar el bloque real.
2.11	TLS: código que corre antes de main	Directorio TLS, AddressOfCallBacks y callbacks previos al entry point.
2.12	MFC: llegar al handler cuando todo pasa por clases	Reconocer MFC y alcanzar el handler de un control por el message map.
2.13	Huella de compilador: saber con qué se hizo	MSVC, MinGW, Delphi, VB6 y MFC por imports, strings y runtime.
2.14	PE fino: los directorios que cuentan la historia	Rich header, recursos, delay-load, .pdata, debug/PDB y Load Config.
2.15	Taxonomía anti-análisis: reconocer, no evadir	Anti-debug, anti-VM, anti-disassembly y anti-dump: detectarlos.
2.16	Ghidra (y Cutter): la alternativa libre, con práctica	El recorrido de IDA replicado en Ghidra: autoanálisis, strings, tipos y decompilador.
2.17	Packers didácticos: UPX, stub, OEP, dump e IAT	Desempaquetado didáctico de UPX sobre binarios propios; llegar al OEP y reconstruir la IAT.
2.18	C++ profundo: RTTI, vtables, STL y excepciones	RTTI, std::string con SSO, contenedores STL y manejo de excepciones.
2.19	WinDbg, TTD y heap	WinDbg, time-travel debugging y diagnóstico de heap (Page Heap, LFH).
2.20	Patch diffing y CVE: leer qué cambió	BinDiff/Diaphora: comparar dos versiones y leer qué arregló un parche (defensivo).

#	Lección	Qué aprendés
2.21	Scripting de debugger: automatizar y dejar evidencia	Automatizar x64dbg (breakpoints, logs) y guardar evidencia; OllyScript como legado.
2.22	(optativa) Malware dinámico defensivo	Laboratorio aislado, red simulada, observación de comportamiento e IOCs.
2.23	(optativa) Shellcode, red y C2: reconocer y describir	Identificar shellcode y describir tráfico de C2; análisis y clasificación, no construcción.
2.24	(optativa) Fuzzing y análisis simbólico	Fuzzear un parser/crackme propio y usar ejecución simbólica para hallar condiciones.
2.25	Ejecución simbólica y concólica	angr/Triton/Miasm y Z3 para recuperar condiciones y simplificar MBA en binarios propios.
2.26	Ofuscación por máquina virtual	Dispatcher, bytecode y handlers; reconocer virtualización y practicar con una mini-VM propia.
2.27	Protectores comerciales y packers multicapa	Reconocer familias y capas (VMProtect, Themida…) sin desempacar software ajeno.
2.28	Neutralizar anti-análisis en laboratorio	ScyllaHide y hooks de Frida sobre binarios propios o malware en VM descartable.
2.29	Instrumentación binaria dinámica: DynamoRIO y Pin	Cobertura (drcov/Lighthouse), conteo y desempaquetado genérico por W⊕X.
2.30	Frida avanzado: Stalker y tracing	Trazar bloques/instrucciones de un hilo, cobertura (frida-drcov) y código en runtime.
2.31	Extensiones de Ghidra y SLEIGH	Scripts (Jython/PyGhidra), headless, P-Code y SLEIGH para modelar bytecode propio.
2.32	Go y Rust en profundidad (amplía 2.13)	pclntab/itab en Go; manglado v0, monomorfización y trait objects en Rust.
2.33	VB6 y p-code (amplía 2.13)	Nativo vs p-code, VBHeader y MSVBVM60; VB Decompiler/WKTVBDE (legado).

Recursos de CLS para esta etapa

- Etapa 2 · Rama optativa 2D — Windows internals (después de 2C)
- Reversing desde cero (tronco principal)
- Curso IDA desde cero (69 documentos)
- Curso de OllyDbg (Videos 23–40)
- Introducción al reversing con IDA Pro (docx)

Evaluación de salida

- **2A** · Explicar qué hace un bloque de 10 instrucciones y qué flags cambia.
- **2A** · Reconstruir un if/else y un bucle a partir del listado.

- **2A** · Poner los tres tipos de breakpoint y justificar cuál usar.
- **2A** · Sincronizar IDA y x64dbg por base o por file offset.
- **2B** · Reconocer tres señales de ofuscación y hallar el bloque real sin NOPear a ciegas.
- **2B** · Encontrar un TLS callback en el PE y demostrar que corre antes de `main`.
- **2B** · Llegar al handler de un control en un binario MFC por una API de texto y el call stack.
- **2B** · Identificar el compilador de un binario y navegar los directorios finos del PE (Rich header, Load Config, debug/PDB).
- **2B** · Clasificar una técnica anti-análisis y detectarla, y leer el mismo binario en IDA y en Ghidra.
- **2C** · Desempaquetar un UPX propio: llegar al OEP, dumpear, reconstruir la IAT y validar con `upx -d`.
- **2C** · Reconocer RTTI, `std::string` (SSO), un `std::vector` y un `try/catch` en un binario C++ propio.
- **2C** · Diagnosticar un bug de heap propio con WinDbg/Page Heap y leer, con un differ, qué cambió un parche.
- **2C** · Automatizar una tarea de análisis con un script de x64dbg y guardar la evidencia.
- **2E** · Resolver un crackme propio con un solver (angr/Z3), reconocer una VM de protección y sacar cobertura con DBI o Frida Stalker.

> **Entregable de la etapa**
>
> Un archivo en tu bitácora con: lecciones completadas, ejercicios resueltos, dudas abiertas y evidencia (capturas, valores, hashes). Sin evidencia no se considera aprobada.

Errores frecuentes

- Confundir F7 con F8 y creer que F8 no ejecuta la función.
- Leer un salto al revés (pensar que el salto es la condición verdadera).
- Usar una copia ilegal de IDA Pro.
- Parchear software ajeno para saltar una licencia.

Qué no entra en esta etapa

Los temas que no tienen material revisado o que exceden el marco se listan en Kernel y otros territorios. No los uses como atajo: cada rama tiene prerrequisitos.

Registros, subregistros y flags

> **Objetivo de la lección**
>
> Al terminar vas a poder nombrar los registros de propósito general y su función típica, descomponer un registro de 32 bits en sus partes de 16 y 8 bits, y explicar qué flag se activa en una suma o resta concreta.

Prerrequisitos: Etapa 1 completa. **Duración sugerida:** 3 horas. Estimación aproximada.

Qué es un registro

Un **registro** es una porción de memoria pequeña y muy rápida que está dentro del procesador. Se usa para operar con los datos que la CPU necesita en el momento. En 32 bits cada registro mide 4 bytes (32 bits); en 64 bits, 8 bytes (64 bits).

El capítulo 03 lo define así: "una porción de memoria pequeña, ultra rápida, que forma parte del procesador y que sirve para intercambiar datos" (`Reversing desde cero, cap. 03`, 00:00:30).

Los registros de propósito general

Registro 32	Registro 64	Nombre	Uso típico
`EAX`	`RAX`	Acumulador	Resultado de funciones y operaciones
`EBX`	`RBX`	Base	Datos generales; a veces puntero a datos
`ECX`	`RCX`	Contador	Bucles, `rep`, shifts variables
`EDX`	`RDX`	Datos	Parte alta de multiplicaciones/divisiones; E/S
`ESI`	`RSI`	Source index	Origen en copias de memoria
`EDI`	`RDI`	Destination index	Destino en copias de memoria
`ESP`	`RSP`	Stack pointer	Cima del stack
`EBP`	`RBP`	Base pointer	Base del marco de la función

Además, `EIP` / `RIP` es el **instruction pointer**: apunta a la instrucción que se va a ejecutar. No se modifica con `mov`; lo cambia el flujo (saltos, llamadas, retornos).

El capítulo 03 asigna funciones concretas: `EAX` guarda el resultado de las funciones, `ECX` actúa como contador en bucles y operaciones repetitivas, `ESI` / `EDI` se usan para copiar bloques de memoria, `ESP` apunta a la cima del stack y `EBP` a la base del marco (00:01:00–00:03:00). Son convenciones, no reglas absolutas: un compilador puede usar `EAX` para cualquier cosa.

Subregistros: el mismo espacio, distintos tamaños

Un registro de 32 bits se puede leer como uno de 16 o como dos de 8. No son registros separados: son **ventanas** sobre el mismo espacio.

Para `EAX`:



Ejemplo con `EAX = 0x008FFB18`:

Vista	Valor	Tamaño
`EAX`	`0x008FFB18`	32 bits
`AX`	`0xFB18`	16 bits (los dos bytes bajos)
`AH`	`0xFB`	8 bits (byte alto de `AX`)
`AL`	`0x18`	8 bits (byte bajo de `AX`)

Esto se aplica a `EAX`, `EBX`, `ECX` y `EDX`. `ESI`, `EDI`, `EBP` y `ESP` tienen versión de 16 bits (`SI`, `DI`, `BP`, `SP`) pero no se dividen en dos de 8. El capítulo 03 lo explica con este mismo ejemplo (00:04:30–00:09:00).

En 64 bits, `RAX` contiene a `EAX`, que contiene a `AX`, que contiene a `AL` / `AH`. Además hay ocho registros nuevos, `R8` a `R15`, con subregistros `R8D` (32), `R8W` (16) y `R8B` (8).

> **Escribir en la parte baja no borra la alta (con matices)**
>
> En x86, escribir en `AL` no cambia el resto de `RAX`. Pero en x64, escribir en `EAX` **sí borra los 32 bits altos de** `RAX`. Es una diferencia que sorprende cuando comparás el código de 32 y 64 bits. Para el análisis inicial, alcanza con saber que los tamaños se solapan y que el debugger te muestra cada vista.

Flags: el resumen de la última operación

Los **flags** son bits de estado que se activan según el resultado de una operación. No son registros de datos: son banderas. Los cuatro que importan en el uso diario:

Flag	Nombre	Se activa cuando...
`ZF`	Zero flag	El resultado es cero
`SF`	Sign flag	El resultado es negativo (bit más alto en 1)
`CF`	Carry flag	Hubo acarreo/desborde en una operación **sin signo**
`OF`	Overflow flag	Hubo desborde en una operación **con signo**

Hay más (`PF` de paridad, `AF` auxiliar, `DF` de dirección), pero casi nunca los mirás al reversear.

Ejemplo 1: `ZF`

```
mov eax, 3
sub eax, 3      ; resultado 0 -> ZF = 1
```

Si en cambio hacés `mov eax, 0`, el flag `ZF` **no** se activa: `mov` no es una operación aritmética y no actualiza los flags. El capítulo 04 lo muestra explícitamente (`Reversing desde cero, cap. 04`, 00:00:30–00:02:00).

Ejemplo 2: `CF`

```
mov eax, 0xFFFFFFFF   ; máximo sin signo
add eax, 5            ; resultado 4, con acarreo
```

El resultado "real" (4.294.967.300) no entra en 32 bits. El procesador guarda `4` y activa `CF`. Para valores sin signo, eso es un desborde (cap. 04, 00:05:00–00:06:00).

Ejemplo 3: `OF` y `SF`

```
mov eax, 0x7FFFFFFF   ; máximo positivo con signo
add eax, 1            ; pasa a 0x80000000
```

El resultado cambió de positivo a negativo: para valores con signo es un desborde, así que se activa `OF`. Además, como el bit más alto quedó en 1, se activa `SF` (cap. 04, 00:07:30–00:09:00).

Lo importante: **el mismo resultado activa flags distintos según cómo lo interpretes**. `0x80000000` es un número enorme sin signo y un número negativo con signo. Los flags no deciden por vos; reflejan ambas lecturas.

Multiplicación y división: registros dobles

`MUL` y `DIV` usan `EDX:EAX` como un registro de 64 bits:

- `MUL operando`: multiplica `EAX` por el operando y guarda el resultado en `EDX:EAX`.

- `DIV operando`: divide `EDX:EAX` por el operando; cociente en `EAX`, resto en `EDX`.

Ejemplo del capítulo 04: `EAX = 4`, `EBX = 2`, `MUL EBX` deja `EAX = 8` y `EDX = 0`. Si el resultado no entra en 32 bits, la parte alta aparece en `EDX` (00:13:00–00:17:00).

`IMUL` / `IDIV` son las versiones con signo. La mecánica es la misma; cambia la interpretación de los bits.

Práctica

1. Completá la tabla para `EBX = 0x11223344`:

Vista	Valor
`EBX`	
`BX`	
`BH`	
`BL`	

1. ¿Qué flag se activa en cada caso?

- `sub eax, eax`
- `add eax, 1` con `EAX = 0xFFFFFFFF`
- `add eax, 1` con `EAX = 0x7FFFFFFF`
- `mov eax, 0`

1. Escribí en x64dbg `mov eax, 0x7FFFFFFF` y `add eax, 1`. Anotá `ZF`, `SF`, `CF` y `OF` antes y después.
2. Explicá por qué `mov` no cambia los flags y `add` sí.
3. ¿Qué registro usarías para contar las vueltas de un bucle? ¿Por qué no `ESP`?

Preguntas de repaso

1. ¿`AX` es un registro distinto de `EAX`? No. Es la parte baja de 16 bits de `EAX`. Escribir en `AX` modifica los dos bytes bajos de `EAX` y deja los altos intactos.

2. ¿Qué diferencia hay entre `CF` y `OF`? `CF` indica desborde en una interpretación sin signo; `OF`, en una interpretación con signo. El mismo resultado puede activar uno, otro o ambos.

3. ¿`EIP` se puede cambiar con `mov`? No directamente. Se cambia con instrucciones de control de flujo (`jmp`, `call`, `ret`, `jcc`) o con el debugger (por ejemplo, "set new origin here").

4. ¿Para qué sirve `EDX` en una división? Para la parte alta del dividendo y para el resto. `DIV` divide `EDX:EAX` por el operando y deja el cociente en `EAX` y el resto en `EDX`.

5. ¿Por qué `mov eax, 0` no activa `ZF`? Porque `mov` es una transferencia de datos, no una operación aritmética. Los flags se actualizan con instrucciones como `add`, `sub`, `cmp`, `test`, `and`, etc.

Antes de avanzar

Tenés que poder:

- Nombrar los ocho registros de propósito general y su función típica.
- Descomponer `EAX` en `AX`, `AH` y `AL` con un ejemplo.
- Explicar `ZF`, `SF`, `CF` y `OF` con un caso cada uno.
- Decir qué registros usa `MUL` / `DIV` para el resultado.

> **Puerta de salida**
>
> Si podés predecir los flags de una operación antes de ejecutarla, estás listo para las instrucciones. Si no, practicá con la calculadora del debugger y papel.

Fuentes de esta lección

- `Reversing desde cero, cap. 03` — registros, subregistros, flags y números con signo.
- `Reversing desde cero, cap. 04` — ejemplos de `ZF`, `CF`, `OF`, `SF`, `MUL` y `DIV`.
- `Videos y material adicional, Video 5` — registros de 32 bits y prefijo `E` en el curso de MASM32.

> **Capas de esta lección**
>
> **Contenido observado en CLS:** la definición de registro, el ejemplo `EAX = 0x008FFB18` y los ejemplos de flags de los capítulos 03 y 04. **Conocimiento externo:** la tabla de registros de 64 bits y la semántica exacta de `CF` / `OF`. **Recomendación editorial:** los ejercicios y la advertencia sobre el borrado de la parte alta en x64.

Instrucciones: mover, calcular y manipular bits

> **Objetivo de la lección**
>
> Al terminar vas a poder leer un bloque de instrucciones y decir qué valor queda en cada registro, distinguiendo transferencias, operaciones aritméticas, lógicas, desplazamientos y accesos a memoria.

Prerrequisitos: registros y flags. **Duración sugerida:** 5 horas. Estimación aproximada.

Cómo se lee una instrucción

Casi todas las instrucciones tienen la forma:

```
MNEMÓNICO destino, fuente
```

La operación se calcula con los dos operandos y el resultado se guarda en el **destino** (el primero). `mov eax, ebx` copia `EBX` en `EAX`; `add eax, ebx` suma y guarda en `EAX`.

Los corchetes significan "el contenido de esta dirección":

```
mov eax, 0x401000      ; EAX = el número 0x401000
mov eax, [0x401000]    ; EAX = lo que hay en la dirección 0x401000
```

Esa diferencia es la que separa un puntero de un valor. El capítulo 02 la explica al mover un valor desde memoria a `EAX`: los bytes `36 00 00 38` en la dirección se convierten en `0x38000036` en el registro (`Reversing desde cero, cap. 02`, 00:07:00–00:08:00).

Transferencias

`MOV`

```
mov eax, 5          ; inmediato -> registro
mov eax, ebx        ; registro -> registro
mov eax, [0x403000] ; memoria -> registro
mov [0x403000], eax ; registro -> memoria
```

No modifica flags. No se puede `mov` de memoria a memoria: hay que pasar por un registro.

`LEA`

`LEA` (*load effective address*) calcula una dirección y la guarda, **sin leer memoria**:

```
lea eax, [ebx + ecx*4 + 8]
```

Si `EBX = 0x1000` y `ECX = 2`, `EAX = 0x1000 + 8 + 8 = 0x1010`. No importa qué haya en `0x1010`: no se lee. Se usa para aritmética de punteros y, de paso, como una multiplicación rápida. El capítulo 12 lo muestra en la práctica (`Reversing desde cero, cap. 12`, 00:08:07).

`MOVSX` y `MOVZX`

Copian un valor más chico a un registro más grande, rellenando los bits altos:

```
; AX = 0x8384
movsx eax, ax   ; EAX = 0xFFFF8384  (rellena con el bit de signo)
movzx eax, ax   ; EAX = 0x00008384  (rellena con ceros)
```

`MOVSX` interpreta el origen como con signo; `MOVZX`, como sin signo. En el capítulo 12, `0x8384` es mayor que `0x7FFF`, así que como `WORD` con signo es negativo y `MOVSX` rellena con `FF` (00:07:21–00:08:00).

`XCHG`

Intercambia los dos operandos. No hay destino ni fuente: los dos cambian.

```
; EAX = 0x20, EBX = 0x40
xchg eax, ebx
; EAX = 0x40, EBX = 0x20
```

> **En el Curso IDA: editar una instrucción (y qué pasa después)**
>
> El Curso IDA edita estas instrucciones desde **Edit → Patch Program → Assemble** sobre el crackme de Cruehead. Un detalle útil: si la instrucción nueva ocupa **menos** bytes que la vieja, el byte que sobra queda mostrado como **dato** (`db 0xC0`) y la función aparece "corrupta"; se lo reemplaza con un `NOP` (`0x90`, relleno que no hace nada) para que vuelva a verse como código. `LEA`, acá arriba, el curso la usa justamente para obtener la dirección de variables y argumentos (Curso IDA · tute 4, Transferencia y tute 5, `LEA`).

Aritmética

`ADD` y `SUB`

```
add eax, ebx    ; EAX = EAX + EBX
sub eax, 4      ; EAX = EAX - 4
```

Actualizan `ZF`, `SF`, `CF` y `OF`. El capítulo 02 las practica con registros y memoria (00:10:00–00:18:00).

`INC` y `DEC`

Suman o restan 1. Son más cortas que `add eax, 1` y **no modifican** `CF` (sí `ZF`, `SF`, `OF`). En el capítulo 06 se ve `INC EAX` con `EAX = 0xFFFFFFFF` (pasa a 0) y `DEC EAX` con `EAX = 1` (queda 0 y activa `ZF`) (`Reversing desde cero, cap. 06`, 00:20:03–00:21:00).

`MUL`, `IMUL`, `DIV`, `IDIV`

Ya las vimos en registros. Para 32 bits: `MUL` usa `EDX:EAX`; `DIV` divide `EDX:EAX`. `IMUL` / `IDIV` son las versiones con signo. En 64 bits, `RDX:RAX`.

Lógica de bits

Instrucción	Qué hace	Ejemplo (8 bits)
`AND`	1 solo si ambos bits son 1	`0x11 & 0x0F = 0x01`
`OR`	1 si alguno es 1	`0x11 OR 0x0F = 0x1F`
`XOR`	1 si son distintos	`0x11 ^ 0x0F = 0x1E`
`NOT`	Invierte todos los bits	`~0x11 = 0xEE` (8 bits) o `0xFFFFFFEE` (32)

El capítulo 08 las explica con tablas de verdad y las verifica en el debugger (`Reversing desde cero, cap. 08`, 00:07:00–00:17:00).

`XOR EAX, EAX` es la forma idiomática de poner un registro en cero: ocupa 2 bytes y activa `ZF`. `MOV EAX, 0` ocupa 5 bytes y no toca flags. Esa diferencia de tamaño es la que hace que los compiladores prefieran `XOR`.

Desplazamientos

```
shl eax, 1    ; desplaza 1 bit a la izquierda = multiplicar por 2
shr eax, 1    ; desplaza 1 bit a la derecha  = dividir por 2 (sin signo)
sar eax, 1    ; desplazamiento aritmético (conserva el signo)
rol eax, 4    ; rota a la izquierda: el bit que sale entra por la derecha
ror eax, 4    ; rota a la derecha
```

Con `EAX = 0x11`:

```
shl eax, 2   -> 0x44     (0x11 * 4)
shr eax, 2   -> 0x04     (0x11 / 4, parte entera)
```

`SHL` / `SHR` **pierden** los bits que salen; `ROL` / `ROR` los reinyectan por el otro extremo. El capítulo 08 lo muestra con `0x11 << 2 = 0x44` y `0x44 >> 2 = 0x11` (00:24:33–00:28:11).

Comparar sin restar: `CMP` y `TEST`

`CMP a, b` hace `a - b`, actualiza flags y **descarta el resultado**. `TEST a, b` hace `a AND b`, actualiza flags y también descarta el resultado.

```
cmp eax, 4      ; ¿EAX == 4?  -> ZF=1 si son iguales
test eax, eax   ; ¿EAX == 0?  -> ZF=1 si es cero
```

`TEST EAX, EAX` es la forma habitual de preguntar "¿este registro es cero?" sin modificar nada. El capítulo 09 lo explica y lo usa con `JZ` (`Reversing desde cero, cap. 09`, 00:31:32–00:33:00).

Pila y cadenas

```
push eax        ; ESP -= 4 ; [ESP] = EAX
pop  ebx        ; EBX = [ESP] ; ESP += 4
pushad          ; guarda EAX, ECX, EDX, EBX, ESP, EBP, ESI, EDI (32 bytes)
popad           ; los restaura
```

`PUSHAD` / `POPAD` se usan para preservar registros alrededor de una llamada. Dato fino: `POPAD` **no** restaura `ESP`, aunque `PUSHAD` lo haya guardado (cap. 12, 00:21:07).

Las instrucciones de cadena operan sobre `ESI` (origen) y `EDI` (destino):

```
rep movsb      ; copia ECX bytes de [ESI] a [EDI]
repe cmpsb     ; compara [ESI] y [EDI] mientras sean iguales
lodsb          ; AL = [ESI]; ESI++
stosb          ; [EDI] = AL; EDI++
repne scasb    ; busca AL en [EDI] mientras no lo encuentre
```

El capítulo 12 las recorre con un ejemplo concreto: `ESI` apunta a `"core"` (`0x402012`), `EDI` a `"core security"` (`0x402031`) y `REPNE SCASB` busca el byte `0x65` (`'e'`) (00:37:11–00:38:30).

Ejemplo paso a paso: un bloque completo

```
mov  eax, 0x11
mov  ecx, 0x0F
xor  eax, ecx
and  eax, 0x1F
shl  eax, 2
mov  ebx, eax
sub  ebx, 0x44
```

Paso	Instrucción	EAX	EBX	Flags relevantes
1	`mov eax, 0x11`	`0x11`	?	sin cambio
2	`mov ecx, 0x0F`	`0x11`	?	sin cambio
3	`xor eax, ecx`	`0x1E`	?	`ZF=0`, `SF=0`
4	`and eax, 0x1F`	`0x1E`	?	`ZF=0`
5	`shl eax, 2`	`0x78`	?	`CF` = último bit que salió
6	`mov ebx, eax`	`0x78`	`0x78`	sin cambio
7	`sub ebx, 0x44`	`0x78`	`0x34`	`ZF=0`, `SF=0`, `CF=0`

Comprobación: `0x11 ^ 0x0F = 0x1E`; `0x1E & 0x1F = 0x1E`; `0x1E << 2 = 0x78`; `0x78 - 0x44 = 0x34`. Todos los valores entran en 8 bits, así que no hay acarreo.

Práctica

1. Ejecutá el bloque anterior en x64dbg y completá la tabla con los valores reales.
2. Cambiá `mov eax, 0x11` por `mov eax, 0xFFFFFFFF` y predecí qué pasa en cada paso.
3. Escribí una secuencia de tres instrucciones que deje `EAX = 0` sin usar `mov`.
4. ¿Qué diferencia hay entre `mov eax, [ebx]` y `lea eax, [ebx]`?
5. Traducí a assembly: `if (x == 0) return;` con `x` en `EAX`.
6. Explicá por qué `INC` no sirve para detectar acarreo sin signo.

Preguntas de repaso

1. ¿Qué significa que el resultado se guarda en el primer operando? Que en `add eax, ebx` el resultado queda en `EAX`, no en `EBX`. El segundo operando se lee pero no se modifica.

2. ¿`LEA` lee memoria? No. Calcula la dirección efectiva de la expresión entre corchetes y la guarda. Es aritmética de punteros.

3. ¿Qué hace `XOR EAX, EAX` y por qué se usa? Pone `EAX` en cero, ocupa menos bytes que `MOV EAX, 0` y actualiza flags. Es la forma idiomática de limpiar un registro.

4. ¿Cuál es la diferencia entre `SHL` y `ROL`? `SHL` desplaza y descarta el bit que sale; `ROL` lo reinyecta por el otro extremo. `SHL` equivale a multiplicar por 2; `ROL` no es una multiplicación.

5. ¿`CMP` guarda el resultado de la resta? No. Solo actualiza flags. Los operandos quedan intactos.

6. ¿Por qué `TEST EAX, EAX` es útil? Porque activa `ZF` si `EAX` es cero, sin modificar el registro. Es la forma estándar de comprobar un valor nulo.

Antes de avanzar

Tenés que poder:

- Leer un bloque de 10 instrucciones y calcular el valor final de los registros.
- Distinguir `mov` de `lea` y `mov` de `movsx` / `movzx`.
- Explicar qué flags cambian `add`, `sub`, `inc`, `and`, `shl` y `cmp`.
- Reconocer las instrucciones de cadena y su relación con `ESI` / `EDI`.

> **Puerta de salida**
>
> Si podés resolver el bloque del ejemplo sin ejecutarlo y después verificarlo en el debugger, estás listo para el flujo de control.

Fuentes de esta lección

- `Reversing desde cero, cap. 02` — `MOV`, `XCHG`, `ADD`, `SUB`, memoria y corchetes.
- `Reversing desde cero, cap. 04` — flags, `MUL` y `DIV`.
- `Reversing desde cero, cap. 06` — `CALL`, `RET`, `JMP`, `PUSH` / `POP`, `INC` / `DEC`.
- `Reversing desde cero, cap. 08` — lógicas y desplazamientos.
- `Reversing desde cero, cap. 09` — `CMP`, `TEST`, `LEAVE`.
- `Reversing desde cero, cap. 12` — práctica con `MOVSX` / `MOVZX`, `LEA`, `LOOP` y cadenas.
- Curso IDA · tutes 4–6 — las mismas instrucciones en IDA: editar con *Assemble*, el byte sobrante como `db` y el relleno `NOP`; `LEA` para direcciones; regla de `ADD` (no se permiten dos operandos en memoria a la vez).

> **Capas de esta lección**
>
> **Contenido observado en CLS:** los valores de los ejemplos (`0x11`, `0x0F`, `0x44`, `0x8384`, `"core"`) y la secuencia de instrucciones de los capítulos 02 a 12. **Conocimiento externo:** la tabla de operaciones y la semántica precisa de cada flag. **Recomendación editorial:** el bloque integrador y los ejercicios.

Punto flotante y SSE: los registros xmm

> **De dónde sale esta lección**
>
> Este tema **no está en el corpus CLS**: es **conocimiento externo** que agregamos para completar la guía. El curso de CLS trabaja casi todo con enteros, así que el punto flotante quedaba sin cubrir. Lo incluimos porque aparece en cualquier binario de Windows real y, sin él, te trabás la primera vez que ves un registro `xmm`.

> **Objetivo de la lección**
>
> Al terminar vas a poder reconocer una operación con `float` / `double` en un listado, leer las instrucciones SSE escalares más comunes (`movss` , `movsd` , `addsd` , `cvtsi2sd` , `ucomisd`) y entender por qué una comparación de flotantes se lee distinto de una de enteros.

Prerrequisitos: instrucciones, flujo y registros (Etapa 2). **Duración sugerida:** 3 horas.

Por qué vas a chocarte con esto

Apenas analices un binario que haga una cuenta con decimales —un precio, un porcentaje, una coordenada, un promedio— vas a ver registros `xmm0` , `xmm1` … e instrucciones terminadas en `ss` , `sd` , `ps` o `pd` que no estaban en la tabla de enteros. No es un tema avanzado: es el día a día de cualquier programa compilado hoy. Además, muchos `memcpy` / `memset` optimizados usan registros `xmm` para mover 16 bytes de una.

Dos mundos: x87 y SSE

Hay dos formas de hacer punto flotante en x86, y conviene distinguirlas porque se ven muy distinto:

	x87 (FPU)	**SSE / SSE2**
Época	Clásico; común en código de 32 bits	Estándar desde ~2003; **obligatorio en x64**
Registros	Pila `st(0)` … `st(7)`	`xmm0` … `xmm7` (32 bits) / `xmm0` … `xmm15` (64 bits)
Estilo	Basado en pila (como una calculadora RPN)	Basado en registros (como los enteros)
Instrucciones	`fld` , `fstp` , `faddp` , `fmul` , `fcomip`	`movss` , `movsd` , `addsd` , `mulsd` , `ucomisd`

En 64 bits el compilador usa **casi siempre SSE**; vas a ver x87 sobre todo en binarios viejos de 32 bits.

IEEE 754 en una cucharada

- Un `float` ocupa 4 bytes; un `double` , 8. No son enteros: tienen signo, exponente y mantisa.
- Por eso las constantes flotantes **no se ven como el número**. `1.0` en `double` es `0x3FF0000000000000` ; `0.5` es `0x3FE0000000000000` . Suelen cargarse desde `.rdata` .

- Si en un `movsd xmm0, [0x...]` el valor cargado es `0x4059000000000000`, no es "un número raro": es `100.0` en `double`. Cualquier conversor IEEE 754 te lo traduce; no hace falta hacerlo a mano.

Las instrucciones SSE que vas a ver

La clave está en el **sufijo**:

- `ss` = *scalar single* → un `float` (4 bytes).
- `sd` = *scalar double* → un `double` (8 bytes).
- `ps` / `pd` = *packed* → varios valores a la vez (vectorización; ver la lección de optimizaciones).

Instrucción	Qué hace
`movss` / `movsd`	mueve un float / un double
`movaps` / `movups`	mueve 16 bytes (alineado / sin alinear)
`addsd`, `subsd`, `mulsd`, `divsd`	aritmética con double
`cvtsi2sd`	entero → double
`cvttsd2si`	double → entero (trunca)
`cvtss2sd` / `cvtsd2ss`	float ↔ double
`comiss` / `ucomisd`	**comparar** flotantes (actualiza EFLAGS)
`xorps xmm0, xmm0`	pone el registro en 0 (truco idiomático)
`pxor`, `andps`, `orps`	lógicas de 128 bits (a veces para el signo)

> **`xorps xmm, xmm` no es "una lógica rara"**
>
> Igual que `xor eax, eax` pone `EAX` en cero, `xorps xmm0, xmm0` pone `xmm0` en cero. Es el modismo del compilador para inicializar un flotante en 0.0.

> **`movaps` exige alineación a 16 bytes (el crash nº 1 al parchear en x64)**
>
> `movaps` requiere que la dirección sea múltiplo de 16 (`addr % 16 == 0`); `movups` no. Si al parchear o inyectar una llamada en x64 dejás `RSP` **desalineado** (típico: 8 bytes de más), el programa muere con `0xC0000005` (`STATUS_ACCESS_VIOLATION`) dentro de `ntdll` / `msvcrt` sobre un `movaps xmmword ptr [rsp+...], xmm0`, aunque la memoria sea perfectamente escribible. Por eso la ABI de Windows x64 exige el stack alineado a 16 bytes antes de cada `call`.

Ejemplo paso a paso: `double` en x64

```
double promedio(double a, double b) {
    return (a + b) / 2.0;
}
```

En x64 de Windows, los `double` llegan en `xmm0` y `xmm1`, y el resultado vuelve en `xmm0`:

```
promedio:
    addsd   xmm0, xmm1              ; xmm0 = a + b
    movsd   xmm1, [rel dos]         ; xmm1 = 2.0  (0x4000000000000000)
    divsd   xmm0, xmm1              ; xmm0 = (a+b) / 2.0
    ret
dos: dq 0x4000000000000000          ; 2.0 en .rdata
```

Lo que tenés que reconocer: `xmm0` / `xmm1` traen los argumentos, `addsd` es la suma, la constante `2.0` se carga de datos y el resultado queda en `xmm0`. No hay `push` / `pop` de flotantes: viajan por registros `xmm`.

Comparar flotantes es distinto

Una comparación de enteros usa `cmp` + `je/jg/jl`. Una de flotantes usa `ucomisd` / `comiss`, que actualiza `ZF`, `PF` y `CF` (no `SF` / `OF`), y después un salto **sin signo** (`ja`, `jae`, `jb`, `jbe`, `je`):

```
ucomisd xmm0, xmm1
jae     es_mayor_o_igual     ; usa CF/ZF, como una comparación sin signo
```

Además aparece el **flag de paridad (`PF`)**: se activa si alguno de los operandos es `NaN` (*not a number*). Por eso a veces ves un `jp` / `jnp` antes del salto real: el compilador está manejando el caso "no es un número". Si ves `ucomisd` seguido de `jp`, no te asustes: es el chequeo de NaN, no la condición principal.

x87 en binarios viejos (lo mínimo)

En código de 32 bits podés encontrarte con la pila del FPU:

```
fld  qword [a]     ; empuja a en st(0)
fadd qword [b]     ; st(0) = a + b
fstp qword [res]   ; guarda st(0) en res y lo saca de la pila
```

Se lee como una calculadora de pila: `fld` empuja, las operaciones trabajan sobre `st(0)`, `fstp` guarda y saca. No necesitás dominarlo; alcanza con reconocer que `f...` **= punto flotante x87** y seguir el valor por la pila `st`.

Práctica

1. Compilá `promedio.c` con `gcc -O0` (x64) y mirá el listado: ubicá `xmm0`, `xmm1`, `addsd`, `divsd` y la constante `2.0` en `.rdata`.
2. Tomá la constante que carga tu binario y convertila con un conversor IEEE 754. ¿Coincide con `2.0`?
3. Agregá una comparación `if (a > b)` y mirá cómo aparece `ucomisd` + salto sin signo (y quizás un `jp`).
4. Compilá una versión que use el **x87** y compará (deberías ver `fld` / `faddp` en vez de `xmm`). **Ojo en Windows x64:** `-mfpmath=387` sin `-m32` se **ignora** (x86-64 exige SSE2 por ABI) y `-m32` falla si tu MinGW no es *multilib*. Comandos que sí funcionan: MSVC de 32 bits (*x86 Native Tools Command Prompt*) `cl /Od /arch:IA32 promedio.c`; o MinGW de 32 bits `i686-w64-mingw32-gcc -O0 -mfpmath=387 -mno-sse -o promedio32.exe promedio.c`.

Preguntas de repaso

<details>
<summary>¿Qué significan los sufijos <code>ss</code>, <code>sd</code>, <code>ps</code> y <code>pd</code>?</summary>

`ss` = un `float` (scalar single); `sd` = un `double` (scalar double); `ps` / `pd` = varios valores a la vez (packed, vectorizado).
</details>

<details>
<summary>Ves <code>xorps xmm0, xmm0</code>. ¿Qué hace?</summary>

Pone `xmm0` en 0.0. Es el modismo para inicializar un flotante en cero, igual que `xor eax, eax` con enteros.
</details>

<details>
<summary>¿Por qué después de <code>ucomisd</code> aparece a veces un <code>jp</code>?</summary>

Porque la comparación de flotantes activa el flag de paridad si hay un `NaN`. El `jp` / `jnp` maneja ese caso antes del salto principal.
</details>

<details>
<summary>Un <code>movsd xmm0, [rel k]</code> carga <code>0x3FF0000000000000</code>. ¿Qué número es?</summary>

`1.0` en `double`. Las constantes flotantes se ven como su codificación IEEE 754, no como el número decimal.
</details>

Antes de avanzar

Tenés que poder:

- Reconocer una operación con `float` / `double` por los registros `xmm` y los sufijos `ss` / `sd`.
- Leer una comparación de flotantes (`ucomisd` + salto sin signo + chequeo de NaN).
- Traducir una constante IEEE 754 con una herramienta.
- Distinguir x87 (`f...`, pila `st`) de SSE (`xmm`).

Puerta de salida

Si podés leer `promedio` en assembly y explicar cómo viajan los argumentos y el resultado en `xmm`, ya no te va a frenar el punto flotante.

Fuentes de esta lección

- *Intel 64 and IA-32 Architectures Software Developer's Manual*, vol. 1 (FPU x87) y vol. 2 (instrucciones SSE/SSE2). Referencia primaria del repertorio de instrucciones.
- *Microsoft x64 calling convention*: paso de `float` / `double` en registros `xmm`.
- Estándar **IEEE 754** para la codificación de `float` y `double`.

Capas de esta lección

Conocimiento externo (agregado por esta edición): todo el contenido (x87, SSE, IEEE 754, convención de `xmm`). **No** proviene del corpus CLS, que trabaja con enteros. **Recomendación editorial:** los ejemplos `promedio` y la práctica con conversor IEEE 754.

Flujo: comparar, saltar y repetir

> **Objetivo de la lección**
>
> Al terminar vas a poder reconstruir un `if/else`, un bucle y un `switch` a partir de un listado de assembly, eligiendo el salto correcto según el dato sea con signo o sin signo.

Prerrequisitos: instrucciones y flags. **Duración sugerida:** 4 horas. Estimación aproximada.

La idea central

El procesador no tiene `if`. Tiene instrucciones que comparan y actualizan flags, y saltos que se toman o no según esos flags. Un `if` en C se convierte en:

```
cmp  eax, 4
jne  else_1      ; si no es igual, salta al else
; bloque del if
jmp  fin
else_1:
; bloque del else
fin:
```

Leer flujo es reconstruir ese patrón: **comparación → salto condicional → bloques → salto incondicional al final**.

`CMP` y `TEST` en detalle

`CMP a, b` calcula `a - b` y descarta el resultado. Los flags quedan como si hubieras restado:

Caso	`ZF`	`CF`	`SF`	`OF`
`a == b`	1	0	según resultado 0	0
`a > b` (sin signo)	0	0	—	—
`a < b` (sin signo)	0	1	—	—
`a > b` (con signo)	0	—	0	0
`a < b` (con signo)	0	—	1	0

`TEST a, b` hace `a AND b` y descarta el resultado. `TEST EAX, EAX` activa `ZF` si `EAX` es cero; es la forma estándar de preguntar "¿es nulo?".

El capítulo 09 lo muestra con `CMP EAX=4, EBX=4` (activa `ZF` y deja los registros intactos) y con `TEST EAX, EAX` seguido de `JZ` (`Reversing desde cero, cap. 09`, 00:01:05–00:06:00 y 00:31:32–00:33:00).

La tabla de saltos condicionales

Instrucción	Salta si...	Uso típico
`JE` / `JZ`	`ZF = 1`	igual / cero
`JNE` / `JNZ`	`ZF = 0`	distinto / no cero
`JA` / `JNBE`	`CF = 0` y `ZF = 0`	mayor, **sin signo**
`JAE` / `JNB` / `JNC`	`CF = 0`	mayor o igual, sin signo
`JB` / `JNAE` / `JC`	`CF = 1`	menor, sin signo
`JBE` / `JNA`	`CF = 1` o `ZF = 1`	menor o igual, sin signo
`JG` / `JNLE`	`ZF = 0` y `SF = OF`	mayor, **con signo**
`JGE` / `JNL`	`SF = OF`	mayor o igual, con signo
`JL` / `JNGE`	`SF ≠ OF`	menor, con signo
`JLE` / `JNG`	`ZF = 1` o `SF ≠ OF`	menor o igual, con signo
`JS` / `JNS`	`SF = 1` / `SF = 0`	signo
`JO` / `JNO`	`OF = 1` / `OF = 0`	desborde con signo

No hace falta memorizarla: alcanza con saber que `J` **+ letra** indica la condición (`E` igual, `N` no, `A` arriba, `B` abajo, `G` mayor, `L` menor) y que las variantes `A/B` son sin signo y `G/L` con signo.

Cómo se codifica un salto (y hasta dónde llega)

Hasta acá vimos *qué* hace cada salto; falta *cómo se escribe en bytes*, porque al parchear —o al leer un salto en el editor hexadecimal— trabajás con esos bytes. El **Curso IDA** lo muestra byte a byte sobre un `CRACKME` propio (Curso IDA · tute 7, Control de flujo):

- **Salto corto (`JMP SHORT`)** → opcode `EB` + **1 byte** de desplazamiento. El destino se calcula como **inicio de la instrucción + 2 + desplazamiento** (el 2 es el largo del salto corto). El alcance es chico: `0x7F` es el **máximo hacia adelante** y `0x80` el **máximo hacia atrás**.
- Un desplazamiento `FE` (−2) hace que el salto caiga **sobre sí mismo**: es el **loop infinito** clásico (`EB FE`).
- **Salto largo (`JMP` / `Jcc` near)** → opcode `E9` + un **dword** de desplazamiento, para alcanzar direcciones lejanas. La distancia se calcula como **destino − inicio − 5** (5 = largo del salto largo).

> **Probarlo sin romper la base de datos**
>
> El tute edita esos bytes con **Patch Byte** y **KeyPatch** dentro de IDA, y antes hace un **snapshot de la IDB** (*View → Database snapshot manager*) para volver atrás con *Restore* si el cambio rompe una función. Hacé lo mismo antes de experimentar: cambiás un `0x05` por `0x7F` o `0x80`, ves el efecto y, si algo se rompe, restaurás.

Ejemplo paso a paso: el mismo dato, dos interpretaciones

```
mov eax, 0xFFFFFFFF
cmp eax, 2
jg  mayor_firmado
ja  mayor_sin_signo
```

- `CMP` calcula `0xFFFFFFFF − 2`. Como número **sin signo**, `0xFFFFFFFF` es 4.294.967.295, mucho mayor que 2: `CF = 0`, `ZF = 0`, así que `JA` **salta**.
- Como número **con signo**, `0xFFFFFFFF` es −1, menor que 2: `SF = 1`, `OF = 0`, `SF ≠ OF`, así que `JG` **no salta**; el que saltaría es `JL`.

Es el ejemplo que usa el capítulo 09 (00:24:30–00:25:30). La lección: **el mismo `cmp` puede producir saltos opuestos según la instrucción que lo sigue**. Si el código usa `JA/JB`, el autor pensó el dato como sin signo; si usa `JG/JL`, como con signo. Ese detalle cambia por completo la reconstrucción de una comparación de contraseñas o de un índice.

Bucles

`while` / `for`

```
while (i < 10) { ...; i++; }
```

Se compila a:

```
mov ecx, 0
loop_start:
cmp ecx, 10
jge loop_end      ; si ecx >= 10, salir
; cuerpo
inc ecx
jmp loop_start
loop_end:
```

El salto de salida es la **negación** de la condición: el cuerpo se ejecuta mientras se cumple, así que se sale cuando deja de cumplirse.

`do/while`

La condición va al final, después del cuerpo. Se ejecuta al menos una vez.

`LOOP`

`LOOP etiqueta` decrementa `ECX` y salta si `ECX ≠ 0`. Es una instrucción compacta pero más lenta que `dec ecx; jnz`. En el capítulo 12 se ve `LOOP` con `ECX` como contador (00:24:06).

Bucles con cadenas

`REP MOVSB`, `REPE CMPSB` y `REPNE SCASB` repiten una operación mientras se cumple una condición y `ECX` no llegue a cero. Son bucles "de hardware" que el procesador ejecuta sin un `jmp` explícito.

`switch`: la tabla de saltos

Un `switch` con casos consecutivos se compila a una **tabla de direcciones**:

```
cmp eax, 3
ja  default
jmp [tabla + eax*4]
```

La tabla contiene las direcciones de cada `case`; `jmp [tabla + eax*4]` salta a la que corresponde al índice. En el desensamblador se ve como un `jmp` con un operando indexado, seguido de una zona de datos con direcciones (a veces en `.rdata`).

Para reconstruirlo: identificá la comparación de rango (`cmp` + `ja default`), la tabla y el orden de las entradas. Si los casos no son consecutivos, el compilador puede usar una cadena de comparaciones en lugar de una tabla.

Decisiones sin salto: `CMOVcc`

No toda decisión produce un salto. La familia `CMOVcc` (*conditional move*) **mueve un valor solo si se cumple una condición**, mirando los mismos flags que los `Jcc`. Por ejemplo, `CMOVZ eax, ebx` copia `EBX` en `EAX` únicamente si `ZF = 1`; si no, `EAX` queda como estaba.

```
cmp  byte ptr [esi], 0x41   ; ¿el primer carácter es 'A'?
mov  ecx, 0x31
cmovz esi_byte, ecx         ; si era 'A' (ZF=1), lo reemplaza por 0x31; si no, lo deja
```

La consecuencia para el reversing es importante: con `CMOVcc` **no hay dos bloques** (uno para el camino bueno y otro para el malo) ni un salto visible. La comparación y la elección ocurren en el mismo bloque, así que no hay un "salto para parchear" evidente. El capítulo 16 usa justamente esto para que un crackme no tenga bloques separados de "chico bueno" y "chico malo": decide con un `CMOVZ` y no con un `Jcc` (`Reversing desde cero, cap. 16`; transcripción .txt, sin marcas de tiempo).

Cuando no encuentres el salto que decide, buscá un `CMOVcc`, un `SETcc` (que deja 0 o 1 en un byte según la condición) o una operación aritmética que use el flag: la decisión puede estar ahí, sin bifurcación.

Ejemplo integrador: el `if` del capítulo 15

El programa de práctica del capítulo 15 pide dos números y aplica:

```
if (a > 50 || b < 50) { error; }
if ((a + b) * 4 == 480) { correcto; } else { incorrecto; }
```

En assembly, la primera condición se parte en dos comparaciones con saltos:

```
cmp eax, 50
jg  error          ; a > 50 -> error
cmp ebx, 50
jl  error          ; b < 50 -> error
```

La segunda usa una suma, un desplazamiento y una comparación:

```
add eax, ebx
shl eax, 2         ; (a+b) * 4
cmp eax, 480
jne incorrecto
```

El capítulo lo reconstruye leyendo los saltos y los comentarios que va agregando en IDA, y luego lo confirma en el debugger con `40` y `80` (`Reversing desde cero, cap. 15`, 00:10:00–00:19:00).

> **El error de leer el salto al revés**
>
> Un salto "hacia el bloque malo" no significa que la condición sea verdadera: puede ser la negación. Antes de escribir el `if`, preguntate **qué camino toma el programa cuando el salto se produce** y **qué camino cuando no**. El capítulo 09 insiste en esto al explicar cada `Jcc` por su condición (00:07:00–00:12:00).

Práctica

1. Traducí a C:

```
cmp eax, 10
jl  menor
mov ebx, 1
jmp fin
menor:
mov ebx, 0
fin:
```

1. Escribí en C un `for` que sume los números del 1 al 10 y compilalo con `-O0`. Identificá el contador, la comparación y el salto de salida.
2. ¿Qué diferencia hay entre `JA` y `JG`? Dame un ejemplo con `0xFFFFFFFF`.
3. Reconstruí un `switch` de tres casos a partir de una tabla de saltos inventada por vos. Dibujá la tabla y el `jmp` indexado.

4. En el programa del capítulo 15, cambiá `40 80` por `50 70`. ¿Sigue dando 480? ¿Qué condición falla?

Preguntas de repaso

1. ¿`CMP` modifica los operandos? No. Solo actualiza flags. Es una resta "de prueba".

2. ¿Cómo se sale de un `while`? Con un salto condicional que se toma cuando la condición deja de cumplirse, o con un `jmp` al final si el cuerpo tiene un `break`.

3. ¿Qué indica `JA` que no indica `JG`? `JA` compara sin signo; `JG`, con signo. Con `0xFFFFFFFF` y `2`, `JA` salta y `JG` no.

4. ¿Cómo se reconoce una tabla de salto? Por un `jmp` con operando indexado (`[tabla + registro*4]`) y una zona de datos con direcciones consecutivas, generalmente precedida por una comparación de rango.

5. ¿`LOOP` usa qué registro? `ECX`. Decrementa y salta si no llegó a cero.

Antes de avanzar

Tenés que poder:

- Reconstruir un `if/else` a partir de `cmp` + `jcc` + `jmp`.
- Elegir `JA/JB` o `JG/JL` según el tipo del dato.
- Reconocer un bucle `while`, un `do/while` y un `for`.
- Explicar una tabla de saltos.

> **Puerta de salida**
>
> Si podés mirar un bloque con dos saltos y decir qué camino toma con un valor concreto, estás listo para las llamadas a funciones.

Fuentes de esta lección

- `Reversing desde cero, cap. 09` — `CMP`, `TEST`, `JE/JNE/JA/JB/JG/JL`, `LEAVE`.
- `Reversing desde cero, cap. 16` — `CMOVcc`: decidir sin salto, un crackme sin bloques separados de bueno/malo.
- `Reversing desde cero, cap. 12` — `LOOP` y comparaciones en un binario real.
- `Reversing desde cero, cap. 15` — reconstrucción de condiciones.
- Curso IDA · tute 7, Control de flujo — codificación de los saltos (`EB` / `E9`), alcance ± `0x7F` / `0x80`, `EB FE` = loop infinito, y cómo parchearlos en IDA con snapshot de la IDB.
- `Videos y material adicional, Video 29` — `IF/ELSE` compilado a `CMP` + saltos.
- `Videos y material adicional, Video 30` — bucles y pila.

Capas de esta lección

Contenido observado en CLS: la tabla de saltos del capítulo 09, el ejemplo `0xFFFFFFFF` contra `2`, el `LOOP` del capítulo 12 y el `if` del capítulo 15. **Conocimiento externo:** la tabla completa de `Jcc` y el patrón de tabla de saltos. **Recomendación editorial:** los ejercicios y el ejemplo integrador.

Llamadas, convenciones y marcos de función

> **Objetivo de la lección**
>
> Al terminar vas a poder seguir un `call` hasta su destino, calcular la dirección de retorno, identificar los argumentos de una función en 32 y 64 bits y decir quién limpia el stack.

Prerrequisitos: flujo, stack y registros. **Duración sugerida:** 4 horas. Estimación aproximada.

`CALL` y `RET`

`call destino` hace dos cosas:

1. Empuja en el stack la dirección de la instrucción **siguiente** (el *return address*).
2. Salta a `destino`.

`ret` hace lo inverso: saca el return address del stack y salta ahí.

Ejemplo del capítulo 06: un `call` en `0x00401013` (5 bytes) deja `0x00401018` en el tope del stack; la rutina hace `nop; ret` y vuelve a `0x00401018` (`Reversing desde cero, cap. 06`, 00:00:53–00:03:10).

```
0x401013: call 0x40102C     ; push 0x401018 ; jmp 0x40102C
0x401018: ...               ; acá vuelve
```

`ret n` (por ejemplo `ret 0x10`) hace lo mismo y además suma `n` a `ESP` después de sacar el return address. Se usa en la convención `stdcall` para que la función llamada limpie los argumentos. En el ejemplo del capítulo 06, `ret 0x10` mueve el stack `0x10 + 4 = 0x14` (00:08:38–00:09:30).

`call` directo e indirecto

```
call 0x401000          ; directo: el destino está en el opcode (E8 + desplazamiento)
call dword ptr [0x403000]  ; indirecto: el destino está en la memoria (FF15)
```

El indirecto es el que se usa para llamar APIs a través de la IAT. En el capítulo 06, `call dword ptr [0x402018]` no salta a `0x402018` sino a lo que hay allí; el capítulo 05 lo muestra con una API de `kernel32` (`Reversing desde cero, cap. 05`, 00:13:00–00:18:00).

Convenciones de llamada

Una **convención de llamada** es el acuerdo sobre tres cosas: dónde van los argumentos, quién limpia el stack y qué registros se preservan.

32 bits

Convención	Argumentos	Limpia el stack	Registros volátiles
`cdecl`	En el stack, de derecha a izquierda	El llamador	`EAX`, `ECX`, `EDX`
`stdcall`	En el stack, de derecha a izquierda	La función (`ret n`)	`EAX`, `ECX`, `EDX`
`fastcall`	`ECX`, `EDX` y el resto en el stack	La función	`EAX`, `ECX`, `EDX`
`thiscall`	`ECX = this`; resto en el stack	La función	`EAX`, `ECX`, `EDX`

`cdecl` es la convención por defecto de C; por eso las funciones variádicas (`printf`) la usan: solo el llamador sabe cuántos argumentos pasó. `stdcall` es la de la API de Windows (`MessageBoxA`, `CreateThread`). `thiscall` es la de los métodos de clase en C++.

Cómo reconocerlas en el listado:

- Si después del `call` hay un `add esp, N`, es `cdecl`: el llamador limpia.
- Si la función termina en `ret N`, es `stdcall` o `fastcall`: la función limpia.
- Si el primer argumento llega en `ECX`, es `fastcall` o `thiscall`.

El video 28 del curso de OllyDbg muestra el mismo concepto con procedimientos propios: `CALL` / `RET`, variables locales y argumentos en la pila (`Videos y material adicional, Video 28`, 00:00:00–00:05:00).

64 bits

En x64 hay una sola convención para el código de Windows: los primeros cuatro argumentos enteros van en `RCX`, `RDX`, `R8` y `R9`; el resto, en el stack. El llamador reserva **32 bytes de shadow space** antes del `call` y limpia el stack después. Los registros `RAX`, `RCX`, `RDX`, `R8`–`R11` son volátiles; `RBX`, `RBP`, `RDI`, `RSI`, `R12`–`R15` se preservan.

El capítulo 18 lo muestra con `main`: los tres argumentos llegan en `RCX`, `RDX` y `R8`, y las variables locales se referencian como `[rsp+0xNN]` (`Reversing desde cero, cap. 18`, 00:13:00–00:20:00).

> **En 64 bits los argumentos no se cargan "en orden"**
>
> Como los argumentos no se apilan uno tras otro con `push`, el compilador puede setear los registros en **cualquier orden**: podés ver que carga `R8` y recién una línea más abajo `RCX`. No deduzcas el primer argumento por su posición en el listado; identificalo por el **registro** (`RCX`, `RDX`, `R8`, `R9`) y por el rol. El capítulo 19 lo muestra con `fgets(buffer, maxcount, stream)`: `RCX` es el buffer, `RDX` el máximo y `R8` el stream, aunque en el código `RCX` aparezca seteado después de `R8` (`Reversing desde cero, cap. 19`; transcripción .txt, sin marcas de tiempo).

x64: el resto de la ABI

El cap. 18 muestra lo básico; la ABI de x64 de Windows tiene cuatro detalles más que conviene conocer (**conocimiento externo**; convención x64 de Microsoft):

- **Alineación de la pila.** En el punto del `call`, `RSP` debe estar alineado a **16 bytes**. Como el `call` empuja 8 bytes (el return address), al entrar a la función `RSP` queda "desalineado en 8"; por eso los prólogos restan una cantidad que deja todo en múltiplo de 16 (y el **shadow space** de 32 bytes también respeta eso). Si ves un `sub rsp, 0x28` típico, es shadow space (0x20) + alineación.
- **Argumentos de punto flotante.** `float` / `double` van en `XMM0` – `XMM3` (misma posición que los enteros: el 1.º en `XMM0` o `RCX`, no ambos). `XMM0` – `XMM5` son **volátiles**; `XMM6` – `XMM15` se **preservan**.
- **Funciones variádicas.** En una variádica (`printf`), cada argumento flotante se pasa **a la vez** en el registro XMM y en el GPR entero correspondiente, y `AL` indica cuántos registros vectoriales se usaron. Por eso una variádica se ve distinta de una función normal con los mismos tipos.
- **Unwind / `.pdata`.** No hay cadena SEH en el stack como en 32 bits: la info de desenrollado (qué registros salva el prólogo, dónde está el handler) vive en tablas `.pdata` / `.xdata` (lo ves en PE fino 2.14). Sirve para **delimitar funciones** sin símbolos y para entender el prólogo/epílogo.

Ejemplo paso a paso: `suma(a, b)` en 32 bits

```
int suma(int a, int b) { return a + b; }
int main(void) { return suma(3, 4); }
```

Compilado con `cdecl`, la llamada se ve así:

```
push 4          ; segundo argumento
push 3          ; primer argumento
call suma
add  esp, 8     ; el llamador limpia los dos argumentos
```

Y la función:

```
suma:
push ebp
mov  ebp, esp
mov  eax, [ebp+8]    ; a
add  eax, [ebp+12]   ; b
pop  ebp
ret
```

Recorrido del stack:

```
antes del call:      [esp] = 3, [esp+4] = 4
después del call:    [esp] = return address
                     [esp+4] = 3
                     [esp+8] = 4
después del prólogo: [ebp] = ebp viejo
                     [ebp+4] = return address
                     [ebp+8] = 3
                     [ebp+12] = 4
```

Si la función fuera `stdcall`, terminaría en `ret 8` y no habría `add esp, 8` después del `call`. Ese es el detalle que te dice quién limpia.

> **En el Curso IDA: leer una función basada en EBP (ejercicios IDA1/IDA2)**
>
> El Curso IDA trae dos ejercicios muy didácticos de funciones basadas en `EBP`: `IDA1.exe` e `IDA2.exe`. El método es el de siempre: se llega al `main` **por la string** ("You are a winner man je"), se usan los **xrefs** (`X`) para ver dónde se tocan las variables y se las **renombra** (en `IDA2` hay dos "cookies" que hay que comparar para caer en la zona del "chico bueno"). Un detalle real a tener en cuenta: `IDA1.exe` está compilado con **DevC++**, que arma las llamadas a las APIs de una forma distinta a la habitual y puede marear al principio (Curso IDA · tute 23, IDA1.exe y tute 24, IDA2 (cookies)).

Recursión

Una función recursiva se llama a sí misma. Cada llamada crea un **nuevo marco** en el stack, con sus propios argumentos y locales. Por eso una recursión sin caso base termina en *stack overflow*: el stack se agota.

```
int fact(int n) { return n <= 1 ? 1 : n * fact(n - 1); }
```

En el debugger vas a ver el mismo `call` repetido con valores decrecientes de `n`, y el call stack creciendo hacia arriba. El panel **Call Stack** te muestra la cadena de marcos; es la forma más rápida de entender una recursión.

Llamadas indirectas y funciones virtuales

```
call [eax]        ; llama a la función apuntada por EAX
call [eax+4]      ; llama a la segunda entrada de una tabla
```

En C++, un objeto con funciones virtuales guarda al principio un puntero a la **vtable**: una tabla de punteros a métodos. `call [eax+4]` puede ser "llamar al segundo método virtual del objeto apuntado por EAX". El capítulo 37 lo desarrolla con una clase de `0x58` bytes y una vtable de cuatro punteros (`Reversing desde cero, cap. 37`, 00:07:38–00:10:48). Es un tema de la rama de C++; en el recorrido base alcanza con reconocer el patrón.

Práctica

1. Compilá `suma.c` con `gcc -O0 -m32` (o Visual Studio en 32 bits) y mirá el listado en x64dbg. Identificá los `push`, el `call`, el `add esp` y el `ret`.
2. Hacé lo mismo con `__stdcall` (en GCC, `__attribute__((stdcall))`) y comprobá si aparece `ret 8`.
3. Compilá la versión de 64 bits y verificá en qué registros llegan `a` y `b`.
4. Escribí una función recursiva de factorial y mirá el Call Stack en el debugger con `n = 3`.

5. En el programa del capítulo 15, identificá si `printf` / `scanf` son `cdecl` o `stdcall` y justificá con el `add esp` posterior.

Preguntas de repaso

1. ¿Qué guarda `call` en el stack? La dirección de la instrucción siguiente al `call`, para que `ret` sepa dónde volver.

2. ¿Quién limpia los argumentos en `cdecl`? ¿Y en `stdcall`? En `cdecl`, el llamador (con `add esp, N` después del `call`). En `stdcall`, la función llamada (con `ret N`).

3. ¿Cómo se pasa `this` en un método de C++ de 32 bits? En `ECX`, según la convención `thiscall`.

4. ¿Cuántos argumentos van en registros en x64? Los primeros cuatro enteros: `RCX`, `RDX`, `R8`, `R9`. El resto va al stack, después de 32 bytes de shadow space.

5. ¿Por qué una recursión infinita produce un stack overflow? Porque cada llamada crea un marco nuevo y el stack tiene un tamaño limitado. Al agotarse, el sistema termina el proceso.

Antes de avanzar

Tenés que poder:

- Calcular la dirección de retorno de un `call`.
- Distinguir `call` directo de indirecto.
- Reconocer `cdecl`, `stdcall` y la convención x64 en un listado.
- Dibujar el marco de una función con dos argumentos.

> **Puerta de salida**
>
> Si podés explicar quién limpia el stack mirando el `ret` o el `add esp`, estás listo para usar el debugger con criterio.

Fuentes de esta lección

- `Reversing desde cero, cap. 06` — `CALL` directo/indirecto, `RET`, `RET n`.
- `Reversing desde cero, cap. 18` — marcos, prólogo/epílogo, convención x64.
- `Reversing desde cero, cap. 19` — argumentos x64 cargados fuera de orden; identificar por registro y rol (`fgets`).
- `Videos y material adicional, Video 28` — procedimientos y variables locales en MASM32.
- `Reversing desde cero, cap. 37` — vtables y llamadas virtuales.
- Curso IDA · tutes 23 y 24 — funciones basadas en EBP (`IDA1.exe` / `IDA2.exe`), el quirk de DevC++ al llamar APIs y el método strings→`main`→xrefs→renombrar.

Capas de esta lección

Contenido observado en CLS: el ejemplo del `call` en `0x00401013`, `ret 0x10`, la convención x64 del capítulo 18 y la vtable del capítulo 37. **Conocimiento externo:** tabla de convenciones `cdecl` / `stdcall` / `fastcall` / `thiscall`. **Recomendación editorial:** el ejemplo `suma` y los ejercicios.

El debugger: x64dbg de punta a punta

> **Objetivo de la lección**
>
> Al terminar vas a poder abrir un programa propio en x64dbg, moverte por el código, ejecutar con F7/F8/F9, poner breakpoints por software, por hardware y de memoria, condicionarlos y dejar un registro de lo observado.

Prerrequisitos: registros, instrucciones, flujo y llamadas. **Duración sugerida:** 5 horas. Estimación aproximada.

Análisis dinámico y estático

Hay dos formas de mirar un programa:

- **Dinámico:** lo ejecutás dentro de un debugger y observás qué hace mientras corre. Te muestra la verdad de la ejecución, pero solo del camino que tomaste.
- **Estático:** lo analizás sin ejecutarlo, con un desensamblador como IDA o Ghidra. Te muestra todo el código, pero no los valores concretos.

El capítulo 11 lo plantea como una decisión de método: el dinámico es más visual y rápido para problemas chicos; el estático permite analizar funciones que no sabés cómo alcanzar y es obligatorio cuando no podés ejecutar (`Reversing desde cero, cap. 11`, 00:00:05–00:01:00).

En la práctica se usan los dos: primero el estático para orientarte, después el dinámico para confirmar.

> **En el Curso IDA: IDA también debuggea, y se practica con crackmes propios**
>
> El Curso IDA refuerza esta misma combinación de estático y dinámico, pero sin salir de IDA: recordá que **IDA no solo desensambla, también soporta varios debuggers**, así que podés estudiar el loader y después ejecutar sobre el mismo programa sin cambiar de herramienta. Lo practica con crackmes chiquitos hechos por el propio autor (`HOLA_REVERSER.exe` , `TEST_REVERSER.exe`) (Curso IDA · tute 10, Loader + debugger; prácticas en tute 9 y tute 12).

La ventana de x64dbg

Al abrir un programa, x64dbg se detiene en el **entry point**: la primera instrucción que ejecuta el código del programa. No es necesariamente `main`; es el punto donde el cargador le pasa el control (cap. 01, 00:02:08).

Las pestañas principales:

Pestaña	Qué muestra
CPU	Desensamblado, registros, flags, stack y dump
Log	Mensajes del debugger y de los breakpoints con log
Notes	Notas que escribís vos
Breakpoints	Lista de breakpoints con condición y estado
Memory Map	Regiones del espacio de direcciones
Call Stack	Cadena de llamadas actual
SEH	Cadena de manejadores de excepciones (32 bits)
Symbols	Módulos cargados y sus funciones
Threads	Hilos del proceso
Handles	Recursos abiertos
Trace	Resultado del trazado
PE Viewer	Cabecera PE del módulo (plugin)

El capítulo 01 hace un recorrido por todas ellas y aclara que al principio no hace falta entenderlas todas (00:04:00–00:07:00).

Ejecutar paso a paso

Tecla	Acción	Qué hace
`F9`	Run	Ejecuta hasta el próximo breakpoint o hasta que el programa termine
`F8`	Step over	Ejecuta la instrucción; si es un `call`, ejecuta la función completa sin entrar
`F7`	Step into	Ejecuta la instrucción; si es un `call`, entra a la función
`F2`	Toggle breakpoint	Pone o saca un breakpoint por software
`Ctrl+F2`	Restart	Reinicia el proceso
`*`	Follow EIP/RIP	Vuelve a la instrucción actual

La diferencia entre F7 y F8 es la más importante al empezar: con F8 pasás "por arriba" de una función; con F7 entrás. El capítulo 01 lo demuestra entrando a `main` con F7 y saliendo con F8 (00:15:16–00:20:00).

> **El "step over" no saltea la función**
>
> `F8` **ejecuta** la función completa, no la ignora. La diferencia es que no te detiene instrucción por instrucción dentro de ella. Si la función tiene efectos (escribe un archivo, muestra un mensaje), esos efectos ocurren igual.

Los tres tipos de breakpoint

Por software

Es el clásico. El debugger reemplaza el primer byte de la instrucción por `0xCC` (la instrucción `INT3`). Cuando el procesador la ejecuta, genera una excepción que el debugger intercepta, muestra el estado y restaura el byte original.

- Se pone con `F2` o con doble clic en la columna de opcodes.
- No hay un límite práctico: podés poner todos los que quieras.
- Es fácil de detectar por un programa: si compara el byte en memoria con el del archivo, ve `0xCC` donde debería haber otra cosa.

Por hardware

Usa los registros de depuración `DR0` a `DR3` del procesador. Hay **cuatro por hilo** (no por proceso: cada hilo tiene su propio contexto de CPU). En x64dbg se aplican al hilo actual, salvo que configures lo contrario.

- Se ponen con clic derecho → *Breakpoint* → *Set hardware on execution*.
- Se marcan con un punto rojo, no con la línea entera.
- No modifican el código, así que son más difíciles de detectar.
- También permiten *on access* (lectura/escritura) sobre 1, 2 o 4 bytes.

El capítulo 07 los muestra y aclara que el hardware "para una instrucción después" del acceso (`Reversing desde cero, cap. 07`, 00:02:34–00:10:00). Es una imprecisión útil de conocer: el *debug exception* se genera después de ejecutar la instrucción que accedió a la dirección vigilada.

> **Una corrección al audio**
>
> El capítulo 07 dice "cuatro por proceso" y "funcionan solamente en el thread principal". Lo correcto es que los registros `DR0` – `DR3` son parte del contexto de cada hilo, así que el límite es por hilo. En la práctica, x64dbg te muestra y aplica los breakpoints de hardware según el hilo que estés mirando. Si no te aparecen, revisá qué hilo está seleccionado.

De memoria

Vigila una región (no solo 1, 2 o 4 bytes) y se dispara cuando el programa la lee o la escribe. Internamente usa protecciones de página: el debugger marca la página como no accesible y recibe la excepción cuando

alguien la toca.

- Se pone seleccionando un rango en el dump → clic derecho → *Breakpoint* → *Memory on access/read/write*.
- Opciones: *single shot* (se deshabilita después del primer impacto) o *restore on hit* (sigue activo).
- Es la herramienta para responder "¿quién lee este buffer?".

El capítulo 07 lo prueba con el buffer donde `GetDlgItemText` escribe el nombre: pone un breakpoint de memoria sobre los primeros bytes y el programa se detiene cuando los lee (00:10:00–00:14:00).

Breakpoints condicionales y con log

Un breakpoint puede tener una **condición** y una **acción**:

- **Condición** (`breakif`): solo se detiene si se cumple. Ejemplo: `EAX == 8`.
- **Log**: escribe un texto en el panel Log cada vez que pasa, sin detenerse. Ejemplo: `valor de EAX = {EAX}`.

En el capítulo 07, Ricardo usa un breakpoint en una función que devuelve la longitud del texto ingresado. Primero lo configura para loguear `{EAX}` y ve `5` cuando escribe `"solid"`. Después le pone la condición `EAX == 8` y comprueba que solo se detiene cuando el nombre tiene 8 caracteres (00:15:00–00:20:00).

```
breakif: EAX == 8
log:     "longitud = {EAX}"
```

Ese patrón —"no me interesa parar, me interesa saber qué valor toma"— es el que vas a usar para entender bucles y comparaciones sin detenerte cien veces.

Etiquetas, comentarios y navegación

- **Etiqueta** (`:`): renombra una dirección para no perderte. En el capítulo 11, Ricardo etiqueta el bloque del mensaje "acceso denegado" como `chico_malo` y el otro como `chico_bueno`.
- **Comentario** (`;`): agrega una nota a una línea.
- **Follow in dump**: muestra en el panel de memoria el contenido de una dirección.
- **Follow in disassembler**: salta a la dirección apuntada.
- **Ctrl+G**: ir a una dirección o expresión.
- **Ctrl+Shift+G**: ir a un file offset (útil para comparar con el archivo).

Las etiquetas y comentarios no cambian el programa: son tu mapa de lectura. El capítulo 11 muestra cómo el análisis visual se apoya en ellas (00:02:16–00:06:00).

Encontrar el código que maneja un botón o un mensaje

En un programa con ventana no hay un `main` lineal: Windows le **envía mensajes** (se hizo clic, se escribió en un campo, se apretó un botón) y el programa los atiende en una función despachadora. Encontrar esa

función es el primer paso para analizar una interfaz.

Lo que hay que saber:

- La función despachadora es el **WinProc** (si la ventana se creó con `CreateWindow`) o la **Dialog procedure** (si se creó con `DialogBoxParam`). Las dos hacen lo mismo: reciben el mensaje y lo reparten con un `switch`.
- El **mensaje** es el **segundo argumento** de esa función. Un `WM_COMMAND` vale `0x111` y es el que se dispara, en general, al apretar un botón; un `WM_LBUTTONDOWN` vale `0x201`.
- El **Control ID** identifica qué botón se tocó. Se puede ver pasando el mouse por el control en una herramienta de inspección de ventanas.

Tres maneras de llegar al handler, de más simple a más robusta:

1. **Buscar el Control ID en estático.** Si el botón tiene ID `0x13A`, buscás ese inmediato en IDA (ver la lección de IDA) y caés dentro del `WinProc`. Falla cuando el ID es un número bajo y común (`1`, `5`, `10`): hay demasiadas ocurrencias.
2. **Breakpoint condicional sobre el mensaje.** Poné un breakpoint donde la función carga el mensaje en un registro y condicionalo a ese mensaje. Podés además loguear los dos argumentos leyéndolos del stack:

```
log:     "msg={2}  id={3}"      ; con getdword(sp+8) y getdword(sp+C)
breakif: getdword(sp+8) == 0x111 && getdword(sp+0xC) == 0x13A
```

1. **Breakpoint en `SendMessage` + breakpoint de ejecución en el código.** Es el método genérico que sirve aunque no sepas nada del programa. En Windows, los mensajes llegan vía `SendMessageW` / `SendMessageA` de `user32.dll` (ahí el mensaje también es el segundo argumento). Poné un breakpoint ahí, condicionado a tu mensaje; cuando pare, poné un **breakpoint de hardware por ejecución sobre toda la sección de código del ejecutable** (su dirección de inicio y su tamaño, que ves en el *Memory Map* o en *Segments*). Al continuar, el programa se detiene en la primera instrucción **de tu código** que corre tras recibir el mensaje: esa es la entrada del WinProc. Después borrás ese breakpoint para seguir.

> **Cómo se registra el WinProc**
>
> Con `CreateWindow`, antes hay un `RegisterClassA` que recibe una estructura `WNDCLASS`; dentro de esa estructura está el puntero al WinProc. Con `DialogBoxParam`, el procedimiento se pasa directamente como uno de sus argumentos. Encontrar `RegisterClassA` o `DialogBoxParam` te lleva, por cualquiera de los dos caminos, a la función que maneja los mensajes.

El capítulo 28 ("Mensajes de Windows") muestra las tres formas sobre varios crackmes propios, incluida la carga de los símbolos de `user32.dll` para poner el breakpoint en `SendMessageW` (`Reversing desde cero, cap. 28`; transcripción .txt, sin marcas de tiempo). El capítulo 24b agrega que el `switch` de mensajes se lee mejor traduciendo los números a sus nombres `WM_*` con la tecla `M` en IDA.

Parchear (solo programas propios)

x64dbg permite modificar instrucciones en memoria y guardar los cambios en un archivo:

1. Seleccioná la instrucción.
2. Clic derecho → *Binary* → *Fill with NOP* (o *Edit* para escribir bytes).
3. *Patches* → *Patch file* para guardar una copia con los cambios.

En el capítulo 11, Ricardo reemplaza un salto condicional por dos `NOP` y guarda una copia del programa que acepta cualquier clave (00:20:00–00:25:00). Es una demostración didáctica sobre un crackme de práctica.

> **El límite legal y ético**
>
> Parchear un programa **no es una técnica neutral**. En esta guía se practica únicamente con programas propios, software abierto o crackmes diseñados para aprender, y se explica el efecto de un parche para entender el flujo. No se parchea software comercial para saltar licencias, DRM, autenticación ni controles de acceso. Si el programa no es tuyo o no tiene licencia para analizarlo, no lo toques.

Anti-debug: qué cambia cuando hay un debugger

Un programa puede detectar que está siendo depurado. Los chequeos más comunes en Windows de 32 bits:

Chequeo	Dónde mira	Qué delata
`IsDebuggerPresent`	`PEB+2`	Byte en 1 si hay debugger
`NtGlobalFlag`	`PEB+0x68`	Flags de heap activados por el debugger
Lista de procesos	`CreateToolhelp32Snapshot`	Encuentra `x64dbg.exe`, `ida.exe`, etc.
`ParentPID`	PID del proceso padre	El padre no es `explorer.exe`
`CloseHandle` inválido	Excepción `0xC0000008`	Comportamiento distinto bajo debugger
`INT3` en memoria	Byte `0xCC`	Breakpoint por software
`ContextRecord` tras una excepción	`DR0` – `DR3` del contexto	Breakpoints de **hardware**

El capítulo 21 muestra el último caso: un programa que provoca una excepción a propósito puede, en su manejador, leer el `ContextRecord` y mirar los registros de depuración `DR0` – `DR3`; si no están en cero, hay breakpoints de hardware puestos y el programa lo detecta (`Reversing desde cero, cap. 21`; transcripción .txt, sin marcas de tiempo). Es un motivo para conocer cómo funcionan las excepciones (ver la rama de vulnerabilidades).

El capítulo 22 explica los dos primeros con el PEB y el comando `hide` de x64dbg (`Reversing desde cero, cap. 22`, 00:02:35–00:07:00). El capítulo 24 muestra un crackme propio con cuatro chequeos y un mensaje que dice cuál lo detectó (`Reversing desde cero, cap. 24`, 00:01:17–00:03:31).

Para esta guía, el objetivo es **entender** esos mecanismos y reconocerlos en un binario, no enseñar a evadirlos. Si un programa se cierra solo bajo el debugger, la pregunta correcta es "¿qué chequeo está fallando?", no "¿qué truco le aplico?".

Práctica

1. Compilá tu programa `check.exe` (Etapa 1) y abrilo en x64dbg.
2. Recorré con F8 hasta `main`. Anotá `EIP`, `ESP` y `EBP`.
3. Poné un breakpoint por software en la comparación `(a+b)*4 == 480`, ejecutá con `40` y `80`, y verificá los valores en el stack.
4. Poné un breakpoint de hardware *on access* sobre la variable `b` y observá cuándo se lee.
5. Poné un breakpoint de memoria sobre el buffer de `scanf` y observá cuándo se escribe.
6. Agregá un breakpoint con log `{EAX}` en la comparación y ejecutá varias veces con valores distintos. Guardá el log.
7. Etiquetá el bloque "Correcto" y el bloque "Incorrecto". Sacá una captura para tu bitácora.

Preguntas de repaso

1. ¿Qué diferencia hay entre F7 y F8? F7 entra a las funciones; F8 las ejecuta sin detenerse dentro. Las dos ejecutan la instrucción actual.

2. ¿Qué byte usa un breakpoint por software? `0xCC`, la instrucción `INT3`. El debugger lo restaura al detenerse.

3. ¿Cuántos breakpoints de hardware hay y de quién son? Cuatro (`DR0` – `DR3`) por hilo. Se aplican al contexto del hilo; si no aparecen, revisá qué hilo está seleccionado.

4. ¿Para qué sirve un breakpoint con `log` y `breakif 0`? Para registrar valores sin detener la ejecución. Es útil en bucles donde el programa pasa muchas veces.

5. ¿Un breakpoint de memoria vigila bytes o páginas? Vigila un rango de bytes, pero internamente trabaja con protecciones de página. Por eso puede detenerse ante un acceso cercano al rango.

6. ¿Qué es el entry point? La primera instrucción del código del programa, donde el cargador le pasa el control. No es necesariamente `main`.

Antes de avanzar

Tenés que poder:

- Abrir un programa, moverte y reconocer las pestañas principales.
- Usar F7, F8 y F9 con intención.
- Poner los tres tipos de breakpoint y explicar en qué se diferencian.
- Configurar un breakpoint condicional con log.
- Guardar un parche en un programa propio y explicar por qué no lo harías en software ajeno.

Puerta de salida

Si podés responder "¿quién escribe este buffer?" con un breakpoint de memoria y "¿cuántas veces pasa por acá y con qué valor?" con uno condicional, ya usás el debugger como herramienta, no como curiosidad.

Fuentes de esta lección

- `Reversing desde cero, cap. 01` — interfaz, ventanas, F7/F8/F9, entry point.
- `Reversing desde cero, cap. 07` — breakpoints por software, hardware, memoria, condicionales y log.
- `Reversing desde cero, cap. 11` — etiquetas, análisis visual y parcheo didáctico.
- `Reversing desde cero, cap. 22` — PEB y detección de debugger.
- `Reversing desde cero, cap. 24` — cuatro chequeos anti-debug en un crackme propio.
- `Reversing desde cero, cap. 28` — WinProc y Dialog procedure, `WM_COMMAND`/Control ID, breakpoint condicional sobre el mensaje y el método `SendMessage` + breakpoint de ejecución en la sección de código.
- `Reversing desde cero, cap. 21` — lectura de `DR0`–`DR3` desde el `ContextRecord` como detección de breakpoints de hardware.
- Curso IDA · tutes 9, 10 y 12 — IDA como debugger (soporta varios) y práctica dinámica sobre crackmes propios (`HOLA_REVERSER`, `TEST_REVERSER`).

Capas de esta lección

Contenido observado en CLS: el recorrido de la interfaz del capítulo 01, los tres breakpoints del capítulo 07, el log `{EAX}` y las etiquetas del capítulo 11, y los chequeos de los capítulos 22 y 24.
Conocimiento externo: el mecanismo `INT3`, los registros `DR` y las protecciones de página.
Recomendación editorial: el límite sobre parcheo y la corrección sobre "cuatro por proceso".

Ampliación editorial basada en fuentes externas — Flujo operativo de trinchera en x64dbg y extracción masiva de cadenas cifradas

El uso ágil del depurador dinámico y la desofuscación rápida de cadenas constituyen dos habilidades prácticas decisivas (x64dbg Documentation):

1. Hoja de Ruta Táctica de Operaciones en x64dbg

- **Atajos de Navegación Esenciales:** `F2` (Poner/quitar breakpoint de software), `F7` (Paso a paso dentro de la función), `F8` (Paso a paso sobre la función), `Ctrl+F9` (Ejecutar hasta el `ret` de la función actual), `F9` (Reanudar ejecución continua) y `Ctrl+G` (Saltar a una dirección o expresión).
- **Follow in Dump:** Clic derecho sobre cualquier registro en el panel de CPU o sobre una dirección en el Stack -> *Follow in Dump*: proyecta inmediatamente el contenido de la memoria apuntada en la vista hexadecimal.
- **Búsqueda de Referencias Intermodulares:** Clic derecho en el desensamblado -> *Search for* -> *Current Module* -> *Intermodular calls*: lista todas las llamadas a APIs importadas por el binario. Permite seleccionar múltiples funciones (ej. `RegOpenKeyEx`, `CreateFileW`, `InternetOpenA`) y presionar `F2` para establecer breakpoints simultáneos en todas ellas.
- **Memory Map (`Alt+M`):** Visualiza todas las páginas de memoria del proceso. Presionar `Ctrl+B` permite buscar cadenas o secuencias de bytes en toda la memoria dinámica. Facilita la detección de bloques con protección `PAGE_EXECUTE_READWRITE` (`ERW`), indicio clásico de payloads inyectados o shellcode.
- **Reconstrucción con Scylla:** Menú *Plugins* -> *Scylla* integrado en x64dbg: automatiza la detección del OEP (*IAT Autosearch*), lectura de importaciones (*Get Imports*), volcado a disco (*Dump*) y corrección de la tabla de importaciones (*Fix Dump*).

2. Los 3 Métodos Estándar para Extraer Cadenas Cifradas en Masa

1. **Método 1 — Breakpoint Condicional con Log en el Retorno del Decodificador (x64dbg):** Cuando un binario tiene decenas de llamadas a una rutina decodificadora XOR o RC4, no se debe revertir una por una a mano. Se localiza la instrucción `ret` de la función decodificadora y se configura un breakpoint:

   ```
   Log text:        "Cadena descifrada: {utf8@rax}"   ; (o {utf16@rax} según corresponda)
   Break condition: 0                                 ; Condición 0: no pausa la ejecución
   ```

   Al presionar `F9`, el programa se ejecuta a velocidad normal y la pestaña *Log* de x64dbg vuelca automáticamente todas las cadenas descifradas secuencialmente.
2. **Método 2 — Scripting Estático en IDAPython / Ghidra:** Si la clave está embebida y el algoritmo es determinista, se escribe un script que localice las referencias cruzadas (XREFs) al buffer cifrado y aplique la operación directamente en la base de datos de IDA:

```
import ida_bytes
def decode_xor(ea, length, key):
    raw = ida_bytes.get_bytes(ea, length)
    return bytes([b ^ key for b in raw]).decode('latin1', errors='ignore')
```

3. **Método 3 — Micro-emulación con Unicorn Engine:** Permite aislar únicamente los 30 o 40 bytes de la rutina decodificadora, inicializar los registros de la CPU virtual con los punteros y longitudes deseadas, y ejecutar el bloque emulado sin interactuar con dependencias del sistema operativo ni requerir un entorno Windows activo.

Ampliación editorial basada en fuentes externas — Patrón de doble breakpoint encadenado para funciones in-place y plugins tácticos

En el flujo de extracción masiva de cadenas cifradas, existe un caso frecuente donde el truco estándar de leer `{utf8@rax}` en el `ret` no funciona:

1. Funciones Decodificadoras void con Buffer In-Place

Cuando la función es `void decodificar(char* buffer, size_t len)`, el buffer se descifra sobre sí mismo (*in-place*) y la función no devuelve ningún puntero en `RAX`. En x64, el puntero al buffer llega en `RCX`, pero `RCX` es un registro volátil (*scratch*) que el compilador suele reutilizar como contador de bucle o índice, perdiéndose su valor antes de llegar al `ret`.

Solución: Técnica de Doble Breakpoint Encadenado con Variable de Usuario ($ptr):

1. **En el prólogo de la función decodificadora (dirección inicial):**

```
Break condition: 0
Command text:    $ptr = rcx    ; Guarda el puntero original antes de que el bucle lo pise
```

2. **En la instrucción `ret` de la función:**

```
Break condition: 0
Log text:        "Cadena descifrada: {utf8@$ptr}"    ; Lee el buffer ya descifrado en
memoria
```

Al reanudar con `F9`, la consola de Log vuelca todas las cadenas limpias sin detener nunca la ejecución.

2. Ecosistema de Plugins Tácticos para x64dbg

- **ScyllaHide:** Plugin indispensable que intercepta a nivel de Ring 3 las APIs de anti-debug más comunes (`PEB.BeingDebugged`, `NtGlobalFlag`, `CheckRemoteDebuggerPresent`, `ProcessDebugPort` y temporizadores `RDTSC` / `GetTickCount`) con un solo clic.
- **xAnalyzer:** Analizador heurístico estático para x64dbg que reconoce automáticamente los prototipos de funciones de Windows y anota los nombres de los argumentos en el desensamblado, emulando la clásica vista contextual de OllyDbg en binarios modernos de 64 bits.
- **OllyDumpEx / SwissArmyKnife:** Permite realizar volcados selectivos de regiones de memoria específicas o secciones individuales directamente desde el *Memory Map* sin necesidad de reconstruir el binario completo.

Nota de alcance arquitectónico — Expresión getdword(sp+8) en WindowProc (x86 vs. x64)

La expresión `getdword(sp+8) == 0x111 && getdword(sp+0xC) == 0x13A` de esta lección aplica a binarios de **32 bits (x86)** (donde `__stdcall` apila `hWnd` en `[esp+4]`, `uMsg` en `[esp+8]` y `wParam` en `[esp+0xC]`) y asume que la palabra alta de `wParam` (código de notificación, p. ej. `BN_CLICKED = 0`) vale cero.

Ampliación editorial basada en fuentes externas — Condiciones para WndProc en x64 y función arg.get() en x64dbg

Según la especificación oficial de WindowProc y WM_COMMAND en Microsoft Learn *(Hecho confirmado)* y la documentación de Expressions & Functions de x64dbg *(Hecho confirmado)*:

- **En binarios de 64 bits (x64):** Los cuatro argumentos llegan en `RCX` (`hWnd`), `RDX` (`uMsg`), `R8` (`wParam`) y `R9` (`lParam`), y el ID del control ocupa solo los 16 bits bajos (`LOWORD(wParam)`):

```
breakif: rdx == 0x111 && (r8 & 0xFFFF) == 0x13A
log:     "WM_COMMAND: notif={r8>>16} id={r8&0xFFFF} hwnd={rcx}"
```

- **Expresión universal (x86 y x64) en el prólogo con `arg.get(n)`:**

```
breakif: arg.get(1) == 0x111 && (arg.get(2) & 0xFFFF) == 0x13A
log:     "uMsg={arg.get(1)} control_id={arg.get(2)&0xFFFF}"
```

IDA y el análisis estático

> **Objetivo de la lección**
>
> Al terminar vas a poder abrir un ejecutable en IDA (o Ghidra), encontrar una cadena, llegar al código que la usa, leer el grafo de una función, renombrar variables y comentar el flujo, y sincronizar direcciones con el debugger.

Prerrequisitos: PE, instrucciones, flujo y llamadas. **Duración sugerida:** 5 horas. Estimación aproximada.

Para qué sirve un desensamblador

Un debugger te muestra el programa **ejecutándose**. Un desensamblador te muestra el programa **completo, sin ejecutarlo**. Esa diferencia importa por tres razones:

1. Hay código que no podés ejecutar: malware, un binario de otra arquitectura, una función a la que no sabés cómo llegar.
2. Hay funciones que se llaman una vez cada mil ejecuciones y no querés esperar a que pasen.
3. El análisis estático te deja **leer todo** el código, no solo el camino que tomaste.

El capítulo 14 lo dice con un ejemplo claro: si estás analizando un virus, no lo corrés para ver qué hace; primero lo mirás estático. Y si estás buscando un bug en una función que no sabés cómo se alcanza, la podés analizar igual (`Reversing desde cero, cap. 14`, 00:03:57–00:05:30).

Abrir el archivo

1. **File** → **Open** y elegí el `.exe`.
2. IDA detecta el formato: `Portable executable for 80386 (PE)` indica un PE de 32 bits; si fuera de 64, lo diría.
3. En el diálogo de carga podés aceptar los valores por defecto. *Manual load* sirve si querés cargar secciones que IDA no trae por defecto (por ejemplo, el header).
4. Si el binario tiene símbolos PDB, IDA te los pide. Si no, cancelá: IDA hará su mejor esfuerzo.
5. Esperá a que termine el autoanálisis (el indicador deja de girar).

El capítulo 14 muestra este recorrido y aclara que la versión gratuita (IDA Free) tiene limitaciones: menos procesadores, un solo decompilador y, según la versión, solo IDC en lugar de Python (00:00:40–00:02:30).

El tutorial escrito de `descargas_sueltas` agrega un dato práctico: para saber si un PE es de 32 o 64 bits antes de abrirlo, mirá los bytes que siguen a la firma `PE`. En el documento, un binario de 64 bits muestra `PE..d†` y uno de 32 bits `PE..L`; la diferencia está en la cabecera opcional (PE32+ vs PE32) (`Introducción al reversing con IDA Pro desde cero, 2016`, sección "Lo primero es lo primero").

> **Sobre las versiones**
>
> El curso usa IDA de 32 bits de una versión de escritorio. La interfaz cambió bastante en las versiones 8 y 9, y las capacidades de IDA Free también. Los conceptos —strings, xrefs, grafo, renombrado, rebase — se mantienen; los botones y atajos pueden variar. Si un atajo no funciona, buscalo en el menú.

Las ventanas que vas a usar

Ventana	Cómo abrirla	Para qué
IDA View	`View → Open subviews → Disassembly`	Código desensamblado
Hex View	`View → Open subviews → Hex`	Bytes del archivo
Strings	`Shift+F12`	Cadenas del binario
Imports	`View → Open subviews → Imports`	Funciones que usa
Exports	`View → Open subviews → Exports`	Funciones que ofrece
Functions	`View → Open subviews → Functions`	Lista de funciones detectadas
Names	`View → Open subviews → Names`	Símbolos y etiquetas
Structures	`View → Open subviews → Structures`	Tipos y structs
Graph	`Espacio` sobre una función	Grafo de bloques

Ejemplo paso a paso: de una cadena al código

Este es el flujo que vas a repetir siempre. Lo hacemos con el programa de práctica del capítulo 15, que muestra el mensaje "el primer número debe ser menor a ese valor".

Paso 1. Buscar la cadena. `Shift+F12` abre la ventana de strings. Con `Ctrl+F` filtrás por texto. También podés usar `View → Open subviews → Strings`.

Paso 2. Ir a la cadena. Doble clic sobre ella: IDA te lleva a la sección `.rdata`, donde está almacenada.

Paso 3. Ver quién la usa. Con el cursor sobre la cadena, apretá `X` (*cross references*). IDA te muestra las instrucciones que la referencian. Elegí una y te lleva al código.

Paso 4. Leer el grafo. Apretá `Espacio` para ver los bloques. Cada bloque es una secuencia de instrucciones sin saltos internos; las flechas verdes/rojas son los caminos. En el capítulo 15, Ricardo pinta los bloques: verde para el camino bueno, salmón para el malo, celeste para el recorrido (`Reversing desde cero, cap. 15`, 00:02:16–00:06:00).

Paso 5. Renombrar y comentar. `N` renombra una función, variable o etiqueta; `;` agrega un comentario; `Y` cambia el tipo. Renombrar `var_18` como `primer_valor` hace que todas las referencias se actualicen y el código se vuelva legible.

Paso 6. Decompilar. `F5` abre el decompilador (si la versión lo incluye). Te da una vista en C aproximada. No es el código original: es una reconstrucción. Sirve para entender la forma general, no para copiar y pegar.

Sincronizar IDA con el debugger

Hay dos problemas clásicos:

1. **La base cambia.** IDA muestra la imagen en su `ImageBase` preferida (por ejemplo `0x400000`), pero el debugger la carga en otra dirección por ASLR. En el capítulo 14, IDA muestra `0x401040` y el debugger `0x410040`; la diferencia es la base.
2. **Las direcciones absolutas no coinciden.** Aunque rebasees, un byte puede estar en una sección con otra alineación.

Soluciones:

- **Rebasar IDA** a la base que ves en el debugger: `Edit → Segments → Rebase program`. En el capítulo 14, Ricardo rebasea de `0x400000` a `0x40000` para que coincida con el debugger (00:05:30–00:08:00).
- **Comparar por file offset.** En el debugger, `Ctrl+Shift+G` te da el offset de una instrucción; en IDA, `Jump → Jump to file offset` te lleva al mismo lugar. En el ejemplo, el offset `0x440` coincide en ambos (00:08:00–00:09:00).
- **Desactivar ASLR** en una copia del binario si necesitás direcciones estables (es lo que hace el capítulo 26b para trabajar cómodo). Es una modificación del archivo: hacela sobre una copia, nunca sobre el original.

Buscar un valor inmediato y las constantes con nombre

A veces no tenés una cadena que te lleve al código, pero sí un número: el identificador de un botón, un código de error, un mensaje de Windows. IDA permite buscarlo.

- **Search → immediate value** (`search for immediate value`) encuentra todas las instrucciones donde aparece ese número. Si el botón "Check" tiene Control ID `0x7D0`, buscás `0x7D0` y IDA te lista cada lugar donde se usa; manteniendo `Shift` podés hacer doble clic para recorrerlos.
- **Constante con nombre** (tecla `M`, *standard symbolic constant*): sobre un número que es en realidad una constante de Windows, `M` te ofrece el nombre simbólico. Un `0x111` se vuelve `WM_COMMAND`; un `0x201`, `WM_LBUTTONDOWN`. Eso hace legible el `switch` que reparte los mensajes de una ventana.
- Si la opción `M` no aparece sobre un `case`, apretá `F5` (decompilador) y probá `M` ahí: suele reconocer la constante igual.

El capítulo 24b hace esto sobre un crackme propio: busca el Control ID con *search immediate value*, llega al `WinProc` y traduce los números de mensaje a `WM_COMMAND` y `WM_LBUTTONDOWN` con las constantes simbólicas (`Reversing desde cero, cap. 24b`; transcripción .txt, sin marcas de tiempo).

Proximity View: ver y buscar caminos entre funciones

El **Proximity View** (menú del grafo, tecla `-` para alejarte) muestra las funciones como un árbol de llamadas en vez de instrucciones. Por defecto muestra dos niveles: lo que la función llama y lo que la llama.

- Los nodos con "corona" son llamadas a la **IAT** (APIs importadas); si una llamada no continúa hacia abajo, es una API que se usa y vuelve (por ejemplo `MessageBox`).
- Doble clic en un nodo lo **centra** y expande dos niveles desde ahí; así seguís solo la rama que te interesa (por ejemplo, de `WinMain` a `WinProc`).
- Clic derecho → *Collapse children / parents* deja una función sola, para no llenar la pantalla.

Lo más útil es **buscar un camino**: con `Add node by name` agregás la función (o variable) destino, y con clic derecho → **Find path** IDA te dibuja la ruta estática que conecta dos funciones, si existe. Sirve cuando el programa es grande y querés saber "¿desde el inicio, cómo se llega hasta la función que muestra *Correcto*?". No siempre hay camino estático (a veces el flujo se arma en tiempo de ejecución), pero cuando lo hay, ahorra horas.

Esto lo muestra el capítulo 24b, que usa *Add node by name* + *Find path* para conectar el inicio del programa con una cadena de la sección de datos, y recomienda activar *use name for addresses* para que el grafo se lea (`Reversing desde cero, cap. 24b`).

Cuando no hay símbolos

Si el binario no trae símbolos, IDA muestra `sub_401000` en vez de `printf`. Peor aún: a veces las DLL están **embebidas** en el `.exe` (compilación estática), así que ni siquiera ves una llamada a una API conocida. Cómo reconocer una función igual:

1. **Por sus argumentos.** Una función con un solo argumento que siempre es una cadena, o con una cadena y después un número, tiene pinta de `printf`. Mirá todas sus referencias: si el patrón de argumentos se repite, confirma la hipótesis.
2. **Siguiendo a una API conocida.** Aunque la DLL esté embebida, `printf` termina llamando a `WriteConsole`. Con el Proximity View podés dejar la función sospechosa sola, agregar `WriteConsoleW` y buscar el camino: si llega, era una envoltura de `printf`.
3. **Confirmando en dinámico.** Si sospechás que una función sin argumentos es `getchar`, ejecutá: si se queda esperando una tecla, era `getchar`.
4. **Con Lumina** (solo IDA con licencia): el servidor compara las funciones con una base y les pone nombre. La versión gratuita o sin licencia no accede al servidor, así que no cuentes con eso.

El capítulo 34 ("Reverseando sin símbolos") usa estas tres primeras técnicas para identificar `printf`, `getchar` y `strcat` en un binario sin símbolos (`Reversing desde cero, cap. 34`). El mismo capítulo resuelve además un keygen; esta guía toma solo las técnicas de **identificación de funciones**, no la generación de seriales.

Reconstruir estructuras y vtables

Cuando una función usa `[reg+offset]` muchas veces sobre la misma base, lo más probable es que esté recorriendo una **estructura**. IDA deja trabajarlas aunque no haya símbolos:

- **Aplicar una estructura a un operando:** parado sobre el offset, apretá `T` (*structure offset*) y elegí la estructura; IDA te dice a qué campo corresponde (por ejemplo, `+0x58` → `F1`). A partir de ahí el listado se lee con nombres en vez de números.

- **Crear una estructura vacía y llenarla:** en la ventana *Structures*, `Insert` crea una estructura; con `D` agregás un campo; con clic derecho → *Array* (sin la tilde *create as array*) la agrandás hasta el tamaño que viste que reserva el programa (por ejemplo, si hace `new` de `0x3C`, creás una de `0x3C`). Después vas nombrando cada campo a medida que descubrís qué guarda (el retorno de `fopen` → `FILE*`, etc.).
- **Padding:** recordá que el tamaño puede incluir relleno por alineación (ver la lección de memoria); no asumas que los campos son contiguos sin huecos.

Para un **objeto de C++**, el primer campo (offset 0) suele ser el puntero a la **vtable**. El cap. 33 arma dos estructuras —una para el objeto y otra para la vtable con los punteros a los métodos virtuales (`suma`, `operator+`, `entrada`, `chequeo`)— y las aplica con `T`. Así, `mov edx, [obj]; call [edx+8]` se lee como "llamar al tercer método virtual".

> **Las llamadas virtuales no dejan xref directa**
>
> Un método virtual se llama a través de la vtable, así que `X` sobre la función solo muestra la referencia **desde la vtable**, no desde el `call`. El cap. 33 resuelve esto agregando una **referencia cruzada manual** (`View → Open subviews → Cross references → Add cross reference`) desde el sitio del `call` hacia el método. Conviene expresarla como **offset** del módulo (`modulo+0x92`) y no como dirección fija, para que ASLR no la rompa. Hecho esto, `X` y el Proximity View ya muestran de dónde viene la llamada.

Dos atajos de navegación que ayudan con listados densos (cap. 20b): con un registro o campo resaltado, `Alt+↓` / `Alt+↑` salta entre sus apariciones; y en una lista de referencias o de strings, `Shift` **+doble clic** te lleva a cada una sin cerrar la ventana.

Con símbolos es más directo (cap. 23). Si el binario trae PDB, la estructura ya está definida en *Local Types* (después de sincronizarla con el IDB) y el **constructor** se reconoce por el patrón `Nombre::Nombre`. Aplicás la struct con `T` y cada `[reg+offset]` muestra su campo por nombre. Ojo con un caso frecuente: cuando un campo se pasa como **buffer** a una función (por ejemplo, el destino de `fread`), vas a ver `lea`/dirección `&campo`, no su contenido —es un puntero al campo, no su valor—. Reconocer las APIs por sus argumentos (`fopen` devuelve `FILE*`, `fread(buf,size,count,stream)`, `LoadLibrary` → `HMODULE`, `GetProcAddress(hmod,"printf")`) termina de dar sentido a la estructura (`Reversing desde cero, cap. 23`; transcripción .txt, sin marcas).

> **x64dbg también entiende estructuras**
>
> No es exclusivo de IDA. En x64dbg, la pestaña *Struct* permite **importar** una estructura con *parse header* (cargás un `.h` con su definición, bien formateado) y luego *Visit type* con el nombre del tipo y la dirección (por ejemplo, la que apunta a una `SYSTEMTIME` que llenó `GetSystemTime`). Es más áspero que en IDA, pero sirve para leer los campos en vivo mientras traceás (cap. 19).

> **En el Curso IDA: una serie entera dedicada a estructuras (25–28)**
>
> El Curso IDA le dedica cuatro partes seguidas al tema. Lo más útil para practicar: el ejercicio `IDA_STRUCT.7z` (ejecutable `ConsoleApplication4.exe`) **viene con su** `.pdb`, pero el autor propone resolverlo **primero sin símbolos** —borrando o renombrando el `.pdb`— porque "es más complejo, pero es más real", y recién después comparar con los símbolos puestos. Es la mejor forma de ejercitar lo de arriba: armar la estructura a mano con `T` / `D` y nombrar campos, contra tenerla servida en *Local Types* (Curso IDA · tutes 25–28, Estructuras).

Patrones de datos más allá de una estructura

Una estructura sola es el caso fácil. En binarios reales vas a ver **colecciones** de estructuras, y cada una deja un patrón propio en el assembly (**conocimiento externo**, salvo lo citado):

- **Arreglo de estructuras.** El acceso a un elemento tiene la forma `base + índice × tamaño + campo`. Lo vas a ver como `imul rax, rcx, 18h` seguido de `[rbx+rax+8]`, o como un índice escalado `[rbx+rcx*8+10h]`. El multiplicador te da el **tamaño del elemento** y el desplazamiento final, el **campo**. Con eso creás la estructura del tamaño correcto y la aplicás como arreglo.
- **Lista enlazada simple.** Un bucle que hace `mov rbx, [rbx+8]` hasta que `rbx` vale cero: el `+8` es el campo *siguiente*. El resto de los accesos sobre `rbx` dentro del bucle son los campos del nodo.
- **Lista doblemente enlazada de Windows (`LIST_ENTRY`).** Windows usa una estructura con dos punteros, `Flink` (siguiente) y `Blink` (anterior), **embebida** dentro de cada elemento, y listas **circulares** con una cabecera. Dos consecuencias para leerla: el puntero apunta al campo `LIST_ENTRY`, no al comienzo del elemento, así que verás un `sub` / `lea` que **resta** el offset del campo para llegar al inicio (lo que en C hace la macro `CONTAINING_RECORD`); y el recorrido termina cuando se vuelve a la **cabecera**, no cuando aparece un `NULL`. Es la misma forma que tiene la lista de módulos del PEB de 2.14.
- **Árboles.** Nodos con dos punteros a hijos y una comparación que decide por cuál bajar; los contenedores ordenados de C++ (`std::map`) se ven así (2.18).

*(Ejemplo **ilustrativo**: recorrido de una lista circular de Windows en x64.)*

```
lea   rdi, [rcx+10h]        ; rdi = &cabecera (LIST_ENTRY dentro de un objeto)
mov   rbx, [rdi]            ; rbx = cabecera.Flink
bucle:
cmp   rbx, rdi              ; ¿volvimos a la cabecera? -> fin (lista circular)
je    fin
lea   rsi, [rbx-20h]        ; inicio del elemento = &entry - offset del campo (CONTAINING_RECORD)
mov   eax, [rsi+8]          ; un campo del elemento
mov   rbx, [rbx]            ; siguiente: entry.Flink
jmp   bucle
fin:
```

Lectura: el campo `LIST_ENTRY` está a `+0x20` dentro de cada elemento (por el `lea rsi, [rbx-20h]`), y el elemento tiene al menos un campo de 4 bytes en `+8`. Con esos dos datos creás la estructura del elemento y la de la cabecera.

Cadenas: codificaciones y por qué no siempre aparecen

La ventana de strings es la puerta de entrada más rápida, pero solo muestra lo que el programa guarda **tal cual** en el archivo. Conviene saber qué se pierde:

- **ANSI frente a UTF-16.** Las funciones de Windows que manejan texto suelen tener una versión **A** (página de códigos, un byte por carácter) y una **W** (Unicode, en Windows UTF-16, dos bytes). En un volcado hexadecimal, un texto ASCII guardado como UTF-16LE se ve como `H.o.l.a.` con ceros intercalados. Si la búsqueda de cadenas está configurada solo para C/ANSI, esas cadenas **no aparecen**: en IDA se agregan los tipos Unicode desde la configuración de la ventana de strings; Ghidra los incluye en *Defined Strings*. Que el programa llame a `CreateFileW` en vez de `CreateFileA` ya te dice qué codificación buscar.
- **Cadenas con longitud.** Algunos lenguajes guardan cadenas como puntero + longitud, sin el `0` final (Go, Rust y otros; ver 2.13). El buscador de cadenas las puede mostrar pegadas unas con otras o cortadas.
- **Cadenas armadas en el stack.** El programa escribe el texto de a pedazos con instrucciones `mov` de inmediatos imprimibles (`mov dword ptr [rsp+20h], 6C6C6548h` = `"Hell"` en little-endian). No hay ninguna cadena en `.rdata`; la pista es una ráfaga de `mov` con constantes que, leídas como bytes, son caracteres.
- **Cadenas cifradas.** Se guardan transformadas y una rutina las descifra justo antes de usarlas (un bucle con `xor` es lo más simple; ver 2.10). Se recuperan siguiendo la rutina, ejecutando hasta después de que descifre o con herramientas que extraen cadenas armadas en el stack y decodificadas (por ejemplo, FLOSS de Mandiant, externa al curso).

Regla práctica: si un programa claramente muestra textos al usuario y la ventana de strings está casi vacía, alguna de estas cuatro situaciones está pasando, y eso ya es un dato del análisis.

Estrategias de lectura

No leas una función de arriba a abajo sin plan. Algunas tácticas que funcionan:

1. **Empezá por las strings.** En programas con interfaz, los mensajes ("acceso denegado", "ingrese la clave") te llevan directo a la lógica.
2. **Buscá las APIs conocidas.** `GetWindowTextA`, `strcmp`, `MessageBoxA`, `CreateFileA` son pistas. En Imports tenés la lista.
3. **Seguí las referencias.** `X` sobre una función te dice desde dónde se llama. A veces el `main` es el nodo con más referencias.
4. **Nombrá mientras leés.** Un binario sin símbolos se vuelve legible cuando renombrás `sub_401000` como `check_password` y `var_8` como `largo`.
5. **Anotá las hipótesis.** Escribí "si el primer valor es <= 50, sigue por acá" y verificalo en el debugger.

El capítulo 15 usa exactamente este método: busca la cadena, llega al código, identifica el `printf`/`scanf`, renombra las variables, deduce las condiciones y las confirma ejecutando (00:06:00–00:19:00).

En el Curso IDA: el mismo método, dicho por Ricardo

El Curso IDA plantea este enfoque desde el arranque: cuando todavía no tenés práctica con el análisis estático, **comentá y renombrá lo poco que vas entendiendo** y resolvé el resto en el debugger; con práctica vas necesitando el debugger cada vez menos, hasta reversear casi sin ejecutar. Y algo clave para no agobiarte: **no se reversea el programa entero**, sino la o las funciones de la zona que necesitás para tu objetivo (Curso IDA · tute 3, El loader y tute 8, Trabajar en el loader).

IDA Free, Ghidra y alternativas

Herramienta	Fortalezas	Límites
IDA Free	La misma base de IDA; buena para empezar	Menos procesadores, decompilador limitado, sin plugins de pago
IDA Pro	Estándar de la industria; decompilador maduro	Licencia cara
Ghidra	Gratis y open source; decompilador integrado; scripting en Java/Python	Interfaz más pesada; consume más recursos
Binary Ninja	Interfaz moderna; buena API	De pago
radare2 / Cutter	Gratis, scriptable, multiplataforma	Curva de aprendizaje alta
x64dbg	Debugger, no desensamblador estático	No reemplaza a IDA para análisis estático

Para este curso, **IDA Free o Ghidra** son suficientes. El capítulo 14 recomienda IDA Free si no se tiene Pro; esta guía agrega Ghidra como alternativa gratuita y activa.

Una recomendación del audio y del documento que no se copia

En el capítulo 14, el autor sugiere buscar una copia "por internet" de IDA Pro si no se puede pagar (00:01:00–00:02:00). El documento de `descargas_sueltas` va más lejos: indica buscar "IDA PRO 6.8 + HEXRAYS" en Google y usar la contraseña de `install_pass.txt`. Esta guía **no** reproduce esas instrucciones ni distribuye claves. Usá IDA Free, Ghidra o una licencia educativa.

Práctica

1. Abrí tu `check.exe` en IDA Free o Ghidra.
2. Buscá la cadena "Correcto" con `Shift+F12`, llegá al código con `X` y leé el grafo.
3. Renombrá la función `main` si no está nombrada, y las variables de los dos números.
4. Agregá comentarios en las dos comparaciones.
5. Si tu versión tiene decompilador, compará la vista `F5` con el assembly. Marcá tres diferencias.
6. Rebaseá IDA a la base que muestra x64dbg y comprobá que una dirección coincide. Si no coincide, usá el file offset.

7. Escribí en tu bitácora: qué string usaste, qué xref elegiste, qué función renombraste y qué hipótesis confirmaste.
8. Agregá a tu programa un mensaje con `MessageBoxW` y otro armado carácter por carácter en un arreglo local. Comprobá cuál aparece en la ventana de strings con la configuración por defecto, cuál solo al activar Unicode y cuál no aparece nunca.
9. Agregá un arreglo de estructuras y recorrelo en un bucle. En el listado, identificá el tamaño del elemento por el multiplicador y creá la estructura correspondiente.

Preguntas de repaso

1. ¿Cuál es la diferencia principal entre IDA y x64dbg? IDA es un desensamblador estático: analiza el archivo sin ejecutarlo. x64dbg es un debugger: ejecuta el programa y muestra su estado.

2. ¿Para qué sirve `X` sobre una cadena? Para ver las referencias cruzadas: las instrucciones que usan esa cadena. Es el camino más rápido de la cadena al código.

3. ¿Qué hace `N`? Renombra un símbolo (función, variable, etiqueta). Todas las referencias se actualizan.

4. ¿Por qué las direcciones de IDA y del debugger no coinciden? Porque ASLR carga la imagen en una base distinta de la preferida. Se soluciona rebaseando o comparando por file offset.

5. ¿El decompilador devuelve el código original? No. Reconstruye una aproximación en C a partir del assembly. Sirve para entender, no para copiar.

6. ¿Qué mirada conviene primero, la estática o la dinámica? Depende. Para un programa con interfaz y strings claras, la estática orienta rápido. Para confirmar un valor concreto, la dinámica es más directa. Lo habitual es alternar.

7. El programa muestra mensajes pero la ventana de strings está casi vacía. ¿Qué cuatro causas revisás? Cadenas UTF-16 no incluidas en la búsqueda, cadenas con longitud sin `0` final, cadenas armadas en el stack con `mov` de inmediatos y cadenas cifradas que se descifran en ejecución.

8. Ves `[rbx+rcx*8+10h]` dentro de un bucle. ¿Qué te dice? Que probablemente es un arreglo de elementos de 8 bytes indexado por `rcx`, y que `+10h` es el desplazamiento del arreglo o de un campo respecto de la base.

9. ¿Por qué en una lista `LIST_ENTRY` de Windows ves una resta antes de leer los campos? Porque el puntero apunta al campo `LIST_ENTRY` embebido, no al inicio del elemento; la resta del offset del campo recupera el inicio (lo que hace `CONTAINING_RECORD`).

Antes de avanzar

Tenés que poder:

- Abrir un binario, buscar una cadena y llegar al código que la usa.
- Leer un grafo y distinguir el camino bueno del malo.
- Renombrar y comentar para que el listado se entienda.
- Sincronizar IDA con el debugger por rebase o por offset.

- Explicar por qué no usás una copia ilegal de IDA.
- Reconocer cadenas UTF-16, armadas en el stack o cifradas, y saber por qué no aparecen en la ventana de strings.
- Reconocer un arreglo de estructuras, una lista enlazada y una lista `LIST_ENTRY` por su patrón en el assembly.

> **Puerta de salida de la Etapa 2**
>
> Si podés analizar tu propio programa de punta a punta —estático y dinámico— y escribir un informe con las direcciones y los valores observados, terminaste la introducción al reversing. La Etapa 3 es una victoria rápida con .NET.

Fuentes de esta lección

- `Reversing desde cero, cap. 14` — interfaz de IDA, rebase, file offset, símbolos, limitaciones de IDA Free.
- `Reversing desde cero, cap. 15` — strings, xrefs, grafo, renombrado, decompilación y verificación dinámica.
- `Reversing desde cero, cap. 24b` — Proximity View, Find Path, search immediate value, constantes simbólicas (`WM_*`) y decompilador para confirmar un `case`.
- `Reversing desde cero, cap. 34` — identificar funciones sin símbolos por argumentos, por el camino a una API conocida y confirmando en dinámico (se omite la parte de keygen).
- `Reversing desde cero, cap. 20b` — aplicar y reconstruir estructuras con `T`, y navegación con `Alt+↑/↓` y `Shift`+doble clic.
- `Reversing desde cero, cap. 33` — objeto con vtable, reconstruir las dos estructuras y agregar cross-references manuales para las llamadas virtuales (se omite la parte de keygen).
- `Reversing desde cero, cap. 23` — estructura con símbolos, constructor `Nombre::Nombre`, `&campo` como buffer y reconocer `fopen` / `fread` / `LoadLibrary` / `GetProcAddress` (se omite la parte de keygen).
- `Reversing desde cero, cap. 19` — importar y visitar estructuras (`SYSTEMTIME`) en x64dbg con parse header / visit type.
- `Videos y material adicional, Video 33` — mapa de memoria y secciones como primer paso del análisis.
- `Videos y material adicional, Video 39` — ExeInfoPE y detección de código nativo/empaquetado.
- Curso IDA desde cero · 69 tutes ilustrados — texto y 3.278 capturas extraídos e indexados por tema (índice en ida-tutes); el método de lectura estática —comentar, renombrar y reversear solo lo necesario— es de los tutes 3 y 8.
- Curso IDA · tutes 25–28, Estructuras — serie dedicada a estructuras en IDA; el ejercicio `IDA_STRUCT` / `ConsoleApplication4.exe` se resuelve primero sin símbolos (lo más real) y después con el `.pdb`.
- `Introducción al reversing con IDA Pro desde cero (docx, 2016)` — tutorial escrito de la parte 1: IDA vs OllyDbg, PE32/PE32+, proximity view, strings, imports, hex dump.
- Microsoft Learn (conocimiento externo): Unicode in the Windows API, Working with Strings, LIST_ENTRY, Singly and doubly linked lists. Herramienta externa: FLOSS.

Capas de esta lección

Contenido observado en CLS: el recorrido del capítulo 14, el ejemplo de `0x440`, las técnicas del capítulo 15 y la advertencia sobre IDA Free. **Conocimiento externo:** comparación con Ghidra y otras herramientas, atajos actuales, patrones de colecciones de datos (arreglos, listas, `LIST_ENTRY`, árboles) y codificaciones y formas de ocultar cadenas. **Ejemplo ilustrativo:** el recorrido de una lista circular. **Recomendación editorial:** la estrategia de lectura y el límite legal sobre copias de IDA Pro.

Cómo cambia el código con optimizaciones

> **De dónde sale esta lección**
>
> Este tema **no está en el corpus CLS**: es **conocimiento externo** que agregamos para completar la guía. El curso de CLS usa ejemplos compilados en modo depuración (`-O0`/Debug), donde el assembly sigue de cerca al código fuente. Los binarios reales vienen optimizados (`-O2`/Release) y se ven muy distintos. Esta lección cubre ese salto, que es donde más se traba quien recién sale de los ejercicios.

> **Objetivo de la lección**
>
> Al terminar vas a poder reconocer los patrones de optimización más comunes —inlining, *strength reduction*, división por constante, vectorización, *tail call*— y no confundirlos con "código raro" o con una protección.

Prerrequisitos: instrucciones, flujo, llamadas (Etapa 2). **Duración sugerida:** 3 horas.

El problema

En `-O0`, cada variable vive en el stack, cada operación es una instrucción y las funciones se llaman tal como las escribiste. En `-O2`, el compilador reescribe todo para que corra más rápido: mete funciones una dentro de otra, reemplaza multiplicaciones por sumas, borra código que "no hace falta" y reordena instrucciones. El resultado es correcto pero **no se parece** al fuente. Reconocer estos patrones es lo que separa leer un ejercicio de leer un programa real.

Strength reduction: multiplicar sin `mul`

El compilador cambia operaciones caras por otras baratas. La más común: multiplicar por una constante usando `lea` y shifts.

```
; x * 2
add eax, eax            ; o:  shl eax, 1
; x * 8
shl eax, 3
; x * 5   ->  x + x*4
lea eax, [rax + rax*4]
; x * 9   ->  x + x*8
lea eax, [rax + rax*8]
```

Si ves un `lea` haciendo cuentas (no calculando una dirección), es casi seguro una multiplicación por constante disfrazada. `lea eax, [rax+rax*4]` es `x*5`, no "un acceso a memoria".

División por constante: el número mágico

Dividir es carísimo, así que, en código optimizado, el compilador **casi nunca** usa `div` para dividir por una constante: lo habitual es que multiplique por un "número mágico" y desplace. Depende del compilador, la arquitectura y las opciones (sin optimizar, o con un divisor que no conoce en tiempo de compilación, sí vas a ver `div` / `idiv`).

```
; unsigned  x / 3
mov   eax, 0xAAAAAAAB     ; número mágico
mul   edx                 ; producto de 64 bits
shr   edx, 1              ; EDX = x / 3
```

Cuando veas una multiplicación por una constante aparentemente arbitraria (`0xAAAAAAAB`, `0x55555556`, `0xCCCCCCCD`…) seguida de un `shr`, es una **división por constante**, no un algoritmo secreto. Hay tablas y calculadoras para recuperar el divisor; lo importante es reconocer el patrón y no perder tiempo buscando sentido a la constante.

Inlining: la función que desaparece

Con `-O2`, una función corta que llamás una vez puede **desaparecer**: el compilador copia su cuerpo donde se usaba (*inlining*). Entonces:

- No vas a encontrar un `call` a esa función.
- El código de la función aparece **mezclado** dentro de quien la llamaba.
- Varias funciones del fuente pueden colapsar en un solo bloque grande.

La consecuencia para el análisis: no asumas "una función del fuente = una función en el binario". Buscá el comportamiento, no el nombre.

Vectorización: SSE donde esperabas un bucle

Un bucle que suma o copia un arreglo puede convertirse en instrucciones **packed** (`addps`, `movdqu`, `paddd`) que procesan varios elementos por vuelta. En vez de un bucle de 100 iteraciones de a 1, ves un bucle de 25 de a 4. Si un bucle "no tiene sentido" porque usa `xmm` y avanza de a 16 bytes, está vectorizado (ver la lección de punto flotante y SSE).

Tail call: un `jmp` donde esperabas `call`

Si lo último que hace una función es llamar a otra y devolver su resultado, el compilador reemplaza `call f; ret` por un simple `jmp f` (*tail call*). Ahorra un marco de stack. El efecto al analizar: una función "termina" saltando a otra con `jmp`, sin `ret` propio. No es un salto sospechoso ni ofuscación: es una llamada final optimizada.

Otros patrones frecuentes

Patrón	Cómo se ve	Qué es
`xor eax, eax`	poner 0	modismo para `eax = 0`
`test eax, eax` + `je`	en vez de `cmp eax, 0`	"¿es cero?"
`cmovcc` / `setcc`	decisión sin salto	el `if` se hizo sin bifurcar (ver lección de flujo)
`lea` con cuentas	aritmética	suma/multiplicación, no acceso a memoria
bloques reordenados	el camino "frío" (error) va al final	mejora la predicción de saltos
constantes plegadas	`mov eax, 480` donde el fuente decía `(a+b)*4`	*constant folding* en tiempo de compilación

Cómo convivir con esto

1. **Aprendé con `-O0`, analizá con `-O2`.** Compilá tu ejemplo en los dos modos y poné los listados lado a lado. Ver la misma lógica en ambas versiones entrena el ojo.
2. **Usá el decompilador.** Ghidra o Hex-Rays deshacen muchos de estos patrones: te muestran `x / 3` en vez del número mágico, y la función inlineada como una sola expresión. No es mágico, pero ahorra tiempo.
3. **Pensá en comportamiento, no en forma.** La pregunta no es "¿por qué hay un `lea` acá?", sino "¿qué valor queda en este registro al final?".

Práctica

1. Compilá una función `int f(int x){ return x*5 + x/3; }` con `-O0` y con `-O2`. Encontrá el `lea [rax+rax*4]` (el `*5`) y el número mágico (el `/3`).
2. Escribí dos funciones chicas donde una llame a la otra y compilá con `-O2`. ¿Desapareció la llamada (inlining)?
3. Hacé un bucle que sume un arreglo de 1000 `int` y compilá con `-O3 -mavx2`. ¿Ves registros `xmm` / `ymm` y avance de a varios elementos?
4. Abrí el `-O2` en Ghidra y compará el decompilado con tu fuente. ¿Recuperó la división por 3?

Preguntas de repaso

▼ **Ves `lea eax, [rax+rax*4]`. ¿Qué es?**

Una multiplicación por 5 (`x + x*4`). `lea` se usa acá para aritmética, no para calcular una dirección.

▼ **Aparece `mov eax, 0xAAAAAAAB` seguido de `mul` y `shr`. ¿Qué está pasando?**

Es una división por constante (por 3) implementada con un número mágico. El compilador evita `div`.

▼ **Una función termina con `jmp otra_funcion` en vez de `call` + `ret`. ¿Es sospechoso?**

> No. Es una *tail call*: la última acción era llamar y devolver, y el compilador la optimizó a un salto.

▼ **No encontrás el `call` a una función que sí está en el fuente. ¿Qué pudo pasar?**

> Inlining: el compilador copió su cuerpo dentro de quien la llamaba, así que no hay llamada separada.

Antes de avanzar

Tenés que poder:

- Reconocer *strength reduction* (`lea`/shifts) y división por constante (número mágico).
- Explicar por qué una función del fuente puede no existir como función en el binario.
- Identificar un bucle vectorizado y una *tail call*.
- Usar un decompilador para deshacer estos patrones.

> **Puerta de salida**
>
> Si podés mirar un listado `-O2` y decir "esto es `x*5`, esto es `x/3`, esta función está inlineada" sin frustrarte, estás listo para binarios reales.

Fuentes de esta lección

- Agner Fog, *Optimizing subroutines in assembly language* y *Instruction tables* (patrones de código generado y costos).
- Documentación de **GCC** (`-O1/-O2/-O3`) y **LLVM/Clang** sobre pases de optimización.
- *Hacker's Delight* (Warren) para división por constantes y trucos aritméticos.

> **Capas de esta lección**
>
> **Conocimiento externo (agregado por esta edición):** todos los patrones de optimización. El corpus CLS usa binarios sin optimizar, así que este salto no estaba cubierto. **Recomendación editorial:** la estrategia "aprendé con Debug, analizá con Release" y los ejercicios.

Herramientas: del OllyDbg histórico al taller actual

> **Objetivo de la lección**
>
> Al terminar vas a poder elegir la herramienta adecuada para identificar, desensamblar, depurar o inspeccionar un binario, justificar por qué algunas del curso son históricas y registrar versiones y hashes de tu taller.

Prerrequisitos: debugger e IDA. **Duración sugerida:** 2 horas. Estimación aproximada.

Por qué hablar de herramientas

El conocimiento de reversing —registros, stack, saltos, PE— envejece poco. Las herramientas envejecen rápido. El curso de CLS mezcla material de 2007, 2016 y 2021, y usa OllyDbg en una parte y x64dbg en otra. Si no distinguís qué es concepto y qué es botón, vas a estudiar una interfaz muerta creyendo que estudias reversing.

El taller mínimo

Tarea	Herramienta	Estado	Nota
Depurar 32 y 64 bits	**x64dbg**	Activa	Gratis, open source, Windows
Desensamblar	**IDA Free**	Activa (versión gratuita)	Limitada pero suficiente para empezar
Desensamblar y decompilar	**Ghidra**	Activa	Gratis, open source, multiplataforma
Editar bytes	**HxD**	Activa	Gratis para uso personal; verificar licencia
Ver cabecera PE	**PE-bear**, **CFF Explorer**, **Detect It Easy**	Activas	Detect It Easy también identifica empaquetado
Ver procesos y handles	**Process Explorer** (Sysinternals)	Activa	Útil para el mapa de memoria y handles
Ver actividad de archivos/registro	**Process Monitor**	Activa	Para análisis dinámico controlado
Ver red	**TCPView**, **Wireshark**	Activas	Solo en laboratorio aislado
Automatizar	**Python** + `pefile`, `capstone`	Activas	Opcional
Detección	**YARA**	Activa	Para reglas defensivas

No necesitás instalar todo el primer día. x64dbg, un editor hexadecimal y un visor de PE alcanzan para las etapas 1 a 4.

El taller actual (2026): más allá de lo mínimo

> **Actualización externa**
>
> El corpus CLS no cubre estas herramientas (son más nuevas que su material). Las agregamos como **conocimiento externo** para que el taller esté al día. Todas corren o se usan sobre **Windows**; no son obligatorias para el recorrido base, pero sí para análisis real.
>
> Verificá siempre la versión y descargá desde el sitio oficial.

Cuando el taller mínimo se te queda corto, estas categorías son las que usa hoy un analista de Windows:

Categoría	Herramienta	Para qué
Instrumentación dinámica	**Frida**	Enganchar (*hook*) funciones de un proceso en vivo, leer/alterar argumentos y resultados con scripts. Potente para ver qué pasa sin trazar a mano.
Time-travel debugging	**WinDbg** + **TTD**	Grabar una ejecución completa y recorrerla **hacia atrás**. Ideal para "¿quién puso este valor?" sin reiniciar mil veces.
Diffing de binarios	**BinDiff**, **Diaphora**	Comparar dos versiones de un binario (por ejemplo, antes y después de un parche) para ver qué cambió. Clave en análisis de parches.
Extracción de strings	**FLOSS** (Mandiant)	Saca strings normales **y** las ofuscadas/apiladas que `strings` no ve.
Triage y capacidades	**CAPA**, **PEStudio**	Identifican qué "sabe hacer" un binario (red, cripto, persistencia) por patrones, sin ejecutarlo.
Clasificación/detección	**YARA**	Reglas para reconocer familias y rasgos. Defensivo.
Emulación y análisis simbólico	**Unicorn**, **Qiling**, **angr**, **Triton**	Ejecutar un fragmento sin el programa completo, o explorar caminos de forma simbólica (p. ej. "¿qué entrada llega a este bloque?").
Desensamblado programable	**Capstone** (leer) + **Keystone** (ensamblar)	Librerías para construir tus propias herramientas en Python.

> **Asistencia con IA (con cautela)**
>
> Hay plugins que usan modelos de lenguaje para **sugerir** nombres de funciones y variables o explicar un bloque decompilado (por ejemplo, asistentes para Ghidra/IDA). Pueden acelerar la lectura, pero **alucinan**: un nombre sugerido es una hipótesis, no un hecho. Usalos como punto de partida y **verificá** cada afirmación contra el código, igual que harías con el renombrado automático de un script.

Elegí por tarea, no por moda: para "¿quién escribe este buffer?" quizás te alcance un breakpoint de memoria en x64dbg; TTD y angr son para cuando el problema lo justifica.

> **En el Curso IDA: plugins y cuidado con la versión**
>
> El Curso IDA dedica una parte a los **plugins** que amplían IDA, con una advertencia que sigue valiendo: algunos no andan en la 6.8 pero sí en la 7.x, así que si no los podés instalar, al menos leé para qué sirven. Y muestra un **bug de la versión 6.8** (corregido en la 6.9) y cómo reconocerlo. Moraleja: **fijate qué versión de IDA tenés** antes de pelearte con un plugin que "no funciona" (Curso IDA · tute 13, Plugins y tute 11, Bug de la 6.8).

OllyDbg: herramienta histórica, no obsoleta como concepto

OllyDbg es un debugger de **ring 3 para binarios de 32 bits**, muy popular en los 2000. El curso de Cayetano de Juan Úbeda lo usa como herramienta principal en los videos 23 a 40, con una máquina virtual Windows 7 de 32 bits (`Videos y material adicional, Video 24`, 00:00:00 y 00:31:47).

Lo que OllyDbg enseña sigue siendo válido:

- Ver el estado de los registros y la pila.
- Poner breakpoints y trazar instrucción por instrucción.
- Leer el flujo de un `if` compilado.
- Identificar una comparación de clave.

Lo que cambió:

- No maneja binarios de 64 bits.
- No se actualiza desde hace años.
- Su interfaz y sus atajos no son los de x64dbg.
- Algunos plugins y técnicas de la época ya no funcionan en Windows modernos.

Conclusión: mirá los videos de OllyDbg por el método y los conceptos, no para aprender la interfaz. Después repetí el mismo ejercicio en x64dbg.

El flujo de trabajo con herramientas

```
1. Identificar      -> Detect It Easy / ExeInfoPE / PE-bear
2. Entender formato -> cabecera PE, secciones, imports
3. Análisis estático-> IDA Free / Ghidra
4. Análisis dinámico-> x64dbg (solo en laboratorio)
5. Documentar       -> bitácora + informe
```

Cada paso tiene una pregunta:

Paso	Pregunta que responde
Identificar	¿Qué tipo de archivo es? ¿Está empacado? ¿Es .NET?
Formato	¿Dónde está el código? ¿Qué imports usa?
Estático	¿Qué hace esta función? ¿Qué cadenas y APIs aparecen?
Dinámico	¿Qué valores toma en esta ejecución? ¿Qué camino sigue?
Documentar	¿Puede otra persona reproducir lo que hice?

El video 33 insiste en que el **mapa de memoria** es lo primero que abre al analizar un binario (`Videos y material adicional, Video 33`, 00:02:00–00:03:00). El video 39 muestra ExeInfoPE mirando cabecera, linker, secciones y entry point (`Videos y material adicional, Video 39`, 00:00:32–00:02:09).

Lo que no cambia

Estos conceptos valen igual en OllyDbg, x64dbg, IDA y Ghidra:

- La diferencia entre dirección virtual, RVA y offset.
- El stack y las convenciones de llamada.
- El significado de `cmp`, `test` y los saltos.
- La IAT y las llamadas indirectas.
- La diferencia entre análisis estático y dinámico.

Si entendés eso, cambiar de herramienta es aprender una interfaz nueva, no empezar de cero.

Práctica: inventario del taller

1. Instalá x64dbg, un editor hexadecimal y un visor de PE. Anotá versión y fecha de descarga.
2. Calculá el SHA-256 de cada instalador y guardalo en tu bitácora.
3. Abrí tu `check.exe` con el visor de PE, con x64dbg y con IDA Free. Completá:

Herramienta	Qué te mostró que las otras no
Visor de PE	
x64dbg	
IDA Free	

1. Buscá en el binario una cadena tuya ("Correcto") con las tres herramientas. ¿Cuál fue más rápida?
2. Escribí en tu bitácora qué herramienta usarías para: (a) saber si un binario está empacado, (b) encontrar quién escribe un buffer, (c) leer una función que nunca se ejecuta.

Preguntas de repaso

1. ¿Por qué OllyDbg se considera histórico? Porque es un debugger de 32 bits que ya no se actualiza. Sigue siendo útil para estudiar el método de la época, pero no es la herramienta recomendada para empezar hoy.

2. ¿x64dbg reemplaza a IDA? No. x64dbg es un debugger; IDA es un desensamblador estático. Se complementan.

3. ¿Qué herramienta usarías para saber si un binario está empacado? Detect It Easy, ExeInfoPE o un visor de PE: miran secciones, imports y entry point. La señal orienta, no concluye.

4. ¿Qué significa que una herramienta esté "activa"? Que recibe mantenimiento y funciona en sistemas actuales. No significa que sea obligatoria ni que sea la mejor para todo.

5. ¿Por qué registrar la versión de las herramientas? Porque un cambio de versión puede cambiar la interfaz o el comportamiento. Un informe reproducible dice con qué versión se hizo.

Antes de avanzar

Tenés que poder:

- Elegir herramienta para identificar, desensamblar, depurar y editar.
- Explicar por qué OllyDbg es histórico y qué se puede aprender de él.
- Registrar versiones y hashes de tu taller.
- Decir qué conceptos no dependen de la herramienta.

> **Puerta de salida**
>
> Si tu bitácora tiene el inventario del taller con versiones y hashes, estás listo para .NET.

Fuentes de esta lección

- `Videos y material adicional, Video 23` — introducción al reversing y limitaciones del análisis.
- `Videos y material adicional, Video 24` — OllyDbg y la VM Windows 7 de 32 bits.
- `Videos y material adicional, Video 33` — PE, secciones e imports.
- `Videos y material adicional, Video 39` — ExeInfoPE.
- `Reversing desde cero, cap. 14` — IDA y sus límites.
- Curso IDA · tutes 11 y 13 — plugins de IDA y el bug de la versión 6.8 (atención a la versión).

Capas de esta lección

Contenido observado en CLS: el uso de OllyDbg en los videos 23–40, la VM Windows 7 y el recorrido de ExeInfoPE. **Conocimiento externo (agregado por esta edición):** el estado actual de las herramientas y todo el "taller actual 2026" (Frida, WinDbg/TTD, BinDiff/Diaphora, FLOSS, CAPA, angr/Unicorn, asistencia con IA). **Recomendación editorial:** el taller mínimo, la tabla de decisión y la cautela con la IA.

Ofuscación nativa: leer lo que no quiere ser leído

> **Objetivo de la lección**
>
> Reconocer las señales de que un binario nativo está **ofuscado** y tener un método para llegar al código real, sin romperlo. **Prerrequisitos:** PE y secciones (1.4), assembly y flujo (2.3–2.4), IDA y debugger (2.6–2.7).

Esto es **reconocimiento defensivo**: entender cómo se ve el código ofuscado para poder analizarlo. No se enseña a ofuscar ni a desempaquetar protectores comerciales.

Qué es la ofuscación nativa

Ofuscar es hacer que el programa **siga funcionando igual** pero sea difícil de leer para una persona y confuso para el desensamblador. A diferencia de un packer, el código puede estar a la vista: lo que cambia es la *forma*. Las técnicas típicas (**conocimiento externo**):

- **Secciones reordenadas o con nombres falsos**, y el *entry point* lejos de lo esperado.
- **Basura entre bloques**: bytes que no se ejecutan pero que el desensamblador intenta interpretar como instrucciones.
- **Saltos al medio de una instrucción**: un `jmp` / `jnz` cae a un byte que no es el inicio real, así el listado lineal muestra una instrucción que *no es* la que se ejecuta.
- **Código muerto y opaco**: `test ebp, ebp` seguido de `jnz` cuando `ebp` nunca es cero — un salto que *siempre* se toma, disfrazado de condicional.
- **Strings ocultas**: con un `00` adelante, armadas en el stack byte a byte, o cifradas y descifradas en runtime.

Cómo se ve en CLS

El capítulo 26 analiza el ejercicio «choteado» y muestra el síntoma clásico: al abrirlo, **IDA lo muestra "todo roto"** y al forzarlo a función queda "todo feo"; en x64dbg tampoco se ve claro. Al mirar el *memory map* aparece la causa: **las secciones están cambiadas** —la primera sección está vacía y el código ejecutable recién está en la segunda—, y eso **engaña al desensamblador**. Con los símbolos cargados se empieza a navegar; sin ellos, la dificultad es la misma pero hay que apoyarse en la traza (`Reversing desde cero, cap. 26`).

El capítulo 30 resuelve la segunda parte y agrega un paso de preparación muy útil para analizar: cargar **todas las secciones** en IDA y, para trabajar con direcciones estables, **poner a cero el** `DLL characteristics` (el bit de `DYNAMICBASE`/ASLR) con *Patch Program → Change byte* y *Apply patches to input file*, de modo que el módulo cargue siempre en su base (aquí `0x400000`) (`Reversing desde cero, cap. 30`). Es un parche **sobre el crackme didáctico** del ejercicio (hecho para practicar), para poder estudiarlo con direcciones estables; no sobre software ajeno.

El método: encontrar el bloque real

1. **Sospechá temprano.** Si IDA no arma funciones, si hay flechas que cruzan todo o si el *memory map* no coincide con lo que esperás, asumí ofuscación.
2. **Mirá las secciones primero** (memory map / segments): dónde está de verdad el código ejecutable y cuál es la base.
3. **Dejá las direcciones quietas**: en tu laboratorio, desactivá ASLR del ejecutable propio para que las direcciones no cambien entre corridas.
4. **Seguí la ejecución, no el listado.** Ante un salto sospechoso, poné un breakpoint y mirá a dónde salta *de verdad*; el desensamblado lineal puede estar mostrando bytes que no se ejecutan.
5. **No NOPees a ciegas.** Reemplazar bytes sin entender rompe el programa y borra la pista. Comentá y renombrá; parchá solo cuando sabés qué hace el byte.

Ejemplo paso a paso: un salto al medio de una instrucción

*(Ejemplo **ilustrativo**: bytes construidos para mostrar la técnica, no un volcado de un binario puntual.)*

Supongamos estos bytes a partir de `0x401000`:

```
401000:  B8 EB FE 90 90        ; así lo decodifica el desensamblador LINEAL
         mov eax, 0x9090FEEB

pero el código que viene antes hace:  jmp 0x401001  (salta UN byte adelante)

401001:  EB FE                 ; reinterpretado desde 0x401001
         jmp short 0x401001    ; EB = jmp corto, FE = -2  → salta a sí mismo
```

Qué pasó: el byte `B8` (opcode de `mov eax, imm32`) hizo que IDA se "comiera" los 4 bytes siguientes como el inmediato. Pero el flujo real nunca ejecuta el `B8`: entra en `0x401001`, donde `EB FE` es un `jmp` a sí mismo (lo que viste en 2.4: desplazamiento `FE` = −2 sobre el final de la instrucción). El listado lineal mostraba una instrucción que **no se ejecuta**; la real aparece solo si decodificás desde el offset correcto. Por eso se sigue la **ejecución** (poné un breakpoint en `0x401001`) y no el listado.

Cadenas ocultas: cómo reconocer la rutina que las descifra

En 2.7 viste por qué una cadena puede no aparecer en la ventana de strings. Cuando la causa es que está **cifrada**, el patrón en el código suele ser reconocible aunque el algoritmo cambie (**conocimiento externo**):

- un **bucle corto** que recorre un buffer con un índice o puntero que avanza de a uno (o de a dos, si es UTF-16);
- una **operación reversible** por byte: `xor` con una constante o con un byte de una clave que se repite (`índice mod largo`), o `add` / `sub` / `rol`;
- un **destino** que después se pasa a una API que espera texto (`CreateFileW`, `RegOpenKeyExW`, `MessageBoxW`, `LoadLibraryW`).

*(Ejemplo **ilustrativo**: descifrado con `xor` y clave de un byte, x64.)*

```
lea   rsi, [rip+datos_cifrados]   ; fuente en .rdata (bytes ilegibles)
lea   rdi, [rsp+40h]              ; destino en el stack
xor   ecx, ecx
bucle:
movzx eax, byte ptr [rsi+rcx]
xor   al, 5Ah                     ; clave de un byte
mov   [rdi+rcx], al
inc   rcx
cmp   rcx, 0Ch                    ; largo fijo: 12 bytes
jb    bucle
mov   byte ptr [rdi+rcx], 0       ; terminador
lea   rcx, [rsp+40h]
call  cs:LoadLibraryA             ; el texto descifrado es un nombre de DLL
```

Cómo se lee: el `xor al, 5Ah` dentro del bucle es la transformación; `0Ch` es el largo; el destino termina como argumento de `LoadLibraryA`, así que el texto es un nombre de módulo. Para recuperarlo hay dos caminos, y conviene hacer los dos: **dinámico**, con un breakpoint en la llamada a `LoadLibraryA` y leyendo `RCX`; **estático**, aplicando la misma operación a los 12 bytes con un script (en IDA, IDAPython; en Ghidra, un script de 2.16). Si coinciden, anotás la cadena en un comentario junto al bucle y renombrás el buffer.

Lo que **no** conviene: ejecutar el programa entero para "ver qué cadenas aparecen" si no sabés qué hace, ni copiar la rutina sin entender de dónde sale la clave (puede depender de un dato del entorno y dar otro resultado en otra máquina).

Aplanamiento del flujo de control

Otra técnica frecuente en ofuscadores de código nativo es el **aplanamiento del flujo de control** (*control-flow flattening*): los bloques básicos de una función se sacan de su orden natural y se ponen **todos al mismo nivel**, dentro de un bucle infinito con un `switch` que elige qué bloque ejecutar según una **variable de estado**. Cada bloque, al terminar, escribe en esa variable el número del bloque siguiente (**conocimiento externo**; el ofuscador de investigación Obfuscator-LLVM lo describe así).

Cómo se ve en el grafo de IDA o Ghidra:

- un **despachador** central (comparaciones o una tabla de saltos sobre la misma variable) al que vuelven todos los bloques;
- muchos bloques "hermanos" sin relación aparente entre sí, todos con una flecha de vuelta al despachador;
- cada bloque termina con un `mov [var_estado], constante` (o con un cálculo que da una constante) antes de volver.

El método para leerlo no es mirar el grafo, que ya no dice nada, sino **reconstruir la secuencia**: para cada bloque, anotá qué valor de estado lo selecciona y qué valor deja al salir. Con esa tabla (estado → bloque → estado siguiente) se recupera el orden real. Los saltos condicionales originales aparecen como bloques que dejan **uno de dos valores** según una comparación. Hacerlo a mano sirve para funciones chicas; para funciones grandes existen complementos que lo automatizan, fuera del alcance de esta lección.

Señal	Qué sospechar	Cómo confirmarlo
Todos los bloques vuelven a un mismo punto	Aplanamiento	Ubicar la variable que compara el despachador
`jnz` tras `test` con registro siempre no nulo	Predicado opaco	Seguir el valor del registro en el debugger
Bucle corto con `xor` sobre un buffer que va a una API	Cadena cifrada	Breakpoint en la API y leer el argumento
Ráfaga de `mov` con constantes imprimibles	Cadena armada en el stack	Leer las constantes como bytes en little-endian
El listado no arma funciones	Basura o saltos al medio	Seguir la ejecución, no el listado

Desactivar ASLR sin romper la cabecera (ni depender de *Manual Load*)

El campo `DllCharacteristics` (2 bytes, WORD) está en el offset **+0x46** del Optional Header (`e_lfanew + 0x5E`), igual en PE32 y PE32+. El bit de ASLR es `IMAGE_DLLCHARACTERISTICS_DYNAMIC_BASE = 0x0040` (en x64 suele venir con `HIGH_ENTROPY_VA = 0x0020`). **No pongas todo el WORD en cero**: pisarías `NXCOMPAT (0x0100)`, `GUARD_CF (0x4000)`, etc. Y ojo: **IDA no carga la cabecera PE** salvo que elijas *Manual Load*, así que ahí no vas a encontrar el campo. Hacelo en PE-bear/CFF Explorer (Optional Header → DllCharacteristics → desmarcá *Dynamic Base*) o con Python sobre una copia:

```
import pefile
pe = pefile.PE("copia_lab.exe")
pe.OPTIONAL_HEADER.DllCharacteristics &= ~0x0060   # limpia DYNAMIC_BASE (0x40) y
HIGH_ENTROPY_VA (0x20)
pe.write("copia_lab_noaslr.exe")
```

Práctica

En tu VM, sobre un **ejercicio propio o un crackme didáctico** (hay varios en el Banco de prácticas):

1. Abrilo en IDA y en x64dbg y anotá tres señales de que algo no se lee normal.
2. Mirá el *memory map*: ¿la sección de código es la primera? ¿La base es la esperada?
3. Encontrá un salto condicional que en realidad sea incondicional (p. ej. `test reg,reg` + `jnz` con el registro siempre distinto de cero) y explicá por qué.
4. Llegá al primer bloque "real" de lógica siguiendo la ejecución, no el listado.
5. En un programa **propio**, guardá un nombre de DLL cifrado con `xor` de un byte y descifralo antes de usarlo. Recuperalo de las dos formas (breakpoint en la API y script sobre los bytes) y compará.
6. Escribí a mano, en C, una función chica con un `if` y un bucle, y una segunda versión "aplanada" con `while (1) { switch (estado) { ... } }`. Compilá ambas y compará los grafos; armá la tabla estado → bloque → estado siguiente de la versión aplanada.

Preguntas de repaso

▼ **¿Por qué un listado lineal puede mostrar una instrucción que no es la que se ejecuta?**

> Porque un salto puede caer **al medio** de lo que el desensamblador tomó como una instrucción; a partir de ese offset los bytes se decodifican distinto. Lo que se ejecuta es la secuencia que sigue el flujo real, no la que el desensamblador alineó de arriba a abajo.

▼ **Viste `test ebp, ebp` seguido de `jnz`. ¿Por qué puede ser ofuscación?**

> Si en ese punto `ebp` nunca vale cero, `ZF` nunca se activa y el `jnz` **se toma siempre**: es un salto incondicional disfrazado de condicional, para que parezca que hay dos caminos cuando hay uno.

▼ **En el «choteado», ¿qué pista da el *memory map*?**

> Que las secciones están **reordenadas**: la primera está vacía y el código ejecutable está en la segunda. Eso explica por qué el desensamblador no arma bien las funciones.

▼ **¿Por qué el capítulo 30 pone a cero el `DLL characteristics`?**

> Para **desactivar ASLR** del ejecutable y que cargue siempre en la misma base (`0x400000`), así las direcciones son estables mientras se lo analiza. Es un parche sobre un binario propio de práctica.

▼ **¿Por qué "no NOPear a ciegas"?**

> Porque reemplazar bytes sin entenderlos rompe la lógica y borra la evidencia de qué hacía el código. Primero se entiende (comentando, trazando); recién después, si hace falta y es un binario propio, se parchea.

▼ **¿Qué tres elementos delatan una rutina que descifra una cadena?**

> Un bucle corto sobre un buffer, una operación reversible por byte (`xor`, `add`, `rol`) y un destino que después se pasa a una API que espera texto.

▼ **¿Por qué conviene recuperar una cadena cifrada de forma estática y dinámica?**

> Porque se controlan entre sí: si el script sobre los bytes y el argumento leído en el breakpoint coinciden, la lectura de la rutina es correcta; si no, la clave puede depender del entorno o la rutina tiene un paso que no viste.

▼ **En un grafo aplanado, ¿qué tabla reconstruís para recuperar el orden?**

> Estado → bloque → estado siguiente: qué valor de la variable selecciona cada bloque y qué valor deja al salir. Los bloques que dejan uno de dos valores según una comparación son los condicionales originales.

Puerta de salida

Podés: (1) nombrar tres señales de ofuscación en un binario; (2) explicar un salto al medio de una instrucción y un salto condicional que siempre se toma; (3) usar el *memory map* para ubicar el código real; (4) llegar al primer bloque de lógica siguiendo la ejecución; (5) reconocer una rutina de descifrado de cadenas y recuperar el texto de dos formas; (6) reconocer un despachador de flujo aplanado y reconstruir el orden de una función chica.

Lo que queda fuera

La ofuscación por **máquina virtual** (el código se traduce a un bytecode propio que interpreta un "procesador" embebido), las expresiones mixtas booleano-aritméticas y la desofuscación automática por ejecución simbólica. Se mencionan para reconocerlas por nombre; analizarlas exige herramientas y tiempo que exceden esta etapa, y en protectores comerciales entran en el límite del curso (no desempacar protectores comerciales).

Fuentes de esta lección

- `Reversing desde cero, cap. 26` — el «choteado» con símbolos: IDA/x64dbg confundidos, secciones invertidas en el memory map.
- `Reversing desde cero, cap. 30` — cargar todas las secciones y poner a cero `DLL characteristics` (ASLR) para direcciones estables.
- Curso IDA · tutes 14–18 — contexto de binarios que "no se leen de una".
- Conocimiento externo: Obfuscator-LLVM, Control Flow Flattening — descripción del aplanamiento con variable de estado y `switch`; FLOSS — tipos de cadenas que extrae (estáticas, armadas en el stack, decodificadas).

Capas de esta lección

Contenido observado en CLS: el «choteado» de los capítulos 26 y 30, las secciones invertidas y el parche de `DLL characteristics`. **Conocimiento externo:** la taxonomía de técnicas (basura, saltos al medio, código opaco, strings ocultas), el patrón de las rutinas de descifrado y el aplanamiento del flujo de control. **Ejemplo ilustrativo:** los bytes del salto al medio y el bucle de descifrado con `xor`. **Recomendación editorial:** el método de cinco pasos, la regla de no NOPear a ciegas, la tabla de señales y la recuperación doble (estática y dinámica).

Ampliación editorial basada en fuentes externas — Criterio de triage estático: Aplanamiento de Flujo (CFF) vs. Máquina Virtual de bytecode (VM)

Aunque desofuscar una máquina virtual queda fuera del alcance del curso, en el inventario estático (Etapa 4) tanto CFF (Obfuscator-LLVM) como un intérprete de VM (Rolles, USENIX WOOT '09) producen un grafo con un gran bucle y un `switch` central *(Hecho confirmado)*. Esta tabla permite distinguirlos en pocos minutos:

Criterio observable	Aplanamiento del Flujo de Control (CFF)	Ofuscación por Máquina Virtual (VM de bytecode)
Contenido de los bloques del switch	Operaciones reales de la función original (llamadas a APIs, acceso a variables locales y comparaciones del negocio).	Micro-operaciones genéricas (*handlers* del CPU virtual: `vADD`, `vPUSH`, `vMOV`, `vNOR`) sin lógica de negocio directa.
Selección del siguiente bloque	Cada bloque asigna una constante fija a una variable local de estado (`mov [rbp-8], 0x3A1F...`) y vuelve al encabezado del `switch`.	Un registro físico actúa como puntero de bytecode (`VIP` / `VPC`), lee el siguiente opcode de una tabla de datos (`lodsb` / `movzx eax, byte ptr [rsi] ; inc rsi`) y salta vía tabla de handlers (`jmp [rbx+rax*8]`).
Reutilización entre funciones	Cada función aplanada tiene su propio bucle y sus propios bloques básicos.	Varias funciones protegidas empujan un offset de bytecode distinto y saltan al mismo intérprete (`VM_Entry`).

TLS: código que corre antes de `main`

> **Objetivo de la lección**
>
> Entender qué es el **Thread Local Storage (TLS)** y, sobre todo, los **TLS callbacks**: código que Windows ejecuta **antes del entry point**, un lugar clásico para esconder lógica. Saber encontrarlo en el PE y en el debugger. **Prerrequisitos:** PE y directorios (1.4), procesos e hilos (1.5), debugger (2.6).

Dos cosas distintas con el mismo nombre

El capítulo 38 es un apéndice teórico sobre TLS y aclara que en un mismo ejercicio (`EjemploTLS`) conviven **dos formas** de guardar datos por hilo (`Reversing desde cero, cap. 38`):

1. **Por API:** `TlsAlloc`, `TlsSetValue`, `TlsGetValue`, `TlsFree`. El programa pide un índice y guarda/lee un valor propio de cada hilo. Es explícito y se ve en los imports.
2. **Por variable `__declspec(thread)`:** el compilador reserva el dato en la sección `.tls` y lo resuelve con ayuda del loader. No hay llamadas visibles: el acceso aparece como lectura de una zona por hilo.

Y hay un **tercer mecanismo, aparte**: los **TLS callbacks**. No dependen de cómo declares las variables —`__declspec(thread)` usa `.tls` pero **no** crea callbacks por sí solo—; los callbacks se registran en el **directorio TLS del PE** y son lo que más importa para el reversing, porque corren antes de `main`.

El directorio TLS y los callbacks

El PE tiene un **directorio TLS** (`IMAGE_TLS_DIRECTORY`) entre sus *data directories*. Su campo clave para el análisis es `AddressOfCallBacks`: un puntero a un arreglo de funciones terminado en cero. Esas funciones son los **TLS callbacks**, y el loader de Windows **las ejecuta antes de que corra el entry point** (antes de `main`), en cada carga y en cada creación/destrucción de hilo.

La consecuencia para quien analiza (**conocimiento externo**, confirmado por el rol de TLS que explica el cap. 38): si ponés el breakpoint solo en el entry point, **te perdés todo lo que pasó en el callback**. Es el escondite preferido para anti-debug temprano o para inicializaciones que no querés que se vean.

Cómo encontrarlo

- **En el PE** (PE-bear, CFF Explorer o IDA): abrí el *data directory* **TLS**; si `AddressOfCallBacks` no es cero, seguilo y vas a ver la lista de callbacks.
- **En IDA:** aparecen como `TlsCallback_0`, `TlsCallback_1`… y suelen quedar fuera del flujo del `main`; búscalos por nombre o desde el directorio TLS.
- **En x64dbg:** activá *Options → Preferences → Events →* ***TLS Callbacks*** para que el debugger **pare en el primer callback**, antes del entry point. Si no, arrancás ya pasado el punto interesante.

Ejemplo paso a paso: leer el directorio TLS

*(Ejemplo **ilustrativo**: valores construidos; en un binario real cambian las direcciones.)*

En PE-bear, el *data directory* número 9 (TLS) apunta a la estructura, y adentro seguís `AddressOfCallBacks`:

```
DataDirectory[9] TLS → RVA 0x3000, size 0x18

IMAGE_TLS_DIRECTORY  (en .rdata)
  StartAddressOfRawData  0x00404000
  EndAddressOfRawData    0x00404008
  AddressOfIndex         0x00404100
  AddressOfCallBacks     0x00403020   ← seguí este puntero
  SizeOfZeroFill         0
  Characteristics        0

En 0x00403020 (arreglo de punteros, terminado en NULL):
  20 10 40 00   00 00 00 00
  → callback[0] = 0x00401020
  → callback[1] = NULL  (fin de la lista)
```

Vas a `0x00401020` y ahí está el `TlsCallback_0`: ese código corre **antes** del entry point. Si `AddressOfCallBacks` fuera `0`, no habría callbacks y bastaría el breakpoint en el entry point.

> **Tamaño del directorio según la arquitectura (x86 vs x64)**
>
> El ejemplo de arriba es de 32 bits. En **PE32**, `IMAGE_TLS_DIRECTORY32` ocupa **0x18 bytes** y `AddressOfCallBacks` está en el offset **+0x0C** (arreglo de DWORDs de 4B). En **PE32+**, `IMAGE_TLS_DIRECTORY64` ocupa **0x28 bytes** y `AddressOfCallBacks` se corre a **+0x18** (arreglo de QWORDs de 8B, terminado en 8 ceros). En ambos casos esos campos son **direcciones virtuales absolutas (VA)**, no RVAs: si hay ASLR, la tabla `.reloc` los ajusta.

Práctica

En tu VM, con un **programa propio**:

1. Compilá algo mínimo con un callback TLS. En MSVC, el código mínimo es:

```c
#include <windows.h>
#include <stdio.h>

volatile int orden = 0;   /* lo escribe el callback; lo lee main */

void NTAPI mi_cb(PVOID h, DWORD motivo, PVOID r) {
    /* OJO: nada de printf acá. El callback corre ANTES de que
       el CRT se inicialice, así que el runtime de C todavía no
       está listo. Solo tocamos una variable. */
    if (motivo == DLL_PROCESS_ATTACH) orden = 1;
}

/* Registrar el callback en .CRT$XLB. Con __declspec(allocate) el símbolo NO
   se elimina aunque compiles con /O1 /O2 /GL (a diferencia de 'const_seg +
   EXTERN_C const', que el linker puede descartar). Incluye variante MinGW. */
#ifdef _MSC_VER
  #ifdef _WIN64
    #pragma comment(linker, "/INCLUDE:_tls_used")    /* x64: un guion bajo */
  #else
    #pragma comment(linker, "/INCLUDE:__tls_used")   /* x86: dos guiones */
  #endif
  #pragma section(".CRT$XLB", long, read)
  __declspec(allocate(".CRT$XLB")) PIMAGE_TLS_CALLBACK p_cb = mi_cb;
#else  /* MinGW-w64 (GCC / Clang) */
  PIMAGE_TLS_CALLBACK p_cb __attribute__((used, section(".CRT$XLB"))) = mi_cb;
#endif

int main(void) {
    printf("orden al entrar a main = %d  (1 => el callback ya corrió)\n", orden);
    return 0;
}
```

1. Abrilo en PE-bear y confirmá que el directorio **TLS** no está vacío y que `AddressOfCallBacks` apunta a tu función.
2. En x64dbg, activá el breakpoint de TLS callbacks y verificá que **para antes de** `main`.
3. Confirmá el orden **sin** imprimir desde el callback: que el callback solo escriba `orden = 1` y que `main` imprima `orden`. Si al entrar a `main` vale 1, el callback corrió antes (imprimir desde el callback puede colgar: el CRT todavía no se inicializó).

Preguntas de repaso

▼ **¿Qué hace especial a un TLS callback frente a cualquier otra función?**

> Que el loader lo ejecuta **antes del entry point** (y en cada alta/baja de hilo), sin que el `main` lo llame. Corre "solo", por estar registrado en el directorio TLS.

▼ **¿Qué campo del directorio TLS mirás primero y por qué?**

> `AddressOfCallBacks`: si no es cero, hay callbacks. Apunta a un arreglo de punteros a función terminado en cero.

▼ **Si solo ponés un breakpoint en el entry point, ¿qué te perdés?**

> Todo lo que haya hecho el/los callbacks TLS, que ya corrieron. Por eso en x64dbg conviene habilitar el breakpoint de TLS callbacks.

▼ **¿En qué se diferencian las dos formas de TLS del `EjemploTLS`?**

> Una usa **APIs** (`TlsAlloc` / `TlsGetValue` / `TlsSetValue`), visible en imports; la otra usa `__declspec(thread)`, resuelta por el compilador y el loader vía la sección `.tls`, sin llamadas explícitas.

▼ **¿Por qué el TLS es un buen escondite para anti-debug?**

> Porque corre antes que el código "principal" y mucha gente empieza a mirar desde el entry point; un chequeo puesto en un callback se ejecuta y decide antes de que el analista llegue.

Puerta de salida

Podés: (1) distinguir TLS por API de `__declspec(thread)`; (2) ubicar el directorio TLS y `AddressOfCallBacks` en un PE; (3) hacer que x64dbg pare en el primer callback; (4) demostrar con tu propio binario que el callback corre antes de `main`.

Fuentes de esta lección

- `Reversing desde cero, cap. 38` — `EjemploTLS`: TLS por API (`TlsAlloc` / `TlsGetValue` / `TlsSetValue`) y por `__declspec(thread)`.

Capas de esta lección

Contenido observado en CLS: las dos formas de TLS del `EjemploTLS` del capítulo 38. **Conocimiento externo:** la estructura `IMAGE_TLS_DIRECTORY`, `AddressOfCallBacks`, el orden de ejecución antes del entry point y el breakpoint de TLS en x64dbg. **Recomendación editorial:** el método de búsqueda y la práctica con binario propio.

MFC: llegar al handler cuando todo pasa por clases

> **Objetivo de la lección**
>
> Reconocer un ejecutable hecho con **MFC** y llegar al **handler** de un control (el botón de un diálogo) aunque el `WinProc` clásico no esté a la vista. **Prerrequisitos:** mensajes de Windows y debugger (2.6), estructuras y vtables (2.7).

Por qué MFC complica la lectura

En los ejercicios anteriores la ventana se hacía **con la API de Windows directa**: el programa llamaba a `RegisterClassA` (que registra la clase y apunta al `WinProc`) y luego a `CreateWindow`; desde el `WinProc` se llegaba fácil a la comparación con los números de mensaje (`WM_COMMAND`, etc.) y de ahí al código que maneja cada evento. Era sencillo.

MFC (Microsoft Foundation Classes) es un conjunto de clases con mucha herencia que envuelve esa misma API. Por debajo sigue pasando lo mismo, pero **por arriba no se ve el `WinProc`**: Visual Studio genera una clase de diálogo (`CDialog` / `CWnd`) y el ruteo de mensajes queda escondido en el framework. Por eso, abrir un binario MFC y buscar el `WinProc` a mano no alcanza (**observado**, capítulo 39: `Reversing desde cero, cap. 39`).

Señales de que es MFC

- Importa `mfc###u.dll` (p. ej. `mfc140u.dll`) y el runtime de VC — **si MFC está enlazado dinámico**. En **enlace estático** esa DLL no aparece: ahí te guiás por los símbolos (`CWnd`, `AfxWinInit`) y las strings.
- Aparecen `AfxWinMain` / `AfxWinInit` y nombres de clases MFC (`CWnd`, `CDialog`, `CWinApp`, `CString`).
- Las APIs de texto que te sirven como ancla siguen estando: `GetWindowTextW`, `SetWindowTextW`, `MessageBoxW`, `SendMessageW`.

El message map: dónde está el ruteo

MFC asocia cada mensaje/control a una función con un **message map**, una tabla que el compilador arma a partir de macros del estilo `BEGIN_MESSAGE_MAP` / `ON_BN_CLICKED` / `END_MESSAGE_MAP`. En el binario queda como un arreglo de entradas `AFX_MSGMAP_ENTRY`, y cada entrada tiene más que "ID y puntero": `nMessage` (el mensaje, p. ej. `WM_COMMAND`), `nCode` (código de notificación), `nID` y `nLastID` (rango de IDs de control), `nSig` (la firma de la función) y `pfn` (el puntero al handler). El handler de "se hizo clic en el botón 1" suele llamarse `OnBnClickedButton1` en el fuente.

Dos caminos para llegar, sin pelearte con la tabla (**recomendación editorial**, apoyada en el método de mensajes del cap. 28):

1. **Por string/API (el más rápido):** buscá la string de la ventana ("clave", "acceso", el mensaje de éxito) o poné un breakpoint en `GetWindowTextW` / `SendMessageW`; al dispararse, el *call stack* te deja dentro del handler del control.

2. **Por el message map:** localizá la tabla del diálogo (cerca del constructor de la clase) y seguí el puntero de la entrada cuyo ID coincide con el del botón.

Ejemplo paso a paso: del botón al handler

*(Ejemplo **ilustrativo**: direcciones y nombres construidos para mostrar la ruta.)*

Ponés un breakpoint en `GetWindowTextW`, tocás **Aceptar** en el diálogo y mirás el *call stack*:

```
USER32!GetWindowTextW
app!CWnd::GetWindowTextW(...)          ; wrapper de MFC
app!CMiDialog::OnBnClickedOk+0x3e      ← el handler (acá estás)
mfc140u!_AfxDispatchCmdMsg+0x52        ; el framework ruteó el mensaje
mfc140u!CWnd::OnCommand+0x...
```

Subís al frame del handler `OnBnClickedOk` y encontrás la decisión:

```
lea   rdx, [rsp+0x40]        ; buffer donde quedó el texto tipeado
lea   rcx, [rsi+8]           ; el control (this)
call  CWnd::GetWindowTextW
lea   rdx, aS3cr3t           ; "s3cr3t"  (la clave esperada)
lea   rcx, [rsp+0x40]
call  wcscmp                 ; la comparación que decide éxito/fracaso
test  eax, eax
jnz   mal                    ; != 0 → clave incorrecta
```

Sin el call stack habría que desenredar el message map; con el breakpoint en la API de texto caés **directo** en `OnBnClickedOk`.

Práctica

En tu VM:

1. Si tenés Visual Studio, creá una **app MFC basada en diálogo** con un botón y un `OnBnClickedButton1` que compare un texto; compilala. Si no, usá un **binario MFC de práctica**.
2. Confirmá que importa `mfc###u.dll` y ubicá `AfxWinMain`.
3. Poné un breakpoint en `GetWindowTextW` (o en `SendMessageW`), tocá el botón y, desde el *call stack*, llegá al handler.
4. Identificá en el handler la comparación que decide éxito/fracaso.

Preguntas de repaso

▼ **¿Por qué en un binario MFC no encontrás el `WinProc` como en los ejercicios con API directa?**

> Porque MFC **envuelve** la API: el ruteo de mensajes lo maneja el framework a través de clases y del message map, en vez de un `WinProc` propio que el programa registre con `RegisterClassA`.

▼ **¿Qué es el message map y qué guarda cada entrada?**

> Una tabla que mapea **mensaje/control → función**. Cada entrada tiene el ID del control, el código de notificación y el **puntero al handler**.

▼ **Nombrá dos señales de que un binario es MFC.**

> Importa `mfc###u.dll` (p. ej. `mfc140u.dll`) y aparecen `AfxWinInit` / `AfxWinMain` y clases como `CDialog` / `CWnd` / `CString`.

▼ **¿Cuál es la forma más rápida de llegar al handler de un botón?**

> Un breakpoint en `GetWindowTextW` / `SendMessageW` (o seguir la string del diálogo): al dispararse, el *call stack* te deja dentro del handler, sin recorrer la tabla a mano.

▼ **Por debajo, ¿MFC cambia lo que hace Windows con los mensajes?**

> No. Sigue siendo el mismo sistema de mensajes de Windows; MFC solo agrega una capa de clases por encima. Las APIs ancla (`GetWindowTextW`, `SendMessageW`) siguen ahí.

Puerta de salida

Podés: (1) reconocer un binario MFC por imports y símbolos; (2) explicar qué es el message map; (3) llegar al handler de un botón por breakpoint en una API de texto y el call stack; (4) señalar la comparación que decide el resultado.

Fuentes de esta lección

- `Reversing desde cero, cap. 39` — el ejercicio MFC «de CITO»: por qué el `WinProc` no está a la vista y qué cambia respecto de usar la API directa.
- `Reversing desde cero, cap. 28` — base: `RegisterClassA`, `WinProc`, `WM_COMMAND` y breakpoint sobre el mensaje.

Capas de esta lección

Contenido observado en CLS: el contraste API-directa vs. MFC del capítulo 39 y el método de mensajes del 28. **Conocimiento externo:** los nombres del message map (`BEGIN_MESSAGE_MAP`, `ON_BN_CLICKED`, `OnBnClickedButton1`), las DLLs `mfc###u.dll` y `AfxWinInit`. **Recomendación editorial:** los dos caminos para llegar al handler y la práctica con binario propio.

Ampliación editorial basada en fuentes externas — Definición en C y offsets de AFX_MSGMAP y AFX_MSGMAP_ENTRY para análisis estático

Cuando un binario MFC está enlazado estáticamente o se analiza sin ejecutarlo en IDA (*Local Types*) o Ghidra (*Parse C Source*), las estructuras oficiales de `<afxwin.h>` (Microsoft Learn: TN006 Message Maps) tienen el siguiente layout en memoria *(Hecho confirmado)*:

```
struct AFX_MSGMAP_ENTRY {
    unsigned int nMessage;   // +0x00 (4B): Mensaje (p. ej. 0x0111 = WM_COMMAND)
    unsigned int nCode;      // +0x04 (4B): Código de notificación (0 = BN_CLICKED)
    unsigned int nID;        // +0x08 (4B): Control ID inicial
    unsigned int nLastID;    // +0x0C (4B): Control ID final (igual a nID si es único)
    uintptr_t    nSig;       // +0x10 (4B en x86 / 8B en x64): Firma AfxSig_* (0 =
AfxSig_end)
    void*        pfn;        // +0x14 en x86 (total 24B) / +0x18 en x64 (total 32B): Puntero
al handler
};
struct AFX_MSGMAP {
    const void* pfnGetBaseMap;               // +0x00: Puntero al mapa base
    const struct AFX_MSGMAP_ENTRY* lpEntries; // +0x04 (x86) / +0x08 (x64): Arreglo
terminado en {0,0,0,0,0,0}
};
```

Recomendación editorial: El método virtual `GetMessageMap()` en la vtable de cualquier clase derivada de `CWnd` devuelve la dirección de su `AFX_MSGMAP` en dos instrucciones (`lea rax, [rip + MiDialog_messageMap] ; ret`).

Huella de compilador: saber con qué se hizo

> **Objetivo de la lección**
>
> Antes de leer una sola instrucción, saber **con qué compilador/runtime** se armó el binario, porque eso cambia todo: dónde está la lógica, cómo se ven las llamadas, qué estructuras esperar. **Prerrequisitos:** PE e imports (1.4), IDA (2.7), MFC (2.12).

Por qué mirar esto primero

El mismo programa en C se ve **distinto** según el compilador, y un programa de Delphi o Visual Basic no se parece en nada a uno de C. Identificar la huella te ahorra horas: te dice si vas a encontrar un `main` clásico, un runtime que rutea todo, p-code en vez de nativo, o una maraña de clases. Es lo primero que hace un analista con experiencia (**recomendación editorial**).

Tabla de huellas

Conocimiento externo (pistas que se confirman en los imports, strings y secciones del PE):

Compilador / runtime	Pistas típicas
MSVC (Visual C++)	**Rich header** en la cabecera (ver 2.14); `__security_cookie`, `__scrt_common_main`, `__acrt_*`; ruta a un `.pdb`; runtime `vcruntime###.dll` / `ucrtbase.dll`
MinGW/GCC	`msvcrt.dll`, `__mingw_*`, `libgcc_s_*`, `___chkstk_ms`, sección `.eh_frame`; **sin** Rich header; strings de rutas tipo `/mingw/...`
Delphi	Strings `Sysutils`, `System`, `Classes`, `TForm`, `TObject`; símbolos con `@`; VMT (tabla de métodos) al inicio del objeto; secciones `.itext` / `.data` particulares
Visual Basic 6	Import de `MSVBVM60.DLL`; funciones `__vba*`; strings `Project1` / `Form1`; puede ser **p-code** (interpretado) o nativo
MFC (sobre MSVC)	`mfc###u.dll` (p. ej. `mfc140u.dll`), `AfxWinInit` / `AfxWinMain`, clases `CWnd` / `CDialog` / `CString`, message map (ver 2.12)
Go	Binario grande y enlazado estáticamente, con pocos imports; cadena `Go build ID`; nombres de paquetes y funciones como `main.main`, `runtime.` o `fmt.` visibles aunque no haya símbolos de depuración; sección o datos con la tabla `pclntab`
Rust	Rutas de código fuente en mensajes de pánico (`src/main.rs`, `library/core/src/...`); símbolos con mangling `_ZN...E` (antiguo) o `_R...` (v0); mensajes como «called `Option::unwrap()` on a `None` value»; con el target `-msvc` puede traer **Rich header** del linker de Microsoft

Regla práctica: **mirá primero los imports** (qué DLLs de runtime trae) y las **strings** (nombres de clases/módulos); con eso ya ubicás la familia.

Dos casos que cambian la estrategia

- **VB6 p-code vs. nativo.** Si es p-code, el código "real" no es x86: es bytecode que interpreta `MSVBVM60.DLL`. Leer el x86 directo no sirve; hay que usar herramientas específicas de VB. Reconocerlo temprano evita perder tiempo.
- **Delphi.** Las APIs no se llaman como en C: el objeto trae su **VMT** al inicio y los métodos se resuelven por tabla. Esperar un `main` con `CreateWindow` directo te confunde.

En CLS esto está **como material, no como lección**: en `Teorias_Numeradas` hay **decenas de títulos de Delphi** y **varios de Visual Basic/VB6, p-code y Java** (índice clasificado). Son conteos **por subcadena del título** (aproximados, sin abrir los contenedores): orientan para profundizar, no son prueba. El único runtime trabajado a fondo en el curso es **MSVC/C++** (y MFC en la 2.12, observado en el cap. 39: `Reversing desde cero, cap. 39`).

Lenguajes que no se leen como C: Go, Rust y Delphi

Los tres producen código nativo x86/x64, así que IDA y Ghidra los abren sin problema. Lo que cambia es **qué esperar** del listado. Todo este apartado es **conocimiento externo** al material de CLS; lo citado viene de la documentación de cada lenguaje.

Go. Un "hola mundo" en Go pesa varios megabytes porque trae enlazado el runtime completo (planificador de goroutines, recolector de basura). Para leerlo:

- **Los nombres de funciones sobreviven.** El runtime de Go necesita una tabla que relaciona direcciones con funciones y líneas (`pclntab`) para armar trazas de pila, perfilar y recolectar basura, así que está en el binario aunque se hayan quitado los símbolos de depuración. Herramientas e IDA/Ghidra recientes la usan para recuperar nombres como `main.main` o `main.verificarClave`. Empezá por las funciones del paquete `main`: el resto suele ser runtime y biblioteca estándar.
- **Las cadenas no terminan en `0`.** Un `string` de Go es puntero + longitud; el compilador guarda muchas cadenas pegadas en una sola zona y el buscador de strings las muestra como un bloque largo. Para saber dónde termina una, buscá la longitud que se carga junto al puntero.
- **Convención propia.** Go no usa la convención x64 de Windows de Llamadas y convenciones para sus funciones internas: tiene su propio ABI (en versiones recientes pasa argumentos en registros, en otro orden). Al comienzo de casi toda función vas a ver una **comprobación de pila** —una comparación del puntero de pila con un límite y un salto a `runtime.morestack`— que no es lógica del programa y se puede saltar con la vista.

Rust. También enlaza estáticamente su biblioteca estándar, pero suele usar el runtime de C de la plataforma.

- **Huella engañosa.** Con el target `x86_64-pc-windows-msvc` el binario lo enlaza el linker de Microsoft: puede tener Rich header e importar `vcruntime140.dll`. Ver esas pistas y concluir "C++ de MSVC" es el error típico; las rutas `.rs` y los mensajes de pánico lo desmienten.
- **Cadenas como puntero + longitud** (`&str`), igual que en Go: sin `0` final.
- **Nombres con mangling.** El esquema antiguo se parece al de C++ de Itanium (`_ZN...` con un hash al final) y el esquema **v0** empieza con `_R`; ambos se pueden demanglear con las herramientas de Rust.

- **Mucho código de comprobación.** Chequeos de límites de arreglos y desbordes que terminan en una función de pánico. Las ramas que van al pánico son el camino de error; la lógica está en el otro.

Delphi (amplía la fila de la tabla).

- **Formularios como recursos.** Los formularios (`TForm`) se guardan como recursos `RCDATA` que empiezan con la firma `TPF0`; ahí están los nombres de los controles y los **nombres de los manejadores de eventos** (por ejemplo `Button1Click`), que te llevan directo a la lógica.
- **Convención `register`.** En Delphi de 32 bits la convención por defecto pasa los tres primeros argumentos en `EAX`, `EDX` y `ECX`, no en el stack: un método recibe `Self` en `EAX`. Si leés el listado esperando `stdcall`, los argumentos "desaparecen".
- **Cadenas con cabecera.** Los `string` largos de Delphi guardan contador de referencias y longitud **antes** del primer carácter, así que el puntero apunta al texto pero hay datos en offsets negativos.
- **RTTI y VMT.** Los nombres de clases y la tabla de métodos virtuales permiten reconstruir jerarquías; existen herramientas específicas para Delphi que lo automatizan.

> **Pistas, no pruebas**
>
> Una huella es un **indicio**, no un certificado. La **ausencia de Rich header** sugiere que no es MSVC, pero no **demuestra** MinGW (podría ser otro linker o estar removido). Y una app **MFC enlazada estáticamente** puede **no** importar `mfc###u.dll`. Confirmá siempre con varias señales juntas.

Ejemplo paso a paso: el mismo `hola.c`, dos compiladores

*(Ejemplo **ilustrativo**: imports representativos; varían según versión.)*

El mismo programa compilado en MSVC y en MinGW deja huellas distintas:

```
MSVC (cl hola.c)                      MinGW (gcc hola.c)
________________________              ____________________________
Rich header: presente                 Rich header: AUSENTE
Imports:                              Imports:
  VCRUNTIME140.dll                      msvcrt.dll
    __C_specific_handler                  __getmainargs, __set_app_type
    memcpy, __current_exception           _onexit, __p__fmode
  api-ms-win-crt-stdio-l1-1-0.dll       KERNEL32.dll
    __stdio_common_vfprintf           Secciones: .text .data .rdata
  KERNEL32.dll                                 .eh_frame  ← GCC
Secciones: .text .rdata .data         Símbolos: __mingw_*, ___chkstk_ms
  (sin .eh_frame)
```

Con solo mirar imports y secciones ya decidís: Rich header + `api-ms-win-crt-*` + `__C_specific_handler` ⇒ **MSVC**; `msvcrt.dll` + `__mingw_*` + `.eh_frame` y sin Rich header ⇒ **MinGW** (recordá la salvedad de arriba: es indicio, no prueba).

Práctica

En tu VM, con **programas propios**:

1. Compilá el **mismo** `hola.c` con **MSVC** (`cl`) y con **MinGW** (`gcc`). Abrí ambos en PE-bear/IDA.
2. Armá una tabla comparando: DLLs de runtime en imports, presencia de Rich header, strings de rutas, secciones (`.eh_frame` vs. no).
3. Si tenés cómo, compilá algo mínimo en **Delphi** o abrí un binario VB6 de práctica y buscá `MSVBVM60.DLL` o `TForm` / `TObject`.
4. Escribí en una línea, para cada binario, "esto es X porque veo Y".
5. Si tenés Go o Rust instalados, compilá un programa propio mínimo que compare una cadena ingresada con una fija. En el de Go, ubicá `main.main` y la comprobación de pila del inicio; en el de Rust, buscá las rutas `.rs` y mirá si hay Rich header. En ambos, encontrá la cadena fija y anotá dónde está su longitud.

> **Lo que queda fuera**
>
> Reconstruir tipos de Go (interfaces, *maps*, canales) o los genéricos y *traits* de Rust, analizar p-code de VB6 y usar herramientas específicas de Delphi. Esta lección enseña a **reconocer** el lenguaje y a no leerlo con expectativas de C; profundizar cada uno es un tema propio.

Preguntas de repaso

<details>
<summary>¿Por qué conviene identificar el compilador antes de leer el código?</summary>

Porque determina **dónde está la lógica y cómo se ve**: un binario Delphi, uno VB6 p-code y uno MSVC se analizan de formas distintas. Saberlo evita buscar un `main` clásico donde no lo hay.
</details>

<details>
<summary>¿Qué pista distingue MSVC de MinGW de un vistazo?</summary>

MSVC trae **Rich header** y runtime `vcruntime` / `ucrtbase` con `__security_cookie`; MinGW usa `msvcrt.dll`, símbolos `__mingw_*`, `.eh_frame` y **no** tiene Rich header.
</details>

<details>
<summary>Ves import de `MSVBVM60.DLL`. ¿Qué implica y qué riesgo hay?</summary>

Es **Visual Basic 6**. Riesgo: puede ser **p-code** (bytecode interpretado por esa DLL), así que leer el x86 directo no te muestra la lógica real; necesitás herramientas de VB.
</details>

<details>
<summary>¿Cómo reconocés Delphi?</summary>

Strings como `Sysutils` / `System` / `Classes` / `TForm` / `TObject`, símbolos con `@` y la **VMT** al inicio de los objetos; las llamadas no se ven como en C.
</details>

<details>
<summary>Un binario tiene Rich header e importa `vcruntime140.dll`, pero aparecen rutas `src/main.rs`. ¿Qué es?</summary>

Probablemente **Rust** compilado con el target `-msvc`: el linker de Microsoft deja el Rich header y el runtime de C, pero las rutas `.rs` y los mensajes de pánico delatan el lenguaje. Es el ejemplo de por qué una sola pista no alcanza.
</details>

▼ **¿Por qué en un binario de Go sin símbolos podés ver nombres como `main.main`?**

> Porque el runtime necesita la tabla `pclntab` (direcciones → funciones y líneas) para trazas de pila, perfilado y recolección de basura, y quitar los símbolos de depuración no la elimina.

▼ **En Delphi de 32 bits, ¿dónde llegan los primeros argumentos de un método?**

> En `EAX`, `EDX` y `ECX` (convención `register`); `Self` viene en `EAX`. Leerlo como `stdcall` hace que los argumentos parezcan faltar.

▼ **¿Qué parte de esto está respaldada por material de CLS y qué es externo?**

> MFC está observado (cap. 39) y hay **títulos** de Delphi/VB/Java en `Teorias_Numeradas` (no leídos). La tabla de huellas de MSVC/MinGW/Delphi/VB6/Go/Rust y el apartado de lenguajes no-C son **conocimiento externo**.

Puerta de salida

Podés: (1) decir con qué compilador/runtime se hizo un binario y por qué; (2) distinguir MSVC de MinGW por imports/Rich header/secciones; (3) reconocer VB6 (y el riesgo de p-code) y Delphi; (4) justificar cada conclusión con una pista concreta del PE; (5) reconocer un binario de Go o de Rust y explicar qué cambia al leerlo (nombres recuperables, cadenas con longitud, comprobaciones del runtime).

Fuentes de esta lección

- `Reversing desde cero, cap. 39` — MFC como capa sobre la API (huella de framework).
- Teorias_Numeradas · clasificación — títulos de Delphi, Visual Basic/VB6, p-code y Java (conteo aproximado por subcadena del título); material para profundizar, no leído.
- Go (conocimiento externo): paquete debug/gosym y diseño de la tabla de símbolos de Go 1.2 — la tabla `pclntab` y por qué la necesita el runtime; A Quick Guide to Go's Assembler.
- Rust (conocimiento externo): Symbol Mangling y esquema v0; tipo primitivo str.
- Delphi (conocimiento externo verificado en Embarcadero RAD Studio DocWiki): Program Control (convención register), Internal Data Formats (cabecera StrRec en offsets negativos) y Using Resource Files (recursos DFM/TPF0).

Capas de esta lección

Contenido observado en CLS: la huella de MFC del capítulo 39 y los conteos por título de `Teorias_Numeradas`. **Conocimiento externo:** la tabla de huellas (Rich header, `MSVBVM60.DLL`, VMT de Delphi, `.eh_frame` de MinGW, Go y Rust), el caso p-code y las particularidades de lectura de Go, Rust y Delphi. **Recomendación editorial:** mirar imports y strings primero y la práctica comparativa MSVC/MinGW.

Ampliación editorial basada en fuentes externas — Metodología de reversing para binarios en Go (pclntab, strings y ABI) y Rust (mangling v0 y enums)

Dada la proliferación de binarios compilados en **Go** y **Rust** en software de sistema, herramientas defensivas y amenazas modernas, su análisis requiere técnicas específicas que difieren sustancialmente del código compilado en C/C++ clásico:

1. Reversing de Binarios en Go

- **Recuperación de símbolos mediante la tabla `pclntab`:** El compilador de Go (`gc`) enlaza una tabla en la sección `.gopclntab` o `.rodata` denominada *Program Counter Line Table* (Go Runtime: symtab.go). Esta estructura asocia cada dirección virtual con el nombre de su función, archivo fuente `.go` y número de línea. En binarios sin símbolos (*stripped*), `pclntab` **permanece intacta** porque el recolector de basura, el planificador y `runtime.panic` la necesitan para generar trazas de pila.
  Herramientas de recuperación: Plugins como `go_parser` (IDA Pro), el analizador integrado *Go Module Analyzer* (Ghidra 11+) o la utilidad `GoReSym` leen el número mágico al inicio de la tabla (`0xFFFFFFFB` en Go 1.2, `0xFFFFFFFA` en Go 1.16, `0xFFFFFFF0` en Go 1.18+ y `0xFFFFFFF1` en Go 1.20+) y renuevan en pocos segundos los nombres de las 2.000+ funciones (revelando de inmediato funciones críticas como `main.main` o `main.verificarLicencia`).
- **Estructura interna de cadenas (`string`):** Un `string` en Go es un par inmutable `struct { const char* str; ptrdiff_t len; }` (16 bytes en x64). No contiene terminador nulo `0x00` . En desensamblado x64 se reconoce el patrón invariable de carga del puntero seguido de su tamaño en un registro secundario:

```
lea rax, [rip + 0x2A150]   ; Puntero al inicio del buffer en .rodata
mov rbx, 0x18              ; Longitud exacta: 24 bytes (sin terminador nulo)
```

- **Convención de llamadas por registros (Go 1.17+ ABI):** A partir de Go 1.17, las funciones internas pasan argumentos y retornos en 9 registros enteros ordenados: `RAX` , `RBX` , `RCX` , `RDI` , `RSI` , `R8` , `R9` , `R10` , `R11` , pasando a la pila solo los excedentes.
- **Prólogo de comprobación de pila:** En el inicio de casi toda función Go aparece una comprobación con el puntero de la goroutine activa (almacenado en `R14` en x64):

```
cmp rsp, qword ptr [r14 + 0x10]   ; Comprueba límite de pila de la goroutine
jbe runtime.morestack             ; Aloca un segmento de pila nuevo si falta espacio
```

2. Reversing de Binarios en Rust

- **Name Mangling v0 (RFC 2603):** Desde Rust 1.59+, los símbolos usan el esquema estándar iniciado con el prefijo `_R` (Rust RFC 2603). Nombres crudos como `_RNvNtCs1234_7mi_app4auth14verificar_pass` se decodifican de inmediato con la herramienta oficial `rustfilt` a `mi_app::auth::verificar_pass` .
- **Tipos algebraicos `Result<T, E>` y `Option<T>` :** En memoria se compilan como estructuras con un byte discriminante (*tag*):

```
struct Option_u32 {
    uint8_t  tag;   // 0 = None, 1 = Some
    uint32_t valor; // Válido únicamente si tag == 1
};
```

En ensamblador se observa la comprobación del byte discriminante previa a operar con el dato (`cmp byte ptr [rsp+0x18], 1 ; jne fallo`).

- **Manejo de pánicos y cadenas de error:** Cuando una función aborta, Rust invoca a `core::panicking::panic_fmt` pasando una estructura que contiene el nombre del archivo fuente `.rs` y la línea exacta como cadenas ASCII legibles en `.rdata`.

Ampliación editorial basada en fuentes externas — Optimización de nicho (Niche Optimization) en Rust para tipos Option sin byte tag

Al revertir binarios compilados en Rust, es fundamental conocer la **optimización de nicho** (*Niche Optimization* o *Null-Pointer Optimization*) implementada por el compilador (The Rustonomicon: Alternative Representations):

- **El principio del nicho:** En tipos como `Option<u32>`, Rust debe reservar un byte extra como discriminante (*tag*) porque cualquier valor de 32 bits es un número válido. Sin embargo, para tipos que **garantizan que nunca pueden ser nulos** (como referencias `&T`, punteros inteligentes `Box<T>`, punteros a función `fn()` o enteros no nulos `core::num::NonZeroU32`), el compilador **elimina por completo el byte** `tag`.
- **Representación en memoria:**
  - `Some(&T)`: Se representa simplemente por la dirección de memoria válida del puntero (8 bytes en x64).
  - `None`: Se representa con el valor nulo exacto (`0x0000000000000000`).
- **Patrón en desensamblado:** Ante estos tipos, el analista **no debe buscar** un `cmp byte ptr [rsp+offset], 1`. En su lugar, el chequeo de `None` se compila de forma idéntica a una comprobación de puntero nulo en C:

```
test rax, rax     ; Comprueba si el Option<&T> o Option<Box<T>> es nulo
jz es_none        ; Si es cero, equivale a la rama None
; ... continúa en la rama Some(&T) usando RAX directamente como puntero
```

Ampliación editorial basada en fuentes externas — ABI register, cabecera StrRec en offsets negativos y firma TPF0 en Delphi

- **Convención `register` en 32 bits vs. x64 (Embarcadero DocWiki: Program Control):** En Delphi x86 (`DCC32`), los tres primeros argumentos de hasta 32 bits van en `EAX` (1.º, o `Self` en métodos), `EDX` (2.º) y `ECX` (3.º); los restantes se apilan de izquierda a derecha y la función llamada limpia el stack (`ret N`). En Delphi x64 (`DCC64`), el compilador usa la ABI estándar de Microsoft (`RCX` , `RDX` , `R8` , `R9`) *(Hecho confirmado)*.
- **Layout en memoria de cadenas largas (Embarcadero DocWiki: Internal Data Formats):** El puntero de cadena (Delphi 2009+) apunta al primer carácter (`offset 0`), precedido por una cabecera de 12 bytes en desplazamientos negativos: `[ptr - 12]` = `codePage` (2 B), `[ptr - 10]` = `elemSize` (2 B), `[ptr - 8]` = `refCnt` (4 B; `-1` en literales constantes) y `[ptr - 4]` = `length` en caracteres (4 B). Por eso `Length(s)` se traduce como `mov eax, [eax - 4]` *(Hecho confirmado)*.
- **Formularios en `.rsrc` (`TPF0`) (Embarcadero DocWiki: Using Resource Files):** Los formularios VCL/FMX compilados en recursos `RT_RCDATA` comienzan con los 4 bytes ASCII `54 50 46 30` (`"TPF0"`) generados por `TWriter` y asocian cada control con el nombre de su método de evento publicado *(Hecho confirmado)*.

PE fino: los directorios que cuentan la historia

> **Objetivo de la lección**
>
> Leer los **directorios del PE** que la lección 1.4 solo nombra, y entender qué pista da cada uno **antes** de llegar al entry point. **Prerrequisitos:** El formato PE (1.4), TLS (2.11), huella de compilador (2.13).

En 1.4 viste la cabecera, las secciones, los imports y la IAT. Acá vamos a los **otros** `DataDirectory`, que muchas veces resuelven el análisis sin ejecutar nada. Todo esto se lee con **PE-bear**, **CFF Explorer** o el navegador de IDA; el recorrido VA↔RVA↔offset es el del capítulo 05 (`Reversing desde cero, cap. 05`). El contenido de esta lección es **conocimiento externo** salvo donde se cite.

Rich header

Entre el `DOS stub` y la cabecera PE, los linkers de Microsoft dejan el **Rich header**: una lista cifrada (XOR con una clave que está ahí mismo) de qué herramientas y versiones tocaron el binario (`@comp.id`). Qué te dice:

- Es casi una **firma de MSVC**: si está, se armó con el toolchain de Microsoft; su **ausencia sugiere** otro compilador (MinGW u otro) o que lo removieron, pero **no lo prueba** (ver 2.13).
- Las versiones ayudan a **datar** el binario y a agrupar muestras de la misma máquina de compilación.

Recursos (`.rsrc`)

El **directorio de recursos** guarda íconos, menús, *dialogs*, *string tables*, *version info* y, a veces, **blobs** (otro ejecutable, una config, una clave). Qué mirar:

- *Version info*: nombre de producto, compañía, versión. Pistas de origen (y, en malware, mentiras).
- *Dialogs* y *string tables*: los textos de la interfaz, que te llevan a la lógica (como las strings del `.rdata`).
- Un recurso **grande o de tipo raro** (RCDATA) suele ser un payload embebido: a mirar con cuidado.

Imports diferidos (*delay-load*)

Además de la tabla de imports normal, el PE puede tener un directorio de **delay-load**: DLLs que **no** se cargan al inicio sino la **primera vez que se usan**. Consecuencia para el análisis: una dependencia puede **no aparecer** en la lista de imports "a primera vista" y cargarse recién en runtime vía `__delayLoadHelper2`. Si buscás una API y no está en los imports normales, revisá delay-load.

Excepciones y unwind (`.pdata`)

En x64, el manejo de excepciones **no** vive en el stack como el SEH de 32 bits: está en tablas `.pdata` / `.xdata` (ver la rama de vulnerabilidades, cap. 36). Para el análisis, esas tablas sirven para:

- **Delimitar funciones** aunque no haya símbolos (cada entrada cubre un rango de código).
- Entender el prólogo/epílogo (qué registros se salvan), útil al reconstruir marcos.

Debug directory y PDB

El **directorio de debug** puede traer la **ruta del** `.pdb` (p. ej. `D:\proyectos\cosa\Release\cosa.pdb`) y un GUID/age para emparejar símbolos. Qué te da:

- La **ruta** filtra información: nombre de proyecto, usuario, estructura de carpetas del que lo compiló.
- El GUID/age permite **cargar los símbolos** correctos (si los tenés o están en un symbol server), lo que vuelve legible un binario (como el caso con `.pdb` de la serie de estructuras del Curso IDA).

Load Config

El `Load Config` **directory** declara mitigaciones y metadatos que el loader usa:

- Puntero al `security cookie` (el `/GS` que viste en vulnerabilidades).
- Tabla **SEH segura** (SafeSEH) en 32 bits.
- Tablas de **CFG** (`GuardCFFunctionTable`): los destinos válidos de llamadas indirectas.

Leerlo te dice **qué defensas tiene** el binario sin ejecutarlo, y complementa la tabla de mitigaciones de la rama de vulnerabilidades.

Cuando los imports no cuentan todo: resolución dinámica de APIs

La tabla de imports solo lista lo que el programa enlazó **al compilar**. Un programa puede obtener la dirección de cualquier función **en ejecución**, y entonces esa API no aparece en los imports. Es una de las primeras cosas que se encuentran al analizar binarios reales, y hay tres formas típicas, de más visible a menos visible (**conocimiento externo** salvo lo citado):

1. `LoadLibrary` + `GetProcAddress`**.** El programa carga la DLL y pide la dirección por nombre. Señales: los dos imports, cadenas con nombres de API (`"VirtualAlloc"`, `"MessageBoxW"`) y, después, una llamada **indirecta** a través de un registro o una variable (`call rax`, `call [rbp-18h]`). La propia documentación de Microsoft usa este mecanismo para funciones sin biblioteca de importación, como las de `ntdll`.
2. **Recorrer el PEB y la tabla de exports.** Sin llamar a ninguna API: el código lee el PEB (`gs:[60h]` en x64, `fs:[30h]` en x86), sigue `Ldr` hasta la lista `InMemoryOrderModuleList` —una lista doblemente enlazada de entradas con la base y el nombre de cada módulo cargado— y, en el módulo elegido, recorre el **directorio de exports** del PE (nombres, ordinales y direcciones) hasta encontrar la función. Señales: accesos a esos offsets fijos, un bucle que compara nombres de módulo y otro que recorre `AddressOfNames`.
3. **Hashing de nombres de API.** Igual que el anterior, pero en vez de comparar el nombre con una cadena, calcula un **hash** de cada nombre exportado y lo compara con una **constante**. No queda ninguna cadena con el nombre de la API en el binario. Señales: un bucle corto que recorre los bytes de un nombre con rotaciones, `xor` o sumas, seguido de un `cmp` contra un valor de 32 bits. Como la misma función de hash produce la misma constante para el mismo nombre, esas constantes se pueden buscar en bases públicas de hashes conocidos para nombrar la API.

Qué registrar en el análisis: qué APIs resuelve dinámicamente (con evidencia: cadena, constante o traza), **dónde** guarda las direcciones (una tabla propia funciona como una IAT paralela) y desde dónde se llaman. En dinámico, un breakpoint en `GetProcAddress` o en el retorno del resolvedor te da la lista real.

Un detalle de lectura relacionado: en binarios modernos vas a ver imports de nombres como `api-ms-win-core-…`. Son **API sets**: nombres de **contratos**, no archivos; el cargador los resuelve contra la DLL que implementa cada función en esa versión de Windows, y puede no existir ningún archivo con ese nombre.

Firma Authenticode: integridad y procedencia

Un PE puede incluir una firma **Authenticode** en la tabla de certificados del directorio de datos; también puede tener firma de catálogo. La firma permite comprobar integridad según las reglas de hash de PE y validar la cadena del certificado, pero **no prueba que el programa sea benigno** ni que su editor sea confiable para cualquier uso. Una firma ausente tampoco demuestra intención maliciosa. La entrada de la tabla de certificados es una excepción importante al modelo general de directorios: su ubicación se expresa como **offset de archivo**, no como RVA.

Al revisar un binario, registrá el resultado de verificación, el sujeto del certificado, la cadena y el sello de tiempo cuando estén disponibles. Interpretá todo como evidencia de identidad e integridad, no como veredicto de seguridad.

Resolución efectiva de DLL

La IAT dice qué módulos y funciones necesita el binario; **no siempre revela qué archivo concreto se cargó**. Si el nombre no especifica ruta, la ubicación efectiva depende del orden de búsqueda de Windows y del contexto: directorio de la aplicación, módulos ya cargados, Known DLLs, API sets, redirecciones y configuración del proceso. Por eso, al analizar un problema de carga, distinguí el nombre importado de la ruta observada en ejecución. En programas propios, usar rutas explícitas y las opciones seguras del cargador reduce ambigüedades; en reversing, documentá la ruta efectiva y el contexto, sin inferirla solo del nombre de la DLL.

Aplicaciones empaquetadas, manifiestos y búsqueda segura

No hay un único orden que puedas aplicar sin identificar el tipo de aplicación y la forma de carga. En apps de escritorio sin paquete, las redirecciones side-by-side declaradas por manifiestos pueden intervenir. En apps empaquetadas participa el **package dependency graph**, cuyas dependencias se buscan según el manifiesto del paquete. También importan la lista de módulos ya cargados, Known DLLs, API sets, redirecciones y si el programa pidió una ruta completa o usó flags de búsqueda.

Para analizarlo, registra si el ejecutable está empaquetado, qué manifiesto y flags de carga observas, el nombre solicitado y el **path real** de la imagen cargada. Confírmalo en un proceso propio con debugger o Process Monitor; no infieras el archivo final por el nombre del import. En software propio, preferí rutas explícitas o LoadLibraryEx con flags de búsqueda seguros y dependencias conocidas.

La capa nativa: de kernel32 a ntdll y la llamada al sistema

Una llamada documentada de Win32 no es la última parada. En los sistemas actuales, buena parte de la implementación de las funciones de `kernel32.dll` vive en `kernelbase.dll` (conocimiento externo), y casi todo lo que necesita al kernel termina en una función **nativa** de `ntdll.dll` con prefijo `Nt` o `Zw`, que hace la **llamada al sistema** y entra en kernel mode. Microsoft documenta que, llamadas desde user mode, las

versiones `Nt` y `Zw` se comportan igual, y que la API nativa es en gran parte interna: funciones como `NtQueryInformationProcess` "pueden cambiar o no estar disponibles" en versiones futuras.

Cómo se ve al analizar:

- En una pila de llamadas, el último frame de user mode suele ser `ntdll!NtAlgo`, no la API que llamó el programa. Para entender una operación de archivo, por ejemplo, la cadena típica es `CreateFileW` → (kernelbase) → `NtCreateFile` → kernel.
- Un *stub* de `ntdll` en x64 es corto: carga el número de servicio en `EAX` y ejecuta `syscall`. Ese número **cambia entre builds** de Windows.
- **Llamadas al sistema directas.** Si encontrás una instrucción `syscall` (o `sysenter` / `int 2Eh` en x86) **dentro del código del propio programa**, fuera de `ntdll`, el programa está saltándose la capa de user mode: no pasa por las funciones de `ntdll` donde un depurador, una herramienta de monitoreo o un producto de seguridad suelen observar. Como los números cambian por build, ese código o los tiene fijos (y queda atado a una versión) o los lee de `ntdll` en ejecución. Es una señal de anti-análisis o evasión para **documentar**, no algo para replicar.

Ejemplo ilustrativo: un resolvedor sin cadenas

*(Listado **ilustrativo** de x64, construido para mostrar el patrón; los offsets siguen las estructuras documentadas `PEB`, `PEB_LDR_DATA` y `LDR_DATA_TABLE_ENTRY`; la constante de hash es inventada.)*

```
mov   rax, gs:[60h]          ; PEB
mov   rax, [rax+18h]         ; PEB->Ldr
mov   rsi, [rax+20h]         ; Ldr->InMemoryOrderModuleList.Flink
siguiente_modulo:
mov   rdi, [rsi+20h]         ; DllBase (offset relativo a InMemoryOrderLinks)
; ... ubicar el directorio de exports de ese módulo ...
siguiente_nombre:
xor   edx, edx               ; hash = 0
hash_loop:
movzx eax, byte ptr [rcx]    ; siguiente carácter del nombre exportado
test  al, al
jz    comparar
ror   edx, 0Dh               ; rotar ...
add   edx, eax               ; ... y sumar: una función de hash simple
inc   rcx
jmp   hash_loop
comparar:
cmp   edx, 1A2B3C4Dh         ; ¿es el hash de la API buscada? (valor ilustrativo)
jne   siguiente_nombre
```

Lectura: no hay `GetProcAddress` ni cadenas con nombres de API; la pista son `gs:[60h]`, la cadena de offsets hacia la lista de módulos, el bucle de hash y la constante. En el informe se anota "resolución dinámica por hash; API identificada como X porque la constante coincide con el hash de su nombre con esta función", con la evidencia de esa coincidencia.

Ejemplo paso a paso: un `Load Config` y un debug directory reales de leer

*(Ejemplo **ilustrativo**: valores construidos; mostrá los campos, no las direcciones exactas.)*

Compilado con `/GS /guard:cf`, PE-bear muestra:

```
Load Config Directory
  Size                          0x0118
  SecurityCookie                0x0040B120   → hay /GS (stack canary)
  SEHandlerTable                0x00000000   → x64: SEH va por .pdata, no acá
  GuardCFCheckFunctionPointer   0x0040B008
  GuardCFFunctionTable          0x0040A800
  GuardFlags                    0x00010500   → CFG activo (INSTRUMENTED)

Debug Directory  (type = CODEVIEW / RSDS)
  PDB: D:\dev\miapp\x64\Release\miapp.pdb
  GUID: 7F3A... age 1
```

Sin ejecutar nada ya sabés: tiene **/GS** y **CFG**, es **x64** (SEH por tablas), y el `.pdb` filtra la ruta de compilación (`D:\dev\miapp\...`: usuario, proyecto, configuración Release). Ese GUID + age es lo que usarías para cargar los símbolos correctos.

Práctica

En tu VM, con un **binario propio** (compilá con /GS /guard:cf en MSVC) y PE-bear:

1. Encontrá el **Rich header** y confirma qué sugiere al compararlo con una compilación propia de MinGW.
2. Abrí el directorio de **recursos** y ubicá version info y, si pusiste uno, un RCDATA.
3. Mirá **Load Config**: ¿apunta a un security cookie? ¿Hay tabla CFG?
4. Mirá el **debug directory**: ¿quedó la ruta del PDB? Anotá qué revela de tu entorno.
5. En un programa propio, obtené una API con LoadLibraryW + GetProcAddress y confirma por qué no figura como import estático.
6. En el depurador, seguí una llamada de tu programa hasta ntdll y registra lo que puedes concluir del stub en tu build, sin generalizar números de servicio entre versiones.
7. Con Get-AuthenticodeSignature (PowerShell) o signtool verify /pa (Windows SDK), registra el estado de firma y los datos del firmante; anota que una firma válida no demuestra que el programa sea benigno.
8. En una aplicación propia, identifica si está empaquetada y qué manifiesto/flags de carga aplican; compara el nombre solicitado de una DLL con la ruta realmente observada en el proceso.

Preguntas de repaso

▼ **¿Qué te dice la presencia (o ausencia) del Rich header?**

> Que el binario se armó (o no) con el toolchain de **MSVC**; además sus `@comp.id` ayudan a datar y agrupar. Su ausencia sugiere MinGW u otro compilador.

▼ **Buscás una API en los imports y no está, pero sabés que el programa la usa. ¿Dónde más mirás?**

> En los **imports diferidos** (delay-load): esa DLL se carga la primera vez que se usa, vía `__delayLoadHelper2`, y no aparece en la tabla de imports normal.

▼ **¿Para qué sirve `.pdata` al analizar un x64 sin símbolos?**

> Para **delimitar funciones** (cada entrada cubre un rango) y entender el prólogo/epílogo; en x64 el unwind vive ahí, no en el stack.

▼ **¿Qué información sensible puede filtrar el debug directory?**

> La **ruta del `.pdb`**: nombre de proyecto, usuario y estructura de carpetas del que compiló; y el GUID/age para emparejar símbolos.

▼ **Un programa usa una API que no figura en sus imports ni en sus cadenas. ¿Qué buscás?**

> Resolución dinámica por **hash**: acceso al PEB (`gs:[60h]` / `fs:[30h]`), recorrido de la lista de módulos y de los exports, un bucle que calcula un hash y un `cmp` contra una constante.

▼ **¿Qué es un import `api-ms-win-core-…`?**

> Un **API set**: el nombre de un contrato que el cargador resuelve contra la DLL que lo implementa; puede no existir un archivo con ese nombre.

▼ **¿Por qué una instrucción `syscall` en el código del propio programa es una señal a documentar?**

> Porque el programa entra al kernel sin pasar por `ntdll`, donde suelen observar depuradores y herramientas; además los números de servicio cambian entre builds.

▼ **¿Qué defensas podés leer en Load Config sin ejecutar el binario?**

> El **security cookie** (`/GS`), la tabla **SafeSEH** (32 bits) y las tablas de **CFG** (destinos válidos de llamadas indirectas).

Puerta de salida

Podés: (1) ubicar y explicar Rich header, recursos, delay-load, `.pdata`, debug/PDB y Load Config y firma Authenticode; (además) explicar por qué la IAT no determina por sí sola la ruta efectiva de una DLL; (2) decir qué pista da cada uno antes del entry point; (3) deducir compilador y defensas de un binario solo leyendo sus directorios; (4) reconocer resolución dinámica de APIs (por nombre, por recorrido del PEB y por hash); (5) ubicar la capa nativa (`ntdll`, `Nt*`, `syscall`) en una pila de llamadas y reconocer una llamada al sistema directa.

Lo que queda fuera

Escribir resolvedores o llamadas al sistema propias, las tablas de números de servicio por versión, el formato completo de `LDR_DATA_TABLE_ENTRY` (la mayoría de sus campos son reservados) y las técnicas para ocultar imports en sí no se desarrollan: la lección enseña a reconocerlas.

Fuentes de esta lección

- Microsoft Learn: Understanding executable file signing, Dynamic-link library search order, Dynamic-link library security.

- `Reversing desde cero, cap. 05` — VA, RVA y offsets: cómo se navega dentro del PE.
- El formato PE (1.4) — base: cabeceras, secciones, imports e IAT, sobre la que se apoya esta lección.
- Curso IDA · tutes 25–28 — uso de `.pdb`/símbolos para volver legible un binario.
- Microsoft Learn: PE Format, GetProcAddress, LoadLibraryW, PEB, PEB_LDR_DATA, Windows API sets, Using Nt and Zw versions of the native system services, NtQueryInformationProcess.

Capas de esta lección

Contenido observado en CLS: el recorrido VA/RVA/offset del capítulo 05, la base de PE de 1.4 y el uso de símbolos del Curso IDA. **Conocimiento externo:** Rich header, directorio de recursos, delay-load, `.pdata`/unwind, debug/PDB, Load Config, resolución dinámica de APIs (incluido el hashing), API sets, Authenticode, búsqueda de DLL en apps empaquetadas y no empaquetadas, resolución efectiva de DLL y la capa nativa. **Ejemplo ilustrativo:** el resolvedor por hash (constante inventada). **Recomendación editorial:** leer los directorios antes del entry point y la práctica con binario propio.

Nota de lectura sobre el listado x64 — Por qué [rsi+20h] apunta a DllBase al recorrer el PEB

En el listado ilustrativo x64 de esta lección, `mov rsi, [rax+20h]` lee `Ldr->InMemoryOrderModuleList.Flink`. Como ese enlace apunta al campo `InMemoryOrderLinks` (ubicado en el offset `+0x10` dentro de `LDR_DATA_TABLE_ENTRY` en x64, o `+0x08` en x86) y no al inicio `+0x00` de la estructura, todos los accesos posteriores con base en `rsi` llevan restado ese desplazamiento.

Ampliación editorial basada en fuentes externas — Tabla completa de offsets de PEB, PEB_LDR_DATA y LDR_DATA_TABLE_ENTRY en x86 y x64

De acuerdo con PEB (winternl.h) y PEB_LDR_DATA (winternl.h) en Microsoft Learn *(Hecho confirmado)*:

Estructura y campo	Offset en x86 (32 bits)	Offset en x64 (64 bits)	Relativo desde InMemoryOrderLinks (x86 / x64)
Puntero al `PEB`	`fs:[0x30]`	`gs:[0x60]`	—
`PEB->Ldr` (`PEB_LDR_DATA*`)	`+0x0C`	`+0x18`	—
`Ldr->InLoadOrderModuleList`	`+0x0C`	`+0x10`	—
`Ldr->InMemoryOrderModuleList`	`+0x14`	`+0x20`	—
`Ldr->InInitializationOrderModuleList`	`+0x1C`	`+0x30`	—
`LDR_DATA_TABLE_ENTRY.InLoadOrderLinks`	`+0x00`	`+0x00`	`-0x08` / `-0x10`
`LDR_DATA_TABLE_ENTRY.InMemoryOrderLinks`	`+0x08`	`+0x10`	`0x00` / `0x00` *(base de RSI)*
`LDR_DATA_TABLE_ENTRY.InInitializationOrderLinks`	`+0x10`	`+0x20`	`+0x08` / `+0x10`
`LDR_DATA_TABLE_ENTRY.DllBase`	**`+0x18`**	**`+0x30`**	**`+0x10` (x86) / `+0x20` (x64)**
`LDR_DATA_TABLE_ENTRY.EntryPoint`	`+0x1C`	`+0x38`	`+0x14` (x86) / `+0x28` (x64)
`LDR_DATA_TABLE_ENTRY.FullDllName` (`UNICODE_STRING`)	`+0x24`	`+0x48`	`+0x1C` (x86) / `+0x38` (x64)
`LDR_DATA_TABLE_ENTRY.BaseDllName` (`UNICODE_STRING`)	`+0x2C`	`+0x58`	`+0x24` (x86) / `+0x48` (x64)

Taxonomía anti-análisis: reconocer, no evadir

> **Objetivo de la lección**
>
> Tener un **mapa** de las técnicas que un binario usa para dificultar su análisis y saber **reconocerlas** cuando aparecen. **Prerrequisitos:** debugger (2.6), PE y TLS (1.4, 2.11), ofuscación (2.10).

Esto es **reconocimiento defensivo**: clasificar y detectar estas técnicas para analizar mejor. No se enseña a **evadirlas** para proteger software comercial; la mayoría de los títulos de `Teorias_Numeradas` que las usan con ese fin quedan fuera del recorrido.

Las cuatro familias

Familia	Qué intenta	Ejemplos de señales
Anti-debug	Detectar que hay un debugger y cambiar de comportamiento o cerrarse	`IsDebuggerPresent`, `PEB.BeingDebugged`, `NtGlobalFlag`, flags del heap, `NtQueryInformationProcess`, chequeos de tiempo (`rdtsc`, `GetTickCount`), detección de breakpoints (`0xCC`, registros `DR`)
Anti-VM / anti-sandbox	Detectar que corre en una máquina virtual o sandbox	Claves de registro de VMware/VBox, direcciones MAC conocidas, drivers/dispositivos, pocos núcleos/poca RAM, nombres de usuario típicos de sandbox
Anti-disassembly	Confundir al desensamblador estático	Instrucciones solapadas, saltos al medio, `jmp` a registro, basura entre bloques (lo viste en 2.10)
Anti-dump	Dificultar volcar el proceso de memoria	Borrar la cabecera PE en memoria, `SizeOfImage` alterado, secciones sin permisos esperados

Qué muestra CLS

El capítulo 22 introduce anti-debug "desde lo más básico": qué es (técnicas para **detectar** un debugger y reaccionar), el comando `hide` de x64dbg para ocultarse, y cómo un programa detecta **breakpoints por software y por hardware** (`Reversing desde cero, cap. 22`).

El capítulo 24 usa una **aplicación propia** que aplica varias técnicas a la vez: al ejecutarla suelta "no encontró debugger", pero si hay un IDA abierto —**aunque no cargue el programa**— lo detecta por el **nombre del proceso/ventana** y muestra "detected by IDA"; lo mismo con `x32dbg` ("detected by x32dbg"). Es el ejemplo perfecto de **detección por entorno**, no solo por breakpoint (`Reversing desde cero, cap. 24`).

Cómo detectarlas al analizar

- **Por imports/strings:** `IsDebuggerPresent`, `CheckRemoteDebuggerPresent`, `NtQueryInformationProcess`, nombres de procesos (`ida`, `x32dbg`, `ollydbg`), claves de VM.
- **Por comportamiento:** el programa "anda distinto" cuando lo abrís con un debugger o dentro de una VM. Ese cambio **es** la pista.

- **Dónde miran temprano:** acordate de los **TLS callbacks** (2.11): un chequeo anti-debug puesto ahí corre antes de `main`.

Existen herramientas que **ocultan** el debugger (el comando `hide` de x64dbg, plugins tipo ScyllaHide). Se mencionan **solo para que las reconozcas** si aparecen; esquivar protecciones queda fuera del marco de esta lección, que es de **reconocimiento y documentación**.

Ejemplo paso a paso: dos formas del mismo chequeo

*(Ejemplo **ilustrativo**: listado representativo de un anti-debug típico.)*

El mismo "¿me están debuggeando?" se ve de dos maneras:

```
; (a) por API — la más fácil de reconocer
call  IsDebuggerPresent          ; kernel32
test  eax, eax
jnz   detectado

; (b) a mano, leyendo el PEB (x64) — sin llamar a ninguna API
mov   rax, gs:[0x60]             ; RAX = puntero al PEB
movzx eax, byte ptr [rax+2]      ; PEB.BeingDebugged (offset 0x02)
test  al, al
jnz   detectado
```

La forma (a) la cazás en los imports (`IsDebuggerPresent`). La (b) **no usa imports**: la pista es el acceso a `gs:[0x60]` (en x86 sería `fs:[0x30]`) y el offset `+2`. Reconocer el patrón del PEB es lo que te permite detectar el anti-debug aunque no haya una API delatora. Lo documentás y seguís; no hace falta evadirlo para entenderlo.

Más variantes del mismo chequeo

Las técnicas cambian de forma, pero cada una deja una **firma** reconocible. Estas tres aparecen a menudo (**conocimiento externo**; lo citado viene de la documentación de Microsoft):

`CheckRemoteDebuggerPresent`. Recibe un handle de proceso y un puntero a un `BOOL`, y escribe `TRUE` si ese proceso está siendo depurado. El "remote" del nombre no significa otra máquina: según Microsoft, el debugger está en **otro proceso**. La señal en el listado es el `call` y la lectura posterior de la variable, no el valor de retorno (que solo indica si la llamada funcionó).

`NtQueryInformationProcess` con `ProcessDebugPort`. Es una función de `ntdll` (la capa nativa de 2.14); Microsoft la documenta como interna y sujeta a cambios, y no tiene biblioteca de importación, así que el programa suele obtenerla con `GetProcAddress`. Con la clase `ProcessDebugPort` (valor 7) devuelve un número de puerto: **distinto de cero** indica que el proceso corre bajo un debugger de modo usuario.

*(Ejemplo **ilustrativo**, x64: la consulta del puerto de depuración.)*

```
; rbx = GetProcAddress(GetModuleHandleW(L"ntdll.dll"), "NtQueryInformationProcess")
call  cs:GetCurrentProcess
mov   rcx, rax                    ; ProcessHandle
mov   edx, 7                      ; ProcessInformationClass = ProcessDebugPort
lea   r8, [rsp+30h]               ; &puerto (DWORD_PTR)
mov   r9d, 8                      ; tamaño del buffer
mov   qword ptr [rsp+20h], 0      ; ReturnLength = NULL
call  rbx
cmp   qword ptr [rsp+30h], 0      ; distinto de 0 -> hay debugger
jnz   detectado
```

Qué delata: la cadena `"NtQueryInformationProcess"` (o su hash, si la resolución está ofuscada), la constante `7` en `EDX` y la comparación del buffer contra cero.

Chequeos de tiempo. Dos lecturas de `rdtsc` (o de `GetTickCount` / `QueryPerformanceCounter`) alrededor de un fragmento corto y una comparación de la diferencia contra un umbral: si paso a paso tardó demasiado, "hay alguien mirando". La firma es el **par de lecturas** con una resta y un `cmp` contra una constante grande.

Anti-VM por `CPUID`. La instrucción `cpuid` con `EAX = 1` devuelve en el bit 31 de `ECX` el indicador de **hipervisor presente**; con `EAX = 0x40000000` un hipervisor devuelve su identificación en `EBX`, `ECX` y `EDX` (en el de Microsoft, los textos `"Micr"`, `"osof"`, `"t Hv"`). Ver `cpuid` seguido de `bt ecx, 31` o de comparaciones contra esas cadenas es una señal anti-VM. **Límite de la señal:** en un Windows con Hyper-V o seguridad basada en virtualización (VBS) activa, el propio sistema corre sobre un hipervisor, así que el bit también aparece en una máquina física. Un programa que lo usa como prueba de "estoy en una VM" se equivoca, y eso también se documenta.

Cómo documentar lo que encontrás

Cada chequeo hallado se registra en una fila, con la misma disciplina de evidencia de la Etapa 4:

Dirección / función	Técnica	Familia	Señal que la delata	Efecto observado	Cómo se confirmó
`0x140001120` (TLS)	`PEB.BeingDebugged`	Anti-debug	`gs:[60h]` + `[rax+2]`	Con debugger muestra "modo seguro"	Breakpoint en el `jnz`, `AL = 1`
`0x1400013A0`	`ProcessDebugPort`	Anti-debug	`GetProcAddress` + `edx = 7`	Igual que la anterior	Valor del buffer con y sin debugger
`0x140001500`	`CPUID` bit 31	Anti-VM	`cpuid` + `bt ecx, 31`	Mensaje "entorno virtual"	Mismo resultado en VM y en host con VBS

*(Filas **ilustrativas**: direcciones y efectos de un programa de práctica hipotético.)*

La columna "cómo se confirmó" es la que distingue una sospecha de un hallazgo: sin ella, la fila es una hipótesis.

Práctica

En tu VM, con un **programa propio** (podés compilar uno que llame a `IsDebuggerPresent` y cambie un mensaje):

1. Hacé una tabla de tres columnas: técnica, familia, cómo la detectaste (import / string / comportamiento).
2. Ejecutá tu binario con y sin debugger y anotá la diferencia.
3. Poné el chequeo en un **TLS callback** y verificá que corre antes de `main`.
4. **Documentá** el cambio de comportamiento: qué muestra el programa con y sin debugger, y qué señal (import, string, lectura del PEB) lo delata. El objetivo es reconocer y dejar registrado, no evadir.
5. Agregá a tu programa `CheckRemoteDebuggerPresent` y la consulta de `ProcessDebugPort` (resolviendo `NtQueryInformationProcess` con `GetProcAddress`). Anotá el valor que devuelve cada una con y sin debugger.
6. Agregá la lectura de `CPUID` (bit 31 y, si está, la identificación del hipervisor) e imprimí el resultado en la VM. Escribí en la tabla de documentación por qué esa señal no prueba por sí sola que el equipo sea virtual.

Preguntas de repaso

▼ **¿Cuál es la diferencia entre anti-debug y anti-disassembly?**

> Anti-debug detecta/estorba la ejecución **dinámica** bajo un debugger; anti-disassembly confunde el análisis **estático** del desensamblador (instrucciones solapadas, saltos al medio). Son familias distintas.

▼ **En el cap. 24, ¿por qué el programa dice "detected by IDA" sin que IDA cargue el binario?**

> Porque detecta por **entorno**: busca el proceso/ventana de IDA abierto en el sistema, no necesita estar siendo debuggeado por él. Es detección por nombre de proceso.

▼ **Nombrá tres señales de anti-debug visibles sin ejecutar.**

> Imports/uso de `IsDebuggerPresent`, `CheckRemoteDebuggerPresent`, `NtQueryInformationProcess`; lectura de `PEB.BeingDebugged` / `NtGlobalFlag`; o strings con nombres de debuggers.

▼ **¿Por qué conviene recordar los TLS callbacks acá?**

> Porque un chequeo anti-debug puesto en un callback corre **antes de** `main`: si solo mirás desde el entry point, ya decidió.

▼ **¿Qué significa un `ProcessDebugPort` distinto de cero y por qué el programa suele obtener la función con `GetProcAddress`?**

> Que el proceso corre bajo un debugger de modo usuario. `NtQueryInformationProcess` es una función interna de `ntdll` sin biblioteca de importación, así que se resuelve en tiempo de ejecución; la cadena con su nombre (o su hash) es una pista.

▼ **¿Por qué el bit de hipervisor de `CPUID` no prueba que el programa corre en una VM?**

> Porque un Windows con Hyper-V o VBS activo corre sobre un hipervisor aunque la máquina sea física, así que el bit también aparece ahí.

▼ **¿Qué columna convierte una fila de la tabla de documentación en un hallazgo?**

> "Cómo se confirmó": el breakpoint, el valor leído o la comparación con y sin debugger. Sin ella, la fila es una hipótesis.

▼ **¿Por qué el cambio de comportamiento con y sin debugger es, en sí, una pista?**

> Porque significa que el programa **chequea** su entorno: si hace algo distinto al correr bajo un debugger (o en una VM), tiene anti-análisis. Ese cambio te dice dónde buscar el chequeo, aunque no lo evites.

Puerta de salida

Podés: (1) clasificar una técnica en anti-debug/anti-VM/anti-disassembly/anti-dump; (2) detectar anti-debug por imports, strings y comportamiento; (3) explicar la detección por entorno del cap. 24; (4) relacionar un chequeo temprano con un TLS callback; (5) reconocer `CheckRemoteDebuggerPresent`, `ProcessDebugPort`, un chequeo de tiempo y un chequeo de `CPUID` por su firma en el listado; (6) registrar cada chequeo en una tabla con su confirmación.

Lo que queda fuera

Cómo neutralizar estos chequeos, los anti-debug de modo kernel y los de protectores comerciales, y las técnicas basadas en excepciones o en manipular hilos (se mencionan como familia, sin detalle). La lección es un mapa para **reconocer y documentar**; no es un catálogo completo.

Fuentes de esta lección

- `Reversing desde cero, cap. 22` — qué es anti-debug, `hide` de x64dbg, detección de breakpoints software y hardware.
- `Reversing desde cero, cap. 24` — app propia con varias técnicas; detección por nombre de proceso ("detected by IDA/x32dbg").
- Teorias_Numeradas · clasificación — títulos de anti-debug (material por título, no leído; muchos con fin de evasión comercial, fuera de alcance).
- Microsoft Learn (conocimiento externo): IsDebuggerPresent, CheckRemoteDebuggerPresent, NtQueryInformationProcess (clase `ProcessDebugPort`, función interna sin biblioteca de importación) y Hypervisor Feature Discovery (bit de hipervisor presente y hoja `0x40000000`).

Capas de esta lección

Contenido observado en CLS: el `hide` y la detección de breakpoints del capítulo 22 y la detección por entorno del 24. **Conocimiento externo:** la taxonomía de las cuatro familias y las señales concretas (PEB, NtGlobalFlag, `ProcessDebugPort`, chequeos de tiempo, `CPUID`, anti-VM, anti-dump). **Ejemplo ilustrativo:** los listados del PEB y de `ProcessDebugPort`, y las filas de la tabla de documentación. **Recomendación editorial:** el enfoque de reconocer y no evadir, la tabla de documentación y la práctica con binario propio.

Ampliación editorial basada en fuentes externas — Evasión en User-Mode: API Hooking de EDRs, Syscalls Directas (Hell's Gate / Halo's Gate) y DLL Unhooking

El análisis de binarios modernos en entornos corporativos exige comprender cómo interactúa el software con los agentes de seguridad (antivirus/EDR) en el espacio de usuario (Microsoft Learn: x64 Architecture & System Calls):

1. Mecanismo de los Inline API Hooks

Los sistemas de seguridad inyectan una DLL en cada proceso para supervisar llamadas sensibles. Para interceptar la ejecución, modifican los primeros bytes de las funciones de `ntdll.dll` reemplazando el prólogo original por un salto incondicional (*inline hook*):

```
; --- ntdll.dll original (Limpia) ---
mov r10, rcx
mov eax, 0x18         ; SSN (System Service Number de NtAllocateVirtualMemory)
test byte ptr [0x7ffe0308], 1
jne short ...
syscall
ret

; --- ntdll.dll hookeada por un EDR ---
jmp qword ptr [rip + 0x10240]   ; Salto (0xE9 o 0xFF 0x25) al motor de análisis del EDR
```

2. Técnicas de Evasión Reversibles

- **Syscalls Directas (Direct Syscalls):** El programa no invoca a `ntdll.dll`; en su lugar, define un stub propio en ensamblador con la instrucción `syscall` y el número SSN correspondiente.
- **Algoritmos Hell's Gate y Halo's Gate:** Como los números SSN cambian entre versiones y compilaciones de Windows:
  - *Hell's Gate (2020):* Parsea la tabla de exportaciones de `ntdll.dll` en memoria, localiza la función deseada y lee los bytes `4C 8B D1 B8 [SSN]`. Si la función tiene un `jmp` (está hookeada), falla.
  - *Halo's Gate:* Supera la limitación inspeccionando las funciones vecinas inmediatamente adyacentes (offsets ±32 bytes). Si la función vecina no está hookeada, lee su SSN y le suma o resta el desplazamiento para deducir el SSN de la función bloqueada.
- **DLL Unhooking:** El binario abre `C:\Windows\System32\ntdll.dll` directamente desde disco con `CreateFileW` + `MapViewOfFile`, localiza la sección `.text` limpia, cambia los permisos de la copia en memoria con `VirtualProtect(PAGE_EXECUTE_READWRITE)` y sobrescribe los hooks con `memcpy`, restaurando el prólogo original de todas las APIs.

3. Criterio de Detección Defensiva

La presencia de instrucciones `syscall` fuera del rango de memoria virtual asignado a `ntdll.dll` o `win32u.dll` constituye una anomalía heurística de máxima prioridad que delata intentos deliberados de evasión de telemetría.

Ampliación editorial basada en fuentes externas — De Direct Syscalls a Indirect Syscalls y Call Stack Spoofing

Aunque las *Direct Syscalls* (Hell's Gate / Halo's Gate) permiten evitar los hooks de `ntdll.dll`, los agentes de seguridad modernos (EDRs) y el propio kernel de Windows implementan contramedidas que detectan esta técnica (Microsoft Learn: x64 Architecture & System Calls y telemetría ETWti):

1. La Limitación de las Syscalls Directas (Inspección de RIP)

Cuando la CPU ejecuta la instrucción `syscall` en Ring 3, el procesador transfiere el control al manejador del kernel (`KiSystemCall64`). Durante esta transición, el kernel captura el registro `RIP` de origen en la estructura del hilo (*trap frame*). Mediante telemetría de kernel (como **ETWti** —*Microsoft-Windows-Threat-Intelligence*— o *ProcessInstrumentationCallback*):

- Si el `RIP` ejecutor reside en la sección `.text` de `ntdll.dll` o `win32u.dll`, la llamada se considera conforme al diseño del sistema.
- Si el `RIP` apunta a la sección `.text` del propio ejecutable anfitrión o a memoria privada (`MEM_PRIVATE`), se genera una alerta inmediata por ejecución de llamadas al sistema anómalas.

2. La Solución: Indirect Syscalls

Para satisfacer la validación del `RIP` sin pasar por los hooks del EDR, el cargador prepara los registros de la llamada en su propio código, pero en lugar de emitir el opcode `0F 05` (`syscall`), realiza un salto (`jmp`) a una instrucción `syscall ; ret` legítima ya existente dentro de `ntdll.dll` :

```
; --- Stub en el binario (Indirect Syscall) ---
mov r10, rcx
mov eax, 0x18                     ; SSN deducido dinámicamente con Halo's Gate
jmp qword ptr [rip + pfn_syscall]  ; Salta a la instrucción "syscall; ret" dentro de ntdll
```

Al ejecutarse el `syscall` físicamente dentro del espacio de memoria de `ntdll.dll`, el `RIP` registrado en el kernel resulta 100 % legítimo.

3. El Problema del Call Stack Walking y Call Stack Spoofing

Aun con *Indirect Syscalls*, los EDRs avanzados inspeccionan la pila de llamadas retrospectiva (*Call Stack Unwinding* guiado por las tablas `.pdata` / `UNWIND_INFO` vistas en la lección 2.14). Si el frame inmediatamente anterior a `ntdll!NtAllocateVirtualMemory` no es una API de alto nivel (como `kernelbase.dll!VirtualAllocEx`), sino memoria no respaldada o un salto anómalo, la llamada es detectada. Para evadir esto, los cargadores avanzados implementan **Call Stack Spoofing** (sintetizando marcos de pila falsos para simular que la llamada se originó en una secuencia legítima de Windows).

Nota sobre Tartarus' Gate: Algunos EDRs modernos colocan su hook `jmp` no en el byte `+0`, sino después de `mov r10, rcx` (en el byte `+3`). La variante *Tartarus' Gate* verifica si el hook está desplazado y ajusta el puntero de lectura para no confundir el prólogo.

Ampliación editorial basada en fuentes externas — PEB.NtGlobalFlag en x86 vs. x64 y clases 0x1E / 0x1F de NtQueryInformationProcess

Según GFlags Flag Table y NtQueryInformationProcess en Microsoft Learn *(Hecho confirmado)*:

- **`PEB.NtGlobalFlag` (valor `0x70`):** Combina `FLG_HEAP_ENABLE_TAIL_CHECK (0x10) | FLG_HEAP_ENABLE_FREE_CHECK (0x20) | FLG_HEAP_VALIDATE_PARAMETERS (0x40) = 0x70`. En **x86** se ubica en `fs:[30h] -> [peb + 0x68]`; en **x64** se desplaza a `gs:[60h] -> [peb + 0xBC]`.
- **`ProcessDebugObjectHandle` (clase `0x1E = 30`):** Consulta el handle al `DEBUG_OBJECT` del kernel. Si no hay debugger, `NtQueryInformationProcess` devuelve `0xC0000353` (`STATUS_PORT_NOT_SET`).
- **`ProcessDebugFlags` (clase `0x1F = 31`):** Consulta `NoDebugInherit`; devuelve `0` si hay depurador activo y `1` si no está siendo depurado.

Ghidra (y Cutter): la alternativa libre, con práctica

> **Objetivo de la lección**
>
> Hacer en **Ghidra** el mismo recorrido que hiciste en IDA, para no depender de una sola herramienta: importar, dejar que autoanalice, seguir strings y referencias, corregir tipos, leer el decompilador y documentar. **Prerrequisitos:** IDA y el análisis estático (2.7), estructuras (2.7).

> **Herramienta externa al corpus**
>
> El curso CLS enseña con IDA, OllyDbg y x64dbg; **Ghidra y radare2 no son material de CLS**, así que esta lección es **conocimiento externo**. En el corpus hay, sin revisar, una copia de **Ghidra 11.3.2** y una carpeta **RADARE** (en `CLS_fuentes_externas/links_chat` y `.../RADARE`); igual conviene **descargar Ghidra del sitio oficial** y verificar el hash antes de usarlo.

Por qué una segunda herramienta

Ghidra es gratis y open source (NSA), trae **decompilador** integrado y corre en cualquier sistema. Saber leer el mismo binario en IDA **y** en Ghidra te da dos ventajas: cuando una decompila mal una función, la otra suele aclararla; y no quedás atado a una licencia. Cutter (la GUI de **radare2**) es otra opción libre, más liviana.

El recorrido equivalente al de IDA

1. **Proyecto e import.** Ghidra trabaja con un *project* que contiene uno o varios binarios. `File → Import File`, elegí el `.exe`; Ghidra detecta el formato PE y el procesador.
2. **Autoanálisis.** Al abrirlo por primera vez, ofrece correr los *analyzers*: dejalos con la opción por defecto la primera vez. Son el equivalente al análisis inicial de IDA (arma funciones, resuelve referencias, nombra imports).
3. **Navegación.** Las ventanas clave: **Symbol Tree** (funciones, imports, exports), **Defined Strings** (el equivalente a las strings de IDA), **Listing** (el desensamblado) y **Decompile** (el pseudo-C, como Hex-Rays).
4. **Seguir el hilo.** Doble clic en una string te lleva a su uso; clic derecho → **References** → **Show References to** es el `X` (xrefs) de IDA. Con eso llegás a la lógica igual que en 2.7.
5. **Corregir tipos.** En el decompilador, clic derecho → **Retype Variable** y **Rename Variable/Function**; para estructuras, el **Data Type Manager**. Es el `T`/renombrado de IDA: cada tipo que corregís hace el pseudo-C más legible.
6. **Documentar.** Comentarios (`;`), *bookmarks* y renombrado quedan en el proyecto. Exportá un resumen de hallazgos para tu informe (Etapa 4).

Tipos y estructuras: donde Ghidra más mejora el pseudo-C

El decompilador es tan bueno como los tipos que le das. Tres herramientas cubren casi todo (**conocimiento externo**, guías de Ghidra):

- **Data Type Manager.** Ventana con los tipos del programa y los **archivos de tipos** (`.gdt`) que trae Ghidra; para un PE aplica automáticamente los de Windows, por eso `CreateFileW` ya aparece con sus parámetros. Ahí creás estructuras a mano (*New → Structure*) con los campos que dedujiste, como en IDA y el análisis estático.
- **Auto Create Structure.** En el decompilador, clic derecho sobre una variable que es un puntero → *Auto Create Structure*: Ghidra mira los accesos `[ptr+N]` de la función y arma una estructura con un campo por offset. Es un punto de partida, no el resultado: renombrá los campos y corregí tamaños cuando sepas qué son.
- **Parse C Source.** Si tenés el encabezado `.h` de tu propio programa (o la definición de una estructura documentada), *File → Parse C Source* lo convierte en tipos de Ghidra. Es lo que más rápido transforma un listado de `param_1 + 0x18` en `ctx->contador`.

Regla de trabajo: cada vez que el pseudo-C muestra aritmética de punteros sobre la misma variable, hay una estructura sin declarar.

Scripting y modo headless

Ghidra expone toda su API para scripts en **Java** y **Python** (Jython en el *Script Manager* y, en versiones recientes, también CPython mediante PyGhidra). Es el equivalente a IDAPython de la Etapa 5. Dentro de un script, `currentProgram` es el programa abierto y hay funciones de conveniencia de la API "plana" (*FlatProgramAPI*), como `getReferencesTo(direccion)`.

*(Ejemplo **ilustrativo**, no ejecutado en esta edición: listar las veinte funciones sin nombre más grandes, para decidir por dónde empezar.)*

```
# Ghidra, Script Manager (Python): funciones FUN_ ordenadas por tamaño
fm = currentProgram.getFunctionManager()
sin_nombre = [f for f in fm.getFunctions(True) if f.getName().startswith("FUN_")]
sin_nombre.sort(key=lambda f: f.getBody().getNumAddresses(), reverse=True)
for f in sin_nombre[:20]:
    print(f.getEntryPoint(), f.getName(), f.getBody().getNumAddresses())
```

El **modo headless** corre el autoanálisis y los scripts sin interfaz, para procesar varios binarios en lote o repetir un análisis exactamente igual:

```
analyzeHeadless C:\proyectos_ghidra LabCLS -import hola.exe -postScript listar_fun.py
```

Eso deja el resultado del script en la salida de consola y el binario analizado en el proyecto, que es evidencia reproducible.

Version Tracking: llevar el trabajo de una versión a otra

Si ya renombraste y comentaste la versión 1 de un programa y aparece la versión 2, no hace falta empezar de cero. **Version Tracking** compara dos programas (origen y destino), busca funciones y datos que se corresponden con distintos **correlacionadores** (bytes idénticos, mismo código con otras direcciones, mismas referencias, mismos símbolos) y te deja **aceptar** cada coincidencia y **trasladar** al destino nombres,

comentarios y tipos. Lo que queda sin correspondencia es justamente lo que cambió. Se usa a fondo en 2.20 Patch diffing; acá alcanza con saber que existe.

Ejemplo paso a paso: la misma función en IDA y en Ghidra

*(Ejemplo **ilustrativo**: pseudo-C representativo de los dos decompiladores.)*

La misma rutina de chequeo, decompilada por cada herramienta antes de renombrar:

```
// IDA (Hex-Rays)                        // Ghidra
int sub_401050(char *a1) {               undefined4 FUN_00401050(char *param_1) {
  if ( strlen(a1) != 6 )                   size_t sVar1 = strlen(param_1);
    return 0;                              if (sVar1 != 6) return 0;
  return strcmp(a1, "s3cr3t") == 0;        iVar2 = strcmp(param_1,"s3cr3t");
}                                          return (uint)(iVar2 == 0);
                                         }
```

Dicen lo mismo con otra prosa: la longitud debe ser 6 y el texto `"s3cr3t"`. Dónde **conviene comparar**: a veces Ghidra resuelve mejor una `switch` o una estructura que Hex-Rays enreda (y al revés). Después de **renombrar** (`FUN_00401050` → `check`, `param_1` → `clave`) y **tipar**, el pseudo-C de Ghidra queda tan legible como el de IDA:

```
bool check(char *clave) {
  if (strlen(clave) != 6) return false;
  return strcmp(clave, "s3cr3t") == 0;
}
```

> **`analyzeHeadless` en Windows: ruta, `-scriptPath` y `-overwrite`**
>
> El ejecutable es `analyzeHeadless.bat`, dentro de `<ghidra>\support\` (no está en el PATH). Si tu script no está en los directorios internos de Ghidra ni en `%USERPROFILE%\ghidra_scripts`, pasá `-scriptPath` o falla con *Script not found*. En Ghidra 11.x conviene ponerle al `.py` la cabecera de runtime (`#@runtime Jython` o `#@runtime PyGhidra`). Y si reimportás el mismo binario al mismo proyecto (p. ej. al repetir la práctica), agregá `-overwrite` (o `-process`) o choca con el archivo ya existente:
>
> ```
> C:\ghidra_11.3.2\support\analyzeHeadless.bat C:\proyectos_ghidra LabCLS ^
>   -import C:\lab\hola.exe -overwrite -scriptPath C:\lab\scripts -postScript listar_fun.py
> ```

Práctica

En tu VM, con el **mismo crackme** que analizaste en IDA (ver Banco de prácticas):

1. Importalo en un proyecto de Ghidra y corré el autoanálisis.

2. Encontrá la misma string clave que usaste en IDA y llegá, por referencias, a la función de decisión.
3. Leé la función en el **decompilador**; renombrá dos variables y confirmá que el pseudo-C mejora.
4. Compará lado a lado con la vista de IDA: ¿coincide el decompilado? ¿Alguna lo hace más claro?
5. (Opcional) Abrí el mismo binario en **Cutter** y repetí el paso 2.
6. Elegí una función de tu programa que reciba un puntero a estructura. Usá *Auto Create Structure*, renombrá los campos y compará el pseudo-C antes y después; si tenés el `.h`, importalo con *Parse C Source* y compará con la estructura automática.
7. Corré el script de ejemplo desde el *Script Manager* y después en modo headless sobre el mismo binario. Guardá la salida como evidencia.

Preguntas de repaso

▼ **¿Qué ventaja concreta da leer el mismo binario en IDA y en Ghidra?**

> Que cuando una **decompila mal** una función, la otra suele aclararla; además no dependés de una licencia. Son dos lecturas del mismo código.

▼ **¿Cuál es el equivalente en Ghidra de las "strings" y de los "xrefs" de IDA?**

> **Defined Strings** para las cadenas y **References → Show References to** (clic derecho) para los xrefs.

▼ **¿Qué hacés la primera vez que abrís un binario en Ghidra?**

> Dejar correr el **autoanálisis** con la opción por defecto: arma funciones, resuelve referencias y nombra imports, como el análisis inicial de IDA.

▼ **¿Cómo mejorás el decompilado de una función?**

> **Renombrando** variables/funciones y **corrigiendo tipos** (Retype Variable, Data Type Manager); cada corrección hace el pseudo-C más legible, igual que `T`/rename en IDA.

▼ **¿Qué hace *Auto Create Structure* y por qué no es el resultado final?**

> Crea una estructura con un campo por cada offset que la función usa sobre ese puntero. Es un borrador: no conoce los nombres ni el tamaño total, y puede faltarle campos que solo se usan en otras funciones.

▼ **¿Para qué sirve el modo headless?**

> Para correr el autoanálisis y scripts sin interfaz: procesar varios binarios en lote o repetir el mismo análisis exactamente igual, lo que lo vuelve reproducible.

▼ **¿Qué te ahorra Version Tracking cuando aparece una versión nueva del programa?**

> Rehacer el trabajo: encuentra las funciones correspondientes y traslada nombres, comentarios y tipos; lo que queda sin correspondencia es lo que cambió.

▼ **¿Por qué conviene bajar Ghidra del sitio oficial aunque haya una copia en el corpus?**

> Porque la copia del corpus **no está revisada**; del sitio oficial, verificando el hash, tenés garantía de integridad. Es la misma precaución que con cualquier herramienta.

Puerta de salida

Podés: (1) importar y autoanalizar un binario en Ghidra; (2) llegar a la función de decisión por strings y referencias; (3) mejorar el decompilado renombrando y tipando; (4) comparar la lectura de IDA y Ghidra del mismo binario; (5) crear y aplicar una estructura con el Data Type Manager o *Auto Create Structure*; (6) correr un script propio desde el *Script Manager* y en modo headless. Con esto cerrás el bloque **2B · Windows intermedio**.

Lo que queda fuera

Escribir analizadores o extensiones de Ghidra, el lenguaje de especificación de procesadores (SLEIGH), el trabajo colaborativo con Ghidra Server y la depuración integrada. Son temas de uso avanzado de la herramienta, no de método.

Fuentes de esta lección

- **Conocimiento externo:** Ghidra (proyecto NSA) y su decompilador; recorrido equivalente al de IDA. Guías oficiales: Ghidra class · Beginner e Intermediate.
- En el corpus (sin revisar): copia de **Ghidra 11.3.2** y carpeta **RADARE** en `CLS_fuentes_externas`. No se usaron como fuente; se mencionan por completitud.
- IDA y el análisis estático (2.7) — el recorrido que esta lección replica en otra herramienta.
- Ghidra (conocimiento externo): Improving Disassembly and Decompilation (presentación en PDF) (tipos y estructuras en el decompilador), Version Tracking y API de Ghidra (`FlatProgramAPI`, `FunctionManager`).

Capas de esta lección

Contenido observado en CLS: solo la existencia de Ghidra 11.3.2 y la carpeta RADARE en el corpus (sin revisar). **Conocimiento externo:** todo el manejo de Ghidra/Cutter (tipos, scripting, headless, Version Tracking) y el paralelo con IDA. **Ejemplo ilustrativo:** el pseudo-C comparado, el script de funciones sin nombre y la línea de `analyzeHeadless` (no ejecutados en esta edición). **Recomendación editorial:** el recorrido paso a paso, la comparación lado a lado y la práctica con el mismo crackme.

Packers didácticos: UPX, stub, OEP, dump e IAT

Objetivo de la lección

Entender qué hace un **packer** y recorrer el desempaquetado **didáctico** de UPX: reconocer el stub, llegar al **OEP**, **dumpear** el proceso y reconstruir la **IAT**. **Prerrequisitos:** PE, secciones e IAT (1.4), PE fino (2.14), debugger (2.6).

Solo UPX didáctico, sobre binarios propios

Esta lección usa **UPX** —un compresor **abierto y reversible**— sobre **programas propios**. UPX trae su propio `upx -d` que descomprime, así que siempre tenés una red de seguridad. **No** se cubre el desempaquetado de **protectores comerciales** (Themida, ACProtect, ASProtect, VMProtect…): eso queda fuera del recorrido, como la Etapa 5 excluida. El objetivo es **entender el mecanismo**, no vulnerar protecciones de terceros.

Qué hace un packer

Un packer toma tu `.exe`, **comprime** (o cifra) el código original y le pone adelante un **stub**: un pequeño programa que, al ejecutarse, **descomprime** el original en memoria y le salta. En disco ves poco; el código real aparece recién en runtime. Señales de UPX en el PE (lo viste en 1.4 y 2.14):

- Secciones llamadas `UPX0` (vacía en disco, grande en memoria: ahí se descomprime) y `UPX1` (el stub + datos comprimidos).
- **Pocos imports** (el stub resuelve el resto en runtime).
- El **entry point** cae en `UPX1`, no en el código original.

El **OEP** (*Original Entry Point*) es la dirección donde empezaba tu programa antes de empacar. Desempaquetar didácticamente es: **llegar al OEP**, **dumpear** la memoria ya descomprimida y **reconstruir la IAT** para que el dump vuelva a ser un `.exe` que corre.

El método (sobre un binario propio)

El capítulo 13 lo plantea como práctica de unpacking **simple**: los ejemplos no tienen IAT redirection ni stolen bytes/code, y su anti-debug es tan básico que alcanza el comando `hide` de x64dbg (`Reversing desde cero, cap. 13`).

1. **Reconocé el packer.** Secciones `UPX0` / `UPX1`, entry en `UPX1`.
2. **Llegá al OEP.** El stub termina restaurando registros y **saltando** al OEP. En UPX el patrón clásico es un `popad` seguido de un salto lejano (`jmp`) a una dirección en la zona que estaba vacía (`UPX0`). Un breakpoint de hardware sobre el stack o seguir hasta ese `jmp` te deja en el OEP.
3. **Dumpeá** el proceso cuando estás en el OEP (con Scylla, o el script que muestra el curso).
4. **Reconstruí la IAT.** El tute 15 del Curso IDA sigue justo acá: después del OEP, **dumpear y reconstruir la IAT** para que el ejecutable vuelva a resolver sus imports (`Curso IDA, tute 15`).

5. **Comprobá con `upx -d`.** Como es UPX, descomprimí el mismo binario con `upx -d` y comparás tu dump con el original: así validás que llegaste bien.

Ejemplo paso a paso: el salto al OEP

*(Ejemplo **ilustrativo** y **de 32 bits (PE32)**: `popad` / `pushad` existen solo en x86, así que este patrón es el de un UPX sobre un binario de 32 bits; en x64 el stub se ve distinto. Direcciones construidas para mostrar el patrón.)*

```
; ...final del stub de UPX, en la sección UPX1...
0045A1F0  popad                     ; restaura los registros que guardó al inicio (pushad)
0045A1F1  jmp     0x00401000        ; salto lejano a UPX0 → ese es el OEP
;--- en 0x00401000 ya estás en el código ORIGINAL ---
00401000  push    ebp
00401001  mov     ebp, esp          ; prólogo típico del main
```

El `popad` + `jmp` fuera de `UPX1` (hacia la zona `UPX0`, antes vacía) es la firma de "acá terminó de descomprimir y salta a tu programa". Parás en `0x00401000`, dumpeás y reconstruís la IAT.

Y en 64 bits, ¿dónde está el `popad`?

No está: `pushad` / `popad` no existen en x64. El stub de un UPX de 64 bits guarda y restaura los registros que necesita con varios `push` / `pop` sueltos, y la firma del final cambia. Lo que **no** cambia es la idea: en algún momento el stub termina de escribir el código original y **salta a una dirección dentro de la región que estaba vacía**. Por eso el método general no busca una instrucción concreta, sino ese salto (**conocimiento externo**):

- **Por el destino:** ubicá en el *memory map* la región que corresponde a `UPX0` y buscá, cerca del final del stub, un `jmp` cuyo destino caiga ahí.
- **Por la ejecución:** poné un **breakpoint de memoria en ejecución** sobre esa región. No salta mientras el stub escribe en ella; salta cuando el procesador ejecuta la primera instrucción ahí, que es el OEP.

La segunda forma funciona igual en 32 y 64 bits, y es la que conviene entender, porque no depende del packer.

Señales genéricas de empaquetado

UPX se reconoce por el nombre de sus secciones, pero un packer cualquiera deja otras huellas que se pueden combinar (**conocimiento externo**; los permisos y constantes están en la documentación de Microsoft):

Señal	Dónde se ve	Por qué sugiere empaquetado
Entropía alta de una sección (cerca de 8 bits por byte)	PE-bear, Detect It Easy o un script	Los datos comprimidos o cifrados parecen aleatorios; el código normal tiene entropía bastante menor
Tamaño **en disco** chico y tamaño **virtual** grande	Tabla de secciones	La sección se llena en memoria al descomprimir
Sección **escribible y ejecutable** a la vez	Flags de la sección (`MEM_WRITE` + `MEM_EXECUTE`)	El stub escribe código y después lo ejecuta en el mismo lugar
Pocos imports, con `LoadLibrary` / `GetProcAddress`	Tabla de imports	Los demás se resuelven en ejecución (2.14)
Entry point fuera de la primera sección de código	Cabecera opcional	El primer código que corre es el stub
En ejecución: `VirtualAlloc` / `VirtualProtect` que dejan memoria ejecutable, escritura en ella y salto	Breakpoints en esas APIs	Es el desempaquetado en vivo

Ninguna alcanza sola: un instalador con datos comprimidos tiene entropía alta sin ser un packer, y un programa legítimo puede cambiar permisos de memoria. Como con la huella de compilador (2.13), se suman señales.

*(Ejemplo **ilustrativo**: cálculo de entropía de Shannon de un bloque de bytes, para entender qué muestran las herramientas.)*

```
import math
from collections import Counter

def entropia(datos: bytes) -> float:
    # bits por byte: 0 si todos los bytes son iguales, 8 si son uniformes
    if not datos:
        return 0.0
    n = len(datos)
    return -sum(c / n * math.log2(c / n) for c in Counter(datos).values())
```

Por qué un dump puede no funcionar

Llegar al OEP y dumpear no garantiza un ejecutable que corra. Las causas típicas, de la más común a la menos (**conocimiento externo**):

- **IAT sin reconstruir:** el dump tiene en la IAT direcciones absolutas de esta ejecución, no una tabla de imports que el loader pueda resolver.
- **Cabeceras de secciones sin ajustar:** en memoria cada sección está en su dirección virtual; si el dumper no reescribe los offsets en disco para que coincidan, el archivo queda desalineado.
- **Base distinta:** si el proceso cargó en otra base por ASLR y el dump no conserva o ajusta las relocalizaciones, las direcciones absolutas del código no coinciden.
- **Estado ya modificado:** si dumpeás después del OEP, las variables globales ya pueden tener valores de esta ejecución.

- **Código que corre antes:** TLS callbacks (2.11) o anti-dump (2.15).

Para **analizar** no siempre hace falta que el dump corra: abierto en IDA o Ghidra, con la base correcta, ya muestra el código descomprimido. Que corra es la prueba de que reconstruiste bien, no un requisito para leerlo.

> **El dump de Scylla crashea al ejecutarlo: es el ASLR, no la IAT**
>
> Al volcar un proceso que cargó con **ASLR activo**, las direcciones absolutas ya quedaron reubicadas a la base de *esa* ejecución, pero el dump conserva el flag `DYNAMIC_BASE (0x0040)` y Windows intenta re-reubicarlo con una `.reloc` ya consumida → `0xC0000005` al hacer doble clic. No reconstruiste mal la IAT: es el ASLR. Solución: en Scylla activá **Update header ImageBase** (que el `ImageBase` del dump sea la base real del volcado) y en PE-bear **desmarcá Dynamic Base** (o poné `Relocations stripped` `0x0001` en File Header). Así el loader lo carga exactamente en la base para la que quedaron fijadas sus direcciones.

Práctica

En tu VM, con un **programa propio** (necesitás UPX, gratis y open source; versiones de referencia: UPX 4.x, x64dbg, Scylla):

1. Compilá un `hola.exe` propio **de 32 bits (x86)** —para que valga el patrón `popad` de arriba— y guardá su hash.
2. Comprimilo: `upx hola.exe`. Mirá en PE-bear las secciones `UPX0` / `UPX1` y cómo cambió el entry point y los imports.
3. En x64dbg (x32dbg), llegá al OEP siguiendo el `popad` / `jmp`. Anotá el OEP como **RVA** (dirección − base de imagen), no como VA absoluta.
4. Dumpeá y reconstruí la IAT (Scylla).
5. **Validá**: `upx -d hola.exe` recupera el original; comparalo con tu dump por **RVA del OEP y comportamiento** (no por la VA absoluta, que puede diferir según dónde cargue). ¿Coinciden el punto de entrada y los imports resueltos?
6. Repetí con un `hola.exe` propio **de 64 bits**: llegá al OEP con un breakpoint de memoria en ejecución sobre la región de `UPX0` y anotá qué instrucción del stub hizo el salto.
7. Calculá la entropía de cada sección del original y del comprimido (con una herramienta o con la función de arriba) y armá una tabla con las señales genéricas que muestra cada uno.

Preguntas de repaso

▼ **¿Qué son las secciones `UPX0` y `UPX1`?**

> `UPX1` trae el **stub** y los datos comprimidos; `UPX0` está **vacía en disco** pero reservada en memoria: ahí el stub **descomprime** el código original. El entry point apunta a `UPX1`.

▼ **¿Qué es el OEP y por qué lo buscás?**

> El *Original Entry Point*: la dirección donde arrancaba tu programa antes de empacar. Al llegar ahí, el código ya está **descomprimido** en memoria, listo para dumpear.

▼ **¿Cuál es el patrón clásico para detectar el salto al OEP en UPX?**

> Un `popad` (restaura los registros que guardó el `pushad` inicial del stub) seguido de un `jmp` **lejano** que sale de `UPX1` hacia la zona `UPX0`. Ese salto cae en el OEP.

▼ **¿Por qué hay que reconstruir la IAT después de dumpear?**

> Porque el stub resolvió los imports en runtime; el dump "crudo" no tiene una tabla de imports válida. Reconstruir la IAT (p. ej. con Scylla) hace que el `.exe` recuperado vuelva a resolver sus funciones.

▼ **En x64 no hay `popad`. ¿Cómo llegás al OEP de un UPX de 64 bits?**

> Buscando el salto a la región que estaba vacía, o con un breakpoint de memoria en ejecución sobre esa región: se dispara en la primera instrucción ejecutada ahí, que es el OEP. Ese método no depende del packer.

▼ **¿Por qué la entropía alta sola no prueba que un binario esté empaquetado?**

> Porque cualquier dato comprimido o cifrado la tiene (un instalador con recursos comprimidos, por ejemplo). Se combina con otras señales: tamaños en disco y virtual, secciones escribibles y ejecutables, pocos imports, entry point en el stub.

▼ **Llegaste al OEP y dumpeaste, pero el dump no arranca. ¿Qué revisás primero?**

> La IAT (¿se reconstruyó?), las cabeceras de secciones (¿coinciden offsets en disco y virtuales?) y la base (¿se conservaron las relocalizaciones?). Para leer el código no hace falta que arranque.

▼ **¿Por qué UPX es seguro para practicar y un protector comercial no entra en esta guía?**

> UPX es **abierto y reversible** (`upx -d` deshace el empaquetado), y se practica sobre **binarios propios**. Desempaquetar protectores comerciales apunta a vulnerar software de terceros: queda fuera del marco educativo/defensivo.

Puerta de salida

Podés: (1) reconocer un binario UPX por sus secciones/entry/imports; (2) explicar qué es el OEP; (3) llegar al OEP por el patrón `popad` / `jmp` y dumpear; (4) decir por qué hay que reconstruir la IAT; (5) validar tu resultado con `upx -d`; (6) llegar al OEP de un UPX de 64 bits sin el patrón `popad`; (7) reconocer un posible empaquetado por señales genéricas y explicar por qué una sola no alcanza; (8) diagnosticar por qué un dump no corre.

Lo que queda fuera

Packers con varias capas, código que se descifra por partes durante la ejecución, redirección de la IAT, *stolen bytes* y, por el límite del curso, todo protector comercial. La lección enseña el mecanismo con un compresor abierto y reversible.

Fuentes de esta lección

- `Reversing desde cero, cap. 13` — unpacking simple: llegar al OEP y dumpear; ejemplos sin IAT redirection/stolen; anti-debug básico (`hide`).
- `Curso IDA, tute 15` — dumpeado y reconstrucción de la IAT después del OEP.
- Curso IDA · tutes 14–18 — serie de unpacking didáctico (UPX, ASPack) con IDA y Scylla.
- Video «10 Unpacking 1 - UPX» del tronco del curso → índice de videos.
- Conocimiento externo: UPX (proyecto oficial); Microsoft Learn: PE Format (flags de sección), VirtualProtect y Memory Protection Constants.

Capas de esta lección

Contenido observado en CLS: el método de OEP del capítulo 13 y la reconstrucción de IAT del tute 15. **Conocimiento externo:** las secciones `UPX0` / `UPX1`, el patrón `popad` / `jmp`, el método por breakpoint de memoria en x64, las señales genéricas (entropía, permisos, tamaños), las causas de un dump fallido y `upx -d`. **Recomendación editorial:** el marco de solo-UPX-didáctico-sobre-binarios-propios y los ejemplos ilustrativos (listado del stub y función de entropía; no hay un laboratorio compilado con hashes reales en esta edición).

Ampliación editorial basada en fuentes externas — Procedimiento paso a paso del breakpoint de hardware en el stack («ESP/RSP trick»)

El paso 2 de esta lección menciona el breakpoint de hardware sobre el stack. Su fundamento combina la propiedad LIFO de la pila con los registros `DR0–DR3` (*Practical Malware Analysis*, Cap. 18; Intel SDM Vol. 3B §17.2) *(Hecho confirmado)*:

1. En el inicio del stub de UPX, ejecutá **una sola vez** `F8` sobre el `pushad` inicial (que guarda los 8 registros en el stack).
2. En el panel de registros de x64dbg, hacé clic derecho en el nuevo valor de `ESP` → *Follow in Dump*.
3. Seleccioná los primeros 4 bytes en el *Dump*, clic derecho → *Breakpoint* → *Hardware, Access* → *Dword*.
4. Presioná `F9` **(Run)**: cuando el stub termine de descomprimir y ejecute `popad` para restaurar esos mismos 4 bytes, el procesador disparará la interrupción de depuración y se detendrá exactamente sobre el `jmp OEP`.

CURSO DE REVERSING · TOMO I

PARTE B

Reversing Avanzado, Ecosistemas y Material de Apoyo

Este volumen continúa el **Bloque 2C (Windows Avanzado)** a partir del análisis profundo de C++ y sus estructuras en memoria (RTTI, vtables, STL y excepciones), cubriendo además las **Etapas 3 (.NET), 4 (Método y Criptografía) y 5 (Malware y Automatización)**, culminando con el **Kit de 5 Plantillas C/C++**, las **Hojas de Referencia Rápida**, el **Glosario A–Z unificado** y las **Preguntas de Repaso**.

> **Entorno activo de laboratorio para la Parte B:**
> VM con Windows x64 aislada, Visual Studio Developer Command Prompt (`cl.exe`), IDA Free o Ghidra 11+, `x64dbg` y `WinDbg Preview` .

C++ profundo: RTTI, vtables, STL y excepciones

> **Objetivo de la lección**
>
> Reconocer en un binario los rasgos de C++ que van más allá de una llamada virtual: **RTTI**, `std::string` (con su optimización SSO), **contenedores STL** y el **manejo de excepciones**.
> **Prerrequisitos:** estructuras y vtables en IDA (2.7), llamadas y marcos (2.5), PE fino (2.14).

En 2.7 ya reconstruiste una **vtable** y leíste una llamada virtual sobre un binario de **32 bits** (`call [edx+8]` = tercer método, porque cada puntero ocupa 4 bytes). El capítulo 33 del curso lo hace con un objeto propio: arma **dos estructuras** —el objeto y su vtable con los punteros a los métodos— y agrega cross-references manuales para las llamadas virtuales (`Reversing desde cero, cap. 33`). Acá vamos a los otros cuatro rasgos. Lo que sigue es **conocimiento externo** salvo donde se cite.

> **Arquitectura y compilador fijados**
>
> Toda esta lección, incluidos el ejemplo y la práctica, se fija a **MSVC x64** (Visual Studio 2022, biblioteca estándar de Microsoft). En x64 cada puntero y cada slot de vtable ocupa **8 bytes**, `this` llega en `RCX` y el RTTI usa **RVA** (desplazamientos de 32 bits desde la base de la imagen) en lugar de punteros absolutos. Cuando algo cambia en x86 se indica aparte. El layout de RTTI, de `std::string` y de los contenedores **no** es parte del estándar de C++: es un detalle de implementación de MSVC que puede variar entre versiones y entre Debug y Release. MinGW/GCC, Clang con libc++ o libstdc++ usan otros layouts y otro RTTI (Itanium ABI).

RTTI (Run-Time Type Information)

Cuando el programa usa `dynamic_cast` o `typeid` (y RTTI no está deshabilitado con `/GR-`), MSVC deja estructuras de **RTTI** colgando de la vtable: **justo antes** del primer método, en `vtable[-1]` (offset `-0x08` en x64, `-0x04` en x86), hay un puntero al *RTTI Complete Object Locator* (COL). El COL lleva al *TypeDescriptor*, que contiene el **nombre decorado** de la clase (por ejemplo, `.?AVMiClase@@`, que se desdecora a `class MiClase`), y a la jerarquía de clases base.

Diferencia importante entre arquitecturas: en **x86**, los campos del COL son punteros absolutos; en **x64**, son **RVA** de 32 bits que hay que sumar a la base de la imagen. Si sumás mal o tratás una RVA como puntero, el TypeDescriptor "no aparece". Para el análisis, RTTI es un **regalo**: te da los nombres reales de las clases sin símbolos. IDA y Ghidra muestran la jerarquía si la reconocen.

`std::string` y la optimización SSO

En MSVC x64 (Release), `std::string` ocupa **32 bytes**: una **unión de 16 bytes** que contiene o bien el **buffer interno** o bien un **puntero** al heap, seguida de **size** (8 bytes) y **capacity** (8 bytes). En x86 los dos contadores son de 4 bytes y el objeto ocupa 24. En Debug el objeto puede ser más grande por el soporte de depuración de iteradores: medilo con `sizeof` en tu build en lugar de suponerlo.

La clave para no confundirse es la **SSO** (*Small String Optimization*): si la cadena entra en 15 caracteres `char` (capacidad 15 más el terminador), se guarda **dentro** del objeto y no hay `new`; si es más larga, los primeros 8 bytes de la unión pasan a ser el puntero al heap. Síntoma en el reversing: para strings cortas no ves una asignación dinámica y el texto aparece **en el stack**, dentro del objeto. Para distinguir los dos casos, mirá `capacity`: si es mayor que 15, la unión contiene un puntero.

Contenedores STL

- `std::vector`: tres punteros — `inicio`, `fin`, `fin_capacidad` — (24 bytes en x64 Release). El tamaño es `(fin - inicio) / sizeof(T)`; la capacidad, `(fin_capacidad - inicio) / sizeof(T)`. Un `push_back` que realoca mueve los tres.
- `std::map` / `std::set`: árbol rojo-negro; vas a ver nodos con punteros a padre/izquierda/derecha, un byte de color y un nodo cabecera.
- En el decompilador, estos patrones se ven como aritmética de punteros repetida sobre la misma base: candidatos a **aplicar una estructura** (como en 2.7).

Excepciones de C++

Un `try/catch` de C++ sobre MSVC se apoya en el manejo de excepciones de Windows (lo viste en vulnerabilidades: SEH en 32 bits, tablas `.pdata` / `.xdata` en 64). Las señales cambian con la arquitectura:

- **x86:** la función registra un marco en la cadena SEH (`fs:[0]`) con un handler que termina en `__CxxFrameHandler3` (u otra variante según la versión).
- **x64:** no hay registro en tiempo de ejecución. La entrada de la función en `.pdata` apunta a su información de desenrollado en `.xdata`, que nombra el handler de C++ (`__CxxFrameHandler3`, o `__CxxFrameHandler4` en versiones recientes de MSVC) y sus tablas. El cuerpo del `catch` suele compilarse como un **funclet** separado.

En ambos casos, `throw` termina en `_CxxThrowException` con una estructura `ThrowInfo` que describe el tipo lanzado. No hace falta reconstruir todo: alcanza con **reconocer** que hay `try/catch` y dónde está el cuerpo del `catch`.

Ejemplo paso a paso: vtable + RTTI (MSVC x64)

*(Ejemplo **ilustrativo**: las direcciones y las RVA están construidas con forma realista para una imagen x64 con base `0x140000000`; no son la salida de un binario compilado en esta edición.)*

```
Base de la imagen: 0x0000000140000000

obj @ 0x000000D51C8FF8A0 (stack):
  +0x00  vptr -> 0x0000000140023A48          ; puntero a la vtable (8 bytes)

vtable @ 0x0000000140023A48:
  -0x08  0x00000001400289F0  -> RTTI Complete Object Locator   ; vtable[-1]
  +0x00  0x0000000140001200  -> MiClase::metodo1               ; slot 0
  +0x08  0x0000000140001250  -> MiClase::metodo2 (virtual)     ; slot 1
```

Siguiendo `vtable[-1]` hasta el nombre de la clase:

```
Complete Object Locator @ 0x00000001400289F0 (campos de 4 bytes):
  +0x00  signature        = 1        ; 1 indica formato x64 con RVA
  +0x04  offset           = 0
  +0x08  cdOffset         = 0
  +0x0C  pTypeDescriptor  = 0x0002F0C8   (RVA)  -> 0x14002F0C8
  +0x10  pClassDescriptor = 0x00028A18   (RVA)
  +0x14  pSelf            = 0x000289F0   (RVA del propio COL)

TypeDescriptor @ 0x0000000140002F0C8:
  +0x00  pVFTable (puntero a la vtable de type_info)
  +0x08  spare
  +0x10  ".?AVMiClase@@"   -> desdecorado: class MiClase
```

Y la llamada virtual que se lee con ese nombre:

```
  mov  rcx, [rsp+30h]        ; rcx = this (obj)
  mov  rax, [rcx]            ; rax = vptr (0x0000000140023A48)
  call qword ptr [rax+8]     ; slot 1 = 0x08 / 8 -> MiClase::metodo2
```

Dos comprobaciones de coherencia que valen para cualquier binario x64: `pSelf` + base debe dar la dirección del propio COL (`0x140000000 + 0x289F0 = 0x1400289F0`), y `pTypeDescriptor` + base debe caer en una zona de datos donde aparezca una cadena que empiece con `.?AV` (clase) o `.?AU` (struct). Si alguna falla, estás leyendo otra cosa.

Con RTTI, `call [rax+8]` deja de ser "el slot 1" y pasa a ser `MiClase::metodo2`: el nombre te lo dio `vtable[-1]`.

Variante x86 (solo para contraste): con MSVC de 32 bits, el mismo objeto tendría el COL en `vtable[-1]` a `-0x04`, el COL usaría punteros absolutos (sin `pSelf`), `this` llegaría en `ECX` (`__thiscall`) y la llamada sería `mov eax,[ecx]` / `call dword ptr [eax+4]`, porque el slot 1 está a 4 bytes. No mezcles offsets de una arquitectura con la otra.

Herencia múltiple y virtual en MSVC

Con herencia simple hay un vptr por objeto. Con **herencia múltiple** aparecen varios, y eso cambia tres lecturas.

Varios vptr y varias vtables. Para `class C : public A, public B` (con `A` y `B` polimórficas), el objeto `C` contiene un subobjeto `A` con su vptr al principio y un subobjeto `B` con **otro** vptr más adelante. `C` tiene entonces dos vtables (en símbolos de MSVC, `C::vftable{for A}` y `C::vftable{for B}`), y **cada una** tiene su propio Complete Object Locator en `vtable[-1]`. El campo `offset` del COL dice a qué distancia del comienzo del objeto completo está **ese** vptr: 0 para el de `A`, distinto de 0 para el de `B`. Así es como `dynamic_cast` y `typeid` encuentran el objeto completo desde cualquier subobjeto.

Conversión de punteros. Pasar de `C*` a `B*` **suma** un desplazamiento (`lea rdx, [rcx+10h]` o similar). Si en el binario ves que un puntero a objeto se ajusta con una constante antes de pasarlo, lo más probable es

un cambio de base, no aritmética de datos.

Thunks de ajuste. Si `C` redefine un método virtual de `B`, la vtable de `B` dentro de `C` no puede apuntar directo a `C::metodo`, porque ese código espera un `this` de tipo `C*` y recibe un `B*`. MSVC pone en ese slot un **adjustor thunk** de dos instrucciones:

```
; ilustrativo, MSVC x64: thunk en la vtable de B dentro de C
sub  rcx, 10h          ; vuelve de B* a C*
jmp  C::metodoDeB      ; salta a la implementación real
```

Cuando sigas un slot y caigas en un `sub rcx, N` seguido de un salto, no es ofuscación: es un ajuste de `this`, y el desplazamiento coincide con el `offset` del COL de esa vtable.

Herencia virtual. Con `class D : virtual public Base`, MSVC agrega un **vbptr** (*virtual base pointer*) que apunta a una **vbtable** con los desplazamientos hasta cada base virtual, porque su posición dentro del objeto completo depende de la clase más derivada. Lo vas a ver como una lectura indirecta antes de acceder a miembros de la base: cargar el vbptr, leer un desplazamiento de la vbtable y sumarlo. Con algunas combinaciones de constructores y funciones virtuales aparecen además campos `vtordisp` (controlados por la opción `/vd`). Para el análisis alcanza con reconocer el patrón; reconstruir el layout completo exige el tipo exacto.

Ejemplo de layout **ilustrativo** (x64, Release; los desplazamientos dependen de los miembros y la alineación reales):

```
C @ obj:
  +0x00  vptr -> C::vftable{for A}   ; COL.offset = 0
  +0x08  int a  (+ relleno)
  +0x10  vptr -> C::vftable{for B}   ; COL.offset = 0x10
  +0x18  int b  (+ relleno)
  +0x20  int c
```

Contraste: el ABI Itanium (GCC, Clang, MinGW)

Los binarios de GCC y Clang fuera de MSVC (incluido MinGW en Windows) siguen, en general, el **C++ ABI Itanium**. A pesar del nombre, es una especificación **genérica**: se escribió para Itanium, se apoya en el ABI de C de cada plataforma, y la propia especificación aclara que no es la definición autoritativa para ninguna plataforma en particular; cada plataforma puede adaptarla y documentar sus diferencias. Conviene separar dos partes.

Vtables, RTTI y nombres (especificación principal).

- El vptr apunta al **address point** de la vtable, que no es su comienzo: antes hay componentes en desplazamientos negativos. En 64 bits, `vptr[-1]` es el **puntero a typeinfo** y `vptr[-2]` es el **offset-to-top** (distancia desde ese vptr hasta el comienzo del objeto completo). Con bases virtuales hay además *vbase offsets* y *vcall offsets* más atrás.
- Un destructor virtual ocupa **dos** entradas consecutivas (destructor del objeto completo y destructor que además libera), a diferencia del único *scalar deleting destructor* de MSVC.

- Los nombres usan el *mangling* Itanium, que empieza con `_Z`: la vtable de `MiClase` es `_ZTV7MiClase`, su typeinfo `_ZTI7MiClase` y el nombre de tipo `_ZTS7MiClase` (cuyo texto es `7MiClase`). Si ves cadenas de este estilo y no `.?AV…@@`, cambiaste de ABI.
- En herencia múltiple también hay vtables secundarias y thunks de ajuste de `this`, con nombres propios del mangling.

Excepciones (específico del destino). El manejo de excepciones está en un documento aparte (ABI-EH): una capa C++ común (`__cxa_throw`, `__cxa_begin_catch` y una función *personality*) apoyada en el mecanismo de desenrollado **de la plataforma**. En sistemas ELF eso suele significar tablas DWARF en `.eh_frame` y datos específicos del lenguaje en `.gcc_except_table`. En Windows, según cómo se compiló el toolchain MinGW, puede usarse desenrollado SJLJ, DWARF o basado en las tablas SEH de Windows (`.pdata` / `.xdata`) con una personality de GCC. No supongas uno: identificalo por las secciones, los símbolos y las DLL de runtime que use el binario.

Señales rápidas para saber qué ABI tenés delante:

Señal	MSVC	GCC/Clang (Itanium)
Nombre de vtable	`??_7MiClase@@6B@`	`_ZTV7MiClase`
RTTI en `vptr[-1]`	Puntero (x86) o puntero a COL con RVA (x64)	Puntero directo a typeinfo
Cadena de tipo	`.?AVMiClase@@`	`7MiClase` (en `_ZTS…`)
Excepciones	`__CxxFrameHandler*`, `_CxxThrowException`	`__cxa_throw`, `__gxx_personality_*`

Práctica

En tu VM, con un **programa propio** compilado con C++ (clases con métodos virtuales, un `std::vector`, un `std::string` corto y uno largo, y un `try/catch`):

1. Compilá con **MSVC x64** (p. ej. VS 2022, `/EHsc`) en **Debug** y en **Release** y compará. Anotá la versión exacta del compilador. Fijá el compilador: el layout de **RTTI** y la **SSO** de `std::string` son específicos de MSVC; con MinGW/Clang/libstdc++ los detalles cambian, así que el ejemplo solo es repetible con el mismo toolchain.
2. Encontrá la vtable de tu clase y, en `vtable[-1]` (`-0x08`), seguí el COL: sumá la RVA de `pTypeDescriptor` a la base de la imagen hasta llegar al nombre de la clase. Verificá `pSelf`.
3. Mirá un `std::string` corto (≤15) y uno largo: confirmá con `capacity` que el corto vive **en el objeto** (SSO) y el largo apunta al heap. Registrá `sizeof(std::string)` en Debug y en Release.
4. Ubicá el `try/catch`: en x64, buscá la entrada de la función en `.pdata`, su información de desenrollado y el handler de C++ que nombra; identificá el funclet del `catch`.
5. Agregá a tu programa una clase con **herencia múltiple** que redefina un método de la segunda base. Encontrá las dos vtables, compará el `offset` de sus COL y ubicá el adjustor thunk en la vtable de la segunda base.
6. Opcional, si tenés el toolchain instalado en la VM: compilá el mismo fuente con MinGW-w64 o Clang con libstdc++/libc++ y compará nombres de vtable, la posición del typeinfo y las funciones de excepciones con la tabla de señales. Anotá la versión exacta de cada compilador.

Preguntas de repaso

▼ **¿Dónde está el RTTI respecto de la vtable y para qué sirve al analizar?**

> En `vtable[-1]` (justo antes del primer método; `-0x08` en x64) hay un puntero al *Complete Object Locator*, que lleva al TypeDescriptor con el **nombre decorado de la clase**. Da los nombres reales sin símbolos.

▼ **En un binario MSVC x64, ¿por qué no podés tratar `pTypeDescriptor` del COL como un puntero?**

> Porque en x64 es una **RVA** de 32 bits: hay que sumarle la base de la imagen para obtener la dirección.

▼ **¿Qué es la SSO de `std::string` y cómo cambia lo que ves?**

> *Small String Optimization*: las cadenas de hasta 15 `char` se guardan **dentro** del objeto (buffer interno), sin `new`. Para strings cortas no ves asignación dinámica y el texto aparece en el stack. Con `capacity` > 15, la unión contiene un puntero al heap.

▼ **¿Cómo reconocés un `std::vector` en el binario?**

> Por **tres punteros** juntos (inicio, fin, fin de capacidad). El tamaño es `(fin - inicio) / sizeof(T)`; una realocación en `push_back` mueve los tres.

▼ **¿Qué señal delata un `try/catch` de C++ en MSVC x64?**

> La información de desenrollado de la función (`.pdata` / `.xdata`) nombra un handler de C++ (`__CxxFrameHandler3` o `__CxxFrameHandler4`), con funclets para el `catch`; los `throw` llegan a `_CxxThrowException` con un `ThrowInfo`. En x86, en cambio, se ve el registro del marco en `fs:[0]`.

▼ **En una clase con herencia múltiple, un slot de vtable lleva a `sub rcx, 10h` y un `jmp`. ¿Qué es?**

> Un adjustor thunk: convierte el `this` del subobjeto base en el `this` del objeto completo antes de saltar a la implementación. El 0x10 debería coincidir con el `offset` del COL de esa vtable.

▼ **¿Para qué sirve el campo `offset` del Complete Object Locator?**

> Indica a qué distancia del comienzo del objeto completo está el vptr de esa vtable; con herencia múltiple cada vtable tiene su COL y su offset.

▼ **En un binario GCC de 64 bits, ¿dónde está el typeinfo respecto del vptr y qué hay antes?**

> En `vptr[-1]` está el puntero a typeinfo y en `vptr[-2]` el offset-to-top; con bases virtuales hay más componentes en desplazamientos negativos.

▼ **¿Por qué no basta con saber que un binario usa el ABI Itanium para saber cómo maneja excepciones?**

> Porque la especificación separa vtables y nombres del manejo de excepciones, que se apoya en el desenrollado de la plataforma: DWARF en ELF, y en MinGW puede ser SJLJ, DWARF o SEH según el toolchain.

▼ **Con RTTI, ¿en qué cambia leer `call [rax+8]` en x64?**

> Deja de ser "el slot 1" (`0x08 / 8`) y pasa a tener **nombre** (p. ej. `MiClase::metodo2`), porque `vtable[-1]` te dio la clase. El análisis se vuelve legible.

Puerta de salida

Podés: (1) seguir el RTTI desde `vtable[-1]` hasta el nombre de la clase en un binario x64, resolviendo las RVA; (2) distinguir un `std::string` con SSO de uno en heap; (3) reconocer un `std::vector` por sus tres punteros; (4) identificar un `try/catch` por su handler de C++; (5) decir qué offsets cambian entre x86 y x64; (6) reconocer varios vptr y adjustor thunks en herencia múltiple; (7) distinguir por sus señales un binario MSVC de uno con ABI Itanium.

Lo que queda fuera

La reconstrucción completa de layouts con herencia virtual (vbtable, `vtordisp`), la jerarquía completa de `ClassHierarchyDescriptor` , `std::unordered_map` , `std::shared_ptr` y otros contenedores, las variantes del ABI Itanium por plataforma (por ejemplo ARM) y el formato de las tablas de excepciones de GCC no se desarrollan. Los layouts descritos para MSVC deben verificarse en cada versión, y los del ABI Itanium en cada plataforma y toolchain.

Fuentes de esta lección

- `Reversing desde cero, cap. 33` — objeto con vtable, reconstruir las dos estructuras y cross-references manuales a los métodos virtuales.
- Curso IDA · tutes 41–43 — strings en stack/heap, `new` / `malloc` y heap en IDA.
- IDA y el análisis estático (2.7) — base de vtables y estructuras sobre la que se apoya esta lección.
- Microsoft Learn: x64 calling convention, x64 exception handling, /GR (Enable Run-Time Type Information), Multiple base classes, /vd (Disable Construction Displacements). El layout interno del COL y de `std::string` no está documentado como contrato por Microsoft: se describe como conocimiento externo de reversing, igual que los adjustor thunks y la vbtable.
- Itanium C++ ABI (vtables, RTTI y mangling) y Itanium C++ ABI: Exception Handling.

Capas de esta lección

Contenido observado en CLS: la vtable y las dos estructuras del capítulo 33 (binario de 32 bits).
Conocimiento externo: RTTI (`vtable[-1]` , Complete Object Locator con RVA en x64, TypeDescriptor), SSO de `std::string` , layout de los contenedores STL de MSVC handlers de excepciones de C++, herencia múltiple/virtual en MSVC y contraste con el ABI Itanium.
Recomendación editorial: el ejemplo ilustrativo fijado a MSVC x64 y la práctica con binario propio (sin laboratorio compilado con hashes en esta edición).

Ampliación editorial basada en fuentes externas — Diferencia entre __CxxFrameHandler3 y __CxxFrameHandler4 (FH4) en MSVC x64

Desde Visual Studio 2019 16.3+ y Visual Studio 2022, MSVC x64 activa por defecto `/d2FH4` (Making C++ Exception Handling Smaller on x64), reduciendo entre un 30 % y un 60 % el tamaño de las tablas en `.xdata` *(Hecho confirmado)*:

- **`__CxxFrameHandler3`:** Apunta a una estructura `FuncInfo` de campos fijos de 32 bits que empieza con la firma mágica `0x19930520`.
- **`__CxxFrameHandler4` (`FuncInfo4`):** Elimina la constante `0x19930520`; comienza con un byte de banderas (`FuncInfoHeader`) y comprime los desplazamientos y conteos con enteros de longitud variable de 1 a 4 bytes. Para compilar una práctica que conserve el formato fijo clásico `FuncInfo`, puede pasarse `/d2FH4-` a `cl.exe` (ver plantilla `cpp_rtti_sso.cpp` en el Kit de plantillas C/C++).

WinDbg, TTD y heap

> **Objetivo de la lección**
>
> Manejar **WinDbg** para lo que x64dbg no da cómodo: **time-travel** (ir hacia atrás) y **diagnóstico de heap**. Encontrar la causa de un error de memoria en un programa **propio**. **Prerrequisitos:** debugger (2.6), llamadas y pila (2.5), memoria (1.2).

WinDbg: por qué y lo mínimo

El Curso IDA integra **WinDbg con IDA**: lo instala y lo maneja desde la interfaz de IDA porque es "un gran debugger" aunque incómodo, casi todo por comandos (`Curso IDA, tute 32`). Comandos que vas a usar siempre:

Comando	Qué hace
`g` / `p` / `t`	correr / step over / step into
`bp dir` / `ba r4 dir`	breakpoint de software / de acceso (hardware)
`k`	call stack
`dt tipo dir`	volcar una estructura con sus campos
`!address dir`	a qué región pertenece una dirección
`!heap`	estado del heap (ver abajo)

TTD: grabar y rebobinar

Time Travel Debugging (TTD, de Microsoft; **conocimiento externo**, es posterior al material de CLS) graba **toda** la ejecución en un trace y después la reproducís **hacia adelante y hacia atrás**. Cambia el método: en vez de "¿quién va a escribir este valor?" y reiniciar mil veces, ponés un breakpoint de acceso y hacés `g-` (go back) hasta la escritura que lo rompió. Para "¿quién corrompió este buffer?", TTD es la herramienta: encontrás la dirección mala y **retrocedés** hasta la instrucción culpable.

Heap: Page Heap y LFH

El heap es donde más cuesta ver la corrupción, porque el síntoma (un crash) aparece **lejos** de la causa. Dos ayudas:

- **Page Heap / Application Verifier** (se activan con `gflags`; **conocimiento externo**): ponen cada bloque al **final de una página**, con una página de guardia detrás. Así un overflow de heap **crashea en el acto**, en la instrucción que escribe de más, no páginas después. El modo *full* consume más memoria, pero te da el crash exacto.
- **LFH** (*Low Fragmentation Heap*): Windows agrupa las alocaciones chicas en *buckets*. El Curso IDA dedica una parte a distinguir si una alocación cae en el **LFH o en el heap estándar**, porque cambia cómo se ven

los bloques (`Curso IDA, tute 48`). En WinDbg lo mirás con `!heap`.

El tute 44 muestra justamente **qué info de heap se obtiene con WinDbg dentro de IDA** (`Curso IDA, tute 44`).

> **Heap spraying: solo como concepto**
>
> El *heap spraying* (rellenar el heap con muchas copias de un dato para que una dirección sea predecible) se explica **para reconocerlo en un informe** —qué dice y qué evidencia buscar—, no se practica su construcción. Esta lección es de **diagnóstico defensivo**.

Ejemplo paso a paso: cazar un overflow de heap con Page Heap + TTD

*(Ejemplo **ilustrativo**: salida construida para mostrar el flujo.)*

Programa propio que hace `memcpy` de 40 bytes en un buffer de 32 del heap. Con Page Heap activo (`gflags /p /enable prog.exe /full`), crashea **en el** `memcpy`:

```
(1a2c.0b30): Access violation - code c0000005
prog!copiar+0x2e:
  mov byte ptr [eax], cl      ; escribe en la página de guardia → crash acá

0:000> !heap -p -a eax
    address 0019f000 found in _DPH_HEAP ... 32 bytes allocated
    stack:  prog!reservar+0x1c   ← dónde se pidió el bloque de 32

0:000> k                         ; cómo llegué
  prog!copiar+0x2e
  prog!procesar+0x51
  prog!main+0x22
```

Con TTD, si no tuvieras el crash exacto, pondrías `ba w1 <fin del buffer>` y harías `g-` hasta la escritura. En los dos casos llegás a `copiar+0x2e` y, por `!heap -p -a`, al bloque de 32 pedido en `reservar`: la causa, no solo el síntoma.

Práctica

En tu VM, con un **programa propio** con un bug de heap (un `memcpy` / `strcpy` que escribe de más en un bloque del heap):

1. Reproducí el crash "normal": vas a ver que explota lejos de la causa.
2. Activá **Page Heap** (`gflags /p /enable prog.exe /full`) y volvé a correrlo: ahora crashea en la escritura culpable.
3. Con `!heap -p -a <dir>` encontrá dónde se pidió ese bloque y con `k` el camino.
4. (Si tenés TTD) grabá, poné `ba w` en el fin del buffer y `g-` hasta la escritura.
5. **Apagá Page Heap al terminar**: `gflags /p /disable prog.exe`. La configuración de GFlags **persiste en el registro** (`Image File Execution Options`); si no la sacás, el programa queda siempre con page heap

(más lento y con más consumo). Verificá con `gflags /p` que no quede ninguno activo.

Preguntas de repaso

▼ **¿Qué problema del heap resuelve Page Heap?**

> Que el crash aparece **lejos** de la causa. Page Heap pone cada bloque pegado a una **página de guardia**, así un overflow crashea **en el acto**, en la instrucción que escribe de más.

▼ **¿En qué cambia TTD el análisis de "¿quién escribió este valor?"**

> Deja **retroceder** (`g-`) en una ejecución grabada hasta la escritura culpable, sin reiniciar una y otra vez. Grabás una vez y navegás hacia adelante y atrás.

▼ **¿Qué es el LFH y por qué importa?**

> El *Low Fragmentation Heap*: Windows agrupa alocaciones chicas en buckets. Importa porque un bloque en LFH se ve distinto de uno en el heap estándar, y eso cambia cómo interpretás la memoria.

▼ **¿Qué te da `!heap -p -a <dir>` en WinDbg?**

> La información del bloque de heap que contiene esa dirección: tamaño y, con Page Heap, el **stack de dónde se alocó**. Te lleva del síntoma a dónde se pidió la memoria.

▼ **¿Por qué el heap spraying está solo como concepto?**

> Porque esta lección es de **diagnóstico defensivo**: interesa reconocer de qué habla un informe y qué evidencia buscar, no construir la técnica.

> **Puerta de salida**
>
> Podés: (1) moverte en WinDbg con `g/p/t`, `bp/ba`, `k`, `dt`, `!heap`; (2) explicar qué resuelve Page Heap y usarlo para cazar un overflow de heap propio; (3) decir cómo TTD ayuda a ir a la causa; (4) distinguir LFH de heap estándar; (5) explicar qué es el heap spraying sin construirlo.

Fuentes de esta lección

- `Curso IDA, tute 32` — WinDbg manejado desde IDA.
- `Curso IDA, tute 44` — info de heap con WinDbg dentro de IDA.
- `Curso IDA, tute 48` — alocación en LFH vs. heap estándar.

Capas de esta lección

Contenido observado en CLS: WinDbg+IDA (tute 32), heap con WinDbg (tute 44) y LFH (tute 48). **Conocimiento externo:** TTD, Page Heap/Application Verifier (`gflags`) y el concepto de heap spraying. **Recomendación editorial:** el ejemplo ilustrativo y la práctica con un bug de heap propio (sin laboratorio compilado con hashes en esta edición).

Ampliación editorial basada en fuentes externas — NT Heap (0xEEFFEEFF) vs. Segment Heap (0xDDEEDDEE) en Windows 10/11

En Windows 10 y 11 coexisten dos backends de heap en user mode (Microsoft Learn: Segment Heap y !heap) *(Hecho confirmado)*: el **NT Heap** clásico (firma `0xEEFFEEFF`) y el **Segment Heap** (firma `0xDDEEDDEE` , activo en servicios del sistema, apps MSIX y ejecutables con `<segmentHeap>true</segmentHeap>` en su manifiesto). En WinDbg moderno, `!heap` muestra la columna `Type` (`NT Heap` vs. `Segment Heap`); cuando activás **Full Page Heap** (`gflags /p /enable prog.exe /full`), las reservas pasan por `_DPH_HEAP_ROOT` y `!heap -p -a <dir>` funciona de forma idéntica en ambos casos (podés practicarlo con `heap_bug.c` del Kit de plantillas C/C++).

Patch diffing y CVE: leer qué cambió

> **Objetivo de la lección**
>
> Comparar **dos versiones** de un binario y entender **qué cambió un parche** y qué riesgo mitiga, sin convertirlo en una práctica de explotación. **Prerrequisitos:** IDA (2.7) y el differ mencionado en flujo (2.4). *Contexto recomendado (no bloqueante):* la rama de vulnerabilidades (5.2) — 2C es la continuación natural de 2B y no exige haber hecho la Etapa 5.

Qué es un differ

Un **differ** toma dos versiones consecutivas de un programa y **empareja** (matchea) sus funciones a pesar de los cambios, para mostrarte **cuáles cambiaron y dónde**. El tute 22 del Curso IDA lo define así y aclara el caso que nos importa: esos cambios pueden ser **un parche de seguridad** que corrige una vulnerabilidad (`Curso IDA, tute 22`). Las herramientas estándar (**conocimiento externo**) son **BinDiff** y **Diaphora**: emparejan por estructura del grafo, hashes de instrucciones, llamadas y nombres, y te dan un porcentaje de similitud por función.

Por qué es **defensivo**: comparar la versión vulnerable con la parcheada te dice **qué chequeo agregaron**, es decir, cuál era el defecto. Es la base del análisis de **1-day** (entender una falla ya corregida) y de verificar que un parche realmente arregla lo que dice. Identificar el cambio **no** es construir el exploit.

El flujo

1. **Conseguí las dos versiones** (antes y después del parche) de un binario que tengas derecho a analizar.
2. **Analizá cada una** en IDA/Ghidra y exportá para el differ.
3. **Corré el differ**: ordená por "más cambiadas / menos similares". Las funciones con 100% de match no te interesan; mirá las que bajaron.
4. **Leé el cambio**: casi siempre es un **chequeo agregado** (una comparación de tamaño, un `if` nuevo antes de un `memcpy`).
5. **Redactá** qué cambió y qué riesgo mitiga, para tu informe.

El caso real del curso: VLC (CVE-2008-4654)

El Curso IDA toma un **CVE público** y lo trabaja como diff de **dos versiones consecutivas de VLC**, usando el número de CVE como guía de qué buscar (`Curso IDA, tute 30`; explicación en el tute 31). El objetivo acá es **entender el defecto y el arreglo** (qué validación faltaba), no desarrollar un exploit; la rama de vulnerabilidades ya enmarca esto en clave defensiva.

Ejemplo paso a paso: una función antes y después del parche

*(Ejemplo **ilustrativo**: pseudo-C construido para mostrar qué resalta un differ.)*

BinDiff marca `leer_registro` con **82%** de similitud (bajó): algo cambió. Abrís las dos:

```
// versión vulnerable                // versión parcheada
int leer_registro(FILE *f) {         int leer_registro(FILE *f) {
  char buf[64];                        char buf[64];
  int n = leer_u16(f);                 int n = leer_u16(f);
                                       if (n > sizeof(buf))        // <-- chequeo AGREGADO
                                           return -1;
  fread(buf, 1, n, f);                 fread(buf, 1, n, f);
  ...                                  ...
}
```

El diff resalta el bloque nuevo: un `if (n > sizeof(buf)) return -1` antes del `fread`. Conclusión para el informe: **el defecto era un overflow de stack** (se leían `n` bytes controlados por el archivo en un buffer de 64 sin validar `n`); **el parche agrega la validación de tamaño**. Eso es todo lo que esta lección busca: leer el cambio, nombrar el riesgo.

Cómo empareja un differ, y dónde se equivoca

Saber cómo trabaja la herramienta te dice cuándo creerle (**conocimiento externo**). **BinDiff** es hoy de código abierto y trabaja sobre exportaciones en formato **BinExport**, que se generan desde IDA, Binary Ninja o Ghidra; **Diaphora** corre como script dentro de IDA. Los dos siguen la misma lógica general:

1. Empiezan por las coincidencias **más seguras**: mismo nombre (si hay símbolos), mismos bytes o el mismo hash de instrucciones.
2. Siguen con atributos **estructurales**: cantidad de bloques y aristas del grafo, llamadas que hace y recibe la función, constantes y cadenas que usa.
3. **Propagan**: si `A` coincide con `A'`, las funciones que `A` llama probablemente coincidan con las que llama `A'`.

Cada par queda con una **similitud** (cuánto se parecen) y una **confianza** (qué tan seguro está el emparejamiento). Una similitud baja con confianza alta es lo que buscás: la misma función, cambiada. Donde se equivoca: funciones **chicas** y parecidas entre sí (envoltorios, *thunks*, métodos triviales) que se emparejan mal; funciones que el compilador **inlineó** en una versión y no en la otra, y que "desaparecen"; y código **reordenado** que cambia el grafo sin cambiar la lógica.

Separar el ruido del parche

En un diff real, la mayoría de las funciones que cambiaron **no** son el parche. Fuentes típicas de ruido:

Ruido	Cómo se ve	Cómo descartarlo
Otra versión del compilador o del enlazador	Cientos de funciones con cambios menores	El cambio está repartido en todo el binario, no concentrado
Otras opciones de optimización	Mismos bloques en otro orden, registros distintos	El grafo cambia pero las llamadas y constantes son las mismas
Direcciones y constantes de enlace	Solo cambian inmediatos que son direcciones	Ignorar diferencias en offsets a datos
Bibliotecas actualizadas	Cambios en funciones del runtime	Mirar primero el código propio del programa

La señal del parche suele ser otra: **un bloque nuevo** con una comparación y una salida temprana, una llamada nueva a una función de validación o una API "segura" en lugar de la anterior (por ejemplo, una variante con tamaño explícito). Mirar también el **diff de imports** ayuda: una función nueva importada a veces nombra el arreglo.

Símbolos públicos: nombres para los binarios de Windows

Para diffear binarios de Windows, Microsoft publica **símbolos públicos** en su servidor de símbolos: el debugger o la herramienta de análisis los descarga según la versión exacta del binario. Traen nombres de funciones y variables globales, no el código fuente ni los tipos privados. Con nombres en las dos versiones, el emparejamiento mejora mucho y el informe puede decir "cambió `FuncionX`" en lugar de "cambió `sub_180012340`". La ruta de símbolos que combina el servidor público de Microsoft con tus propios PDB se arma igual que en la depuración de kernel de 6.3; IDA y Ghidra también pueden descargarlos.

Ghidra Version Tracking como alternativa

Version Tracking de Ghidra (2.16) hace un trabajo parecido con otro enfoque: corrés **correlacionadores** (bytes exactos, instrucciones exactas, mismas referencias a datos, mismos símbolos, y otros más permisivos), revisás las coincidencias propuestas, las **aceptás** y trasladás nombres y comentarios del origen al destino. Lo que queda sin aceptar es el candidato a cambio. Es más manual que BinDiff, pero todo queda dentro de un solo proyecto y no necesita IDA.

> **Paso a paso según tu herramienta (IDA Free no carga BinExport)**
>
> BinDiff necesita **BinExport**, un plugin **nativo** que **IDA Free no carga** (es para el SDK de IDA Pro). Por eso, sin pagar:
>
> - **Ghidra Version Tracking** (ver sección anterior): el *Version Tracking Wizard* (source = v1, destination = v2) → *Add to Existing Session* con **Exact Function Bytes/Instructions Match** → filtrá las parcialmente emparejadas: la que cambió salta de inmediato.
> - **Diaphora en IDA (dos fases):** corré `diaphora.py` sobre `v1` y exportá `v1.sqlite`; después corrélo sobre `v2` apuntando a `v1.sqlite` y revisá la pestaña **Partial** (Ratio < 1.00).

Práctica

En tu VM:

1. Compilá un programa **propio** con un bug simple (un `fread` / `memcpy` sin validar tamaño). Guardá el binario **"v1"**.
2. Arreglalo (agregá la validación) y compilá **"v2"**.
3. Corré **BinDiff** o **Diaphora** entre v1 y v2; ordená por menor similitud.
4. Abrí la función que cambió y escribí dos líneas: **qué** cambió y **qué riesgo** mitiga.
5. Compilá una **v2b** con el mismo arreglo pero otra opción de optimización (por ejemplo `/O1` en lugar de `/O2`). Diffeá v1 contra v2b, contá cuántas funciones cambiaron y explicá cuáles son ruido y cuál es el parche.
6. Repetí el diff v1/v2 con **Version Tracking** de Ghidra y compará con el resultado de BinDiff o Diaphora: ¿encontraron la misma función?

Preguntas de repaso

▼ **¿Qué hace un differ y para qué sirve en defensa?**

> Empareja las funciones de **dos versiones** de un binario y marca las que cambiaron. En defensa sirve para ver **qué arregló un parche** (qué chequeo agregaron = cuál era el defecto): análisis de 1-day y verificación de parches.

▼ **Al correr BinDiff, ¿qué funciones mirás?**

> Las de **menor similitud** (las que cambiaron). Las que matchean al 100% no aportan; el parche está en las que bajaron.

▼ **En el ejemplo, ¿cuál era el defecto y cuál el arreglo?**

> El defecto: leer `n` bytes (controlados por el archivo) en un buffer de 64 **sin validar** `n` → overflow de stack. El arreglo: un `if (n > sizeof(buf)) return -1` antes del `fread`.

▼ **¿Qué aporta el número de CVE al hacer el diff de VLC?**

> Es una **guía**: te dice qué clase de falla y en qué zona buscar, para no comparar a ciegas las dos versiones.

▼ **En un diff, ¿qué combinación de similitud y confianza buscás?**

> Similitud **baja** con confianza **alta**: el differ está seguro de que es la misma función y de que cambió. Similitud baja con confianza baja puede ser un mal emparejamiento.

▼ **Diffeaste dos versiones y cambiaron trescientas funciones. ¿Qué hacés?**

> Separar el ruido: cambios repartidos por todo el binario suelen ser otra versión del compilador u otras optimizaciones. Buscás el cambio concentrado: un bloque nuevo con comparación y salida temprana, una validación nueva o un import nuevo.

▼ **¿Qué aportan los símbolos públicos de Microsoft al diffear binarios de Windows?**

> Nombres de funciones y variables globales en las dos versiones: mejoran el emparejamiento y permiten describir el cambio por nombre. No incluyen código fuente ni tipos privados.

▼ **¿Dónde está el límite de esta lección?**

> En **leer el cambio y nombrar el riesgo**. Identificar el defecto no es construir el exploit; eso queda fuera (la guía es defensiva).

Puerta de salida

Podés: (1) explicar qué es un differ y por qué es defensivo; (2) correr BinDiff/Diaphora entre dos versiones propias y encontrar la función cambiada; (3) leer el chequeo agregado y nombrar el defecto que mitiga; (4) explicar el rol del CVE en el caso VLC; (5) explicar cómo empareja un differ y en qué casos se equivoca; (6) separar el ruido de compilación del cambio real; (7) usar símbolos públicos o Version Tracking para mejorar el diff.

Lo que queda fuera

El diff de parches mensuales completos de Windows, la extracción de archivos de los paquetes de actualización, el diff de controladores de kernel y, por el marco del curso, cualquier paso desde "entendí el defecto" hacia "construí un exploit".

Fuentes de esta lección

- `Curso IDA, tute 22` — qué es un differ; empareja funciones entre versiones; el caso del parche de seguridad.
- `Curso IDA, tute 30` — VLC / CVE-2008-4654 como diff de dos versiones (explicación en el tute 31).
- Rama vulnerabilidades (5.2) — el marco defensivo del defecto y las mitigaciones.
- Conocimiento externo: BinDiff (código abierto; trabaja sobre BinExport desde IDA, Binary Ninja o Ghidra), Diaphora, Ghidra Version Tracking y Microsoft Learn: Microsoft public symbols.

Capas de esta lección

Contenido observado en CLS: la definición de differ del capítulo 22 y el caso VLC/CVE de los tutes 30–31. **Conocimiento externo:** BinDiff, Diaphora y Version Tracking, cómo emparejan funciones, el ruido de compilación y los símbolos públicos de Microsoft. **Recomendación editorial:** el ejemplo ilustrativo, la tabla de ruido, el flujo de 1-day defensivo y la práctica con dos versiones propias (sin laboratorio compilado con hashes en esta edición).

Scripting de debugger: automatizar y dejar evidencia

> **Objetivo de la lección**
>
> Dejar de repetir a mano la misma secuencia en el debugger: **automatizarla con un script** de x64dbg y que ese script **guarde evidencia** reproducible. Reconocer y adaptar un **OllyScript** antiguo.
> **Prerrequisitos:** debugger (2.6). *Contexto recomendado (no bloqueante):* método de análisis (4.1) y automatización con IDAPython (5.3) — útiles como referencia, pero 2C no exige haber hecho las etapas 4 o 5.

Por qué scriptear el debugger

Cuando una tarea se repite —poner el mismo breakpoint, leer el mismo registro, anotar el mismo valor en cada corrida— conviene **automatizarla**: es más rápido, no te equivocás y queda **reproducible** (cualquiera corre el script y obtiene lo mismo). Es el mismo espíritu que viste con **IDAPython** en la automatización de la Etapa 5, pero del lado **dinámico**: en vez de recorrer el binario estático, manejás la ejecución.

x64dbg: el lenguaje de scripts

x64dbg trae una pestaña **Script** con un lenguaje de comandos simple (**conocimiento externo**). Los que más vas a usar:

Comando	Qué hace
`bp dir` / `bphws dir, "r"`	breakpoint de software / de hardware
`SetBreakpointLog dir, "..."`	texto que se escribe al log cuando se dispara ese bp
`SetBreakpointCondition dir, 0`	*break-condition* 0 = el bp **loggea pero no pausa**
`run` / `erun`	correr hasta el bp / correr pasando excepciones
`log "texto {reg}"`	escribir en el log (con formato de registros/memoria)
`$resultado = [rsp+8]`	variables del script, leer memoria

El formato de `log` permite volcar memoria y registros: `{utf16@rcx}` imprime el string Unicode **cuya dirección está en** `RCX`. Ojo con los corchetes: `@rcx` usa el **valor del registro** como dirección; `@[rcx]` haría una **lectura de más** (tomaría lo que hay *en* `RCX` como dirección). Con un breakpoint que **loggea sin pausar** armás una **traza** automática.

Ejemplo paso a paso: loggear cada archivo que abre el programa

*(Ejemplo **ilustrativo**: script de x64dbg para registrar llamadas y guardar evidencia.)*

```
// script.txt — correr desde la pestaña Script de x64dbg
bp CreateFileW                                          // breakpoint en la API
SetBreakpointLog CreateFileW, "[CreateFileW] {utf16@rcx}" // qué loggear al dispararse
SetBreakpointCondition CreateFileW, 0                   // 0 = loggea pero NO pausa
run                                                     // una sola vez; todo queda en el log

// el log queda con algo como:
// [CreateFileW] C:\Users\...\config.ini
// [CreateFileW] C:\Users\...\datos.bin
```

Cada vez que el programa llama a `CreateFileW`, el breakpoint **loggea el nombre del archivo** (el primer argumento, en `RCX`) y **continúa solo**, porque la *break-condition* es 0. Esa es la forma recomendada: no metas `run` dentro de un comando de breakpoint (x64dbg lo desaconseja por inestable); dejá que el bp loggee y no pause. Al final copiás el **log** como evidencia (Etapa 4): un registro reproducible de qué archivos tocó, sin frenar a mano en cada llamada.

Trazas condicionales: registrar cada instrucción que cumple algo

Un breakpoint con log te dice **cuándo** se llamó a algo. A veces la pregunta es otra: "¿por qué camino pasó entre este punto y aquel?" o "¿en qué instrucción `RAX` tomó este valor?". Para eso x64dbg tiene **trazas condicionales** (menú *Debug*: *Trace into until condition* / *Trace over until condition*): el debugger ejecuta paso a paso por vos y en cada instrucción evalúa expresiones (**conocimiento externo**, documentación de x64dbg):

Campo	Qué hace	Valor por defecto
Break condition	Si da 1, la traza se detiene	`0` (no se detiene por condición)
Log text	Texto con formato que se escribe al log	vacío
Log condition	Si da 1, se escribe el *log text*	`1` (siempre)
Command text / *condition*	Comando que se ejecuta cuando la condición da 1	la condición por defecto es la *break condition*

El log de la traza se puede mandar a un archivo (desde el diálogo o con el comando `TraceSetLogFile`), y ese archivo es la evidencia.

*(Ejemplo **ilustrativo**, no ejecutado en esta edición: valores de los campos para una traza "trace over" desde el inicio de una función de verificación.)*

```
Break condition:  cip == 0x140001290        // fin de la función: ahí se detiene
Log text:         {p:cip} {i:cip}  rax={rax}
Log condition:    1                         // loggear todas las instrucciones
```

El resultado es la lista de instrucciones **realmente ejecutadas**, con el valor de `RAX` en cada una. Comparar dos trazas (una con entrada válida y otra con inválida) muestra exactamente **dónde se separan los caminos**, que es el dato que buscabas en el método de la Etapa 4. Dos precauciones: una traza paso a paso es **mucho**

más lenta que la ejecución normal, así que acotala entre dos direcciones y usá *trace over* para no meterte en las APIs; y el archivo crece rápido, así que logueá solo lo que vas a leer.

Instrumentación dinámica: Frida

Un debugger detiene el proceso (o lo traza instrucción por instrucción). La **instrumentación dinámica** hace otra cosa: **inyecta código propio** en el proceso para que se ejecute al entrar y salir de las funciones que te interesan, sin pausar. **Frida** es la herramienta libre más difundida de este tipo (**conocimiento externo**): inyecta un motor de JavaScript en el proceso y te da una API para enganchar funciones.

La pieza central es `Interceptor.attach(target, callbacks)`: `target` es la dirección de la función y `callbacks` tiene `onEnter(args)`, que recibe los argumentos, y `onLeave(retval)`, que recibe el valor de retorno. La dirección de una función exportada se obtiene con `Process.getModuleByName(nombre).getExportByName(funcion)`.

*(Ejemplo **ilustrativo**, no ejecutado en esta edición: el mismo registro de `CreateFileW` del ejemplo de x64dbg, ahora con Frida.)*

```
// agente.js: registrar qué archivos abre el programa y qué handle recibe
const f = Process.getModuleByName('kernel32.dll').getExportByName('CreateFileW');
Interceptor.attach(f, {
  onEnter(args) {
    this.nombre = args[0].readUtf16String();   // primer argumento: LPCWSTR
  },
  onLeave(retval) {
    console.log('[CreateFileW] ' + this.nombre + ' -> ' + retval);
  }
});
```

Se carga con la herramienta de línea de comandos de Frida al lanzar **tu programa** dentro de la VM. Comparado con el script de x64dbg, ganás el valor de retorno en la misma línea (`this` guarda datos entre `onEnter` y `onLeave`) y la posibilidad de procesar los datos en JavaScript; perdés la vista de registros y memoria que da el debugger.

Herramienta	Cuándo conviene	Costo
Breakpoint con log (x64dbg)	Saber cuándo y con qué argumentos se llama a algo	Bajo; pausa interna en cada disparo
Traza condicional (x64dbg)	Ver el camino exacto entre dos puntos	Muy lento; acotarlo
Instrumentación (Frida)	Registrar entradas y salidas de muchas funciones, procesar datos al vuelo	Inyecta código: modifica el proceso
TTD (2.19)	Grabar una ejecución y recorrerla hacia atrás	Archivos grandes; solo WinDbg

> **Instrumentar modifica el proceso**
>
> Frida inyecta una biblioteca y reescribe el inicio de las funciones enganchadas. Usala solo sobre **programas propios o didácticos, en la VM**. Un programa con controles anti-análisis (2.15) puede notar la inyección y comportarse distinto, y eso es un dato para documentar, no algo para evadir. No la uses sobre software de terceros, juegos en línea ni procesos protegidos del sistema.

OllyDbg como legado

OllyDbg (el de los videos 23–40 del curso, ver 2.9 Herramientas) tiene su propio **OllyScript** (`.txt` con comandos como `bp`, `run`, `log`, `cmp`, `jne`). Es de 32 bits y está discontinuado, pero vas a **encontrar scripts viejos** de OllyScript en material de la escena. El objetivo acá no es escribir OllyScript nuevo, sino **reconocer** qué hace uno antiguo y, si hace falta, **adaptar** su lógica a x64dbg. En `Teorias_Numeradas` hay varios títulos sobre scripting de OllyDbg/x64dbg (material por título, no revisado; ver índice).

Práctica

En tu VM, con un **crackme propio o didáctico** (ver Banco de prácticas):

1. Elegí una tarea repetitiva (p. ej. loggear cada `strcmp` / `CreateFileW`, o leer un registro en un punto).
2. Escribí un script de x64dbg que la haga sola y escriba al **log**.
3. Corré el script y **guardá el log** como evidencia.
4. Tomá un OllyScript viejo (si conseguís uno) y escribí en dos líneas qué hace y cómo se traduciría a x64dbg.
5. Sobre tu crackme propio, hacé una traza condicional *trace over* de la función de verificación con una entrada válida y otra inválida, mandando el log a archivo. Compará los dos archivos y señalá la primera instrucción donde se separan.
6. Si instalás Frida en la VM, registrá con un agente las llamadas a `CreateFileW` (o a la comparación de cadenas) de tu programa propio y compará la salida con la del script de x64dbg.

Preguntas de repaso

<details>
<summary>¿Por qué scriptear una tarea del debugger en vez de hacerla a mano?</summary>

Porque es más rápido, evita errores de repetición y queda **reproducible**: el script documenta exactamente qué se hizo y cualquiera obtiene el mismo resultado.
</details>

<details>
<summary>¿Cómo hacés que un breakpoint loggee sin pausar la ejecución?</summary>

Le ponés un **texto de log** (`SetBreakpointLog`) y la **break-condition en 0** (`SetBreakpointCondition dir, 0`): el bp escribe al log cada vez que se dispara y continúa solo. Es lo recomendado; meter `run` dentro de un comando de breakpoint es inestable.
</details>

▼ **¿Cómo loggeás el nombre de archivo en una llamada a `CreateFileW`?**

> Con el formato de memoria del log: el primer argumento (`RCX` en x64) tiene la **dirección** del nombre; `{utf16@rcx}` vuelca ese string Unicode al log. (Con `@[rcx]` harías una lectura de más.)

▼ **¿Qué te da una traza condicional que no te da un breakpoint con log?**

> El **camino completo** de instrucciones ejecutadas entre dos puntos, con los valores que pidas en cada una. Comparando dos trazas ves dónde se separan los caminos. A cambio es muy lenta, por eso se acota.

▼ **En Frida, ¿para qué sirven `onEnter` y `onLeave`?**

> `onEnter(args)` corre al entrar a la función y recibe los argumentos; `onLeave(retval)` corre al salir y recibe el valor de retorno. Con `this` se guardan datos de la entrada para usarlos a la salida.

▼ **¿Por qué instrumentar con Frida no es "solo mirar"?**

> Porque inyecta una biblioteca y modifica el inicio de las funciones enganchadas: el proceso cambia y un programa con controles anti-análisis puede notarlo. Por eso se usa sobre programas propios en la VM y se documenta.

▼ **¿Qué relación tiene esto con IDAPython (5.3)?**

> Es la misma idea de **automatizar**, pero en dinámico: IDAPython recorre el binario estático; el script de x64dbg maneja la **ejecución** (breakpoints, registros, memoria en vivo).

▼ **¿Qué hacés con un OllyScript antiguo?**

> Lo **reconocés y adaptás**: entendés su lógica (bp, run, comparaciones) y la traducís a x64dbg. No conviene escribir OllyScript nuevo (OllyDbg está discontinuado y es de 32 bits).

Puerta de salida

Podés: (1) escribir un script de x64dbg que automatice una tarea y escriba al log; (2) hacer que un breakpoint loggee sin pausar (log + break-condition 0); (3) guardar el log como evidencia reproducible; (4) reconocer un OllyScript y explicar cómo adaptarlo; (5) acotar una traza condicional y comparar dos trazas; (6) explicar qué hace un agente de Frida con `Interceptor.attach` y qué cambia en el proceso. **Con esto cerrás el núcleo del bloque 2C.**

Lo que queda fuera

Stalker (el trazador de Frida) y la reescritura de funciones con `Interceptor.replace`, los marcos de instrumentación binaria como DynamoRIO o Intel Pin y el uso de Frida en Android o iOS. La lección enseña a registrar evidencia sin cambiar la lógica del programa.

Fuentes de esta lección

- Automatización con IDAPython (5.3) — el paralelo estático del scripting; misma idea de automatizar y reproducir.
- Herramientas (2.9) — OllyDbg en su contexto histórico (videos 23–40).
- Teorias_Numeradas — títulos sobre scripting de x64dbg/OllyDbg (material por título, no revisado).
- x64dbg (conocimiento externo): Conditional Breakpoint y Conditional Tracing — campos de condición, log y comando; `TraceSetLogFile`.
- Frida (conocimiento externo): documentación y JavaScript API — `Interceptor.attach`, `onEnter` / `onLeave`, `Process.getModuleByName`, `getExportByName`, `readUtf16String`.

Capas de esta lección

Contenido observado en CLS: OllyDbg en los videos del curso y la automatización con IDAPython de la Etapa 5; títulos de scripting en `Teorias_Numeradas`. **Conocimiento externo:** el lenguaje de scripts de x64dbg (`SetBreakpointLog`, `SetBreakpointCondition`, `log`, formatos de memoria), las trazas condicionales, Frida y OllyScript. **Recomendación editorial:** los ejemplos ilustrativos (script, campos de traza, agente de Frida; ninguno ejecutado en esta edición), la tabla de herramientas y la práctica de guardar evidencia (sin laboratorio compilado con hashes en esta edición).

Malware dinámico defensivo

Ruta optativa, fuera de la puerta obligatoria

Esta lección es una **especialización**, no un requisito para aprobar 2C. Tiene sus propios prerrequisitos y su propio entregable. **Prerrequisitos:** rama malware estática (5.1), análisis dinámico seguro (4.4), debugger (2.6).

Reglas no negociables del laboratorio

- **VM aislada**, sin carpetas compartidas, con la red en **host-only** o apagada; **snapshot** antes de cada corrida y revertir después.
- **Nunca** en tu máquina real ni en una VM con datos tuyos o credenciales.
- Trabajá con una **muestra preparada para el curso o un simulador benigno** que *imite* comportamiento (crea archivos, toca el registro, abre un socket). **No** descargues ni ejecutes malware real para practicar.
- El objetivo es **observar y documentar**, no "detonar por curiosidad".

La rama de malware te enseñó el **triage estático** (sin ejecutar). Acá sumás la capa **dinámica**: ver qué hace una muestra *mientras corre*, en un entorno controlado. El propio curso marca el límite con el caso **Shelter** (video 37), que ejecuta una muestra y queda **fuera** del recorrido de principiante justamente por eso (`Videos y material adicional, Video 37`). Hacelo solo cuando ya tenés el método y el laboratorio.

Las cuatro superficies que se observan

Todo esto es **conocimiento externo** (herramientas y método estándar de análisis de comportamiento):

- **Archivos:** qué crea, copia o borra. Herramienta: **Procmon** (filtro por operación de archivo), o un *before/after* con listados.
- **Registro:** claves de **persistencia** (`Run` , servicios, tareas programadas) y configuración. Herramienta: **Regshot** (snapshot antes/después) o Procmon.
- **Procesos:** procesos hijos, inyección en otros procesos, servicios. Herramienta: **Process Monitor/Explorer**, Autoruns para persistencia.
- **Red:** a dónde intenta conectarse (C2), qué resuelve por DNS, qué pide por HTTP. Clave: **red simulada** con **INetSim** o **FakeNet-NG**, que responden como si hubiera Internet **sin** salir de verdad; capturás el tráfico con Wireshark.

Un *runner* como **Noriben** (sobre Procmon) junta todo esto en un reporte; es cómodo pero el método importa más que la herramienta.

Del comportamiento a los IOC

El entregable es un **informe de indicadores (IOCs)**: archivos creados (nombre + hash), claves de registro de persistencia, procesos hijos, dominios/IPs de C2, mutex. Esos IOC son lo que alimenta una regla **YARA** o una detección (lo viste en 5.1). El informe sigue la estructura de la plantilla: entorno, hashes, evidencia (capturas de Procmon/Wireshark) y límites.

Ejemplo paso a paso: una corrida típica

*(Ejemplo **ilustrativo**: salida construida de un simulador benigno, no de malware real.)*

```
Snapshot → ejecutar muestra_demo.exe en la VM (red = FakeNet-NG)

Procmon (filtrado):
  CreateFile   C:\Users\lab\AppData\Roaming\svc\host.exe   (SUCCESS)   ← se copia
  RegSetValue  HKCU\...\Run\Updater = %APPDATA%\svc\host.exe           ← persistencia
  Process Create  host.exe (hijo)

FakeNet / Wireshark:
  DNS  A  update.ejemplo-malware-demo.test      ← dominio C2 (simulado)
  HTTP GET http://update.ejemplo-malware-demo.test/gate.php?id=...

IOCs:
  file  %APPDATA%\svc\host.exe   sha256=...
  reg   HKCU\...\Run\Updater
  net   update.ejemplo-malware-demo.test
```

Con esa corrida ya tenés persistencia (clave `Run`), el binario soltado y el dominio de C2: tres IOCs para una regla de detección, sin exponer la red real.

Práctica

En tu VM aislada, con **snapshot** y una **muestra preparada para el curso o un simulador propio** (podés escribir un programa benigno que cree un archivo en `%APPDATA%`, ponga una clave `Run` y haga un GET a un dominio de prueba):

1. Tomá snapshot. Levantá **FakeNet-NG** o **INetSim**.
2. Corré la muestra con **Procmon** y **Regshot** capturando.
3. Extraé los IOCs (archivos, registro, procesos, red) con sus hashes.
4. Escribí un informe corto y **revertí el snapshot**.

Preguntas de repaso

▼ **¿Por qué se usa una red simulada (INetSim/FakeNet) en vez de Internet real?**

> Para ver a dónde **intenta** conectarse la muestra y qué pide, **sin** darle salida real: la red falsa responde como si hubiera Internet, así capturás el C2 sin exponerte ni avisar al atacante.

▼ **Nombrá las cuatro superficies que observás en dinámico.**

> **Archivos**, **registro** (persistencia), **procesos** (hijos/inyección) y **red** (C2/DNS/HTTP).

▼ **¿Qué es un IOC y para qué sirve acá?**

> Un *indicador de compromiso*: archivo+hash, clave de registro, dominio/IP, mutex. Sirve para **detectar** (alimentar una YARA o una regla) y para el informe.

▼ **¿Dónde se corre esto y con qué resguardos?**

> En una **VM aislada** (red host-only/apagada, sin carpetas compartidas), con **snapshot** antes y revert después; nunca en el host ni con malware real descargado para la ocasión.

▼ **¿Por qué el curso deja el caso Shelter (video 37) fuera del recorrido base?**

> Porque **ejecuta una muestra**: no es una práctica de principiante. Requiere laboratorio y método; por eso esto es una ruta **optativa y avanzada**.

Puerta de salida (optativa)

Podés: (1) montar un laboratorio aislado con red simulada y snapshots; (2) observar las cuatro superficies con Procmon/Regshot/Wireshark; (3) extraer IOCs con hashes; (4) redactar un informe de comportamiento. Todo sobre una muestra de curso o un simulador, nunca malware real en el host.

Fuentes de esta lección

- `Videos y material adicional, Video 37` — el caso Shelter: ejecutar una muestra; por qué queda fuera del recorrido base.
- Rama malware (triage estático) (5.1) — la base defensiva sin ejecutar, sobre la que se apoya esta ruta.
- Análisis dinámico seguro (4.4) — cuándo ejecutar y cuándo no, y cómo contener.

Capas de esta lección

Contenido observado en CLS: el caso Shelter (video 37) y la rama malware estática. **Conocimiento externo:** Procmon, Regshot, INetSim/FakeNet-NG, Noriben, el modelo de cuatro superficies y los IOCs. **Recomendación editorial:** las reglas de laboratorio, el enfoque de muestra-de-curso/simulador y el ejemplo ilustrativo (no hay un laboratorio real ni muestras en esta edición).

Ampliación editorial basada en fuentes externas — Anatomía de la inyección de procesos en memoria: Process Hollowing, Early Bird APC y Reflective Loading

En el análisis de software dinámico y amenazas defensivas, las técnicas de inyección en user-mode buscan ejecutar código dentro del espacio de direcciones virtual de un proceso legítimo para enmascarar su presencia (Microsoft Learn: Process Creation Flags):

Técnica de Inyección	Secuencia de APIs Clave	Comportamiento y Huella en Memoria	Detección y Caza
Inyección Clásica (CreateRemoteThread)	`OpenProcess` -> `VirtualAllocEx(RWX)` -> `WriteProcessMemory` -> `CreateRemoteThread`	Aloca un bloque independiente en un proceso ya en ejecución y crea un hilo remoto.	Interceptada por telemetría del kernel (`PsSetCreateThreadNotifyRoutine`) y monitores de API.
Process Hollowing (RunPE)	`CreateProcess(CREATE_SUSPENDED)` -> `NtUnmapViewOfSection` -> `VirtualAllocEx` -> `WriteProcessMemory` -> `SetThreadContext` -> `ResumeThread`	Vacia el binario legítimo original de la memoria del proceso y proyecta un ejecutable completamente nuevo.	Discrepancia entre la memoria mapeada y el archivo en disco (detectable con `PE-sieve` o inspección de VAD).
Early Bird APC Injection	`CreateProcess(CREATE_SUSPENDED)` -> `VirtualAllocEx` -> `WriteProcessMemory` -> `QueueUserAPC` -> `ResumeThread`	Encola una Asynchronous Procedure Call (APC) antes de que inicie el hilo principal, ejecutándose en el primer estado de alerta.	Monitoreo de llamadas a `QueueUserAPC` dirigidas al hilo primario de procesos recién creados.
Reflective DLL Injection	`VirtualAllocEx` -> `WriteProcessMemory` -> `CreateRemoteThread(offset_ReflectiveLoader)`	La DLL inyectada incluye su propio loader interno que resuelve imports y relocalizaciones sin usar `LoadLibrary` .	La DLL no figura en las listas del PEB (`InLoadOrderModuleList`), dejando páginas de memoria huérfanas con permisos de ejecución.

Flujo paso a paso de Process Hollowing (RunPE)

1. **Creación suspendida:** El cargador inicia un proceso del sistema benigno (ej. `svchost.exe`) pasando el flag `CREATE_SUSPENDED (0x00000004)` a `CreateProcessW` .
2. **Desmapeo:** Invoca `NtUnmapViewOfSection(pi.hProcess, baseImageOriginal)` para desalojar el ejecutable legítimo de la memoria virtual.
3. **Reserva y escritura:** Asigna memoria con `VirtualAllocEx` en la dirección base preferida del nuevo binario (`ImageBase`) y escribe las cabeceras PE y secciones con `WriteProcessMemory` .
4. **Reajuste de contexto:** Llama a `GetThreadContext(pi.hThread, &ctx)` , sobrescribe el registro del EntryPoint (`ctx.Rcx` en x64 o `ctx.Eax` en x86) con la nueva dirección del OEP, aplica `SetThreadContext` y despierta el proceso con `ResumeThread` .

Shellcode, red y C2: reconocer y describir

> **Ruta optativa · solo análisis y clasificación**
>
> Especialización, no requisito de 2C. Acá se **reconoce y describe** —a alto nivel— shellcode y tráfico de C2 en un entorno controlado. **No** se explica cómo escribir shellcode ni cómo montar un C2: esta guía es defensiva y se queda en identificar y clasificar. **Prerrequisitos:** PE e imports (1.4), malware dinámico defensivo (2.22), criptografía/encoding (4.2).

Cómo se reconoce que algo es shellcode

Shellcode es **código máquina suelto**, sin envoltura PE: no tiene cabecera, secciones ni tabla de imports. Para el análisis alcanza con **reconocerlo** por sus rasgos, sin entrar en cómo está hecho por dentro (**conocimiento externo**):

- **No es un PE:** aparece como un **blob de bytes** dentro de un buffer, un recurso (RCDATA), un documento o tráfico de red, no como un `.exe` / `.dll`.
- **No tiene IAT PE:** al ser un blob sin formato PE, no trae la tabla de imports PE. Eso **no demuestra** que resuelva APIs por su cuenta: puede recibir funciones o datos desde un cargador, no usar APIs, usar llamadas al sistema, o resolverlas en tiempo de ejecución. La ausencia de IAT por sí sola no identifica cuál de esos casos aplica.
- **Autosuficiente:** strings armadas al vuelo o cifradas, y saltos/llamadas por direcciones calculadas en vez de referencias con nombre.

El Curso IDA menciona shellcode en el contexto de exploiting (p. ej. el tute 40, que además avisa que técnicas viejas pueden no andar en Windows 10 moderno). Lo tomamos solo como **algo a identificar**, no a producir (`Curso IDA, tute 40`).

Analizar un blob sin ejecutarlo en tu sistema

La forma segura de ver **qué hace** un shellcode es un **emulador** que lo corre en una caja y te reporta su comportamiento, sin tocar tu Windows (**conocimiento externo**):

- `scdbg` (basado en libemu) o **blobrunner + debugger en la VM**: emulan el blob y listan **qué APIs resolvió y llamó**.
- Vos leés el **reporte** y deducís la intención: si llama a algo como `WinExec` / `CreateProcess`, ejecuta un programa; si descarga y corre, es un *downloader*; si reserva memoria y escribe en otro proceso, es inyección.

No hace falta entender byte a byte cómo se autoresuelve: te quedás con el **qué hace** que reporta el emulador.

C2: describir el tráfico

C2 (*command and control*) es cómo una muestra habla con el atacante. En el laboratorio de 2.22, con **red simulada** (INetSim/FakeNet) y Wireshark, **describís** el canal sin darle salida real:

- **Beacon:** conexiones periódicas a un dominio/IP (cada N segundos).
- **Protocolo:** HTTP(S) con User-Agent raro, DNS con subdominios largos (indicio de *DNS tunneling*), o un puerto TCP fijo.
- **Datos:** muchas veces base64 o XOR sobre el contenido (lo reconocés con 4.2 criptografía).

El entregable es una **descripción + IOCs de red** (dominios, IPs, patrón del beacon), no una réplica del canal.

Ejemplo: cómo se lee el reporte de un emulador

*(Ejemplo **ilustrativo**: salida resumida de un emulador tipo `scdbg`, no un blob real.)*

```
scdbg sobre blob_demo.bin (emulado, NO ejecutado en el host):
  APIs resueltas/llamadas:
    LoadLibraryA("wininet.dll")
    InternetOpenA / InternetOpenUrlA  → http://descarga.ejemplo-demo.test/p.bin
    CreateFileA("%TEMP%\p.bin")  → WriteFile
    WinExec("%TEMP%\p.bin")
  Lectura: es un DOWNLOADER → baja p.bin y lo ejecuta.
  IOCs: descarga.ejemplo-demo.test ; %TEMP%\p.bin
```

Con solo **leer qué APIs resolvió** el emulador, clasificás el shellcode (downloader) y sacás los IOCs, sin analizar su mecánica interna ni ejecutarlo en tu sistema.

Práctica

En tu VM aislada, con un **blob de curso** (o un programa benigno que simule el comportamiento):

1. Comprobá si el blob tiene envoltura PE; tratá la ausencia de cabecera/imports solo como un indicio y usá el contexto y el análisis como evidencia adicional.
2. Pasalo por `scdbg` (o un emulador equivalente) y leé qué APIs resuelve.
3. Clasificalo: ¿ejecuta, descarga, inyecta? Justificá con las APIs del reporte.
4. Si hay red, describí el beacon en el laboratorio simulado y anotá los IOCs.

Preguntas de repaso

▼ **¿Qué distingue a un shellcode de un `.exe` normal?**

> No tiene envoltura **PE** (cabecera, secciones ni IAT PE): es código suelto dentro de un buffer, recurso o tráfico. Eso no permite concluir por sí solo cómo obtiene funciones; puede depender de un cargador, no usar APIs o resolverlas durante la ejecución.

▼ **¿Cómo ves qué hace un shellcode sin ejecutarlo en tu máquina?**

> Con un **emulador** (`scdbg`/libemu, o blobrunner en la VM): corre el blob en una caja y reporta las **APIs que resuelve y llama**; vos leés ese reporte para clasificarlo.

▼ **Si el reporte muestra `InternetOpenUrlA` + `CreateFile` + `WinExec`, ¿qué es?**

> Un **downloader**: baja un archivo y lo ejecuta. La combinación de APIs delata la intención.

▼ **¿Qué es un beacon de C2 y cómo lo describís?**

> Conexiones **periódicas** de la muestra hacia su servidor. Lo describís por el **patrón** (cada N segundos), el protocolo (HTTP/DNS) y el dominio/IP, usando la red simulada; no replicás el canal.

▼ **¿Cuál es el límite de esta lección?**

> **Reconocer y clasificar**. No se enseña a escribir shellcode ni a montar un C2; el trabajo se queda en análisis defensivo e IOCs.

Puerta de salida (optativa)

Podés: (1) explicar por qué la ausencia de PE/IAT no basta para demostrar que un blob es shellcode ni cómo resuelve funciones; (2) usar un emulador para observar y clasificar comportamiento; (3) describir un beacon de C2 en el laboratorio simulado; (4) entregar IOCs de red. Siempre en análisis, nunca en construcción.

Fuentes de esta lección

- `Curso IDA, tute 40` — menciona shellcode en contexto de exploiting (y que técnicas viejas pueden no andar en Win10).
- Malware dinámico defensivo (2.22) — el laboratorio aislado y la red simulada donde se observa el C2.
- Reconocer criptografía y encoding (4.2) — base64/XOR en el tráfico y los payloads.

Capas de esta lección

Contenido observado en CLS: la mención de shellcode del tute 40. **Conocimiento externo:** la identificación de código sin envoltura PE, aclarando que la ausencia de IAT no demuestra por sí sola resolución dinámica, los emuladores (`scdbg`/libemu) y el modelo de C2 (beacon, protocolo, IOCs). **Recomendación editorial:** el enfoque estricto de reconocer-y-clasificar, el ejemplo ilustrativo de salida de emulador y el límite de no construir.

Fuzzing y análisis simbólico

> **Ruta optativa · sobre binarios propios**
>
> Especialización, no requisito de 2C. Se practica con un **parser o crackme propio** y busca **encontrar y entender fallos** (bugs de robustez) y condiciones de entrada, en clave defensiva. **Prerrequisitos:** C/compilación (1.5), WinDbg/heap (2.19), método de análisis (4.1).

Dos técnicas que **automatizan** la búsqueda de problemas, cada una por un camino distinto:

Fuzzing: romper a fuerza de entradas

El fuzzing encuentra fallos en el parser, pero no sustituye la reconstrucción de su estructura: para deducir campos y framing desde muestras, consulta 4.5.

Fuzzear es alimentar al programa con **muchísimas entradas** (mutadas o generadas) y ver cuáles lo hacen **crashear o colgarse**. Un crash es una pista de un bug (overflow, desreferencia nula, consumo de memoria). El flujo (**conocimiento externo**):

1. **Elegí un objetivo chico:** un **parser propio** (lee un formato de archivo o un input de texto) o un crackme. Cuanto más acotado, mejor.
2. **Semillas:** juntá entradas válidas de ejemplo (*corpus* inicial).
3. **Fuzzer:** herramientas como **AFL++**, **libFuzzer** (si compilás con instrumentación) o **WinAFL** en Windows. Mutan el corpus y maximizan la **cobertura** de código.
4. **Guardá todo:** cada entrada que provoca un crash se **guarda** tal cual, para reproducir.
5. **Triage:** reproducís el crash (con Page Heap de 2.19 si es de memoria), lo minimizás y explicás la causa.

La **reproducibilidad** es el punto: un fuzzing sirve si podés volver a disparar el crash con la entrada guardada y señalar la línea.

Reducir el corpus y minimizar un crash son tareas distintas

El corpus puede crecer con casos que recorren los mismos caminos. En libFuzzer, -merge=1 permite construir un corpus reducido conservando las entradas que aportan cobertura nueva; no promete conservar todas las propiedades semánticas ni elegir el archivo más corto para cada camino. Conserva el corpus original y versiona el corpus curado por separado.

Minimizar un **crash concreto** busca una entrada más pequeña que siga reproduciendo ese mismo fallo; libFuzzer ofrece -minimize_crash=1 con un límite de ejecuciones/tiempo. Guarda siempre el crash original, la versión minimizada, hashes y el comando exacto. Los flags cambian entre fuzzers: consulta la documentación de la herramienta y no uses una opción de corpus como sustituto de reproducir el fallo.

Análisis simbólico: resolver en vez de adivinar

En vez de probar entradas al azar, la **ejecución simbólica** trata la entrada como una **incógnita** y, con un *solver* (SMT), calcula **qué valor** hace que la ejecución tome un camino. Sirve para "¿qué entrada llega a este bloque?" sin fuerza bruta (**conocimiento externo**). Herramientas: **angr**, **Triton**, **Miasm** (el "taller actual" de 2.9 las nombra). Dónde brilla:

- Un crackme con una comparación compleja: angr puede **hallar la clave** resolviendo las restricciones hasta el "chico bueno".
- Encontrar la entrada que alcanza un bloque de error que el fuzzer no pega por azar.

Límite práctico: la **explosión de caminos**. En binarios grandes el árbol de estados crece muchísimo; por eso conviene **acotar** (empezar desde una función, fijar longitudes) y combinarlo con fuzzing.

Ejemplo: fuzzing + simbólico sobre un parser propio

*(Ejemplo **ilustrativo**: flujo y salida construidos.)*

```
Objetivo: parser propio  parse_registro(archivo)   (lee un header + N bytes)

Fuzzing (WinAFL, 2 h, corpus de 5 archivos válidos):
  crashes/ id:000  → len declarado 0xFFFF con archivo de 20 bytes
  Triage con Page Heap: AV en parse_registro+0x3e (lee fuera del buffer)
  Causa: confía en el 'len' del header sin validarlo contra el tamaño real.

Simbólico (angr) para alcanzar el bloque "formato_v2":
  entrada hallada: primeros 4 bytes = "RG02"  → toma ese camino
```

El fuzzer encontró un bug de robustez (no valida `len`); angr encontró **qué bytes** activan una rama concreta. Los dos resultados son **reproducibles** y señalan una línea.

libFuzzer necesita la firma `LLVMFuzzerTestOneInput` y un flag de compilación

Los flags `-merge=1` / `-minimize_crash=1` son del **binario instrumentado**, no de un `main()` normal. Hay que exportar `int LLVMFuzzerTestOneInput(const uint8_t *Data, size_t Size)` y compilar con el runtime de fuzzing:

```
:: MSVC (VS 2022 Developer Command Prompt x64):
cl /fsanitize=fuzzer /fsanitize=address /Zi /Od fuzz_clsf.c /Fe:fuzz_clsf.exe
:: Clang en Windows:
clang -g -O1 -fsanitize=fuzzer,address fuzz_clsf.c -o fuzz_clsf.exe
:: Ejecutar y minimizar un crash:
mkdir corpus && fuzz_clsf.exe corpus -max_total_time=60
fuzz_clsf.exe -minimize_crash=1 -max_total_time=30 crash-xxxx
```

El harness mínimo valida la cabecera y deja un *out-of-bounds* a propósito si comentás el chequeo de longitud:

```
int LLVMFuzzerTestOneInput(const uint8_t *Data, size_t Size) {
    if (Size < sizeof(CabeceraCLSF)) return 0;
    const CabeceraCLSF *h = (const CabeceraCLSF *)Data;
    if (memcmp(h->magic, "CLSF", 4) != 0 || h->version != 1) return 0;
    if ((size_t)h->hdr_len + h->body_len > Size) return 0;   // comentá esto y ASan caza el
OOB
    volatile uint8_t sum = 0;
    for (uint16_t i = 0; i < h->body_len; ++i) sum ^= Data[h->hdr_len + i];
    return 0;
}
```

Práctica

En tu VM, con un **parser propio** y datos de prueba inventados:

1. Definí un harness determinista que procese una entrada y termine sin depender de red, reloj, rutas personales ni servicios externos.
2. Prepará un corpus pequeño de archivos válidos y anotá qué estructura cubre cada uno. Guardá una copia inmutable del corpus inicial.
3. Activá cobertura si la herramienta lo permite; ejecutá con límites razonables de tiempo y memoria. Conservá cada entrada que produzca un fallo.
4. Reproducí un crash sin el fuzzer, minimizá esa entrada para conservar el mismo fallo y registra versión de herramienta, comando, hash del original y del caso reducido, y pila/síntoma.
5. Reducí por separado el corpus preservando cobertura (en libFuzzer, -merge=1); compara cobertura y cantidad de casos. No borres el corpus original ni los crashes.
6. Corregí el parser y añadí el caso reducido como regresión; comprobá que los ejemplos válidos siguen pasando.

7. Como ejercicio separado, usá angr sobre un binario propio pequeño para hallar una entrada que alcance una rama concreta; explicá qué restricciones resolvió y qué quedó fuera del modelo.

Preguntas de repaso

▼ **¿Qué es fuzzing y qué buscás con un crash?**

> Alimentar el programa con **muchas entradas** (mutadas/generadas) para ver cuáles lo rompen. Un **crash** es la pista de un bug (overflow, null deref, etc.); lo guardás para reproducir y explicar la causa.

▼ **¿Por qué guardás cada entrada que crashea?**

> Por **reproducibilidad**: un hallazgo sirve si podés volver a disparar el crash con esa entrada exacta, minimizarla y señalar la línea culpable.

▼ **¿En qué se diferencia el análisis simbólico del fuzzing?**

> El fuzzing **prueba** entradas (al azar/mutadas); el simbólico trata la entrada como **incógnita** y un *solver* **calcula** qué valor toma un camino. Uno adivina con cobertura; el otro resuelve restricciones.

▼ **¿Cuál es el límite típico del análisis simbólico?**

> La **explosión de caminos**: el árbol de estados crece muchísimo en binarios grandes. Se acota (desde una función, fijando longitudes) y se combina con fuzzing.

▼ **¿Qué herramientas nombrás para cada técnica?**

> Fuzzing: **AFL++**, **libFuzzer**, **WinAFL**. Simbólico: **angr**, **Triton**, **Miasm**.

> **Puerta de salida (optativa)**
>
> Podés: (1) fuzzear un parser/crackme propio y guardar los crashes; (2) reproducir y explicar un crash con Page Heap; (3) usar ejecución simbólica para hallar la entrada que alcanza un bloque; (4) minimizar un crash reproducible y un corpus conservando cobertura; explicar la explosión de caminos y cuándo combinar ambas.

Fuentes de esta lección

- LLVM: libFuzzer: coverage-guided fuzz testing.
- Herramientas: el taller actual (2.9) — menciona angr, Unicorn, Triton y Miasm en el taller moderno.
- WinDbg, TTD y heap (2.19) — Page Heap para reproducir y diagnosticar los crashes de memoria del fuzzing.

Capas de esta lección

Contenido observado en CLS: la mención de angr/Triton/Miasm en el taller actual (2.9).
Conocimiento externo: fuzzing (AFL++/libFuzzer/WinAFL), ejecución simbólica (angr) y la explosión de caminos. **Recomendación editorial:** el enfoque sobre binarios propios, el ejemplo ilustrativo y la práctica de triage reproducible.

Ejecución simbólica y concólica: desofuscar con un solver

> **Objetivo de la lección**
>
> Entender qué es la **ejecución simbólica** y la **concólica**, y usarlas para recuperar las condiciones de un binario propio ofuscado (qué entrada lo lleva a una rama) y para **simplificar** expresiones enredadas. Saber **cuándo no sirve** y por qué. **Prerrequisitos:** fundamentos de simbólico (2.24), ofuscación nativa (2.10), instrucciones (2.2) y scripting (2.21). *Contexto recomendado:* método de análisis (4.1).

Esto es **análisis**, no evasión: se usa el solver sobre **binarios propios o crackmes didácticos** para entender su lógica. No se producen cracks, keygens ni parches de software comercial.

Qué es la ejecución simbólica

En la ejecución **concreta** (la de siempre), una variable tiene un valor: `eax = 0x41`. En la **simbólica**, la variable es una *incógnita* —la llamamos `x`— y el motor va anotando, instrucción por instrucción, **qué operaciones se le hicieron**. Cuando el programa llega a un salto condicional, en vez de tomar un camino, el motor guarda la **restricción** (*path constraint*) que haría cierto ese salto y puede explorar **los dos lados** (**conocimiento externo**).

Al final, cada camino es una lista de restricciones del estilo «`(x ^ 0x23) + 7 == 0x62`». Esa lista se le entrega a un **solver SMT** (típicamente **Z3**), que responde una de dos cosas: *insatisfacible* (ninguna entrada llega acá) o *satisfacible* y, de paso, **un valor concreto de** `x` que cumple. Ese valor es, muchas veces, la «clave» que buscabas: en vez de leer y revertir el algoritmo a mano, le preguntás al solver qué entrada produce el resultado.

Por qué «concólica» (concrete + symbolic)

El simbólico puro se ahoga cuando el programa llama al sistema operativo, lee un archivo o depende del entorno: no todo se puede dejar como incógnita. La **ejecución concólica** mezcla las dos: mantiene simbólico **lo que te interesa** (la entrada del usuario) y **concretiza** el resto (una syscall devuelve un valor fijo, una librería se reemplaza por un modelo). Así el árbol de caminos no explota por cosas que no te importan. En la práctica, casi siempre vas a trabajar en modo concólico.

Los tres motores que vas a encontrar

Motor	Cómo trabaja	Cuándo elegirlo
angr (Python)	Levanta el binario a VEX IR y emula con estados simbólicos; `SimulationManager` gestiona los caminos	El más completo para «encontrá la entrada que llega acá»; mucho ejemplo de crackmes
Triton (C++/Python)	Construye el AST simbólico de cada instrucción x86/x64 y lo resuelve con Z3; se engancha a una traza	Cuando ya tenés una *traza* concreta y querés razonar sobre ella (ideal junto a DBI, 2.29)
Miasm (Python)	IR propio, ensambla/desensambla, emula y hace simbólico; buen *lifting*	Reconstrucción de flujo y desofuscación de rutinas puntuales

Los tres apoyan abajo en un solver: **Z3** (de Microsoft Research) es el más usado. No hace falta escribir Z3 a mano para empezar —angr lo maneja por vos—, pero conviene entender que ahí abajo hay un demostrador de teoremas sobre *bitvectors*.

El enemigo: explosión de caminos

Cada salto condicional **duplica** la cantidad de estados a seguir. Un bucle que corre 32 veces con una rama adentro son `2³²` caminos: inmanejable. La ofuscación **empeora** esto a propósito —el **aplanamiento de flujo** (2.10) convierte un programa lineal en un *switch* gigante, y los **predicados opacos** agregan ramas falsas—. Mitigaciones:

- **Exploración dirigida:** decile al motor *adónde* querés llegar (`find=`) y *qué* evitar (`avoid=`), para que no explore todo.
- **Concretizar:** fijá la longitud de la entrada, modelá las syscalls, saltá funciones de librería con *SimProcedures*.
- **Veritesting / fusión de estados:** angr puede unir caminos equivalentes para no multiplicarlos.
- **Límites:** cota de pasos, de memoria y de tiempo; si no cierra, cambiá de técnica (trazá con DBI, 2.29, y pasale la traza a Triton).

Simplificar expresiones y MBA

Una técnica de ofuscación frecuente es el **MBA** (*mixed boolean-arithmetic*): reemplazar `x + y` por una maraña equivalente de `and`, `or`, `xor` y sumas. El solver sirve para **probar que dos expresiones son iguales** y así deshacer la maraña. La identidad clásica:

```
# Z3: demostrar que una expresión MBA es, en realidad, una suma
from z3 import BitVec, prove
x, y = BitVec("x", 32), BitVec("y", 32)
prove((x ^ y) + 2 * (x & y) == x + y)   # imprime: proved
```

Si `prove` dice *proved*, esa expresión complicada era `x + y` disfrazada. Lo mismo sirve para confirmar que un chequeo enrevesado equivale a un simple `cmp`.

Ejemplo paso a paso: resolver un crackme propio con angr

*(Ejemplo **ilustrativo**: código de referencia sobre un binario propio; no es un volcado de una corrida verificada en esta edición.)*

Como el curso es **solo Windows**, no entramos por `stdin` ni leemos la salida con `posix.dumps` (eso es de binarios ELF/Linux; en un PE de MSVC devuelve vacío y `simgr.found` queda siempre `[]`). Arrancamos **directo en la función de validación** con `blank_state`, ponemos la entrada en memoria y dirigimos la búsqueda por **direcciones de bloque** (el que devuelve «correcto» vs. el que devuelve «incorrecto»):

```
# resolver.py — crackme PROPIO, PE de Windows x64.
# Entra directo en validar(const char *clave), sin pasar por el CRT.
import angr, claripy

proj = angr.Project("./crackme_propio.exe", auto_load_libs=False)

ADDR_VALIDAR = 0x140001000   # prólogo de validar()
ADDR_EXITO   = 0x140001085   # bloque que devuelve 1 (Correcto)
ADDR_FALLO   = 0x140001090   # bloque que devuelve 0 (Incorrecto)
BUF_ADDR     = 0x140050000   # memoria escribible donde alojar la clave

clave = claripy.BVS("clave", 16 * 8)
estado = proj.factory.blank_state(addr=ADDR_VALIDAR)

# escribir el bitvector en memoria y apuntar RCX (1.er arg en Windows x64) al buffer
estado.memory.store(BUF_ADDR, clave)
estado.regs.rcx = BUF_ADDR

# 15 bytes ASCII imprimible + terminador nulo
for b in clave.chop(8)[:-1]:
    estado.solver.add(b >= 0x20, b <= 0x7e)
estado.solver.add(clave.chop(8)[-1] == 0x00)

simgr = proj.factory.simulation_manager(estado)
simgr.explore(find=ADDR_EXITO, avoid=ADDR_FALLO)

if simgr.found:
    s = simgr.found[0]
    print("clave:", s.solver.eval(clave, cast_to=bytes).rstrip(b"\x00").decode())
```

Qué hizo: arrancó el estado **en el prólogo de `validar()`** (con `blank_state`, no en el *entry point*, para no modelar el CRT), puso 16 bytes simbólicos en una dirección escribible y apuntó `RCX` ahí (primer argumento en la ABI de Windows x64), y dirigió la búsqueda a la **dirección** del bloque «correcto» evitando la del «incorrecto» (esas direcciones las sacás en IDA/Ghidra). Si **no** encuentra nada o tarda infinito, la lección está en el *porqué*: explosión de caminos, un bucle sin cota, o una comprobación que depende de algo que no modelaste.

En Windows: no modeles el CRT, entrá por la función de validación

Alimentar la entrada por `stdin` en un `.exe` de MSVC arrastra a angr por el runtime de C (`ReadFile`, buffers de `conio`…) y dispara *timeouts* o explosión de caminos que no te interesan. Dos salidas: (1) pasá la entrada por `sys.argv[1]` en vez de stdin; (2) la del ejemplo de arriba — `blank_state(addr=…)` en el prólogo de la validación (no `entry_state`, que es para el *entry point*), con la entrada en memoria y `RCX` apuntándola—. Y nunca guíes `find` / `avoid` con `posix.dumps(1)` / `stdin`: eso lee el descriptor POSIX de Linux; en un PE de Windows la salida va por `WriteFile` / `ucrtbase`, así que dirigí por **direcciones de bloque**.

Práctica

En tu VM, con un **crackme propio** (compilá uno en C con una comprobación aritmética sobre la entrada):

1. Escribí la comprobación a propósito como una cuenta (p. ej. `(entrada[i] ^ 0x5A) + i == tabla[i]`) y compilalo.
2. Resolvelo con angr usando `find` / `avoid`; verificá que la clave que da el solver es aceptada por el programa real.
3. Agregá un bucle grande o una rama extra y observá cómo cambia el tiempo: explicá qué pasó con los caminos.
4. Tomá una línea con MBA (usá la identidad de arriba) y demostrá con Z3 a qué operación simple equivale.

Preguntas de repaso

▼ **¿Qué le pedís en el fondo a un solver SMT?**

> Que decida si un conjunto de restricciones sobre *bitvectors* es **satisfacible** y, si lo es, que te dé **un valor concreto** de las incógnitas que las cumple. Ese valor suele ser la entrada que buscabas.

▼ **¿Por qué casi siempre se trabaja en modo concólico y no simbólico puro?**

> Porque dejar **todo** simbólico (syscalls, entorno, librerías) hace explotar el árbol de caminos. El concólico mantiene simbólica solo la entrada de interés y **concretiza** el resto, acotando el problema.

▼ **¿Cómo empeora la ofuscación la ejecución simbólica?**

> El **aplanamiento de flujo** multiplica las ramas (un *switch* gigante) y los **predicados opacos** agregan caminos falsos. Más ramas = más estados = explosión de caminos. Por eso se usa exploración dirigida, fusión de estados y límites.

▼ **¿Para qué sirve `prove` de Z3 frente a un MBA?**

> Para **demostrar que la expresión enrevesada es igual** a una operación simple (p. ej. que `(x^y)+2*(x&y)` es `x+y`). Así deshacés la maraña con una garantía matemática, no por inspección a ojo.

▼ **El motor no encuentra solución ni termina. ¿Qué revisás?**

> Si hay un **bucle sin cota**, si falta **concretizar** algo (longitud de entrada, una syscall), si conviene **dirigir** con `find` / `avoid`, o si el binario está virtualizado (2.26) y hay que trazarlo con DBI (2.29) antes.

> **Puerta de salida**
>
> Podés: (1) explicar qué es un estado simbólico y un *path constraint*; (2) resolver un crackme propio con angr dirigiendo la búsqueda; (3) nombrar dos causas de explosión de caminos y sus mitigaciones; (4) demostrar con Z3 que un MBA equivale a una operación simple.

Fuentes de esta lección

- docs.angr.io — `SimulationManager`, `explore(find=, avoid=)`, *SimProcedures*, veritesting (conocimiento externo, documentación primaria).
- triton-library.github.io — AST simbólico por instrucción y resolución con Z3.
- github.com/cea-sec/miasm — IR, emulación y simbólico.
- github.com/Z3Prover/z3 — el solver SMT; API de Python para *bitvectors*.
- 2.24 Fuzzing y simbólico — la introducción sobre la que se apoya esta lección.

> **Capas de esta lección**
>
> **Contenido observado en CLS:** la ofuscación y el aplanamiento de flujo de los capítulos 26/30 (vistos en 2.10), como el problema que motiva el solver. **Conocimiento externo (agregado por esta edición):** ejecución simbólica/concólica, angr/Triton/Miasm/Z3, explosión de caminos y MBA. **Recomendación editorial:** los ejemplos y la práctica son ilustrativos; no hay laboratorio compilado con hashes en esta edición.

Ofuscación por máquina virtual: dispatcher, bytecode y handlers

Objetivo de la lección

Reconocer un binario protegido por virtualización de código e identificar sus piezas (*dispatcher*, puntero de instrucción virtual, tabla de *handlers* y región de *bytecode*), y entender a alto nivel cómo se **devirtualiza**. La práctica es sobre una **mini-VM propia**. **Prerrequisitos:** ofuscación nativa (2.10) y ejecución simbólica (2.25).

Límite declarado

Esta lección enseña a **reconocer y entender** la virtualización. **No** se desvirtualizan protectores comerciales (VMProtect, Themida/WinLicense) ni se practica sobre software ajeno: eso queda fuera de alcance. Se practica sobre una VM didáctica que vos mismo escribís.

Qué es virtualizar código

En vez de proteger el x86 original, el protector lo **traduce a un bytecode propio** —un juego de instrucciones inventado, distinto en cada binario— y embebe un **intérprete** que lo ejecuta. El resultado: cuando abrís la función en IDA/Ghidra, el x86 original **ya no está**; lo que ves es el intérprete corriendo un «programa» que no entendés, porque su lenguaje es secreto (**conocimiento externo**). Es la protección nativa más dura que vas a cruzar.

Anatomía de una VM de protección

- **Bytecode:** una región de datos (a menudo cifrada) con el «programa» virtual.
- **VIP** (*virtual instruction pointer*): puntero al bytecode; hace de «EIP» de la VM.
- **VSP** (*virtual stack pointer*): la VM suele ser de pila; opera sobre su propio stack.
- **Contexto virtual:** un bloque de memoria con los «registros» virtuales.
- **VMEnter / VMExit:** al entrar, guarda los registros nativos (típico `pushad` / `pushfd`); al salir, los restaura. Marca la frontera entre lo nativo y lo virtual.
- **Dispatcher:** el gran bucle *fetch → decode → dispatch*: lee el opcode que apunta el VIP y salta al handler que lo implementa, normalmente por una **tabla** (salto indirecto).
- **Handlers:** muchas rutinas chicas, una por operación virtual (sumar, xor, cargar, saltar…). Terminan volviendo al dispatcher.

Cómo reconocerla (las señales)

1. Un **bucle central enorme** con un **salto indirecto** a una tabla de direcciones.
2. **Muchos bloques pequeños** parecidos entre sí, todos terminando en el mismo lugar (vuelta al dispatcher): son los handlers.

3. Un **guardado masivo de contexto** al entrar y restaurarlo al salir (frontera VMEnter/VMExit).
4. Una **región de datos** que se recorre con un puntero que solo avanza (el VIP sobre el bytecode).
5. El *decompiler* produce algo gigante y sin sentido para una función que debería ser corta.

> **Si no ves un despachador central: *threaded code***
>
> No todas las VM convergen en un único `switch`. En protectores agresivos (VMProtect, Tigress en modo duro) se usa *indirect threading*: **cada handler termina con su propio *fetch* **del siguiente opcode y** salta directo al siguiente handler** (`jmp [tabla_handlers + rbx*8]`), sin volver a un despachador común. Si en IDA no aparece un bucle que converja en un solo bloque, buscá handlers encadenados que saltan de uno a otro por una tabla: también es una VM, solo que sin dispatcher central.

Devirtualizar: la idea, a alto nivel

No se «desvirtualiza con un botón». El flujo de trabajo de investigación es:

1. **Ubicar** el dispatcher y la tabla de handlers; aislar la frontera VMEnter/VMExit.
2. **Trazar** una corrida (con DBI, 2.29, o Triton) para ver qué handlers se ejecutan y en qué orden.
3. **Dar semántica** a cada handler: qué operación x86 equivale (este suma, este hace xor, este salta).
4. **Levantar** la traza a una IR (Triton/Miasm) y **simplificar** (deshacer MBA con el solver, 2.25), hasta recuperar la lógica original.

Es trabajoso y específico de cada VM. Para aprender el mecanismo —sin tocar un protector comercial— lo mejor es construir y analizar una VM propia.

Ejemplo paso a paso: una mini-VM didáctica

*(Ejemplo **ilustrativo**: VM propia en C para ver el patrón* fetch-decode-dispatch*, no un volcado de un protector.)*

```
// mini_vm.c — intérprete de un bytecode propio (didáctico)
enum { OP_SET=1, OP_XOR, OP_ADD, OP_CMP, OP_JNZ, OP_HALT };

// "programa" virtual: SET acc,0x41 ; XOR acc,0x23 ; CMP acc,0x62 ; HALT
unsigned char bytecode[] = { OP_SET,0x41,  OP_XOR,0x23,  OP_CMP,0x62,  OP_HALT };

int vm(unsigned char *vip) {       // vip = puntero de instrucción virtual
    int acc = 0, zf = 0;
    for (;;) {
        unsigned char op = *vip++;         // FETCH
        switch (op) {                       // DECODE + DISPATCH (la tabla)
            case OP_SET: acc  = *vip++;             break;   // handler SET
            case OP_XOR: acc ^= *vip++;             break;   // handler XOR
            case OP_ADD: acc += *vip++;             break;   // handler ADD
            case OP_CMP: zf   = (acc == *vip++);    break;   // handler CMP
            case OP_JNZ: { char d=*vip++; if(!zf) vip+=d; } break;
            case OP_HALT: return acc;                        // VMExit
        }
    }
}
```

Reconocé las piezas: `vip` es el VIP, el `for(;;)` + `switch` es el **dispatcher**, cada `case` es un **handler** y `bytecode[]` es el programa virtual. Un protector real tiene handlers cifrados, decenas de operaciones, el bytecode ofuscado y la frontera VMEnter/VMExit con `pushad`; pero el **esqueleto es este**. Si lo compilás y lo abrís en IDA, vas a ver el bucle con el salto a tabla: esa es la huella que hay que aprender a reconocer.

Práctica

En tu VM, sobre tu **propia** mini-VM:

1. Compilá `mini_vm.c` y abrilo en IDA/Ghidra; señalá el dispatcher, la tabla de saltos y la región de bytecode.
2. Agregá dos operaciones nuevas (p. ej. `OP_SHL` y `OP_SUB`) y mirá cómo crece la tabla de handlers.
3. A partir del binario (sin mirar el fuente), **reconstruí el programa virtual**: qué hace cada opcode y qué calcula `bytecode[]`.
4. Escribí en dos renglones por qué un *decompiler* no puede recuperar la lógica original mirando solo el intérprete.

Preguntas de repaso

▼ **¿Qué le pasó al x86 original cuando una función está virtualizada?**

> Se **tradujo a un bytecode propio** del protector y se reemplazó por un **intérprete** que lo ejecuta. El x86 original ya no aparece en el listado; ves el intérprete corriendo un «programa» en un lenguaje secreto.

▼ **Nombrá tres piezas de una VM de protección.**

> **Dispatcher** (el bucle fetch-decode-dispatch), **tabla de handlers** (una rutina por operación virtual), **VIP** (puntero al bytecode). También VSP, contexto virtual y la frontera VMEnter/VMExit.

▼ **¿Cuál es la señal más característica en el listado?**

> Un **bucle central** con un **salto indirecto a una tabla** y **muchos handlers chicos** que vuelven todos al mismo punto. Más el guardado masivo de contexto al entrar.

▼ **¿Por qué no se «desvirtualiza con un botón»?**

> Porque el juego de instrucciones es **distinto en cada binario**. Hay que darle semántica a cada handler, trazar una corrida, levantar a IR y simplificar. Es trabajo específico de cada VM.

▼ **¿Dónde queda la línea de lo que se practica?**

> Sobre una **VM propia** y para **reconocer** la técnica en general. Desvirtualizar protectores comerciales o software ajeno está **fuera de alcance**.

Puerta de salida

Podés: (1) explicar qué significa virtualizar código y por qué desaparece el x86 original; (2) identificar dispatcher, tabla de handlers, VIP y bytecode en tu mini-VM; (3) listar las señales de virtualización en un listado; (4) describir el flujo de devirtualización a alto nivel (trazar → dar semántica → levantar → simplificar).

Fuentes de esta lección

- Rolles, «Unpacking Virtualization Obfuscators» (USENIX WOOT 2009) — anatomía dispatcher/handlers y estrategia de devirtualización (conocimiento externo).
- tigress.wtf — obfuscador académico (Collberg) que incluye virtualización; sirve para generar muestras propias de estudio.
- Triton y Miasm — *lifting* y simplificación de trazas.
- 2.10 Ofuscación nativa y 2.25 Simbólico — el aplanamiento de flujo y el solver que se usan al simplificar.

Capas de esta lección

Contenido observado en CLS: la ofuscación nativa de 2.10 como escalón previo. **Conocimiento externo (agregado por esta edición):** la arquitectura de las VM de protección y el flujo de devirtualización, basados en literatura pública. **Recomendación editorial:** la mini-VM, los ejemplos y la práctica son ilustrativos; no se analizó ningún protector comercial.

Protectores comerciales y packers multicapa: reconocer, no evadir

Objetivo de la lección

Identificar qué familia de protector tiene un binario y reconocer una protección **multicapa** (packer sobre cifrado sobre VM), y **planear** un análisis seguro. **Prerrequisitos:** packers didácticos (2.17), taxonomía anti-análisis (2.15) y ofuscación por VM (2.26).

Encuadre legal y de alcance

Esta lección es de **reconocimiento**. **No** se desempaquetan protectores comerciales (Themida, WinLicense, VMProtect, Enigma, ACProtect) como práctica, ni se producen cracks, keygens o parches. La práctica usa **binarios propios** empaquetados con herramientas libres, o malware estudiado **en estático**. Desempaquetar protectores está fuera de alcance.

Packer vs protector

Un **packer** comprime el ejecutable y lo descomprime en memoria al arrancar (su objetivo original era el tamaño). Un **protector** suma **capas de defensa**: cifrado, anti-debug, anti-VM, anti-dump, verificación de integridad y, en la gama alta, **virtualización** (2.26) de las funciones críticas. En 2.17 desempaquetaste un UPX propio llegando al **OEP** (*original entry point*); acá subimos de dificultad para **reconocer** lo que no vas a poder —ni deber— desempaquetar.

Familias y sus huellas

Familia	Tipo	Huellas típicas (conocimiento externo)
UPX	Packer	Secciones `UPX0` / `UPX1`, IAT mínima, alta entropía; se reconoce con `upx -t`
ASPack / PECompact	Packer	Secciones `.aspack` / `.adata`; stub chico
ASProtect	Protector	SEH como flujo, IAT redirigida, trucos de robo de código
Enigma / Obsidium	Protector	Licenciamiento, anti-debug fuerte, VM parcial
Themida / WinLicense	Protector	Anti-todo, virtualización pesada, nombres de sección aleatorios
VMProtect	Protector (VM)	Virtualización dominante, poco código nativo, sección de bytecode
.NET Reactor / ConfuserEx	Protector .NET	Nombres *mangled*, control-flow en IL, anti-tamper (ver 3.1)

Las capas de una protección dura

Un protector serio no es una capa, son varias apiladas:

1. **Compresión / cifrado del cuerpo:** el código real no está legible en disco (entropía alta, casi sin imports).
2. **Stub de arranque:** descifra/descomprime en memoria y reconstruye la IAT.
3. **Anti-análisis:** chequeos de debugger/VM/tiempo (2.15) que abortan si te detectan.
4. **Anti-dump y anti-attach:** borran encabezados en memoria, falsean tamaños, detectan *attach*.
5. **Virtualización** de las rutinas críticas (licencia, checks): lo más duro de todo.

Por eso el **OEP** que en UPX era un salto limpio acá **no existe** como punto único: hay código robado, re-cifrado por tramos y funciones que nunca vuelven a ser x86 nativo. Reconocer esto es saber *cuándo parar*.

Herramientas de reconocimiento

- **Detect It Easy (DIE):** identifica packer/protector/compilador por firmas y muestra la **entropía por sección**.
- **PE-bear:** inspección de secciones, encabezados y directorios sin ejecutar nada.
- **Entropía:** una sección con entropía ~7,9/8 es casi seguro comprimida o cifrada; una sección de código normal ronda 6–6,5.
- **Imports:** una IAT con solo `LoadLibrary` / `GetProcAddress` grita «empaquetado».

> **IAT vacía: *splicing* y emulación de APIs**
>
> Los protectores duros van más allá de redirigir la IAT: la dejan **vacía o con entradas ficticias** y reemplazan cada llamada a una API importada por un *stub* propio que **emula la API** o ejecuta la *syscall* directamente. Por eso una IAT casi inexistente en un binario que claramente usa el sistema (abre archivos, red, registro) no es un error de lectura: es la huella de que los imports se resuelven y se envuelven en runtime. Reconstruir esa tabla es parte de lo que hace difícil —y, en software comercial, fuera de alcance— el análisis.

Ejemplo paso a paso: leer las capas de un binario propio

*(Ejemplo **ilustrativo**: valores típicos de un binario propio empaquetado; no un volcado verificado.)*

```
# empaquetar un binario PROPIO con una herramienta libre y mirar sus huellas
upx -9 hola_propio.exe                 # capa 1: compresión (didáctica)

# luego, con Detect It Easy en modo consola:
diec --entropy hola_propio.exe
# Sección   Entropía   Estado
# UPX0        0.00      (vacía en disco; se llena en memoria)
# UPX1        7.91      packed    <- alta: comprimida
# .rsrc       4.20      not packed

diec hola_propio.exe
# packer: UPX(4.x)[NRV,best]    <- identificación por firma
```

Qué leíste **sin ejecutar**: dos secciones con nombres de UPX, una vacía en disco y otra con entropía ~7,9 (comprimida), y la firma del packer. Si encima hubiera una **segunda capa** propia (p. ej. un XOR del cuerpo

hecho por vos para practicar), DIE mostraría entropía alta en otra sección y **no** reconocería firma: esa es la pista de «multicapa / a medida». El **plan** de análisis se escribe antes de tocar nada: qué capa resuelvo primero, qué espero ver, dónde está el riesgo.

Práctica

En tu VM, con binarios **propios**:

1. Compilá un «hola mundo» propio, empaquetalo con UPX e identificalo con DIE; anotá secciones y entropías.
2. Agregá una **segunda capa propia** (cifrá el cuerpo con un XOR en un stub que vos escribís) y volvé a medir: ¿qué cambió en la entropía y en la firma?
3. Tomá tres nombres de protector de la tabla y describí, de memoria, una huella de cada uno.
4. Escribí un **plan de análisis** de media página para un binario multicapa hipotético: orden de capas, qué herramienta en cada paso y en qué punto decidirías que está *fuera de alcance*.

Preguntas de repaso

▼ **¿Qué diferencia a un packer de un protector?**

> El packer **comprime** (y descomprime al arrancar). El protector agrega **capas de defensa**: cifrado, anti-debug/anti-VM, anti-dump, integridad y, en la gama alta, **virtualización** de funciones críticas.

▼ **¿Por qué la entropía por sección es una buena primera pista?**

> Porque datos comprimidos o cifrados tienen entropía **cercana a 8**, mientras que el código normal ronda 6–6,5. Una sección en ~7,9 casi seguro está empaquetada o cifrada, y se ve **sin ejecutar** el binario.

▼ **En un protector duro, ¿por qué el OEP «no existe» como en UPX?**

> Porque hay **código robado**, re-cifrado por tramos y funciones **virtualizadas** que nunca vuelven a ser x86 nativo. No hay un salto único y limpio al código original como en un packer de compresión.

▼ **¿Qué delata una IAT con casi solo `LoadLibrary` / `GetProcAddress`?**

> Que el binario **resuelve sus imports en runtime** (los reconstruye el stub tras descomprimir): señal clásica de empaquetado. La IAT «real» no está en el PE de disco.

▼ **¿Dónde está el límite de esta lección?**

> En **reconocer e inventariar** la protección y **planear**. Desempaquetar protectores comerciales o software ajeno está fuera de alcance; la práctica es sobre binarios propios y malware en estático.

Puerta de salida

Podés: (1) distinguir packer de protector; (2) identificar una familia por sus huellas con DIE/PE-bear; (3) leer entropía e imports para sospechar empaquetado sin ejecutar; (4) enumerar las capas de una protección dura y escribir un plan de análisis que incluya *cuándo parar*.

Fuentes de esta lección

- github.com/horsicq/Detect-It-Easy — identificación de packer/protector y entropía por sección (conocimiento externo).
- github.com/hasherezade/pe-bear — inspección estática de PE.
- upx.github.io — el packer libre que se usa para empaquetar binarios propios de práctica.
- 2.17 Packers didácticos — el OEP/dump/IAT sobre UPX propio, escalón previo.
- 2.15 Anti-análisis y 2.26 VM — las capas de defensa que apila un protector.

Capas de esta lección

Contenido observado en CLS: el desempaquetado didáctico de UPX (2.17) y la exclusión declarada de protectores comerciales en Fuentes y límites. **Conocimiento externo (agregado por esta edición):** la taxonomía de familias, sus huellas y las herramientas de reconocimiento. **Recomendación editorial:** los ejemplos y la práctica son ilustrativos y sobre binarios propios.

Neutralizar anti-análisis en laboratorio

Objetivo de la lección

Pasar de **reconocer** un chequeo anti-análisis (lo que viste en 2.15) a **neutralizarlo de forma controlada** para poder analizar: parches, plugins de ocultación y *hooks*, sobre **binarios propios** o malware en **VM descartable**. **Prerrequisitos:** taxonomía anti-análisis (2.15), scripting de debugger (2.21), debugger (2.6) y el laboratorio de malware dinámico defensivo (2.22).

Riesgo: neutralizar un chequeo puede ejecutar el payload

Muchos binarios maliciosos **esconden su comportamiento detrás del anti-análisis**: si hacés que crean que nadie los observa, **corren su carga real**. Hacé esto solo en una **VM descartable con snapshot**, sin red o con red controlada, tal como se define en 4.4 Dinámico seguro. Para **aprender la técnica**, usá binarios propios con chequeos que escribís vos.

Por qué, a veces, hay que neutralizar

En 2.15 aprendiste a **detectar** que un binario se defiende. Pero para analizarlo en dinámico a veces necesitás que **no aborte**: un programa que llama a `IsDebuggerPresent` y sale si estás enganchado no se deja estudiar bajo debugger. Neutralizar el chequeo —hacer que devuelva «no hay debugger»— te deja seguir. La regla de oro: **neutralizás para observar, no para romper protecciones de software ajeno**.

Chequeos y su contramedida

Chequeo (2.15)	Dónde mira	Contramedida en lab
`IsDebuggerPresent`	PEB `BeingDebugged` (byte en PEB+2)	Hook que devuelve 0; o poner a 0 el byte del PEB
`CheckRemoteDebuggerPresent` / `NtQueryInformationProcess(ProcessDebugPort)`	*debug port* del proceso	Forzar el resultado a «sin debugger» (ScyllaHide)
`NtGlobalFlag` / *heap* `ForceFlags`	Flags que cambian bajo debugger	Normalizar los flags al valor «sin debugger»
Tiempos (`rdtsc`, `QueryPerformanceCounter`, `GetTickCount`)	Mide cuánto tardó entre dos puntos	Acortar el delta (hook que devuelve tiempos cercanos)
Anti-VM (`cpuid` bit de hipervisor, artefactos)	Detecta que corre en VM	Ocultar artefactos de la VM; parchear el chequeo
Hardware breakpoints (DR0–DR3)	Lee el contexto de depuración	Usar bp de software; limpiar DRx vía hook

> **Un VEH que lee DR0–DR3 te delata al poner un breakpoint de hardware**
>
> En x64, un truco frecuente es registrar un *vectored exception handler* (`AddVectoredExceptionHandler`) que, ante cualquier excepción, inspecciona `ContextRecord->Dr0..Dr3`: si alguno es distinto de cero, hay un breakpoint de hardware y el binario aborta. Un bp de hardware tuyo en x64dbg cae en la trampa. Para neutralizarlo en lab: enganchá `NtGetContextThread` (o el VEH registrado) con Frida y **forzá** `Dr0..Dr3 = 0` en el `CONTEXT` antes de que el código los evalúe. Alternativa: usá solo breakpoints de software en ese tramo.

Herramientas

- **ScyllaHide / TitanHide:** plugins (de x64dbg el primero; driver el segundo) que **ocultan el debugger** interceptando las APIs y estructuras de arriba. Es el camino rápido para «hacer invisible» el debugger sin parchear a mano.
- **Frida:** *hooks* en runtime para reemplazar el valor que devuelve una API concreta (fino y reproducible).
- **Parche estático** en x64dbg: cuando el chequeo está en el propio binario propio (p. ej. forzar el salto), con *Patch → Apply patches*. Igual que en 2.10, **no a ciegas**.

Ejemplo paso a paso: neutralizar tres chequeos propios con Frida

*(Ejemplo **ilustrativo**: script sobre un binario propio con chequeos que escribiste vos, en VM.)*

```
// ocultar.js — en VM, sobre binario PROPIO. frida -f .\propio.exe -l ocultar.js
const k32 = Process.getModuleByName('kernel32.dll');
const ntdll = Process.getModuleByName('ntdll.dll');

// 1) IsDebuggerPresent -> siempre 0
Interceptor.replace(
  k32.getExportByName('IsDebuggerPresent'),
  new NativeCallback(() => 0, 'int', []));

// 2) limpiar PEB.BeingDebugged (+0x02) y PEB.NtGlobalFlag (+0xBC en x64 / +0x68 en x86).
//    En Frida no hay un objeto 'gs'; se obtiene el PEB llamando a ntdll!RtlGetCurrentPeb.
const RtlGetCurrentPeb = new NativeFunction(
  ntdll.getExportByName('RtlGetCurrentPeb'), 'pointer', []);
const peb = RtlGetCurrentPeb();
peb.add(0x02).writeU8(0);                                  // BeingDebugged = 0
const offNtGlobalFlag = (Process.pointerSize === 8) ? 0xBC : 0x68;
peb.add(offNtGlobalFlag).writeU32(0);                      // NtGlobalFlag = 0

// 3) chequeo de tiempo: devolver un delta chico y estable
let ticks = 100000;
Interceptor.replace(
  k32.getExportByName('GetTickCount64'),
  new NativeCallback(() => { ticks += 10; return uint64(ticks); }, 'uint64', []));

console.log('[*] PEB limpiado en ' + peb + '; hooks anti-debug activos.');
```

Con estos tres *hooks*, tu binario de práctica —que sin esto salía con «debugger detectado»— sigue hasta el **flujo protegido** y podés analizarlo. En un debugger, **ScyllaHide** hace lo equivalente con un par de clics. Lo importante es entender **qué** estás falseando y **por qué**, para no engañarte a vos mismo sobre lo que hace el binario.

Práctica

En tu VM, con un **binario propio**:

1. Compilá un programa con tres chequeos (`IsDebuggerPresent`, lectura de `BeingDebugged` en el PEB y un chequeo de tiempo con `rdtsc`) que imprima «protegido» solo si ninguno detecta nada.
2. Verificá que bajo debugger **detecta** y sale.
3. Neutralizalos con ScyllaHide o con un script de Frida y confirmá que ahora llega a «protegido».
4. Escribí dos renglones sobre por qué este mismo procedimiento, aplicado a malware, se hace **solo en VM descartable**.

Preguntas de repaso

▼ **¿Cuál es el riesgo específico de neutralizar anti-análisis en malware?**

> Que el binario **esconde su payload detrás del chequeo**: al hacerle creer que no lo observan, **ejecuta su carga real**. Por eso se hace solo en VM descartable con snapshot y red controlada (4.4).

▼ **¿Qué mira `IsDebuggerPresent` y cómo se neutraliza?**

> Lee el byte `BeingDebugged` **del PEB** (PEB+2). Se neutraliza con un *hook* que devuelve 0, o poniendo a 0 ese byte del PEB; ScyllaHide lo hace automáticamente.

▼ **¿Por qué un chequeo de tiempo detecta al analista?**

> Porque entre dos puntos mide un **delta** (con `rdtsc` / `QueryPerformanceCounter`); bajo debugger o con breakpoints, el tiempo se dispara. Se neutraliza **achicando el delta** que devuelve el contador.

▼ **¿Qué hace ScyllaHide que te ahorra parchear a mano?**

> Intercepta las APIs y estructuras del PEB que usan los chequeos (`IsDebuggerPresent`, `NtQueryInformationProcess`, `NtGlobalFlag`, etc.) y les devuelve valores de «sin debugger», **ocultando el debugger** de forma centralizada.

▼ **¿Por qué «no parchear a ciegas» sigue valiendo acá?**

> Porque falsear un valor sin entenderlo puede ocultarte el comportamiento real o romper la lógica. Primero entendés qué chequea y qué estás cambiando; recién después neutralizás, y solo lo necesario.

Puerta de salida

Podés: (1) explicar por qué neutralizar un chequeo puede disparar el payload y qué recaudos exige; (2) relacionar cada chequeo de 2.15 con su contramedida; (3) ocultar el debugger con ScyllaHide o un hook de Frida sobre un binario propio; (4) justificar por qué se hace solo en VM descartable.

Fuentes de esta lección

- github.com/x64dbg/ScyllaHide — ocultación de debugger (PEB, APIs, NtGlobalFlag) (conocimiento externo).
- frida.re/docs — `Interceptor.replace` / `attach` para falsear el resultado de una API.
- github.com/LordNoteworthy/al-khaser — catálogo de chequeos anti-análisis para generar binarios propios de práctica.
- 2.15 Anti-análisis (reconocer) y 4.4 Dinámico seguro (la VM descartable).

Capas de esta lección

Contenido observado en CLS: los chequeos y el encuadre de reconocimiento de 2.15; el dinámico seguro de la Etapa 4. **Conocimiento externo (agregado por esta edición):** las contramedidas concretas, ScyllaHide/TitanHide y los hooks de Frida. **Recomendación editorial:** ejemplos y práctica ilustrativos, sobre binarios propios y malware solo en VM descartable.

Instrumentación binaria dinámica: DynamoRIO y Pin

> **Objetivo de la lección**
>
> Entender qué es la **DBI** (instrumentación binaria dinámica) y usarla para **contar instrucciones**, **medir cobertura** y **detectar código auto-modificado** (desempaquetado genérico), sobre binarios propios. **Prerrequisitos:** scripting de debugger (2.21) y packers didácticos (2.17).

Esto es **análisis dinámico instrumentado** sobre programas propios o crackmes. El malware, si se instrumenta, va en **VM descartable** (4.4).

Qué es la DBI

Una herramienta de DBI (**conocimiento externo**) toma el proceso mientras corre, **re-compila su código en bloques** (JIT) justo antes de ejecutarlos e **inserta tu código** en los puntos que elijas: antes de cada instrucción, al entrar a una función, cuando se escribe memoria. A diferencia de un debugger (que para en breakpoints), la DBI observa **todo** a escala de instrucción y es **mucho más rápida**, porque no hay *trap* por instrucción. El precio: tu código corre dentro del mismo proceso, así que hay que cuidar la *transparencia* (no alterar el comportamiento observado).

Los dos frameworks

Framework	Cómo se programa	Herramientas que ya trae
Intel Pin	*pintools* en C++; registrás *callbacks* por instrucción, *trace*, rutina o imagen (`INS_AddInstrumentFunction`, `TRACE_`, `IMG_`)	Ejemplos: `inscount`, `pinatrace`, `itrace`
DynamoRIO	*clients* en C; API de eventos (`dr_register_bb_event`)	`drcov` (cobertura), `drltrace` (llamadas a librería), `drcachesim`

Para empezar no hace falta escribir un *client*: `drcov` y `drltrace` de DynamoRIO resuelven el 80% de los casos de reversing sin compilar nada.

Para qué sirve en reversing

- **Cobertura:** qué bloques se ejecutaron con una entrada dada. Se vuelca con `drcov` y se colorea en IDA/Ghidra/Binary Ninja con **Lighthouse**: ves «por dónde pasó» el programa.
- **Fuzzing dirigido:** esa misma cobertura es la señal que usan los fuzzers (2.24) para saber si una entrada llegó a código nuevo.
- **Desempaquetado genérico:** un packer **escribe** una página y luego **ejecuta** desde ella. Una herramienta DBI que marca las páginas escritas y avisa cuando se ejecuta una de ellas encuentra el **OEP** sin conocer el packer (regla W⊕X).
- **Conteo y perfil:** cuántas instrucciones corre una rutina (útil para hallar la función «cara», p. ej. la de descifrado).

> **Cobertura ≠ traza: elegí según el objetivo**
>
> Son dos niveles de detalle distintos. La **cobertura** (`drcov`) registra *qué bloques básicos* se ejecutaron —basta para colorear en Lighthouse y comparar caminos, y es barata—. La **traza de instrucciones** registra *cada instrucción con los valores de los registros* (p. ej. `pinatrace` de Pin, o `drcachesim` / `drtrace` de DynamoRIO): es mucho más pesada, pero es la que necesitás para **reconstruir un algoritmo** —seguir cómo se transforma un dato paso a paso, como una rutina de descifrado—. Regla: para «¿por dónde pasó?» usá cobertura; para «¿qué le hizo al dato?» usá traza.

Ejemplo paso a paso: cobertura con drcov y OEP por W⊕X

*(Ejemplo **ilustrativo**: comandos típicos sobre un binario propio; no un volcado verificado.)*

```
# 1) cobertura de un binario PROPIO con DynamoRIO (no hay que compilar nada)
drrun -t drcov -- .\crackme_propio.exe entrada1
#   -> genera  drcov.crackme_propio.exe.<pid>.0000.proc.log
#   abrilo en Lighthouse (IDA/Ghidra) para colorear los bloques ejecutados

# 2) contar instrucciones con un pintool de ejemplo de Intel Pin
pin -t inscount0.dll -- .\crackme_propio.exe
#   -> Count 1234567   (cuántas instrucciones corrió)
```

```
// idea de un client/pintool de "unpacking genérico" (resumen conceptual)
//  - marcar cada página donde el programa ESCRIBE
//  - en cada bloque a ejecutar, si su página fue escrita antes -> candidato a OEP
on_memory_write(addr):      paginas_escritas.add(page_of(addr))
on_basic_block(bb):         if page_of(bb) in paginas_escritas:
                                print("posible OEP:", bb); dump_module()
```

La cobertura de `drcov` + Lighthouse te muestra **qué caminos tomó** cada entrada (comparás dos corridas y ves la rama que cambió). El esquema W⊕X encuentra el OEP de un UPX propio **sin** saber que era UPX: es el mismo OEP que en 2.17 buscabas a mano, ahora detectado por la regla «se ejecutó una página que antes se escribió».

Práctica

En tu VM, con binarios **propios**:

1. Sacá la cobertura con `drcov` de tu crackme para dos entradas distintas y coloreala en Lighthouse; identificá la rama que separa «bien» de «mal».
2. Contá las instrucciones con un pintool de ejemplo (`inscount`) y encontrá la rutina que más ejecuta (candidata a descifrado/bucle).
3. Sobre tu «hola mundo» empaquetado con UPX (de 2.17), explicá cómo la regla W⊕X marcaría el OEP.
4. Escribí dos diferencias prácticas entre instrumentar con DBI y depurar con breakpoints.

Preguntas de repaso

▼ **¿En qué se diferencia la DBI de un debugger?**

> La DBI **re-compila el código (JIT) e inserta tu código** en cada punto de interés, observando a escala de instrucción y **mucho más rápido** (sin *trap* por instrucción). El debugger para en breakpoints puntuales.

▼ **¿Qué herramienta de DynamoRIO usás para cobertura y con qué la visualizás?**

> `drcov` genera el archivo de cobertura; **Lighthouse** lo colorea sobre IDA/Ghidra/Binary Ninja para ver qué bloques se ejecutaron.

▼ **¿Cómo encuentra la DBI un OEP sin conocer el packer?**

> Con la regla **W⊕X**: marca las páginas donde el programa **escribe** y avisa cuando se **ejecuta** una de ellas. Ese salto a código recién escrito es el candidato a OEP, sea cual sea el packer.

▼ **¿Qué relación hay entre cobertura y fuzzing?**

> La cobertura (qué bloques nuevos tocó una entrada) es la **señal** que guía al fuzzer (2.24): si una entrada llega a código nuevo, vale la pena mutarla. La DBI puede proveer esa cobertura.

▼ **¿Por qué importa la «transparencia» en DBI?**

> Porque tu código corre **dentro del mismo proceso**: si altera tiempos, memoria o APIs observables, el binario puede comportarse distinto (o detectarte). Hay que medir sin cambiar lo que se mide.

> **Puerta de salida**
>
> Podés: (1) explicar qué hace la DBI y en qué se diferencia de un debugger; (2) sacar cobertura con `drcov` y leerla en Lighthouse; (3) contar instrucciones con un pintool; (4) describir el desempaquetado genérico por W⊕X.

Fuentes de esta lección

- Intel Pin · Pin User Guide — `INS_AddInstrumentFunction`, granularidades y pintools de ejemplo (conocimiento externo).
- dynamorio.org — API de clients y herramientas `drcov` / `drltrace` / `drcachesim`.
- github.com/gaasedelen/lighthouse — visualizar cobertura en IDA/Ghidra/Binary Ninja.
- 2.17 Packers (OEP a mano) y 2.24 Fuzzing (la cobertura como señal).

Capas de esta lección

Contenido observado en CLS: el OEP/dump de 2.17 como problema que la DBI automatiza. **Conocimiento externo (agregado por esta edición):** Pin, DynamoRIO, `drcov`/Lighthouse y la regla W⊕X. **Recomendación editorial:** ejemplos y práctica ilustrativos, sobre binarios propios.

Frida avanzado: Stalker y tracing de código

> **Objetivo de la lección**
>
> Dar el salto de los *hooks* puntuales (que ya usaste) al **tracing de todo un hilo** con **Stalker**: seguir instrucciones y bloques, medir **cobertura** y detectar **código generado en runtime**. Saber cuándo conviene Stalker y cuándo la DBI nativa. **Prerrequisitos:** scripting de debugger y Frida básico (2.21) y DBI (2.29).

Análisis dinámico sobre binarios propios o crackmes; el malware se instrumenta solo en **VM descartable** (4.4).

De los hooks al tracing

En 2.21 usaste `Interceptor` para engancharte a una función concreta (entrar, leer argumentos, salir). Eso es puntual. **Stalker** es otra cosa: es el motor de *tracing* de Frida que **sigue el flujo completo** de un hilo, reescribiendo el código **bloque por bloque** a medida que se ejecuta e informándote de cada bloque, llamada o retorno (**conocimiento externo**). Es la versión «scriptable en JavaScript» de lo que viste con Pin/DynamoRIO.

Qué te da Stalker

- **Eventos:** `exec` (cada instrucción), `block` / `compile` (cada bloque básico), `call` y `ret`. Elegís la granularidad según cuánto *overhead* tolerás.
- **`transform`:** podés **reescribir** el flujo mientras se traza (insertar *callouts*, saltar, instrumentar). Es el gancho más potente.
- **Cobertura:** con `frida-drcov` generás un archivo de cobertura **compatible con Lighthouse** (el mismo visor de 2.29).
- **Código en runtime:** como Stalker ve el código *cuando se va a ejecutar*, detecta bloques en regiones recién escritas (útil para OEP y descifrado en memoria).

Stalker vs DBI nativa (2.29)

	Frida Stalker	Pin / DynamoRIO
Se programa en	JavaScript (rápido de iterar)	C/C++ (compilar el tool)
Plataformas	Windows, Linux, macOS, Android, iOS	sobre todo desktop x86/x64 (+ ARM en DynamoRIO)
Overhead / transparencia	Mayor *overhead*; menos transparente	Más rápido y maduro para trazas largas
Fuerte en	enganchar APIs + trazar en el mismo script	trazas masivas, cobertura a gran escala

Regla práctica: si ya estás en Frida haciendo *hooks* y querés cobertura o una traza acotada, usá Stalker. Si necesitás trazar millones de instrucciones o integrarte con un fuzzer nativo, andá a Pin/DynamoRIO.

Ejemplo paso a paso: trazar los bloques de un hilo

*(Ejemplo **ilustrativo**: script sobre un binario propio en VM; `frida -f .\propio.exe -l stalker.js`.)*

```
// stalker.js — trazar SOLO dentro de la función objetivo (frida -f .\propio.exe -l stalker.js)
// Clave: activá Stalker al ENTRAR a la función y detenelo al salir. Si hacés follow/unfollow
// en la carga del script, Stalker corre y se apaga antes de que el hilo llegue a main -> imprime
0.
const objetivo = DebugSymbol.fromName('validar').address;  // o
Module.getBaseAddress('propio.exe').add(0x1000)
let total = 0;

Interceptor.attach(objetivo, {
  onEnter() {
    this.tid = this.threadId;
    Stalker.follow(this.tid, {
      events: { compile: true },        // un evento por bloque básico compilado
      onReceive(raw) {
        // annotate:false devuelve pares limpios [inicio, fin] (sin el tag 'compile')
        const bloques = Stalker.parse(raw, { annotate: false, stringify: false });
        total += bloques.length;
        for (const [ini, fin] of bloques) {
          // ini/fin son NativePointer reales del bloque básico compilado
        }
      }
    });
  },
  onLeave() {
    Stalker.unfollow(this.tid);
    Stalker.flush();
    console.log('[*] bloques ejecutados dentro de validar:', total);
  }
});
```

Con `compile: true` recibís un evento por **bloque básico** (barato): es la **cobertura**. Con `exec: true` tendrías uno por **instrucción** (caro, pero fino): es la **traza** que sirve para reconstruir un algoritmo (la misma distinción cobertura/traza de 2.29). Para cobertura real no reinventes el parser: `frida-drcov` ya emite el formato de Lighthouse. Para ver la transición **stub → OEP** en un packer propio, marcás las regiones que se escriben y avisás cuando Stalker compila un bloque dentro de una de ellas: ahí saltó al código desempaquetado.

Práctica

En tu VM, con binarios **propios**:

1. Seguí con Stalker el hilo principal de tu crackme y contá los bloques de una corrida «clave correcta» vs «incorrecta»; ¿qué bloque aparece solo en una?
2. Generá cobertura con `frida-drcov` y abrila en Lighthouse; comparala con la que sacaste con `drcov` nativo en 2.29.
3. Sobre tu binario empaquetado con UPX, detectá con Stalker el salto a una región recién escrita (OEP).
4. Anotá dos casos en los que preferirías Pin/DynamoRIO antes que Stalker.

Preguntas de repaso

▼ **¿Qué diferencia a `Interceptor` de `Stalker`?**

> `Interceptor` engancha funciones **puntuales**; `Stalker` **traza el flujo completo** de un hilo, bloque por bloque, informando cada instrucción/bloque/llamada/retorno.

▼ **¿Por qué elegirías `compile` / `block` en vez de `exec`?**

> Porque `exec` emite un evento **por instrucción** (mucho *overhead*), mientras que `compile` / `block` emiten uno **por bloque básico**: suficiente para cobertura y mucho más barato.

▼ **¿Cómo detecta Stalker código generado en runtime?**

> Porque ve el código **justo cuando se va a ejecutar**: si un bloque cae en una región que se **escribió antes**, es código nuevo (descifrado o desempaquetado), candidato a OEP.

▼ **¿Qué ventaja da que la cobertura sea formato Lighthouse?**

> Que la **visualizás igual** que la de `drcov` nativo (2.29), coloreando los bloques en IDA/Ghidra. Podés comparar trazas de Stalker y de Pin/DynamoRIO con el mismo visor.

▼ **¿Cuándo preferís Pin/DynamoRIO a Stalker?**

> Para **trazas muy largas** (millones de instrucciones), cuando importa el *overhead*/transparencia, o para integrarte con un fuzzer nativo. Stalker gana cuando ya estás en Frida y querés iterar rápido en JavaScript.

Puerta de salida

Podés: (1) explicar la diferencia entre hook puntual y tracing; (2) seguir un hilo con Stalker y elegir granularidad; (3) sacar cobertura con `frida-drcov` y leerla en Lighthouse; (4) detectar el salto al OEP por bloques en regiones recién escritas, y decidir entre Stalker y DBI nativa.

Fuentes de esta lección

- frida.re/docs/stalker — `Stalker.follow`, eventos, `transform` (conocimiento externo).
- frida-drcov (Lighthouse) — emitir cobertura en formato Lighthouse.
- 2.21 Scripting (Frida básico) y 2.29 DBI (la comparación con Pin/DynamoRIO).

Capas de esta lección

Contenido observado en CLS: la automatización dinámica de 2.21 como base. **Conocimiento externo (agregado por esta edición):** Stalker, sus eventos y `transform`, y la cobertura con frida-drcov. **Recomendación editorial:** ejemplos y práctica ilustrativos, sobre binarios propios.

Extensiones de Ghidra y SLEIGH

> **Objetivo de la lección**
>
> Automatizar Ghidra con **scripts** (descifrar strings, renombrar, resolver imports) y entender **P-Code** y **SLEIGH** lo suficiente para leer el IR y, si hace falta, **modelar un bytecode propio** (el de tu mini-VM de 2.26). **Prerrequisitos:** Ghidra/Cutter (2.16) y C++ profundo (2.18).

Ampliación de 2.16: ahí recorriste Ghidra «a mano»; acá lo **programás**.

Por qué extender Ghidra

Dos motivos (**conocimiento externo**): **automatizar** lo repetitivo (descifrar cientos de strings, renombrar por patrón, anotar structs) y **enseñarle algo que no sabe** (una arquitectura rara, un bytecode a medida). Lo primero son *scripts*; lo segundo es **SLEIGH**.

Scripts: Jython, PyGhidra y Java

- **Jython (Python 2):** el scripting histórico, integrado en el *Script Manager*. Cómodo para tareas rápidas; el lenguaje es viejo.
- **PyGhidra (Python 3):** desde **Ghidra 11+** viene integrado y es el **estándar actual**: corre CPython 3 real contra la API, así que podés usar librerías de Python 3 dentro del script (`z3-solver` para resolver una condición, `requests`, `capstone`…). En versiones previas se lograba lo mismo con Ghidrathon. Jython queda para scripts viejos.
- **Java (GhidraScript):** la API nativa, la más completa y la de los *plugins*/analizadores.

Las piezas de la API que más vas a tocar: `FlatProgramAPI` (`getInstructionAt`, `getBytes`, `createFunction`, `setEOLComment`, `getSymbolAt`) y la interfaz de decompilado `DecompInterface` (pedir el C decompilado de una función desde el script).

Headless: Ghidra sin interfaz

Para procesar muchos binarios o integrar en CI, `analyzeHeadless` corre el análisis y tus scripts **sin GUI**:

```
analyzeHeadless C:\proyectos miProyecto ^
  -import .\muestra_propia.exe ^
  -postScript descifrar_strings.py ^
  -scriptPath .\scripts
```

P-Code: el IR de Ghidra

Ghidra no razona sobre x86 directamente: traduce cada instrucción a **P-Code**, un lenguaje intermedio de operaciones chiquitas (`COPY`, `INT_ADD`, `LOAD`, `STORE`, `CBRANCH`…). Hay dos niveles: **raw** (crudo, 1 instrucción → varias ops) y **refined/high** (tras el análisis, el que alimenta el decompilador). Trabajar sobre P-

Code te da análisis de flujo de datos **independiente de la arquitectura**: el mismo script sirve para x86 y para ARM.

SLEIGH: enseñarle una arquitectura (o tu bytecode)

SLEIGH es el lenguaje con que Ghidra describe un procesador: un `.slaspec` (cómo se decodifican los bytes y qué P-Code produce cada instrucción), más `.pspec` / `.cspec` / `.ldefs` (registros, convención de llamada, variantes). Modelar una instrucción es decir tres cosas: **qué bits** la identifican, **cómo se muestra** (el *display*) y **qué hace** (su semántica en P-Code). Esto es lo que te deja **liftear el bytecode de tu mini-VM** (2.26) y que Ghidra lo decompile como si fuera código normal.

```
# fragmento SLEIGH (ESQUEMÁTICO) para el OP_XOR de la mini-VM de 2.26
# opcode de 8 bits + inmediato de 8 bits; 'acc' es un registro virtual
define token instr (16) op=(8,15) imm=(0,7);
:XOR imm is op=0x02 & imm { acc = acc ^ imm; }     # semántica = P-Code
:ADD imm is op=0x03 & imm { acc = acc + imm; }
```

Ejemplo paso a paso: descifrar strings con un script

*(Ejemplo **ilustrativo**: script Jython sobre un binario propio; Window → Script Manager.)*

```
# descifrar_strings.py (Jython) — XOR 0x5A y dejar el claro como comentario
addr = toAddr(0x00403020)
n = 16
data = getBytes(addr, n)                       # byte[] (con signo)
claro = "".join(chr((b & 0xff) ^ 0x5A) for b in data)
setEOLComment(addr, "desc: " + claro)          # queda anotado en el listado
print("descifrado:", claro)
```

Qué hiciste: leíste 16 bytes, los pasaste por `XOR 0x5A` y dejaste el texto claro **anotado en el propio listado**, de forma reproducible. Escalado a un bucle sobre todas las referencias a la rutina de descifrado, resolvés cientos de strings en segundos —lo que a mano sería una tarde—.

Práctica

En tu VM, sobre binarios **propios**:

1. Compilá un binario que guarde sus strings con `XOR 0x5A` y escribí un script Ghidra que las descifre y comente todas.
2. Corré ese mismo script en modo **headless** sobre dos binarios de una.
3. Pedí con `DecompInterface` el C decompilado de una función y guardalo a archivo.
4. Escribí (esquemático) las líneas SLEIGH de dos opcodes más de tu mini-VM (2.26) y explicá qué P-Code generarían.

Preguntas de repaso

▼ **¿Cuándo escribís un script y cuándo tocás SLEIGH?**

> **Script** para automatizar sobre una arquitectura que Ghidra **ya entiende** (descifrar, renombrar, anotar). **SLEIGH** cuando hay que **enseñarle una arquitectura o un bytecode nuevo** que no trae.

▼ **¿Qué es P-Code y por qué conviene razonar sobre él?**

> Es el **IR** de Ghidra: cada instrucción se traduce a operaciones chicas (`INT_ADD`, `LOAD`…). Razonar sobre P-Code es **independiente de la arquitectura**: el mismo análisis vale para x86 y ARM.

▼ **¿Qué tres cosas define una instrucción en SLEIGH?**

> **Qué bits** la identifican (el patrón del token), **cómo se muestra** (el display/mnemónico) y **qué hace** (su semántica expresada en P-Code).

▼ **¿Para qué sirve `analyzeHeadless`?**

> Para correr el análisis y tus scripts **sin la GUI**: procesar lotes de binarios, integrar en CI o automatizar pipelines de triage.

▼ **¿Cómo se conecta esta lección con la VM de 2.26?**

> Con SLEIGH podés **modelar el bytecode de tu mini-VM** para que Ghidra lo desensamble y decompile como código normal: es la vía de *lifting* para devirtualizar lo que vos mismo virtualizaste.

> **Puerta de salida**
>
> Podés: (1) escribir un script de Ghidra (Jython/PyGhidra) que descifre y anote; (2) correrlo en headless; (3) explicar qué es P-Code y sus niveles; (4) describir cómo SLEIGH modela una instrucción y para qué liftearías tu bytecode propio.

Fuentes de esta lección

- ghidra.re · API (FlatProgramAPI, DecompInterface) — scripting en Java/Jython (conocimiento externo).
- SLEIGH language docs — tokens, display y semántica P-Code.
- github.com/mandiant/Ghidrathon — Python 3 en Ghidra (alternativa a Jython).
- 2.16 Ghidra/Cutter (recorrido a mano) y 2.26 VM (el bytecode que se liftea).

> **Capas de esta lección**
>
> **Contenido observado en CLS:** el recorrido de Ghidra de 2.16 como base. **Conocimiento externo (agregado por esta edición):** la API de scripting, headless, P-Code y SLEIGH. **Recomendación editorial:** los ejemplos (incluida la línea SLEIGH, marcada como esquemática) y la práctica son ilustrativos.

Go y Rust en profundidad

> **Objetivo de la lección**
>
> Ir más allá de «reconocer el compilador» (lo que viste en 2.13) y **recuperar la lógica** de un binario Go o Rust: nombres de función, tipos, interfaces/traits y cómo se ven sus estructuras en memoria.
> **Prerrequisitos:** huella de compilador (2.13) y C++ profundo (2.18, para vtables y layouts).

Esto es una **ampliación profunda** de 2.13: ahí ya identificabas que un binario era Go o Rust; acá aprendés a leerlos.

Por qué cuestan (y por qué hay esperanza)

Go y Rust producen binarios **estáticos y grandes**, con su propio *runtime* adentro y sin los símbolos «clásicos» de C. La buena noticia: ambos dejan **metadatos recuperables**. En Go son casi un regalo; en Rust hay que desmanglar y entender la monomorfización (**conocimiento externo**).

Go: el runtime que se deja leer

- `gopclntab` **(pclntab):** una tabla que mapea direcciones a **nombres de función**, pensada para los *stack traces*. Aunque el binario esté «stripeado», esta tabla suele estar: recuperás casi todos los nombres. Es el primer paso siempre.
- `moduledata`**:** la estructura raíz del runtime; desde ahí se llega a la pclntab y a las tablas de tipos.
- **Tipos (**`_type`**/rtype):** Go guarda descriptores de tipo para reflexión; con ellos recuperás nombres de structs y campos.
- **Interfaces (**`itab`**):** una interfaz es un par `(itab, data)`; el `itab` tiene el tipo concreto y la **tabla de métodos**. Reconocer este par te dice qué método se llama en un despacho dinámico.
- **Strings:** `(puntero, longitud)`, **sin terminador NUL**. En el listado vas a ver una carga de puntero seguida de una longitud inmediata.
- **Convención de llamada:** hasta Go 1.16 los argumentos iban por **stack**; desde **Go 1.17** hay ABI por **registros** (amd64). Saber la versión cambia cómo leés los argumentos.

Herramienta clave: **GoReSym** (Mandiant) recupera pclntab, tipos y metadatos y los exporta para IDA/Ghidra.

Rust: manglado, monomorfización y traits

- **Name mangling:** dos esquemas. El **legacy** (estilo Itanium, `_ZN...E` con longitudes y un hash) y el **v0** (`_R...`, RFC 2603). Se desmanglan con `rustfilt` (o el soporte de Ghidra/IDA).
- **Monomorfización:** los genéricos no existen en runtime: el compilador genera **una copia por cada tipo** usado. Por eso un `Vec<T>` aparece muchas veces, una por `T`: no te confundas creyendo que son funciones distintas.
- **Trait objects (**`dyn Trait`**):** un *fat pointer* = **puntero a datos + puntero a vtable**. La vtable trae *drop*, tamaño, alineación y los métodos. Es el despacho dinámico de Rust y se lee parecido a una vtable de

C++ (2.18).

- **Strings y colecciones:** `&str` es `(ptr, len)` UTF-8; `String` es `(ptr, len, cap)`; `Vec<T>` igual. **Sin NUL**.
- **`Option` / `Result`:** con *niche optimization*, un `Option<&T>` no ocupa un byte extra: usa el valor `0` (puntero nulo) como `None`.
- **Los *panics* son oráculos: **en modo seguro, Rust inyecta antes de indexar un array/slice una llamada a `core::panicking::panic_bounds_check` (y a `panic_fmt` / `panic` en otros casos), muchas veces con el `file:line` del fuente embebido. Esas comprobaciones te** revelan la longitud esperada de los buffers** y marcan dónde empieza cada acceso a datos: un regalo para reconstruir estructuras.

Cómo se ve cada cosa en memoria

Concepto	Go	Rust
String	`(ptr, len)` sin NUL	`&str (ptr,len)` · `String (ptr,len,cap)`
Despacho dinámico	interfaz = `(itab, data)`	`dyn Trait` = `(data, vtable)`
Nombres recuperables	pclntab (casi siempre)	símbolos manglados → `rustfilt`
Genéricos	parcial (desde 1.18, por *shape*)	monomorfizados: una copia por tipo

Ejemplo paso a paso: recuperar nombres y un trait object

*(Ejemplo **ilustrativo**: comandos sobre binarios propios compilados por vos.)*

```
# GO: recuperar nombres aunque esté "stripeado"
GoReSym.exe -t -p .\mi_programa_go.exe > go_meta.json
#   -> funciones (pclntab) + tipos; se importan en IDA/Ghidra

# RUST: desmanglar un símbolo
echo _ZN4core3fmt9Formatter3pad17h0123456789abcdefE | rustfilt
#   -> core::fmt::Formatter::pad
```

En Go, con los nombres de la pclntab cargados, el binario «stripeado» vuelve a tener etiquetas (`main.main`, `main.validar`…) y navegás como si tuviera símbolos. En Rust, al desmanglar ves el *path* real (`core::fmt::Formatter::pad`); y cuando encontrás un par **(puntero a datos, puntero a vtable)** pasándose junto, es un `dyn Trait`: la vtable te dice qué métodos hay.

Práctica

En tu VM, con binarios **propios**:

1. Compilá un programa en Go, «stripealo» (`-ldflags "-s -w"`) y recuperá los nombres con GoReSym; confirmá que reaparecen `main.*`.
2. Identificá una **interfaz** en el Go: encontrá el par `(itab, data)` y el método que se llama.

3. Compilá un programa en Rust con un `dyn Trait`; desmanglá sus símbolos con `rustfilt` y ubicá la vtable del trait.
4. Mostrá en ambos cómo se ve un string (ptr+len, sin NUL) frente a un string C (con NUL) de 2.1.

Preguntas de repaso

<details>
<summary>¿Qué recuperás de la <code>gopclntab</code> y por qué es el primer paso en Go?</summary>

Los **nombres de función** (mapa dirección → nombre), pensados para *stack traces*. Suele estar aun en binarios «stripeados», así que recuperás casi todas las etiquetas y navegás como con símbolos.
</details>

<details>
<summary>¿Qué es una interfaz de Go en memoria?</summary>

Un par `(itab, data)`: el `itab` tiene el **tipo concreto** y la **tabla de métodos**; `data` apunta al valor. Ese par te dice qué método resuelve un despacho dinámico.
</details>

<details>
<summary>¿Por qué una misma función genérica de Rust aparece muchas veces?</summary>

Por la **monomorfización**: Rust genera una copia por cada tipo concreto con el que se instancia el genérico. No son funciones distintas, son instancias del mismo genérico.
</details>

<details>
<summary>¿Cómo se ve un <code>dyn Trait</code> y a qué se parece?</summary>

A un **fat pointer**: puntero a datos + puntero a **vtable**. La vtable trae drop, tamaño, alineación y los métodos. Se lee parecido a una vtable de C++ (2.18).
</details>

<details>
<summary>¿Qué dos esquemas de manglado tiene Rust y cómo los leés?</summary>

El **legacy** (`_ZN...E`, estilo Itanium con hash) y el **v0** (`_R...`, RFC 2603). Se desmanglan con `rustfilt` o el soporte del desensamblador.
</details>

Puerta de salida

Podés: (1) recuperar nombres de un Go «stripeado» con la pclntab; (2) leer una interfaz Go como par (itab, data); (3) desmanglar símbolos Rust y reconocer monomorfización; (4) identificar un `dyn Trait` por su vtable y describir el layout de strings en ambos.

Fuentes de esta lección

- github.com/mandiant/GoReSym — recuperar pclntab, tipos y metadatos de Go (conocimiento externo).
- go.dev · runtime/symtab.go — formato de la pclntab y moduledata.
- RFC 2603 · Rust symbol mangling v0 y rustfilt.
- 2.13 Huella de compilador (identificación) y 2.18 C++ profundo (vtables y layouts).

Capas de esta lección

Contenido observado en CLS: la identificación de Go/Rust/Delphi incorporada a 2.13 en la revisión de esta edición. **Conocimiento externo (agregado por esta edición):** pclntab/itab, manglado v0, monomorfización, trait objects y layouts, con documentación primaria. **Recomendación editorial:** ejemplos y práctica ilustrativos, sobre binarios propios.

VB6 y p-code: nativo vs interpretado

> **Objetivo de la lección**
>
> Distinguir un ejecutable **Visual Basic 6 nativo** de uno en **p-code**, leer la **VBHeader** para orientarte, reconocer las llamadas al runtime `MSVBVM60` y saber qué herramienta usar en cada caso.
> **Prerrequisitos:** huella de compilador (2.13).

> **Tecnología legada**
>
> VB6 es de ~1998 y está discontinuado, pero **sigue apareciendo**: malware viejo, crackmes de la escena y software heredado. Se incluye como contexto histórico, igual que OllyDbg y MASM32 en 2.9.

Por qué VB6 es distinto

Un ejecutable VB6 no es C compilado: apoya casi todo en un **runtime**, `MSVBVM60.dll` (*Microsoft Visual Basic Virtual Machine*). Y puede estar compilado de dos maneras muy distintas —nativo o p-code—, que se analizan de forma diferente (**conocimiento externo**).

La VBHeader: tu punto de entrada

El *entry point* de un VB6 es casi siempre el mismo *stub*: llama a `ThunRTMain` de `MSVBVM60` pasándole un puntero a la **VBHeader** (firma `VB5!`). Desde esa estructura se llega a:

- `ProjectInfo` — datos del proyecto y tabla de objetos.
- `ObjectTable` — los forms y clases, con sus controles y **eventos**.
- La info que te dice si hay **tablas de p-code** o código nativo por evento.

Encontrar la VBHeader es el «abrir la puerta»: a partir de ahí reconstruís la estructura del programa (qué forms, qué botones, qué maneja cada evento).

Nativo vs p-code

	VB6 nativo	VB6 p-code
Qué contiene	x86 real, con **muchísimas** llamadas a `MSVBVM60`	**Bytecode** interpretado por el motor dentro de `MSVBVM60`
En IDA/Ghidra	Se lee como x86 (plagado de *thunks* `__vba*`)	Ves el **intérprete**, no la lógica (como una VM, 2.26)
Herramienta	Desensamblador normal + VB Decompiler	VB Decompiler / P32Dasm; debug con **WKTVBDE**

La clave: en **nativo**, la lógica está en el x86 (aunque enterrada entre llamadas al runtime); en **p-code**, el x86 que ves es el *intérprete* genérico y la lógica está en un bytecode —muy parecido conceptualmente a la virtualización de 2.26, pero **estándar y documentado** por la comunidad, no secreto—.

> **Ante un p-code, IDA/Ghidra no alcanzan**
>
> Con un binario en p-code, un desensamblador clásico solo te muestra el **intérprete de VB** (código de `MSVBVM60`), no tu lógica. La única vía operativa es una herramienta especializada —**VB Decompiler**— que entiende el bytecode de VB y reconstruye el pseudocódigo y los formularios. No pierdas tiempo intentando leer la lógica en el listado x86: ahí no está.

Reconocer las llamadas del runtime

En VB6 nativo, los nombres `__vba*` son el mapa de la lógica (aunque no tengas símbolos propios):

- `__vbaStrCmp`, `__vbaStrComp` — comparación de strings (típico de un *check* de serial).
- `__vbaVarTstEq`, `__vbaVarTstNe` — comparación de `Variant`.
- `rtcMsgBox` — un `MsgBox` («clave correcta/incorrecta»).
- `__vbaStrCat`, `__vbaStrMove` — armado de strings.

Ejemplo paso a paso: un check de serial en VB6 nativo

*(Ejemplo **ilustrativo**: patrón típico de un crackme VB6 propio/didáctico.)*

```
; en VB6 nativo, el "check" se delata por las llamadas al runtime
push   offset serial_ingresado
push   offset serial_esperado
call   __vbaStrCmp            ; compara los dos strings
test   eax, eax               ; 0 = iguales
jnz    short serial_malo
push   offset msg_ok
call   rtcMsgBox              ; "Correcto"
```

No hace falta entender todo el runtime: la secuencia **`__vbaStrCmp` → `test` → salto → `rtcMsgBox`** es la firma de un chequeo de serial. En p-code, en cambio, esto no se ve como x86: hay que abrirlo con **VB Decompiler**, que reconstruye los forms y el bytecode de cada evento, y para seguir la ejecución se usa **WKTVBDE**, un debugger específico de p-code.

Práctica

En tu VM, con binarios **propios o crackmes didácticos**:

1. Tomá (o compilá, si tenés el IDE) un VB6 y clasificalo **nativo vs p-code** con DIE y con la VBHeader.
2. En uno nativo, ubicá la VBHeader desde el entry point y listá los forms/eventos.
3. Encontrá un chequeo con `__vbaStrCmp` + `rtcMsgBox` y explicá la lógica.
4. Abrí un p-code en VB Decompiler y describí por qué IDA no mostraba esa lógica.

Preguntas de repaso

▼ **¿Qué papel cumple `MSVBVM60.dll`?**

> Es el **runtime** de VB6: provee las funciones (`__vba*`, `rtc*`) y, en los binarios p-code, el **intérprete** que ejecuta el bytecode. Casi todo VB6 depende de él.

▼ **¿Para qué buscás la VBHeader?**

> Es la estructura (firma `VB5!`) a la que apunta el stub de arranque; desde ella llegás a `ProjectInfo` y `ObjectTable` para reconstruir forms, controles y eventos, y ver si hay p-code.

▼ **¿Cuál es la diferencia práctica entre nativo y p-code al analizar?**

> En **nativo** la lógica está en el x86 (entre llamadas `__vba*`) y se lee con un desensamblador normal. En **p-code** el x86 es el intérprete; la lógica está en bytecode y hay que usar VB Decompiler / WKTVBDE.

▼ **¿Por qué el p-code se parece a la virtualización de 2.26?**

> Porque también hay un **intérprete** ejecutando un **bytecode** en vez de x86 directo. La diferencia: el p-code de VB6 es **estándar y documentado**, no un lenguaje secreto por binario.

▼ **¿Qué secuencia delata un check de serial en VB6 nativo?**

> `__vbaStrCmp` (comparar) → `test`/salto → `rtcMsgBox` (mensaje). Reconocer esos nombres del runtime te da la lógica sin entender todo el motor.

Puerta de salida

Podés: (1) clasificar un VB6 como nativo o p-code; (2) ubicar la VBHeader y reconstruir forms/eventos; (3) leer un chequeo por las llamadas `__vba*` / `rtc*`; (4) explicar por qué el p-code necesita VB Decompiler/WKTVBDE y en qué se parece a una VM.

Fuentes de esta lección

- vb-decompiler.org — reconstrucción de forms, nativo y p-code (conocimiento externo).
- Detect It Easy — identifica VB6 y si es nativo o p-code.
- Formato **VBHeader**/ `ProjectInfo` / `ObjectTable` — documentación de la comunidad de reversing de VB6 (P32Dasm, WKTVBDE).
- 2.13 Huella de compilador y 2.26 VM (el paralelo intérprete/bytecode).

Capas de esta lección

Contenido observado en CLS: VB6 como una de las huellas de compilador de 2.13 y el contexto legado del material (VB en Teorias_Numeradas). **Conocimiento externo (agregado por esta edición):** VBHeader, nativo vs p-code, las llamadas del runtime y las herramientas. **Recomendación editorial:** ejemplos y práctica ilustrativos, sobre binarios propios/didácticos.

Etapa 3 · .NET como primera victoria

> **Objetivo medible**
>
> Entender el runtime .NET, leer IL y comparar fuente, IL y decompilación en un programa propio.

Prerrequisitos: Etapa 2. **Duración estimada:** 7 a 10 horas. Es una estimación de esta guía, derivada de la suma de las lecciones; no es una duración declarada por CLS.

Lecciones

#	Lección	Qué aprendés
3.1	.NET: runtime, ensamblados y metadatos	CLR, IL, JIT, ensamblado y decompiladores.
3.2	Leer IL y comparar fuente, IL y decompilación	Pila de evaluación, opcodes y reconstrucción.

Recursos de CLS para esta etapa

- Reversing desde cero (contexto de análisis)
- Teorias_Numeradas (títulos .NET, pendiente)

Evaluación de salida

- Explicar C# → IL → JIT → CPU.
- Leer un método IL de 20 instrucciones y decir qué devuelve.
- Compilar Debug y Release y marcar tres diferencias.
- Reconocer señales de ofuscación.

> **Entregable de la etapa**
>
> Un archivo en tu bitácora con: lecciones completadas, ejercicios resueltos, dudas abiertas y evidencia (capturas, valores, hashes). Sin evidencia no se considera aprobada.

Errores frecuentes

- Creer que el decompilador devuelve el código fuente exacto.
- Asumir que .NET es inseguro por naturaleza.
- Analizar software comercial para saltar licencias.

Qué no entra en esta etapa

Los temas que no tienen material revisado o que exceden el marco se listan en Kernel y otros territorios. No los uses como atajo: cada rama tiene prerrequisitos.

.NET: runtime, ensamblados y metadatos

> **Objetivo de la lección**
>
> Al terminar vas a poder explicar qué es el CLR, qué contiene un ensamblado .NET, en qué se diferencia el código administrado del nativo y por qué un decompilador puede reconstruir casi el código fuente, y reconocer los límites de Native AOT y C++/CLI mixto.

Prerrequisitos: Etapa 2 completa. **Duración sugerida:** 3 horas. Estimación aproximada.

Una advertencia sobre las fuentes

En CLS **no hay un curso de .NET leído**. Lo que existe es:

- El video 38 de «Videos y material adicional», que explica el código intermedio (MSIL) y por qué OllyDbg no alcanza para analizarlo (`Videos y material adicional, Video 38`, 00:00:16).
- El video 40, que usa .NET Reflector para decompilar un binario VB.NET y ver un formulario, un botón y una caja de texto (`Videos y material adicional, Video 40`, 00:03:20).
- Títulos de la colección `Teorias_Numeradas` que mencionan .NET/dnSpy, **sin contenido leído**: son contenedores `.zip` que no se abrieron.

Por eso esta etapa combina **contenido observado** (los dos videos) con **conocimiento externo** y **recomendación editorial**. No hay una ficha de un tutorial .NET de CLS porque no se encontró uno legible.

Qué es .NET

.NET es una plataforma de ejecución. Un programa escrito en C#, VB.NET o F# no se compila directamente a código máquina: se compila a un lenguaje intermedio llamado **IL** (o **MSIL**, *Microsoft Intermediate Language*), y se guarda en un **ensamblado** (un `.exe` o `.dll`) junto con **metadatos**.

Cuando ejecutás el programa, el **CLR** (*Common Language Runtime*) toma el IL, lo compila a código nativo con el **JIT** (*just-in-time compiler*) y lo ejecuta. El resultado es código máquina, pero generado en tiempo de ejecución.

```
C#  --compilador-->  IL + metadatos  --CLR/JIT-->  código nativo  --> CPU
```

El video 38 lo explica así: los lenguajes .NET no traducen a código nativo sino a un código intermedio, y por eso un debugger nativo como OllyDbg no alcanza para analizarlos (00:00:16–00:01:30).

El ensamblado por dentro

Un ensamblado .NET es un archivo PE, pero con una diferencia clave: incluye una **cabecera CLR** y una región de metadatos. En .NET Framework, el punto de entrada del PE es un *stub* que llama al runtime

(`_CorExeMain` en `mscoree.dll`); en .NET moderno, un host nativo (`apphost`) carga el runtime y lo arranca. En los dos casos, el código que ves en el ensamblado es IL, no el `Main` compilado a máquina.

Contenido típico:

Parte	Qué guarda
Manifest	Nombre del ensamblado, versión, cultura, referencias a otros ensamblados
Metadatos	Tipos, métodos, campos, propiedades, parámetros, atributos
IL	El código de cada método
Recursos	Iconos, cadenas, imágenes
Relocaciones / PE	Lo mínimo para que Windows lo cargue

Los metadatos son la razón por la que .NET es una "primera victoria": incluyen **nombres**. Un método se llama `CheckPassword`, no `sub_401000`. Un campo se llama `intentos`, no `[ebp-8]`. Salvo que el binario esté ofuscado, el decompilador puede reconstruir una versión muy parecida al código original.

Código administrado vs nativo

Aspecto	Nativo (C/C++)	.NET (C#/VB)
Unidad de ejecución	Código máquina	IL, compilado por JIT
Nombres	Se pierden al compilar (salvo PDB)	Quedan en los metadatos
Decompilación	Assembly difícil de leer	Casi código fuente
Memoria	Manual (malloc/free)	Recolector de basura
Portabilidad	Depende de la CPU	El mismo IL corre en varias
Herramienta típica	IDA, x64dbg	ILSpy, dnSpy, dotPeek

Excepciones al modelo IL/JIT: Native AOT y C++/CLI

El recorrido IL → JIT describe el caso común, no todos los ejecutables relacionados con .NET. **Native AOT** compila IL a código nativo durante la publicación y no usa JIT al arrancar; el resultado es específico de una plataforma y puede ser más difícil de decompilar como IL. En reversing, primero identificá si hay cabecera CLR y metadatos suficientes; no asumas que todo binario hecho con .NET conserva el mismo contenido administrado.

En Native AOT, la reflexión no desaparece por completo, pero debe ser compatible con análisis estático y trimming. Miembros a los que solo se llega mediante nombres construidos en ejecución pueden requerir anotaciones o preservación explícita; de otro modo, el linker puede quitarlos. Native AOT no admite carga dinámica de ensamblados (por ejemplo, Assembly.LoadFile) ni generación de código en runtime (System.Reflection.Emit). Para reversing, ILSpy puede no tener IL recuperable como en un ensamblado JIT;

busca código nativo, metadatos/símbolos conservados y llamadas al runtime como evidencias complementarias.

C++/CLI puede producir ensamblados de modo mixto, con código nativo y administrado; C++/CLI no es una opción de publicación compatible con Native AOT. Esto introduce dos mecanismos de carga e inicialización y hace relevante el *loader lock*: el código administrado no debe ejecutarse desde `DllMain`. Para el analista, la consecuencia es práctica: un PE puede combinar metadatos CLR con secciones y código nativo; usá herramientas administradas y nativas y documentá qué parte estás observando.

Eso no significa que .NET sea "inseguro" ni que "cualquiera puede ver el código". Significa que la **ofuscación** es la protección habitual: renombrar símbolos, cifrar strings, introducir flujo falso. Analizar un binario ofuscado es más difícil, pero el modelo de ejecución sigue siendo el mismo.

Herramientas

Herramienta	Estado	Para qué
ILSpy	Activa, open source	Decompilador de referencia; también `ilspycmd`
dnSpy	Sin mantenimiento desde hace años	Debugger + decompilador; existen forks como **dnSpyEx**
dotPeek	Gratis (JetBrains)	Decompilador cómodo
.NET Reflector	De pago	El histórico que usa el video 40
ildasm	Incluido en el SDK de Windows	Volcar IL y metadatos
Visual Studio	Community gratis	Compilar y depurar tus propios programas

Para empezar, **ILSpy** alcanza. Si necesitás depurar IL paso a paso, dnSpyEx o el debugger de Visual Studio sobre tu propio código son mejores opciones que un debugger nativo.

> **Sobre dnSpy**
>
> dnSpy fue durante años la herramienta más usada para analizar .NET. Su desarrollo se detuvo y quedó como proyecto archivado. Existen bifurcaciones mantenidas (por ejemplo dnSpyEx). Si un tutorial viejo te manda a descargar "dnSpy", verificá de dónde lo bajás y qué versión es. Para leer código, ILSpy es más simple y está activo.

Ejemplo paso a paso: tu primer ensamblado

Si tenés el SDK de .NET instalado, creá un proyecto de consola:

```
dotnet new console -n HolaNet
cd HolaNet
dotnet build
```

El código generado es:

```
Console.WriteLine("Hello, World!");
```

Abrí `bin/Debug/net8.0/HolaNet.dll` en ILSpy. Vas a ver:

- El ensamblado `HolaNet`, con versión y referencias.
- La clase `Program`.
- El método `Main`, con su IL.
- La cadena `"Hello, World!"` en el manifiesto de recursos.

Qué mirar:

1. El **árbol de tipos** a la izquierda: namespace, clase, métodos.
2. El **panel de IL** (si tu versión lo muestra): las instrucciones del método.
3. El **panel de metadatos**: tokens, firmas, referencias.

Ese recorrido es el equivalente .NET de abrir un `.exe` en IDA: primero la estructura, después el código.

Práctica

1. Instalá el SDK de .NET y creá un proyecto de consola con una clase y dos métodos.
2. Compilalo en Debug y en Release. Abrí las dos versiones en ILSpy y compará el `Main`.
3. Anotá tres diferencias entre Debug y Release (por ejemplo, variables locales, saltos, llamadas).
4. Buscá una cadena que hayas escrito en el código. ¿Aparece en el ensamblado? ¿En qué forma?
5. Escribí en tu bitácora qué es el CLR, qué es el IL y por qué los metadatos facilitan el análisis.

Preguntas de repaso

1. ¿Qué es el CLR? El runtime de .NET: carga el ensamblado, compila el IL a código nativo con el JIT y administra memoria, excepciones y seguridad.

2. ¿Qué diferencia hay entre IL y código máquina? El IL es un lenguaje intermedio independiente de la CPU; el código máquina es específico del procesador. El JIT traduce IL a máquina en tiempo de ejecución.

3. ¿Por qué un decompilador .NET produce código casi fuente? Porque los metadatos conservan nombres, tipos, firmas y estructura, y el IL es una representación de alto nivel del programa.

4. ¿Un binario .NET ofuscado sigue siendo .NET? Sí. La ofuscación cambia nombres y estructura, pero el modelo de ejecución (IL + metadatos + CLR) es el mismo.

5. ¿Qué herramienta usarías para ver el IL de un método? ILSpy, ildasm o el panel de IL de un decompilador. Para depurar, dnSpyEx o Visual Studio.

Antes de avanzar

Tenés que poder:

- Explicar el recorrido C# → IL → JIT → CPU.
- Nombrar las partes de un ensamblado.
- Decir por qué los metadatos hacen más fácil el análisis.
- Abrir un `.dll` propio en ILSpy y encontrar un método.

> **Puerta de salida**
>
> Si podés compilar tu propio programa y encontrarlo en ILSpy con nombres y todo, estás listo para leer IL.

Fuentes de esta lección

- Microsoft Learn: Native AOT deployment, Initialization of mixed assemblies (C++/CLI).
- `Videos y material adicional, Video 38` — código intermedio, MSIL, VB/C# y límites de OllyDbg.
- `Videos y material adicional, Video 40` — .NET Reflector sobre un binario VB.NET.
- `teorias_numeradas_por_etapa.csv` — títulos con .NET/dnSpy, **pendientes de revisar** (contenedores no abiertos).

> **Capas de esta lección**
>
> **Contenido observado en CLS:** la explicación del video 38 y el uso de .NET Reflector en el video 40. **Conocimiento externo:** CLR, JIT, metadatos, herramientas actuales, Native AOT (carga dinámica, reflexión y trimming) y ensamblados mixtos C++/CLI. **Recomendación editorial:** el ejemplo con el SDK y la advertencia sobre dnSpy.

Ampliación editorial basada en fuentes externas — Cómo distinguir Single-File Bundle y ReadyToRun (R2R) frente a Native AOT en .NET 6/8/9+

En .NET moderno, ver un `.exe` sin cabecera CLR en PE-bear no implica necesariamente que sea **Native AOT** (Single-file deployment y ReadyToRun deployment) *(Hecho confirmado)*:

Formato de publicación	Estructura del PE	¿Conserva IL y metadatos?	Cómo analizarlo
Single-File Deployment (`PublishSingleFile`)	`apphost.exe` nativo que lleva concatenado un *Bundle* con las `.dll` administradas (marcador `.net bundle` / `hostfxr`).	**Sí**, completos dentro de las DLLs embebidas.	Abrirlo directamente en **ILSpy 8.x/9.x**, que extrae y decompila las DLLs del bundle.
ReadyToRun (R2R) (`PublishReadyToRun`)	PE con cabecera CLR cuyo `ManagedNativeHeader` apunta a `READYTORUN_HEADER` (firma `0x00525452` = `"RTR"`).	**Sí**, conserva el IL original junto al código nativo precompilado.	Se lee como C#/IL normal en ILSpy o dnSpyEx.
Native AOT (`PublishAot`)	PE 100 % nativo compilado con `ilc` , sin JIT ni bundle de IL.	**No hay IL**.	Se analiza como nativo en IDA o Ghidra.

Leer IL y comparar fuente, IL y decompilación

> **Objetivo de la lección**
>
> Al terminar vas a poder leer el IL de un método sencillo, seguir la pila de evaluación instrucción por instrucción y comparar tres vistas del mismo código: la fuente que escribiste, el IL que generó el compilador y lo que reconstruye el decompilador.

Prerrequisitos: runtime y ensamblados .NET. **Duración sugerida:** 4 horas. Estimación aproximada.

IL es una máquina de pila

A diferencia del assembly x86, que usa registros, el IL es una **máquina de pila**: las instrucciones empujan valores a una pila de evaluación y las operaciones los sacan, calculan y empujan el resultado.

```
ldarg.0        -> push a
ldarg.1        -> push b
add            -> pop b, pop a, push (a+b)
```

Eso hace que el IL sea más fácil de leer que el assembly nativo: no hay que seguir registros, solo la pila. El precio es que hay muchas instrucciones `ld*` / `st*`.

El método de ejemplo

Escribí este método en una clase de consola:

```
static bool Check(int a, int b)
{
    if (a > 50 || b < 50)
        return false;
    return (a + b) * 4 == 480;
}
```

Compilalo en **Debug** (sin optimizaciones) y abrilo en ILSpy. El IL va a ser parecido a este (los offsets exactos cambian según el compilador y la versión):

```
.method private hidebysig static bool Check(int32 a, int32 b) cil managed
{
    .maxstack 2

    ldarg.0
    ldc.i4.s 50
    cgt
    brtrue.s L_false

    ldarg.1
    ldc.i4.s 50
    clt
    brtrue.s L_false

    ldarg.0
    ldarg.1
    add
    ldc.i4.2
    shl
    ldc.i4 480
    ceq
    ret

L_false:
    ldc.i4.0
    ret
}
```

Lectura línea por línea

IL	Qué hace	Pila después
`ldarg.0`	Empuja el primer argumento (`a`)	`[a]`
`ldc.i4.s 50`	Empuja la constante 50	`[a, 50]`
`cgt`	Compara: ¿`a > 50`? Empuja 1 o 0	`[a>50]`
`brtrue.s L_false`	Si el tope es distinto de 0, salta a `L_false`	`[]`
`ldarg.1`	Empuja `b`	`[b]`
`ldc.i4.s 50`	Empuja 50	`[b, 50]`
`clt`	¿`b < 50`? Empuja 1 o 0	`[b<50]`
`brtrue.s L_false`	Si es verdadero, salta a `L_false`	`[]`
`ldarg.0`	Empuja `a`	`[a]`
`ldarg.1`	Empuja `b`	`[a, b]`
`add`	Suma	`[a+b]`
`ldc.i4.2`	Empuja 2	`[a+b, 2]`
`shl`	Desplaza a la izquierda 2 bits (×4)	`[(a+b)*4]`
`ldc.i4 480`	Empuja 480	`[(a+b)*4, 480]`
`ceq`	¿Son iguales? Empuja 1 o 0	`[resultado]`
`ret`	Devuelve el tope	`[]`
`L_false: ldc.i4.0`	Empuja 0 (false)	`[0]`
`ret`	Devuelve 0	`[]`

Comprobación con `a = 40`, `b = 80`:

1. `40 > 50` → 0. No salta.
2. `80 < 50` → 0. No salta.
3. `40 + 80 = 120`.
4. `120 << 2 = 480`.
5. `480 == 480` → 1 (`true`).

El método devuelve `true`. Con `a = 60`, el primer `cgt` da 1 y salta a `L_false`: devuelve `false` sin evaluar el resto.

> `cgt`, `clt`, `ceq`
>
> Son las comparaciones de IL: *compare greater than*, *compare less than*, *compare equal*. Empujan 1 si la condición se cumple y 0 si no. No modifican flags como `CMP`; el resultado es un valor en la pila.

Las tres vistas del mismo método

Fuente (lo que escribiste):

```
if (a > 50 || b < 50)
    return false;
return (a + b) * 4 == 480;
```

IL (lo que generó el compilador): el listado de arriba, con `brtrue` y una etiqueta `L_false`.

Decompilación (lo que reconstruye ILSpy):

```
private static bool Check(int a, int b)
{
    if (a <= 50 && b >= 50)
    {
        return (a + b) * 4 == 480;
    }
    return false;
}
```

Las tres son equivalentes, pero **no son iguales**. El decompilador invirtió la condición (`a > 50 || b < 50` se convirtió en `a <= 50 && b >= 50` por De Morgan), movió el `return false` al final y agregó llaves. Eso es normal: reconstruye una lógica equivalente, no el texto original.

Cuando analices un binario ajeno, no esperes "el código fuente". Esperá **una versión legible y equivalente**, que puede diferir en estilo, nombres y estructura.

Metadatos: dónde están los nombres

En ILSpy, además del código, podés ver:

- **Assembly**: nombre, versión, cultura, referencias (`System.Runtime`, `System.Console`).
- **Namespace** y **Type**: `HolaNet.Program`.
- **Method**: nombre, firma, atributos (`private`, `static`).
- **Tokens**: identificadores numéricos que el runtime usa para referenciar tipos y métodos (por ejemplo, un `MethodDef` con su token). No los necesitás para leer, pero explican por qué el decompilador puede reconstruir las referencias.

Si el binario está ofuscado, los nombres pueden ser `\u0001`, `A`, `B`, `method_0`. La estructura sigue ahí; lo que se pierde es la legibilidad.

Debug vs Release

Compilá el mismo método en Release y compará:

- En Debug, las variables locales y los puntos de secuencia están pensados para depurar; el IL es más "literal".
- En Release, el compilador puede inline-ar métodos chicos, reordenar, eliminar variables y simplificar condiciones.
- Las líneas y nombres de variables locales pueden desaparecer si no hay PDB.

Para aprender, usá Debug. Para analizar un binario real, asumí Release y no te sorprendas si el decompilador muestra un código más raro.

Ofuscación: qué cambia y qué no

Técnica	Qué hace	Cómo se reconoce
Renombrado	Cambia nombres por símbolos ilegibles	`\u0001`, `A`, `method_0`
Cifrado de strings	Guarda las cadenas cifradas y las descifra en runtime	Strings ausentes o ilegibles; llamadas a un método de descifrado
Flujo falso	Agrega saltos y bloques que nunca se ejecutan	Grafos con muchos bloques muertos
State machine	Convierte métodos en una máquina de estados	`switch` sobre un estado, `MoveNext`

El decompilador sigue siendo útil, pero hay que **verificar**. Una condición reconstruida puede ser incorrecta si el flujo está ofuscado. La regla es la misma que en nativo: contrastar estático con dinámico, sobre un programa propio o autorizado.

Práctica

1. Compilá el método `Check` en Debug y en Release. Pegá las dos versiones de IL y marcá tres diferencias.
2. Escribí un método con un `for` que sume 1..10. Encontrá en el IL el salto que vuelve al inicio del bucle.
3. Escribí una propiedad `public int Edad { get; set; }` y buscá el getter y el setter en ILSpy. ¿Qué métodos genera el compilador?
4. Escribí un método `async` que espere un segundo e imprima algo. Buscá la máquina de estados que genera el compilador. No hace falta entenderla entera: identificá `MoveNext`.
5. Compará la decompilación con tu fuente. Marcá al menos una diferencia de estilo y explicá por qué es equivalente.
6. Escribí en tu bitácora: qué método analizaste, qué IL te sorprendió y qué parte no entendés todavía.

Preguntas de repaso

1. ¿IL usa registros? No. Es una máquina de pila: las instrucciones empujan y sacan valores de una pila de evaluación.

2. ¿Qué hace `cgt`? Compara los dos valores del tope: empuja 1 si el primero es mayor que el segundo, 0 si no.

3. ¿Por qué el decompilador muestra una condición distinta de la fuente? Porque reconstruye una lógica equivalente a partir del IL. Puede aplicar De Morgan, reordenar retornos o simplificar expresiones.

4. ¿Qué se pierde en Release? Variables locales, líneas y algunas simplificaciones que el compilador aplica. Los nombres de tipos y métodos se conservan, salvo ofuscación.

5. ¿Qué hace `shl` en el IL? Desplaza a la izquierda la cantidad de bits indicada. `shl 2` equivale a multiplicar por 4.

6. ¿La ofuscación impide analizar un binario .NET? No. Lo hace más difícil. El IL y los metadatos siguen ahí; hay que verificar la reconstrucción con ejecución controlada.

Antes de avanzar

Tenés que poder:

- Leer un método IL de 20 instrucciones y decir qué devuelve.
- Seguir la pila de evaluación con un valor concreto.
- Comparar fuente, IL y decompilación y explicar una diferencia.
- Reconocer señales de ofuscación.

> **Puerta de salida de la Etapa 3**
>
> Si podés compilar tu propio método, leer su IL y explicar qué hace el decompilador, terminaste .NET. La Etapa 4 es el método de análisis completo, con informe.

Fuentes de esta lección

- `Videos y material adicional, Video 38` — MSIL y por qué el código intermedio requiere otras herramientas.
- `Videos y material adicional, Video 40` — .NET Reflector y reconstrucción de un formulario VB.NET.
- `teorias_numeradas_por_etapa.csv` — títulos .NET/dnSpy **pendientes de revisar**.

> **Capas de esta lección**
>
> **Contenido observado en CLS:** el video 38 (MSIL) y el video 40 (decompilación VB.NET). **Conocimiento externo:** conjunto de instrucciones IL, pila de evaluación y comportamiento de los decompiladores. **Recomendación editorial:** el método `Check`, la tabla línea por línea y los ejercicios.

Etapa 4 · Práctica guiada

> **Objetivo medible**
>
> Analizar programas propios con método, reconstruir un formato o protocolo desde muestras, contener la ejecución y escribir un informe reproducible.

Prerrequisitos: Etapas 0 a 3. **Duración estimada:** 19 a 25 horas. Es una estimación de esta guía, derivada de la suma de las lecciones; no es una duración declarada por CLS.

Lecciones

#	Lección	Qué aprendés
4.1	Método de análisis: de la pregunta al hallazgo	Ciclo de cinco pasos, hipótesis y confirmación.
4.2	Reconocer criptografía y encoding en un binario	XOR, base64, constantes de AES/MD5/RC4 y CryptoAPI/CNG.
4.3	El informe reproducible	Estructura, evidencia, límites y divulgación.
4.4	Análisis dinámico seguro: cuándo ejecutar y cuándo no	Matriz de decisión, contención y respuesta.
4.5	Reconstruir formatos y protocolos propios	Campos, longitudes, framing, estados y transformaciones desde muestras controladas.
4.6	Protocolos de terceros: reconstruir un protocolo propietario	Framing, serialización (TLV/Protobuf), hooks en send/recv y disector de Wireshark; solo sistemas propios o autorizados.

Recursos de CLS para esta etapa

- Reversing desde cero (ejercicios 1 y 2)
- Inventario y estados de cuarentena

Evaluación de salida

- Analizar un programa propio de punta a punta sin mirar el fuente.
- Reconstruir desde muestras propias un formato binario o un framing simple, justificando campos, longitudes y estados.
- Confirmar una condición con breakpoints y valores concretos.
- Escribir un informe con hash, entorno, evidencia y límites.
- Explicar por qué el caso Shelter no es una práctica de principiante.

Entregable de la etapa

Un archivo en tu bitácora con: lecciones completadas, ejercicios resueltos, dudas abiertas y evidencia (capturas, valores, hashes). Sin evidencia no se considera aprobada.

Errores frecuentes

- Parchear antes de entender.
- Ejecutar una muestra dudosa en el host.
- Escribir conclusiones sin evidencia.

Qué no entra en esta etapa

Los temas que no tienen material revisado o que exceden el marco se listan en Kernel y otros territorios. No los uses como atajo: cada rama tiene prerrequisitos.

Método de análisis: de la pregunta al hallazgo

> **Objetivo de la lección**
>
> Al terminar vas a poder analizar un programa de práctica con un método explícito: formular una pregunta, inspeccionar el binario, proponer una hipótesis, confirmarla con el debugger y escribir una conclusión que otra persona pueda reproducir.

Prerrequisitos: etapas 0 a 3. **Duración sugerida:** 6 horas (4 para el ciclo con `check.exe` y 2 para el formato propio). Estimación aproximada.

Por qué un método y no "ir probando"

Cuando empezás, el debugger invita a tocar botones: F8, F7, breakpoints, parches. Eso funciona para un programa de 20 líneas y fracasa con uno de 20.000. Un método te evita dos trampas:

1. **Perderte en el camino equivocado.** Sin una pregunta, cualquier instrucción parece interesante.
2. **Confundir correlación con explicación.** Ver que un salto se toma no significa entender por qué.

El capítulo 11 lo dice a su manera: primero buscás lo más fácil (las strings, los mensajes), y si no alcanza, cambiás de estrategia. El análisis visual es rápido, pero hay que saber qué se está buscando (`Reversing desde cero, cap. 11`, 00:00:05–00:02:00).

El ciclo de cinco pasos

```
1. Pregunta     -> ¿qué quiero saber?
2. Inventario   -> ¿qué es este archivo?
3. Estático     -> ¿dónde está la lógica?
4. Hipótesis    -> ¿qué creo que hace y por qué?
5. Confirmación -> ¿cómo lo pruebo?
```

Después se escribe la conclusión y se anotan los límites. Si la confirmación falla, se vuelve al paso 3 con una hipótesis nueva. Ese ida y vuelta es el trabajo real.

Paso 1: la pregunta

Una pregunta útil es específica y verificable. No es "¿qué hace este programa?", sino:

- ¿Qué condición tiene que cumplir la entrada para que muestre "Correcto"?
- ¿Qué compara exactamente la instrucción de la dirección `0x401234`?
- ¿De dónde sale el valor que se usa como índice?

Para el programa de práctica de esta lección, la pregunta es: **¿qué pares de números hacen que `check.exe` imprima "Correcto"?**

Paso 2: inventario

Antes de abrir el debugger, respondé:

Pregunta	Herramienta	Respuesta (ejemplo)
¿Qué tipo de archivo es?	Visor de PE	PE32, ejecutable de consola
¿Es nativo o .NET?	Detect It Easy	Nativo (C)
¿Está empacado?	Secciones/imports	No: `.text` normal, imports de `msvcrt`
¿Qué cadenas muestra?	`strings` o IDA	"Primer valor", "Segundo valor", "Correcto", "Incorrecto"
¿Qué imports usa?	Visor de PE	`printf`, `scanf`, `puts`
¿Tiene símbolos?	PDB	Sí, si lo compilaste en Debug

Ese inventario ya te dice mucho: un programa de consola que usa `scanf` y `puts`, sin empaquetar, con cadenas claras. No hace falta empezar por el entry point.

Árbol de decisión de triage inicial («¿Por dónde empiezo?»)

Cuando te enfrentás a un binario desconocido (un crackme, un programa propio compilado meses atrás o un componente de laboratorio), los primeros diez minutos del **Paso 2 (Inventario)** definen qué herramientas abrir y a qué lección del curso acudir. Usar IDA sobre un ensamblado .NET puro o buscar strings en un binario empaquetado con UPX hace perder horas. Este árbol de decisión conecta los tres niveles del curso (Principiante, Intermedio y Avanzado) en un flujo único (y se complementa con el Hilo conductor multi-nivel y kit de plantillas C/C++):


FASE 1 · CONTENEDOR, ARQUITECTURA Y RUNTIME (PE-bear / DIE · 2 minutos)
¿NO empieza con "MZ" / "PE\0\0"?
¿Es archivo de datos / captura de mensajes? ⟶ Lección 4.5 (Formatos y protocolos)
¿Es código máquina sin cabecera (posición independiente)? ⟶ Lección 2.23 (Shellcode)
¿Es PE32 (0x10B, 32 bits) o PE32+ (0x20B, 64 bits)? (Lecciones 1.4 y 2.5)
32 bits (x86): argumentos en stack (cdecl/stdcall) o ECX/EDX (fastcall/thiscall);
en Windows x64 corre bajo WOW64 (ver Lección 2D.2).
64 bits (x64): argumentos en RCX, RDX, R8, R9 + shadow space de 32 bytes.
¿Tiene cabecera CLR (.NET) o marcador ".net bundle" (hostfxr)? (Lección 3.1)
SÍ (IL puro, ReadyToRun "RTR\0" o Single-File Bundle):
Abrirlo en ILSpy / dnSpyEx (Etapa 3: Lecciones 3.1 y 3.2).
NO (Nativo puro o .NET Native AOT):
Avanzar a la FASE 2.
FASE 2 · ENTROPÍA, SECCIONES, IMPORTS Y HUELLA DE COMPILADOR (3 minutos)
¿Entropía alta (> 6,8), secciones renombradas (UPX0/UPX1), permisos W+X
o primera sección con RawSize = 0 e IAT mínima (LoadLibraryA + GetProcAddress)?
SÍ: Posible PACKER (Lección 2.17).
En UPX x86: "ESP trick" (F8 sobre pushad + HW breakpoint en [ESP]).
En UPX x64: breakpoint de memoria en ejecución sobre la sección vacía.
En el OEP: dumpear y reconstruir IAT con Scylla (si es VM comercial: ver 2.10 / 5.4).
IAT casi vacía pero entropía normal:
Resolución dinámica por PEB / API Hashing (Lección 2.14):
buscar lecturas de fs:[30h] / gs:[60h], recorrido de InMemoryOrderModuleList
([rsi+20h] = DllBase en x64) y constantes de hash de 32 bits.
¿Qué compilador o runtime delata la huella? (Lección 2.13)
MSVC C/C++ (Rich header + vcruntime140.dll / ucrtbase.dll + __security_cookie):
¿Hay vtables con RTTI en vtable[-1] o __CxxFrameHandler3/4 en .pdata/.xdata?
⟶ Lección 2.18 (C++ profundo: RTTI, std::string SSO, STL y excepciones).
MFC (mfc140u.dll o estático con CWnd/CDialog):
Lección 2.12: localizar GetMessageMap() ⟶ AFX_MSGMAP y AFX_MSGMAP_ENTRY.
MinGW / GCC (msvcrt.dll + __mingw_* + sección .eh_frame, sin Rich header):
Lección 2.13: ubicar main pasado a __getmainargs.
Delphi (recursos RCDATA con firma "TPF0", TForm/TObject):
Lección 2.13: ABI register en x86 (EAX=Self, EDX, ECX) y longitud en [ptr-4].
Go (binario grande estático, "Go build ID", tabla pclntab):
Lección 2.13: ir a main.main; recordar que string = {ptr, len} sin byte 0.
Rust (rutas "src/main.rs", mensajes de pánico Option::unwrap, símbolos _ZN / _R):
Lección 2.13: separar ramas de pánico del flujo principal; string = {ptr, len}.
FASE 3 · PRE-MAIN, OFUSCACIÓN Y ROL DEL COMPONENTE (5 minutos)
¿El directorio TLS (AddressOfCallBacks) tiene punteros?
SÍ: Inspeccionar TLS Callbacks antes de main (Lección 2.11); suelen ejecutar
chequeos anti-debug (IsDebuggerPresent, PEB.NtGlobalFlag, NtQueryInformationProcess, 2.15).
¿La ventana de Strings (Shift+F12) no muestra los mensajes esperados?
Buscar cadenas UTF-16 (funciones W, Lección 2.7), cadenas armadas en el stack (mov byte)
o bucles de descifrado XOR / RC4 / ChaCha20 / AES previos al uso (Lecciones 2.10 y 4.2).
¿El grafo de la función en IDA/Ghidra es un único bucle gigante con switch?
Distinguir Aplanamiento de Flujo (CFF: bloques con lógica real y variable de estado)
frente a Máquina Virtual (VM: handlers genéricos leídos por VIP) (Lección 2.10).
¿Cuál es el rol del binario y tu objetivo de análisis?
Programa de consola/GUI estándar ⟶ Ciclo de 5 pasos de esta lección (4.1) + 2.6 / 2.7 / 2.16.
Servicio de Windows (StartServiceCtrlDispatcher) ⟶ Lección 2D.4 (Servicios).
Persistencia en Registro / Tareas / WMI ⟶ Lecciones 2D.3, 2D.5 y 2D.9.
Servidor o cliente COM / DCOM / RPC / Named Pipe ⟶ Lecciones 2D.6 a 2D.10 y 4.5.
Diagnóstico de crash, corrupción de heap o comparación con parche ⟶ Lecciones 2.19, 2.20, 2D.11 y 5.2.
Driver de kernel (.sys, importa ntoskrnl.exe / WdfVersionBind) ⟶ Etapa 6 (6.1 a 6.6).

Nivel	Señal observada en los primeros 5 minutos	Qué significa (hipótesis inicial)	Acción inmediata y lección de referencia
Principiante	Imports claros (`scanf`, `printf`, `MessageBoxW`) y strings visibles en `.rdata`	Binario C/C++ sin empaquetar ni ofuscar	Buscar la string clave con `Shift+F12`, saltar por xref (`X`) y aplicar el ciclo de 5 pasos (2.7, 4.1).
Principiante	Las strings de éxito y error están en el **mismo bloque básico** sin salto condicional entre ellas	Selección sin salto (*branchless*) producida por el compilador (`-O2`)	Buscar una instrucción `CMOVcc` (`cmovne` / `cmove`) que elige qué puntero de cadena pasar a `puts` / `printf` (2.4, 2.8, 4.1).
Principiante	Importa `mscoree.dll` (`_CorExeMain`) o contiene metadatos `.NET` / marcador `.net bundle`	Aplicación administrada .NET (IL, ReadyToRun o Single-File)	Abrir en **ILSpy** o **dnSpyEx** para leer C# e IL antes de tocar un desensamblador nativo (3.1, 3.2).
Intermedio	Secciones `UPX0` (`RawSize = 0`) y `UPX1` (entropía > 7,5, permisos `W+X`), pocos imports	Ejecutable empaquetado con UPX	Llegar al OEP (*ESP trick* en x86 o breakpoint de memoria en ejecución sobre `UPX0` en x64), dumpear y reconstruir IAT con Scylla (2.17).
Intermedio	Mi breakpoint en `main` nunca se alcanza, o una variable global ya llega alterada al entrar a `main`	Código ejecutado antes del entry point principal	Revisar el directorio **TLS (`AddressOfCallBacks`)** en PE-bear/IDA y poner breakpoint en cada callback (2.11, 2.15).
Intermedio	Casi sin imports ni strings, pero lee `gs:[60h]` → `[rax+18h]` → `[rax+20h]` y rota enteros (`ror edx, 0Dh`)	Resolución dinámica de APIs por recorrido del PEB y *API hashing*	Poner breakpoint en el `call reg` del resolvedor o cruzar constantes con listas de hashes de exportaciones (2.14).
Intermedio	Constantes `0x637c777b` (AES), `"expand 32-byte k"` (ChaCha20) o `0x9E3779B9` (TEA) en `.rdata`/código	Transformación criptográfica estándar o cifrado de configuración/mensajes	Etiquetar la función criptográfica por su constante mágica e interceptar su entrada/salida en vez de analizar las rondas internas (4.2, 2.21).
Avanzado	Llamadas indirectas `call qword ptr [rax+18h]` donde `rax` proviene de `[rcx]`	Despacho virtual C++ (`vtable`) o interfaz COM (`IUnknown`)	Ubicar la `vtable` en `.rdata`, leer `vtable[-1]` (`Complete Object Locator`, RVAs en x64) para recuperar el nombre de la clase (2.18, 2D.6).
Avanzado	El proceso muere dentro de `ntdll!RtlFreeHeap` lejos de donde ocurrió el error real	Corrupción de heap previa (desborde, *use-after-free* o *double free*)	Activar **Full Page Heap** (`gflags /p /enable app.exe /full`) o grabar traza **TTD** en WinDbg y retroceder con `ba w4` + `g-` (2.19).
Avanzado	Archivo `.sys` que importa `WdfVersionBind` y registra `EvtIoDeviceControl`	Driver de modo kernel basado en KMDF	Desglosar cada constante `CTL_CODE` del `switch` (DeviceType, Access, Function, Method), revisar `SDDL` y auditar tamaños mínimos y usos de `METHOD_NEITHER` (6.2, 6.4).

Paso 3: análisis estático

Abrí el binario en IDA Free o Ghidra y seguí las cadenas:

1. `Shift+F12`, buscá "Correcto".
2. `X` sobre la cadena para ver quién la usa.
3. Leé el bloque: vas a encontrar una comparación y dos caminos.
4. Renombrá: `check`, `primer_valor`, `segundo_valor`, `camino_ok`, `camino_error`.
5. Anotá la condición que deduce el grafo.

En el programa del capítulo 15, el análisis estático muestra:

- `scanf("%d", &a)` y `scanf("%d", &b)`;
- `if (a > 50 || b < 50) error;`
- `if ((a + b) * 4 == 480) correcto; else incorrecto;`

El capítulo 15 hace exactamente este recorrido, con `0x32` (50) y `0x1E0` (480), y lo confirma después en el debugger (`Reversing desde cero, cap. 15`, 00:06:00–00:16:00).

Paso 4: la hipótesis

Una hipótesis se escribe como una afirmación falsable:

> "El programa acepta cualquier par `(a, b)` con `a <= 50`, `b >= 50` y `(a + b) * 4 == 480`. Con `a = 40` y `b = 80` debería imprimir 'Correcto'; con `a = 40` y `b = 70` debería imprimir 'Incorrecto'."

Falsable significa que podés imaginar un resultado que la contradiga. "El programa compara números" no es falsable; "compara contra 50 y contra 480" sí.

Paso 5: confirmación dinámica

Ahora sí, el debugger. El objetivo no es "ver qué pasa", sino **comprobar la hipótesis**.

Preparación

1. Copiá el programa a la VM.
2. Abrilo en x64dbg.
3. Buscá la cadena "Correcto" y seguí la referencia hasta el bloque.
4. Etiquetá `camino_ok` y `camino_error`.
5. Poné un breakpoint en la comparación final (`cmp eax, 0x1E0`).

Ejecución con `40` y `80`

1. `F9` para correr.
2. Ingresá `40` y `80`.

3. El programa se detiene en el breakpoint.
4. Mirá `EAX`: debería ser `0x1E0` (480).
5. Mirá `ZF`: debería ser 1.
6. `F8` para ejecutar el salto: no se toma el salto a `camino_error`.
7. `F9`: imprime "Correcto".

Ejecución con `40` y `70`

1. Reiniciá (`Ctrl+F2`).
2. Ingresá `40` y `70`.
3. En el breakpoint, `EAX` debería ser `0x1B8` (440).
4. `ZF` = 0; el salto se toma; imprime "Incorrecto".

Ejecución con `60` y `80`

1. Reiniciá.
2. Ingresá `60` y `80`.
3. El programa ni siquiera llega a la comparación final: falla la primera condición (`a > 50`).
4. Confirmá dónde se detiene y qué salto se toma.

> **Tu compilador puede generar otro código**
>
> Las direcciones y la forma exacta dependen del compilador, las optimizaciones y la arquitectura. Lo que no cambia es la **lógica**: dos comparaciones de rango, una suma, un desplazamiento y una comparación final. Si tu listado no coincide, buscá esas operaciones, no una dirección mágica.

Dos casos que afinan el método

El ciclo de arriba sirve para la mayoría de los programas, pero conviene conocer dos situaciones que lo ponen a prueba.

La decisión no está donde la buscás (cap. 16). Lo habitual es un bloque "bueno" y un bloque "malo" separados por un salto. Pero un programa puede imprimir el mensaje correcto o el incorrecto **desde el mismo bloque**, eligiendo con un `CMOVcc` (ver la lección de flujo) qué puntero de cadena usar. Entonces no hay un salto evidente para entender —ni para parchear—. La lección de método: si las cadenas de éxito y error están en el mismo bloque, buscá un movimiento condicional, no un `Jcc`. En ese mismo ejercicio aparece un `strlen` *inline* (un bucle que incrementa un puntero hasta el byte `0` y resta inicio de final) en vez de una llamada a la API: reconocer funciones estándar escritas a mano es parte del método (`Reversing desde cero, cap. 16`; transcripción .txt, sin marcas).

Sin símbolos, igual se puede (cap. 20). El capítulo 20 resuelve un crackme **sin PDB**, y es un buen modelo del ciclo: ejecutar y anotar las cadenas; abrir en IDA; buscar la cadena de error con `Shift+F12` y llegar al código con `X`; **pintar** el bloque malo (salmón), el bueno (verde) y el camino (celeste); como la función es chica, analizarla **desde el inicio** hasta la comparación; renombrar funciones y variables mientras se lee,

poniendo `printf?` con signo de pregunta cuando no hay certeza; deducir el algoritmo (ahí, el serial es la suma de los caracteres del nombre) y confirmarlo en el debugger con *popups* de valores (`Reversing desde cero, cap. 20`). Un detalle útil de ese capítulo: en 32 bits, **el primer argumento es el que está más cerca del `call`**; y sobre una variable del stack, `A` la convierte en array/define su tamaño y `H` muestra el valor en decimal.

> **Cuando el resultado depende del tiempo o del entorno**
>
> A veces el valor correcto no es fijo: el capítulo 19 analiza un programa de 64 bits cuya clave se calcula a partir de `GetSystemTime` (depende del minuto actual). Entonces no se puede "ver el valor y volver a ingresarlo", porque para cuando lo tipeás ya cambió. Ahí la confirmación dinámica no alcanza por sí sola: hay que **entender el algoritmo** (o automatizarlo). Es un recordatorio de que estático y dinámico se complementan, y de que una sola corrida no demuestra el caso general (`Reversing desde cero, cap. 19`).

Cuando la pregunta es un formato o un protocolo propio

Este apartado muestra la reconstrucción de **un archivo propio** dentro del ciclo general. La lección 4.5 · Reconstruir formatos y protocolos propios sistematiza el trabajo con muestras, framing, protocolos con estado y validación independiente.

Muchas preguntas reales no son "¿qué condición acepta?" sino "¿cómo es el archivo que guarda este programa?" o "¿qué significan los mensajes que manda?". El ciclo de cinco pasos sirve igual; cambian las herramientas de cada paso. Este apartado es **recomendación editorial** apoyada en **conocimiento externo**; el ejemplo es **ilustrativo** y supone un programa **propio** que guarda su configuración en un archivo binario.

1. Pregunta. Concreta: "¿qué campos tiene el archivo `.blc` y qué valida el programa al abrirlo?".

2. Inventario. Dos o tres archivos generados por el propio programa con datos distintos, y un volcado hexadecimal de cada uno. Las primeras cosas a buscar: bytes fijos al inicio (**número mágico**), números que coinciden con el tamaño del archivo o de un bloque (**campos de longitud**), texto legible (UTF-16 incluido, ver IDA y el análisis estático) y zonas de alta entropía (posible compresión o cifrado, ver criptografía y encoding).

3. Estático: encontrar el parser. El código que lee el formato está detrás de las llamadas que traen los bytes: `ReadFile`, `fread`, `MapViewOfFile` para archivos; `recv` / `WSARecv` para red. Desde la xref, seguí el buffer: cada comparación contra una constante es una validación (el mágico, una versión) y cada lectura `[buffer+N]` con un tamaño fijo es un campo. Una lectura seguida de un salto que usa ese valor como cantidad o desplazamiento es un campo de longitud u offset.

4. Hipótesis: la tabla de campos. Escribí la estructura como tabla, con la evidencia de cada fila:

Offset	Tamaño	Tipo	Significado (hipótesis)	Evidencia
`0x00`	4	bytes	Mágico `BLOC`	Comparado con `0x434F4C42` antes de seguir
`0x04`	2	uint16 LE	Versión	`cmp ax, 2` y rama de error "versión no soportada"
`0x06`	2	uint16 LE	Cantidad de registros	Controla el contador del bucle
`0x08`	4	uint32 LE	Checksum del resto	Comparado con el resultado de una función con tabla de 256 entradas (CRC32)
`0x0C`	variable	registros	Datos	Leídos de a `0x20` bytes dentro del bucle

5. Confirmación diferencial. Con una **copia** del archivo, cambiá **un solo byte** por vez y observá: si cambiar la versión dispara el error de versión, la fila está confirmada; si cambiar un dato hace fallar el checksum, ya sabés que lo cubre. La prueba final es escribir un **lector independiente** (un script de Python con el módulo `struct`, o una descripción declarativa en un lenguaje como Kaitai Struct) que lea los tres archivos y dé los mismos valores que muestra el programa.

Cuidados: trabajá sobre copias; el programa puede reescribir o "reparar" el archivo al abrirlo. Para protocolos de red, el mismo método se aplica a capturas de tu propio cliente y servidor en la VM aislada, sin apuntar a servicios de terceros. Si un campo está cifrado o comprimido, la tabla lo dice así ("bloque cifrado, clave no recuperada") en lugar de inventarle estructura.

Cómo se escribe la conclusión

Una conclusión útil tiene cuatro partes:

1. **Respuesta a la pregunta.** "El programa acepta pares con `a <= 50`, `b >= 50` y `(a+b)*4 == 480`."
2. **Evidencia.** "Breakpoint en `0x...`: con (40,80), `EAX=0x1E0`, `ZF=1`, imprime 'Correcto'; con (40,70), `EAX=0x1B8`, `ZF=0`, imprime 'Incorrecto'."
3. **Método.** "Análisis estático en IDA Free + confirmación en x64dbg; binario compilado con gcc -O0."
4. **Límites.** "No se probaron todos los pares; la conclusión vale para esta compilación."

Eso es un hallazgo reproducible. No es "lo crackeé": es "entendí la condición y la verifiqué".

Errores frecuentes

Error	Por qué pasa	Cómo evitarlo
Buscar solo strings	El programa puede no tener mensajes claros	Probá imports, xrefs y el entry point
Confundir dirección virtual con offset	ASLR y secciones	Usá RVA u offset, o rebaseá
Leer un salto al revés	`jne` salta cuando **no** son iguales	Preguntá qué camino toma el salto
Confundir signo	`JA` vs `JG`	Mirá el tipo del dato y la instrucción
Parchear como primera opción	Es más rápido que entender	Primero explicá; después decidí si hace falta
No anotar	Te olvidás de lo que probaste	Bitácora en el momento

Práctica

1. Analizá tu `check.exe` con el ciclo de cinco pasos. Escribí la pregunta, el inventario, la hipótesis y la confirmación.
2. Probá tres pares válidos y tres inválidos. Anotá los valores de `EAX` y `ZF` en cada caso.
3. Cambiá el programa: hacé que la condición sea `(a + b) * 2 == 240`. Recompilá y repetí el análisis sin mirar el código fuente. ¿Pudiste deducir la nueva constante?
4. Agregá una tercera condición (por ejemplo `a != b`) y analizala.
5. Escribí un párrafo de conclusión con las cuatro partes.
6. Escribí un programa propio que guarde tres o cuatro valores en un archivo binario con un mágico, una versión y un campo de longitud. Sin mirar el fuente, armá la tabla de campos con evidencia, confirmala cambiando un byte por vez sobre copias y escribí un lector en Python con `struct` que dé los mismos valores.

Preguntas de repaso

1. ¿Cuál es la diferencia entre una hipótesis y una observación? La hipótesis es una afirmación que hacés antes de confirmar; la observación es lo que el debugger te muestra. La hipótesis se contrasta con la observación.

2. ¿Por qué conviene empezar por las strings? Porque en programas con interfaz o mensajes, las cadenas te llevan rápido a la lógica y a los bloques de éxito/error.

3. ¿Qué hacés si la confirmación falla? Volvés al análisis estático con una hipótesis nueva. No parcheás para que "funcione": entendés por qué falló.

4. ¿Por qué hay que anotar la compilación y las direcciones? Porque el mismo código compilado con otra versión u optimización puede generar otro listado. Sin esos datos, el informe no es reproducible.

5. ¿Qué significa que un hallazgo sea reproducible? Que otra persona, con el mismo binario y el mismo método, puede obtener el mismo resultado.

6. Querés entender el formato de archivo de un programa. ¿Dónde empezás a buscar el parser? En las xrefs a las APIs que traen los bytes (`ReadFile`, `fread`, `MapViewOfFile`; `recv` para red) y siguiendo el buffer: las comparaciones con constantes son validaciones y las lecturas a offsets fijos, campos.

7. ¿Qué es una confirmación diferencial y por qué se hace sobre copias? Cambiar un solo byte por vez y observar qué validación falla o qué valor cambia. Se hace sobre copias porque el programa puede reescribir el archivo y porque el original es la referencia.

8. ¿Cuándo das por confirmada la tabla de campos? Cuando un lector independiente (un script propio) lee varios archivos y obtiene los mismos valores que muestra el programa; lo que no se pudo explicar queda anotado como tal.

Antes de avanzar

Tenés que poder:

- Formular una pregunta específica y una hipótesis falsable.
- Hacer el inventario de un binario antes de ejecutarlo.
- Confirmar una condición con un breakpoint y valores concretos.
- Escribir una conclusión con evidencia y límites.
- Reconstruir el formato de un archivo propio como tabla de campos con evidencia y confirmarlo con un lector independiente.

> **Puerta de salida**
>
> Si podés analizar un programa propio y explicar la condición sin mirar el fuente, aprobaste el método. La lección siguiente convierte ese trabajo en un informe.

> **Lo que queda fuera**
>
> El objetivo es reconocer y documentar compresión, checksums, hashes, MAC y cifrado, no romper cifrado ni recuperar claves. Tampoco se reconstruyen protocolos de servicios de terceros. La lección 4.5 enseña el procedimiento con formatos y comunicaciones de programas propios; el *fuzzing* guiado por formato se presenta en la optativa 2.24.

Fuentes de esta lección

- `Reversing desde cero, cap. 11` — análisis dinámico, strings, etiquetas y hipótesis.
- `Reversing desde cero, cap. 15` — análisis estático, condiciones `0x32` / `0x1E0` y confirmación.
- `Reversing desde cero, cap. 16` — decisión con `CMOVcc` en el mismo bloque y `strlen` inline.

- `Reversing desde cero, cap. 19` — clave dependiente del tiempo (`GetSystemTime`): por qué no se reproduce ingresando el mismo valor.
- `Reversing desde cero, cap. 20` — modelo del ciclo sin símbolos: strings, xref, pintar, analizar desde el inicio, renombrar con `?`.
- `plantilla_informe.md` — plantilla base para el informe.
- Conocimiento externo: struct — Interpret bytes as packed binary data (documentación de Python) y Kaitai Struct (lenguaje declarativo para describir formatos binarios).

> **Capas de esta lección**
>
> **Contenido observado en CLS:** el método y los valores de los capítulos 11 y 15 (`0x32` , `0x1E0` , `40` / `80`), el caso `CMOVcc` / `strlen` inline del 16, el modelo sin símbolos del 20 y la clave por tiempo del 19. **Conocimiento externo:** ciclo hipótesis-confirmación, criterios de reproducibilidad y la reconstrucción de formatos (mágicos, campos de longitud, checksums). **Ejemplo ilustrativo:** el formato `.blc` y su tabla de campos. **Recomendación editorial:** el programa `check.exe` , la tabla de errores y la confirmación diferencial con lector independiente.

Reconocer criptografía y encoding en un binario

> **De dónde sale esta lección**
>
> Este tema **no está en el corpus CLS**: es **conocimiento externo** que agregamos para completar la guía. Se incluye porque reconocer cripto y encoding es rutina en análisis de malware de Windows y en CTF. El enfoque es **defensivo y de identificación**: reconocer qué algoritmo hay, no implementar criptografía ni romperla.

> **Objetivo de la lección**
>
> Al terminar vas a poder distinguir encoding de cifrado, reconocer un XOR y un base64 en un listado, e identificar algoritmos conocidos (AES, RC4, MD5/SHA, TEA, CRC32) por sus **constantes y tablas**.

Prerrequisitos: instrucciones, flujo, método (Etapas 2 y 4). **Duración sugerida:** 3 horas.

Encoding no es cifrado

Es la primera distinción, y la que más se confunde:

- **Encoding** (base64, hex, URL-encoding) transforma datos para transportarlos. **No tiene clave**: cualquiera lo revierte. Si algo está en base64, está "ofuscado", no "cifrado".
- **Cifrado** usa una **clave**. Sin la clave no se recupera el texto (salvo error de implementación).
- **Hashing** (MD5, SHA) es de **una sola vía**: no se "descifra". Se usa para comparar o verificar.

Para el análisis, la pregunta no es "¿cómo lo rompo?", sino "¿qué transformación es y con qué parámetros?".

XOR: el disfraz más común

El XOR byte a byte contra una clave fija es el modismo número uno para esconder strings en malware:

```
loop:
    mov   al, [esi]
    xor   al, 0x5A        ; clave de 1 byte
    mov   [edi], al
    inc   esi
    inc   edi
    loop  loop
```

Señales de XOR:

- Un bucle corto con `xor` contra una constante o contra bytes de otra zona.
- Strings que **no aparecen** con `strings` / `Shift+F12` pero que el programa "tiene" (están XOR-eadas y se desXOR-ean en runtime).

- Datos en `.data` que parecen basura hasta que corre una rutina.

El XOR de 1 byte es trivial de deshacer (hay 255 claves posibles). Para análisis, muchas veces alcanza con un breakpoint **después** del bucle para leer el texto ya desXOR-eado en memoria, sin tocar el algoritmo.

Base64: la tabla que lo delata

Base64 se reconoce por su **alfabeto** de 64 caracteres, que casi siempre aparece como un string en `.rdata`:

```
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/
```

Más el `=` de relleno. Si ves ese alfabeto (o una variante con `-_` en vez de `+/`, que es base64 "url-safe"), hay base64. Procesa de a **4 caracteres** → **3 bytes**. Un alfabeto reordenado es base64 "custom": misma mecánica, otra tabla.

Algoritmos conocidos: las constantes los delatan

Casi todos los algoritmos famosos tienen **constantes o tablas fijas**. Encontrarlas es identificar el algoritmo sin leer una sola instrucción de la rutina:

Algoritmo	Qué buscar
AES (Rijndael)	La S-box, que empieza `63 7C 77 7B F2 6B 6F C5 30 01 67 2B …`; o las T-tables
MD5	Init `67452301 EFCDAB89 98BADCFE 10325476`; tabla de senos `D76AA478 …`
SHA-1	Init `67452301 EFCDAB89 98BADCFE 10325476 C3D2E1F0`
SHA-256	Init `6A09E667 BB67AE85 3C6EF372 …`; constantes `428A2F98 71374491 …`
RC4	Un arreglo de **256 bytes** inicializado `0,1,2,…,255` y un bucle de *swaps* (KSA/PRGA)
TEA / XTEA	La constante *delta* `0x9E3779B9` (número áureo)
CRC32	El polinomio `0xEDB88320` o una tabla de 256 entradas de 32 bits
DES	Tablas de permutación fijas (IP, S-boxes)

Un bucle que recorre un arreglo de 256 bytes haciendo intercambios casi seguro es **RC4**. Cuatro valores `67452301…` juntos casi seguro son **MD5 o SHA-1**. No hace falta entender el algoritmo: alcanza con nombrarlo y buscar su especificación.

Herramientas que automatizan el reconocimiento

- **FindCrypt / findcrypt-yara** (plugin de IDA/Ghidra): busca estas constantes y te dice qué algoritmo hay.
- **signsrch**: firmas de cientos de algoritmos.
- **CAPA** (Mandiant): identifica capacidades, incluida criptografía, por patrones.
- **YARA** con reglas de cripto: para clasificar muestras.

- **Detect It Easy / entropía**: una sección con entropía muy alta (cercana a 8 bits/byte) suele estar **cifrada o comprimida/empaquetada**; es una señal, no una prueba.

> **En Windows, muchas veces no hay que reconocer nada**
>
> Antes de pelear con una rutina de cripto, mirá los **imports**: si el programa usa la criptografía del sistema (CryptoAPI en `advapi32.dll` — `CryptEncrypt`, `CryptHashData` — o CNG en `bcrypt.dll` — `BCryptEncrypt` —), el algoritmo y la clave pasan por esas APIs. Un breakpoint en esas funciones te muestra los datos en claro sin reimplementar nada. El trabajo de reconocer constantes queda para cuando el programa trae su **propia** implementación embebida.

Un método de reconocimiento

1. Corré `strings` y FindCrypt/CAPA primero: muchas veces resuelven la identificación en segundos.
2. Si hay strings que "faltan", buscá un bucle con `xor` y poné un breakpoint después para leerlas en claro.
3. Si hay un alfabeto de 64 caracteres, es base64; mirá si es estándar o custom.
4. Si hay constantes de la tabla de arriba, nombrá el algoritmo y buscá su especificación.
5. Documentá **qué** algoritmo es y **con qué parámetros** (clave embebida, IV, tabla custom). No necesitás reimplementarlo para el informe.

> **Marco de uso**
>
> Identificar criptografía para entender un programa, detectar malware o resolver un CTF es análisis legítimo. Usar esto para romper el cifrado de software o datos ajenos, o para producir malware, no entra en esta guía. El objetivo es **reconocer y documentar**, no vulnerar.

> **Práctica**
>
> 1. Escribí un programa propio que guarde una string XOR-eada con una clave de 1 byte y la desXOR-ee al arrancar. Analizalo: encontrá el bucle `xor` y leé la string en memoria después.
> 2. Meté el alfabeto base64 en un programa y comprobá que aparece como string en `.rdata`.
> 3. Bajá una implementación de referencia de MD5 (de código abierto), compilala y buscá las constantes `67452301…` en el binario con tu desensamblador.
> 4. Corré FindCrypt o CAPA sobre esos binarios propios y compará con lo que encontraste a mano.

Preguntas de repaso

<details>
<summary>¿Cuál es la diferencia entre encoding y cifrado?</summary>

El encoding (base64, hex) no tiene clave y cualquiera lo revierte; el cifrado usa una clave y sin ella no se recupera el texto. El hashing, además, es de una sola vía.
</details>

▼ **¿Cómo reconocés base64 en un binario?**

> Por su alfabeto de 64 caracteres (`A–Z a–z 0–9 + /`) en `.rdata` y el `=` de relleno; procesa de a 4 caracteres → 3 bytes.

▼ **Encontrás un arreglo de 256 bytes inicializado `0,1,2,…` y un bucle de intercambios. ¿Qué algoritmo es probable?**

> RC4 (su fase de inicialización, KSA).

▼ **Ves las constantes `67452301 EFCDAB89 98BADCFE 10325476`. ¿Qué sugieren?**

> MD5 o SHA-1 (comparten ese estado inicial). Hay que mirar el resto para distinguirlos.

Antes de avanzar

Tenés que poder:

- Distinguir encoding, cifrado y hashing.
- Reconocer un XOR y un base64 en un listado.
- Identificar un algoritmo conocido por sus constantes/tablas.
- Usar FindCrypt/CAPA y la entropía como ayuda.

> **Puerta de salida**
>
> Si podés mirar un binario, decir "acá hay base64 y acá un XOR con clave de 1 byte, y esta tabla es la S-box de AES" y documentarlo, cumpliste el objetivo.

Fuentes de esta lección

- *Practical Malware Analysis* (Sikorski & Honig), capítulo de criptografía: XOR, base64 y reconocimiento de algoritmos.
- Proyectos **FindCrypt**, **signsrch** y **CAPA** (Mandiant) como referencia de firmas.
- Especificaciones públicas: FIPS 197 (AES), RFC 1321 (MD5), FIPS 180-4 (SHA), RFC 4648 (base64).

> **Capas de esta lección**
>
> **Conocimiento externo (agregado por esta edición):** todo el contenido (XOR, base64, constantes de algoritmos, CryptoAPI/CNG, herramientas). No proviene de CLS. **Recomendación editorial:** el método de reconocimiento y el encuadre defensivo.

Ampliación editorial basada en fuentes externas — Criptografía asimétrica (RSA/ECC) y tabla de intercepción en APIs nativas de Windows (CryptoAPI y CNG)

El software moderno rara vez implementa algoritmos criptográficos desde cero; en su lugar, utiliza esquemas híbridos apoyados en las bibliotecas criptográficas nativas de Windows (Microsoft Learn: Cryptography Next Generation y CryptoAPI):

1. Esquemas Criptográficos Híbridos en la Práctica

La carga útil o los datos de configuración se cifran con un algoritmo simétrico de alta velocidad (AES-CBC, AES-GCM o ChaCha20) utilizando una clave de sesión generada al vuelo. A continuación, esa clave de sesión se cifra con una **clave pública asimétrica** (RSA o Curvas Elípticas ECC) embebida en el binario. Solo quien posee la clave privada correspondiente puede descifrar la clave de sesión y acceder a los datos.

2. Reconocimiento de Claves Asimétricas en Binarios

- **RSA:** Se detecta por la presencia del exponente público estándar `e = 65537` (`0x00010001`, codificado en little-endian como `01 00 01 00` en memoria) y módulos grandes `N` de 256 bytes (2048 bits) o 512 bytes (4096 bits) en `.rdata`.
- **Estructura `RSAPUBKEY` (CryptoAPI clásica):**

```
struct RSAPUBKEY {
    DWORD magic;  // 0x31415352 = ASCII "RSA1" (clave pública) / 0x32415352 = "RSA2"
(privada)
    DWORD bitlen; // Tamaño en bits (ej. 0x00000800 = 2048 bits)
    DWORD pubexp; // Exponente público (ej. 0x00010001 = 65537)
};
```

- **Curvas Elípticas (ECC / ECDSA / ECDH):** Se identifican por constantes de curvas estándares como `secp256r1` (NIST P-256) o `Curve25519` (constante de campo primo `2^255 - 19`).

3. Tabla de APIs Criptográficas Nativas y Puntos de Intercepción en Depuración

Operación	CryptoAPI Clásica (advapi32.dll)	CNG Moderna (bcrypt.dll)	Estrategia de Intercepción en Depurador
Inicialización	`CryptAcquireContextW`	`BCryptOpenAlgorithmProvider`	Identifica el algoritmo pasando cadenas como `BCRYPT_AES_ALGORITHM` o `BCRYPT_RSA_ALGORITHM`.
Importación de Clave	`CryptImportKey`	`BCryptImportKeyPair` / `BCryptGenerateSymmetricKey`	Poner breakpoint para volcar el buffer de la clave en memoria (estructura `BLOBHEADER` o clave simétrica cruda).
Cifrado de Datos	`CryptEncrypt`	`BCryptEncrypt`	**Punto crítico de intercepción:** Poner breakpoint en la entrada. El argumento `pbData` (CryptoAPI) o `pbInput` (`RDX` en x64 para CNG) contiene el **texto plano antes de ser cifrado**.
Descifrado de Datos	`CryptDecrypt`	`BCryptDecrypt`	Poner breakpoint en el **retorno de la función**. El buffer de salida contendrá el texto descifrado en memoria.

Ampliación editorial basada en fuentes externas — La trampa del Endianness en CNG vs. CryptoAPI y script de conversión en Python

Una de las discrepancias arquitectónicas más frecuentes al extraer claves criptográficas de memoria radica en la convención de orden de bytes (Microsoft Learn: CNG Key Storage):

Característica	CryptoAPI Clásica (advapi32.dll)	CNG Moderna (bcrypt.dll / ncrypt.dll)
Estructura del encabezado	`PUBLICKEYSTRUC` (`BLOBHEADER`) + `RSAPUBKEY`	`BCRYPT_RSAKEY_BLOB` / `BCRYPT_ECCKEY_BLOB`
Magic Bytes de clave pública RSA	`0x31415352` (ASCII `"RSA1"`)	`0x31415352` (`BCRYPT_RSAPUBLIC_MAGIC` = `"RSA1"`)
Magic Bytes de clave privada RSA	`0x32415352` (ASCII `"RSA2"`)	`0x32415352` (`BCRYPT_RSAPRIVATE_MAGIC` = `"RSA2"`)
Magic Bytes de Curvas Elípticas (P-256)	No soportado de forma estándar	`0x314B4345` (`"ECK1"` para ECDH) / `0x31534345` (`"ECS1"` para ECDSA)
Orden de Bytes (Endianness) del Módulo y Claves	**Little-Endian** (el byte menos significativo va primero)	**Big-Endian** (formato estándar de red / RFC)

Reconocimiento Visual de Claves ASN.1 DER (X.509)

Cuando un binario no utiliza APIs nativas de Windows y compila su propia librería (como mbedTLS u OpenSSL), las claves públicas suelen embeberse en formato ASN.1 DER binario. Se reconocen al instante por la secuencia invariable que codifica el OID de `rsaEncryption` (`1.2.840.113549.1.1.1`):

```
30 82 .. .. 30 0D 06 09 2A 86 48 86 F7 0D 01 01 01 05 00 03 82 .. .. 00 30 82 ..
```

Script de Extracción y Conversión a PEM (Python / PyCryptodome)

Para convertir un volcado hexadecimal obtenido en x64dbg de una estructura `RSAPUBKEY` (CryptoAPI) a una clave pública `.pem` estándar utilizable en scripts defensivos, se debe invertir el orden de los bytes del módulo:

```
# Reconstrucción de clave pública RSA desde volcado de CryptoAPI (Little-Endian)
from Crypto.PublicKey import RSA

def parse_cryptoapi_rsapubkey(data: bytes) -> RSA.RsaKey:
    # data: buffer que comienza en RSAPUBKEY (después del BLOBHEADER de 8 bytes)
    magic = data[0:4]          # b'RSA1' (0x31415352)
    bitlen = int.from_bytes(data[4:8], 'little')
    pubexp = int.from_bytes(data[8:12], 'little')
    modlen = bitlen // 8
    # En CryptoAPI el módulo viene en little-endian; se invierte a big-endian:
    modulus = int.from_bytes(data[12:12+modlen][::-1], 'big')
    return RSA.construct((modulus, pubexp))

# Exportar a PEM: print(key.export_key().decode())
```

Ampliación editorial basada en fuentes externas — Reconocimiento de ChaCha20 / Salsa20 (RFC 8439) por sus constantes y operaciones ARX

En binarios modernos de Windows, el cifrado de flujo **ChaCha20** (IETF RFC 8439) es sumamente frecuente y no utiliza tablas S-box *(Hecho confirmado)*:

- **Constante mágica de 16 bytes (clave de 256 bits):** La cadena ASCII `"expand 32-byte k"` o los cuatro `DWORD` little-endian `0x61707865`, `0x3320646e`, `0x79622d32`, `0x6b206574` en las palabras `0..3` de un estado de 16 palabras (64 bytes). (Para clave de 128 bits: `"expand 16-byte k"`).
- **Firma en el código (Quarter-Round ARX):** Un bucle de 20 rondas (10 iteraciones dobles) compuesto exclusivamente por sumas de 32 bits (`add`), `xor` y rotaciones (`rol`) con conteos de **16, 12, 8 y 7 bits** (o 7, 9, 13 y 18 en Salsa20).

El informe reproducible

> **Objetivo de la lección**
>
> Al terminar vas a poder escribir un informe de análisis con entorno, hashes, método, evidencia, conclusión y límites, y revisar el informe de otra persona con una lista de verificación.

Prerrequisitos: método de análisis. **Duración sugerida:** 3 horas. Estimación aproximada.

Para qué sirve un informe

Un análisis sin registro es una anécdota. Sirve para vos mientras lo recordás; no sirve para un equipo, no se puede auditar y no se puede repetir. El informe convierte una sesión de debugger en **conocimiento verificable**.

Hay tres destinatarios posibles:

1. **Vos mismo, en dos semanas.** ¿Qué binario era? ¿Con qué versión? ¿Qué probaste?
2. **Otra persona del equipo.** ¿Puede repetir el experimento sin preguntarte nada?
3. **Un tercero que necesita decidir.** ¿Es seguro? ¿Hay un riesgo? ¿Qué mitigación recomendarías?

El nivel de detalle cambia según el destinatario, pero la estructura es la misma.

Estructura mínima

```
1. Identificación
   - Archivo, tamaño, SHA-256, origen, fecha de recepción
   - Entorno: SO, VM, herramientas y versiones, snapshot
2. Alcance y pregunta
   - Qué se analizó y qué quedó fuera
   - Pregunta concreta que guía el trabajo
3. Método
   - Estático: herramienta, pasos, direcciones
   - Dinámico: breakpoints, valores, condiciones
4. Observaciones
   - Tabla de evidencia: qué se hizo, qué se observó, dónde está la prueba
5. Conclusión
   - Respuesta directa, con grado de certeza
6. Límites
   - Qué no se probó, qué supuestos quedan, qué podría cambiar el resultado
7. Recomendaciones
   - Defensivas: detección, mitigación, monitoreo (si corresponde)
8. Anexos
   - Logs, capturas, scripts, hashes de artefactos
```

La plantilla del proyecto está en `plantilla_informe.md`. Esta lección la amplía con criterios de evidencia.

Ejemplo: informe corto de `check.exe`

Este es un informe de ejemplo sobre el programa de práctica de la lección anterior. Los valores son los que se observaron en esa práctica; las direcciones son ilustrativas y pueden variar en tu compilación.

```
INFORME DE ANÁLISIS — check.exe
Fecha: 2026-10-26
Analista: (tu nombre)

1. IDENTIFICACIÓN
Archivo: check.exe (52 KB)
SHA-256: <hash calculado con Get-FileHash>
Origen: compilado por el analista a partir de check.c (gcc -O0 -g)
Entorno: Windows 11 en VM, snapshot 00-limpia, x64dbg 2025.x, IDA Free 8.x
Alcance: determinar la condición de aceptación de dos números enteros.
Fuera de alcance: comportamiento fuera de la VM, otras compilaciones.

2. MÉTODO
Estático: búsqueda de la cadena "Correcto" (Shift+F12), xref (X),
lectura del grafo, renombrado de variables.
Dinámico: breakpoint en la comparación final, ejecución con
(40,80), (40,70) y (60,80).

3. OBSERVACIONES
| Entrada | EAX en la comparación | ZF | Salida |
| 40, 80  | 0x1E0                 | 1  | Correcto |
| 40, 70  | 0x1B8                 | 0  | Incorrecto |
| 60, 80  | no llega              | —  | Incorrecto (falla a>50) |

4. CONCLUSIÓN
El programa acepta pares (a,b) con a <= 50, b >= 50 y (a+b)*4 == 480.
Certeza: alta para esta compilación; no se probó exhaustivamente el espacio.

5. LÍMITES
No se analizaron otras compilaciones ni variantes del código.
No se evaluó el comportamiento con entradas no numéricas.
```

Ese informe es corto, pero responde: qué, cómo, con qué, qué se vio y qué falta.

Qué hace que una evidencia sea buena

Buena evidencia	Mala evidencia
Valor exacto, dirección, timestamp	"Se ve que compara"
Captura con la ventana y el valor visibles	Captura recortada sin contexto
Comando reproducible	"Lo hice con una herramienta"
Hash del binario	"El archivo que me pasaron"
Diferencia entre observado y supuesto	Mezcla de ambos sin aclarar

Una captura vale si se entiende **qué** muestra y **dónde** está el dato. Si recortás la ventana y no se ve la dirección ni el registro, no sirve como evidencia.

Trazabilidad: el hash y la cadena de custodia

El SHA-256 del binario es la forma de decir "este análisis corresponde exactamente a este archivo". Si el archivo cambia, el hash cambia.

Buenas prácticas:

1. Calculá el hash **al recibir** el archivo y anotalo.
2. Guardá una copia de trabajo con permisos de lectura.
3. No modifiques el original; si parcheás, trabajá sobre una copia y anotá el hash de la copia.
4. Registrá quién tuvo acceso y cuándo, si el contexto lo requiere.

Para malware, la cadena de custodia es más estricta: el archivo no sale del laboratorio, no se comparte por correo ni se sube a servicios públicos sin evaluar el riesgo.

Cómo se redacta la conclusión

Una conclusión no es un resumen del método. Es una **respuesta**. Tiene que poder leerse sola.

Mal	Bien
"Se analizó el binario y se encontraron comparaciones."	"El programa acepta `a <= 50`, `b >= 50` y `(a+b)*4 == 480`."
"Parece que hay un chequeo."	"El chequeo está en la comparación de `EAX` con `0x1E0`; con (40,80) `ZF=1`."
"Es seguro."	"No se observó actividad de red ni escritura fuera de la carpeta de trabajo en la VM."

La segunda columna es más útil porque **afirma algo que se puede contradecir con evidencia**.

Divulgación responsable

Si en un análisis autorizado encontrás una vulnerabilidad:

1. **No publiques el exploit** ni los detalles técnicos antes de que el proveedor pueda responder.
2. Documentá el impacto, la versión afectada y los pasos mínimos para reproducirlo.
3. Contactá al proveedor por un canal privado; guardá la evidencia.
4. Acordá un plazo razonable (por ejemplo, 90 días) antes de publicar.
5. No accedas a datos de terceros, no escales privilegios y no dejes puertas abiertas.

Esta guía no enseña a explotar sistemas ajenos. Los ejercicios de vulnerabilidades se hacen en programas propios o laboratorios diseñados para eso.

Práctica

1. Escribí el informe completo de tu `check.exe` con la estructura de arriba.
2. Incluí una tabla de evidencia con al menos tres filas.
3. Agregá una sección de límites con dos supuestos que no verificaste.
4. Intercambiá el informe con un compañero (o releelo al día siguiente) y aplicá esta lista:

Punto	Sí/No
¿Está el hash del binario?	
¿Está el entorno y la versión de las herramientas?	
¿La pregunta es específica?	
¿Cada conclusión tiene evidencia asociada?	
¿Se distingue observado de supuesto?	
¿Se puede repetir el experimento con lo escrito?	
¿Los límites son honestos?	

Preguntas de repaso

1. ¿Por qué el hash es parte del informe? Porque identifica sin ambigüedad el archivo analizado. Si el hash cambia, el informe ya no aplica a ese archivo.

2. ¿Qué diferencia hay entre una observación y una conclusión? La observación es lo que viste ("EAX=0x1E0"); la conclusión es lo que inferís ("la condición es (a+b)*4=480").

3. ¿Para qué sirve la sección de límites? Para que el lector sepa qué no se probó y no sobreestime el alcance del análisis.

4. ¿Qué hace reproducible un informe? Que incluya el binario identificado por hash, el entorno, los pasos y los valores observados, de modo que otra persona pueda repetirlos.

5. ¿Qué se hace si se encuentra una vulnerabilidad? Documentarla, reportarla en privado al proveedor, no publicar el exploit y acordar un plazo de divulgación.

Antes de avanzar

Tenés que poder:

- Escribir un informe con las ocho secciones.
- Distinguir evidencia de opinión.
- Calcular y registrar el hash del binario.
- Explicar la diferencia entre divulgar y publicar un exploit.

Puerta de salida

Si tu informe puede repetirlo otra persona sin preguntarte nada, está listo. Si no, falta evidencia o falta método.

Fuentes de esta lección

- `plantilla_informe.md` — plantilla base del proyecto.
- `Inventario limpio, LEEME` — uso de SHA-256 y criterio de no ejecución.
- `Reversing desde cero, cap. 11` — método de análisis y documentación visual.
- `Reversing desde cero, cap. 15` — valores y confirmación que alimentan el informe de ejemplo.

Capas de esta lección

Contenido observado en CLS: el método de los capítulos 11 y 15 y el uso de hashes del inventario. **Conocimiento externo:** estructura de informes técnicos y divulgación responsable. **Recomendación editorial:** la plantilla ampliada, la tabla de evidencia y el informe de ejemplo.

Análisis dinámico seguro: cuándo ejecutar y cuándo no

> **Objetivo de la lección**
>
> Al terminar vas a poder decidir si un programa se analiza solo en estático o se ejecuta en un laboratorio, preparar la observación con herramientas de monitoreo y actuar si el comportamiento se sale de lo previsto.

Prerrequisitos: laboratorio (Etapa 0) y método (Etapa 4). **Duración sugerida:** 3 horas. Estimación aproximada.

La decisión más importante

No todo se ejecuta. Antes de abrir un programa en el debugger, hacete tres preguntas:

1. **¿Tengo permiso?** ¿Es mío, es de código abierto, es un crackme de práctica, o tengo autorización escrita?
2. **¿Puedo contenerlo?** ¿Está en una VM aislada, con snapshot, sin carpetas compartidas y sin red personal?
3. **¿Necesito ejecutarlo?** ¿La pregunta se responde con análisis estático, o el comportamiento dinámico es imprescindible?

Si alguna respuesta es "no", el programa no se ejecuta. Se analiza estático, se documenta y se deriva a quien corresponda.

Situación	Qué hacer
Programa propio o de práctica	Ejecutar en VM, con snapshot
Software abierto que querés entender	Ejecutar en VM, con la licencia respetada
Binario de origen desconocido	Estático primero; dinámico solo en laboratorio dedicado
Malware confirmado	Estático y detección; dinámico solo con protocolo defensivo
Software comercial con protección	No eludir protecciones; analizar solo lo permitido
Archivo en cuarentena o excluido	No ejecutar; respetar la clasificación

Antes de ejecutar: línea base

Una observación sin línea base no sirve. Antes de correr el programa, registrá el estado del sistema:

- Procesos en ejecución.
- Conexiones de red.
- Archivos en la carpeta de trabajo.
- Claves de registro relevantes (si vas a observar persistencia).
- Hora de inicio.

Herramientas: **Process Explorer** (procesos, handles), **Process Monitor** (archivos, registro, procesos), **TCPView** (conexiones), **Autoruns** (persistencia). Todas son de Sysinternals o equivalentes gratuitos.

En la VM:

```
1. Restaurar snapshot 00-limpia.
2. Anotar hora y estado.
3. Iniciar Process Monitor con filtro por nombre del proceso.
4. Iniciar TCPView.
5. Ejecutar el programa.
6. Observar, anotar y detenerse.
7. Cerrar el programa, detener las capturas y restaurar snapshot.
```

Qué observar

Categoría	Qué mirar	Herramienta
Archivos	Creación, lectura, escritura, borrado	Process Monitor
Registro	Claves creadas/modificadas	Process Monitor
Procesos	Hijos, inyecciones, elevación	Process Explorer
Red	Conexiones, DNS, puertos	TCPView, Wireshark
Memoria	Regiones RWX, módulos cargados	x64dbg, Process Explorer
Persistencia	Run keys, servicios, tareas	Autoruns

No hace falta observar todo a la vez. Elegí dos o tres categorías según la pregunta.

El caso Shelter: lo que CLS muestra y lo que no se copia

El video 37 analiza el malware Shelter dentro de una VM Windows 7 de 32 bits, con otra VM Kali Linux para simular el ataque. El propio autor distingue análisis estático y dinámico y dice que en su caso va a ejecutar la muestra (`Videos y material adicional, Video 37`, 00:24:01–00:24:21).

Ese video es un **caso observado**, no un modelo para esta guía. Ejecuta malware, monta un reverse shell y usa una segunda VM para el ataque. Para un principiante, el orden correcto es:

1. Triage estático: tipo de archivo, secciones, imports, strings, hashes.
2. Detección: reglas YARA, firmas, comparación con VirusTotal **sin subir muestras confidenciales**.
3. Documentación de indicadores.
4. Solo si hay un propósito defensivo claro y un laboratorio dedicado, análisis dinámico controlado.

El video 36 inyecta una puerta trasera de Metasploit en una calculadora y usa una VM "por seguridad" (`Videos y material adicional, Video 36`, 00:29:21–00:29:42). Tampoco es una práctica de esta guía.

> **Lo que no se hace**
>
> No se ejecuta malware en el host. No se conecta la VM de análisis a la red doméstica. No se comparten carpetas. No se usan cuentas personales. No se sube una muestra a un servicio público sin evaluar si es confidencial. No se copia el malware fuera del laboratorio.

Anti-debug: una señal, no un obstacle a vencer

Si un programa detecta el debugger y se cierra, eso **es información**. Puede indicar:

- Que el programa tiene protecciones anti-análisis.
- Que está empaquetado.
- Que su comportamiento cambia según el entorno.

Los chequeos más comunes están en el capítulo 22 (PEB, `IsDebuggerPresent`, `NtGlobalFlag`) y en el capítulo 24 (lista de procesos, ParentPID, `CloseHandle` inválido). El capítulo 24 usa un crackme propio que muestra qué técnica lo detectó (`Reversing desde cero, cap. 24`, 00:01:17–00:03:31).

Para esta guía:

- **Estudiá** los mecanismos para reconocerlos y explicar por qué un binario se comporta distinto.
- **No** aprendas a evadir las protecciones de software comercial.
- **No** uses técnicas de ocultamiento para analizar muestras fuera de un laboratorio autorizado.
- Si el objetivo es defender, la pregunta es "¿cómo detecto este comportamiento?", no "¿cómo lo esquivo?".

Si algo sale mal

Un plan de contención tiene que existir antes de ejecutar:

1. **Detener la VM** (no suspenderla): apagado forzado si hace falta.
2. **Desconectar la red** de la VM desde el hipervisor.
3. **Restaurar la snapshot** limpia.
4. **Revisar el host**: procesos, conexiones, archivos compartidos, cuentas.
5. **Anotar** qué pasó, a qué hora y qué se observó.
6. **Reportar** si hubo exposición de datos o credenciales.
7. **Cambiar** las credenciales que pudieron quedar expuestas.

Si el programa logró escribir en una carpeta compartida o conectar a la red local, el aislamiento falló. Eso se documenta y se corrige antes de la próxima práctica.

Práctica

1. Ejecutá tu `check.exe` en la VM con Process Monitor filtrando por el nombre del proceso. Anotá qué archivos abre y qué claves de registro lee.

2. Ejecutá una versión que abra un archivo (`fopen`) y observá la operación de escritura.
3. Hacé una lista de línea base antes y después. ¿Qué cambió?
4. Elegí un archivo del proyecto marcado `cuarentena` y hacé **solo** triage estático: tipo, secciones, imports, strings, hash. No lo ejecutes.
5. Escribí un procedimiento de 10 líneas para "qué hago si la VM se comporta raro".

Preguntas de repaso

1. ¿Cuándo NO se ejecuta un programa? Cuando no tenés permiso, no podés contenerlo o no necesitás el comportamiento dinámico. También cuando está en cuarentena o excluido.

2. ¿Por qué se toma una línea base antes de ejecutar? Porque sin el estado inicial no podés atribuir los cambios al programa. La comparación antes/después es la evidencia.

3. ¿Qué indica que un programa detecte el debugger? Que tiene protecciones anti-análisis o que su comportamiento depende del entorno. Es un dato para documentar, no un obstáculo para "saltar".

4. ¿Qué hace primero si una muestra resulta sospechosa? Triage estático, cálculo de hash, documentación de indicadores y, si corresponde, derivación. No se ejecuta en el host.

5. ¿Qué hace si la VM se conecta a la red local? Se desconecta desde el hipervisor, se restaura la snapshot, se revisa el host y se documenta el fallo de aislamiento.

Antes de avanzar

Tenés que poder:

- Decidir si un programa se ejecuta o no, con justificación.
- Tomar una línea base y observar con Process Monitor/TCPView.
- Explicar por qué el video 37 no es un modelo de práctica.
- Describir un plan de contención.

> **Puerta de salida de la Etapa 4**
>
> Si podés analizar, documentar y contener una práctica completa, terminaste el recorrido base. Las ramas de la Etapa 5 son opcionales y exigen prerrequisitos.

Fuentes de esta lección

- `Videos y material adicional, Video 37` — caso Shelter: VM, Kali y ejecución de malware; se cita como caso observado.
- `Videos y material adicional, Video 36` — XOR, crypter y backdoor en VM.
- `Reversing desde cero, cap. 22` — PEB y detección de debugger.
- `Reversing desde cero, cap. 24` — cuatro chequeos anti-debug.
- `Inventario limpio, LEEME` — estados `cuarentena` y criterio de no ejecución.

Capas de esta lección

Contenido observado en CLS: el laboratorio con dos VMs del video 37, la inyección del video 36 y los chequeos de los capítulos 22 y 24. **Conocimiento externo:** observación con Process Monitor/TCPView y planes de contención. **Recomendación editorial:** la matriz de decisión y el protocolo de incidentes.

Reconstruir formatos y protocolos propios

Objetivo de la lección

Dado un conjunto pequeño de archivos o mensajes generados por un programa propio, formular y comprobar una estructura: campos, offsets, longitudes, endianness, framing y estados. Distinguir checksum, hash, MAC, compresión y cifrado como hipótesis diferentes, y anotar qué evidencia sostiene cada conclusión.

Prerrequisitos: Bits y endianness (1.1), formato PE (1.4), método de análisis (4.1), IPC de Windows como referencia. **Duración sugerida:** 6 horas.

Alcance y muestras

Usá archivos y programas **propios** o diseñados para la práctica. Para un protocolo, trabajá con dos procesos tuyos en una VM, una named pipe local o loopback. No envíes datos a servicios de terceros. **Capas:** conocimiento externo; ejemplo CLSF ilustrativo y sintético; laboratorio no verificado. La práctica propuesta usa programas propios y comunicación local.

Del síntoma a una pregunta comprobable

«¿Qué significa este blob?» es demasiado amplio. Convertí la duda en una pregunta que permita diseñar una prueba:

- ¿Los primeros cuatro bytes son una firma constante?
- ¿Este entero describe la longitud del cuerpo, del encabezado o del archivo completo?
- ¿Dos mensajes consecutivos se separan por longitud, delimitador o estado de la conexión?
- ¿Qué campo cambia cuando cambia solo el contenido?
- ¿El bloque final detecta corrupción accidental, autentica el mensaje o solo parece aleatorio?

Conservá cada muestra original sin modificarla, calculá su hash y anotá cómo se obtuvo. Trabajá sobre copias. Una tabla de diferencias entre muestras vale más que una interpretación basada en un solo archivo.

Procedimiento de reconstrucción

1. **Inventario.** Anotá tamaño, hash, origen de laboratorio, herramienta que lo produjo y si el espécimen es archivo, mensaje, flujo o captura. Identificá la envoltura conocida y separala del formato interno.
2. **Muestras controladas.** Generá pares que difieran en una sola propiedad: una cadena más larga, otro valor, un registro adicional o una versión distinta. Incluí casos válidos vacíos y límites que tu programa admita.
3. **Mapa de bytes.** Numera offsets desde cero; registra bytes, ASCII/UTF-16, enteros little-endian/big-endian, alineación y regiones repetidas. No decidas el tipo de un campo por su apariencia aislada.

4. **Hipótesis.** Para cada campo escribe hipótesis, evidencia, alternativa y prueba que la refutaría. «Parece una longitud» no es conclusión hasta comprobar varios tamaños.
5. **Framing y estructura.** Separa cabecera, registros, cuerpo y tráiler. Prueba longitudes prefijadas, tamaños fijos, delimitadores, terminadores, conteos y esquemas TLV. Comprueba si las longitudes incluyen o excluyen cabecera/tráiler.
6. **Estado y secuencia.** Dibuja estados y transiciones: conexión, saludo, petición, respuesta, error y cierre. Relaciona cada mensaje con el estado previo; un opcode puede significar cosas distintas según el estado.
7. **Transformaciones.** Investiga por separado codificación, compresión, checksum/CRC, hash, MAC y cifrado. Usa perturbaciones controladas y evidencia del programa propio; entropía alta por sí sola no demuestra cifrado ni compresión.
8. **Validación independiente.** Comprueba la hipótesis con muestras no usadas para formularla, límites y entradas malformadas inocuas. Escribe un esquema y un parser de referencia mínimo.

Endianness, longitudes y TLV

Un entero de dos bytes 05 00 vale 5 en little-endian; 00 05 vale 5 en big-endian. En tráfico IP, los campos numéricos de Winsock usan orden de red (big-endian), aunque el host x86/x64 use little-endian. Un formato puede mezclar órdenes: documenta el orden **por campo**.

En un esquema **TLV** (type, length, value), cada elemento indica tipo, longitud y valor. Confirma el ancho de tipo/longitud, si la longitud incluye el encabezado y cómo se representa un elemento desconocido.

Ejemplo trabajado: formato sintético CLSF

*(Ejemplo **ilustrativo**, generado para la lección; no proviene de un archivo CLS.)* Supongamos un programa propio que escribe estos bytes:

```
Offset  Bytes                                   Lectura provisional
00      43 4C 53 46                              firma ASCII «CLSF»
04      01                                       versión candidata = 1
05      00                                       flags candidatos = 0
06      0C 00                                    tamaño de cabecera candidato = 12 (LE)
08      05 00                                    tamaño de cuerpo candidato = 5 (LE)
0A      2A 00                                    campo variable = 42 (LE)
0C      48 45 4C 4C 4F                           cuerpo ASCII «HELLO»
```

La muestra completa tiene 17 bytes. Es tentador llamar «secuencia» al campo en offset 0A, pero una sola muestra no lo justifica. Generamos variantes cambiando una cosa cada vez:

Muestra	Cambios controlados	Observación
A	Cuerpo HELLO, campo 2A 00	Longitud 5; total 17 bytes
B	Mismo cuerpo, campo 2B 00	Solo cambia el campo en 0A; podría ser contador, ID o checksum
C	Cuerpo BYE!!, campo 2A 00	Cambia solo el cuerpo; el campo 0A permanece
D	Cuerpo HELLO!!!, campo 2A 00	Cuerpo crece tres bytes y el campo 08 pasa de 5 a 8

A y D apoyan la hipótesis «offset 08 es longitud del cuerpo» y que empieza en 0C. A/B/C sugieren que 0A no es un checksum simple del cuerpo, pero no identifican su significado. Solo al observar varias operaciones del programa propio —por ejemplo, que incrementa entre mensajes— podríamos etiquetarlo como contador. Cada etiqueta queda como hipótesis hasta probarla.

Framing y máquina de estados

En un archivo, el inicio y final vienen dados por el archivo. En un protocolo hay que averiguar **dónde termina cada mensaje**:

- **Tamaño fijo:** cada mensaje tiene la misma longitud.
- **Longitud prefijada:** un encabezado anuncia cuántos bytes siguen; determina si cuenta desde el encabezado o solo el cuerpo.
- **Delimitador:** un byte o secuencia separa mensajes; prueba cómo se escapa si aparece en el cuerpo.
- **Estado o longitud implícita:** el final depende de una operación o fase del protocolo.

TCP es un flujo de bytes: los límites de una llamada send no son automáticamente límites de mensaje para quien recibe. El framing pertenece al protocolo de aplicación. Named pipes de Windows pueden operar en modo byte o mensaje; identifica el modo antes de interpretar lecturas, y documenta también la estructura interna definida por la aplicación.

```
DESCONECTADO --conectar--> ESPERA_SALUDO
ESPERA_SALUDO --saludo válido--> LISTO
LISTO --petición--> ESPERA_RESPUESTA
ESPERA_RESPUESTA --respuesta/error--> LISTO
cualquier estado --cierre--> DESCONECTADO
```

El diagrama es una hipótesis hasta contrastarla con trazas de una ejecución propia. Anota estado previo, bytes, respuesta y efecto observable para cada transición.

No confundir checksum, hash, MAC, compresión y cifrado

- **Checksum/CRC:** detecta ciertas corrupciones accidentales; no autentica al emisor.
- **Hash:** resumen de tamaño fijo; sin secreto no prueba quién creó el mensaje.
- **MAC:** etiqueta con clave compartida que permite verificar integridad/autenticidad bajo ese secreto.
- **Cifrado:** transforma datos usando una clave para ocultar contenido; no implica por sí mismo integridad.
- **Compresión:** transforma la representación para reducirla; firmas y cambios de tamaño dan pistas, pero entropía alta no prueba compresión ni cifrado.
- **Codificación:** cambia la representación (por ejemplo, base64 o UTF-16) sin ser cifrado.

Cambia un solo byte y compara el tráiler en muchas muestras. Después cambia una propiedad semántica y repite. Si no puedes validar una fórmula o identificar una llamada concreta en el programa propio, registra «campo no resuelto» en vez de adivinar CRC/AES.

Práctica

En una VM, con **un escritor/parser propio** y, si eliges la rama de protocolo, dos procesos tuyos:

1. Genera al menos cuatro muestras con una sola variable distinta por par. Conserva originales y hashes.
2. Completa una tabla de offsets, ancho, orden de bytes, valor observado, hipótesis y confianza.
3. Identifica el framing y dibuja una máquina de estados para un intercambio local. Si usas named pipe, registra su modo byte/mensaje.
4. Investiga un posible tráiler variando un byte y el contenido; distingue qué puedes concluir y qué no sobre checksum, hash, MAC, cifrado o compresión.
5. Escribe un esquema breve y un parser de referencia para muestras válidas. Añade límites y casos malformados inocuos.
6. Reserva dos muestras para validación: predice sus offsets, longitudes y transiciones antes de abrirlas.

Entregable

Un mini-informe con hashes, tabla de campos/offsets, diagrama de estados si es protocolo, esquema, pruebas, hipótesis refutadas y límites. Marca cada campo como **confirmado**, **probable** o **sin resolver**, con su evidencia.

Preguntas de repaso

▼ **¿Por qué un mensaje TCP no termina necesariamente donde terminó la llamada send?**

> TCP entrega un flujo de bytes; el protocolo de aplicación define el framing. Los segmentos y llamadas de envío no garantizan los límites que verá cada lectura.

▼ **¿Qué experimento ayuda a identificar un campo de longitud?**

> Cambiar solo el tamaño del cuerpo en varias muestras y comprobar si el campo cambia coherentemente y si cuenta encabezado o archivo completo.

▼ **¿Que un tráiler cambie al modificar el cuerpo demuestra que es un CRC?**

> No. También podría ser hash, MAC, cifrado u otro campo dependiente del contenido. Hay que probar la hipótesis con más muestras y evidencia.

▼ **¿Qué diferencia hay entre minimizar una muestra y confirmar un formato?**

> Minimizar reduce una entrada manteniendo un comportamiento o cobertura; confirmar exige que una hipótesis estructural prediga muestras independientes y casos límite.

Puerta de salida

Podés reconstruir un formato o protocolo propio desde muestras controladas: marcar campos y offsets, explicar endianness/longitudes/framing, dibujar estados cuando corresponda, separar transformaciones candidatas y validar el esquema con muestras reservadas. No hace falta adivinar todos los campos; sí declarar qué evidencia falta.

Fuentes

- Microsoft Learn: Byte Ordering, Named Pipes, Named Pipe Type, Read, and Wait Modes.
- IETF: RFC 9293, Transmission Control Protocol (TCP).
- Método de análisis, IPC en Windows y fuzzing/análisis simbólico.

Protocolos de terceros: reconstruir un protocolo propietario

Objetivo de la lección

Reconstruir un **protocolo de red propietario** combinando **capturas** de tráfico y el **reversing del cliente**: descubrir el *framing*, la serialización y la máquina de estados, y documentarlo con un **disector de Wireshark**. **Prerrequisitos:** reconstruir formatos y protocolos (formatos y protocolos), ofuscación (2.10), scripting/Frida (2.21) y simbólico (2.25).

Solo sistemas propios o autorizados

Interceptar o descifrar tráfico de servicios ajenos sin permiso es ilegal. Esta lección se practica con un **cliente/servidor propio** o en un entorno con **autorización explícita**. No se reconstruyen protocolos para eludir controles de acceso ni para atacar servicios de terceros.

De formatos a protocolos

En la lección de formatos reconstruiste un **formato de archivo**: bytes quietos en disco. Un **protocolo** agrega dos cosas: **tiempo** (los mensajes van y vienen en una secuencia) y **estado** (un *handshake*, una autenticación, un *keepalive*). Lo reconstruís cruzando **lo que ves en la red** con **lo que hace el binario** (**conocimiento externo**).

1 · Capturar y aislar

Con **Wireshark** aislás la conversación (por IP/puerto) y mirás la dirección de cada mensaje. Si el tráfico está **cifrado** (TLS), **no se rompe la cripto**: se obtienen las claves del **propio cliente** (variable `SSLKEYLOGFILE`, o un *hook* que vuelca el buffer antes de cifrar). Analizás el texto claro del lado que controlás, no el de un tercero.

2 · Framing: dónde empieza y termina un mensaje

TCP es un chorro de bytes; el protocolo define cómo cortarlo. Los tres patrones:

- **Prefijo de longitud:** los primeros 2–4 bytes dicen cuánto mide el resto (el más común).
- **Delimitador:** un byte/secuencia marca el fin (p. ej. `\n`).
- **Tamaño fijo:** cada mensaje mide lo mismo.

3 · Serialización: cómo se empaquetan los campos

- **TLV** (*type-length-value*): un byte de tipo, una longitud y el valor. Muy frecuente y fácil de reconocer.
- **Protobuf sin `.proto`:** se reconstruye el esquema con `protobuf-inspector` (los *field numbers* y *wire types* se infieren de los bytes).

- **MessagePack / JSON / binario a medida:** cada uno con su huella.

4 · Correlación tráfico ↔ binario (el paso clave)

El salto de calidad es **enganchar la capa de aplicación**, no la red: poné *breakpoints* o *hooks* en `send` / `recv` / `WSASend` / `WSARecv` y mirá el buffer **antes de cifrar** (en el `send`) y **después de descifrar** (en el `recv`). Así ves el mensaje en claro y, subiendo por la pila de llamadas, **qué función lo arma**: ahí está la estructura real del protocolo.

5 · Máquina de estados y documentar

Con varios mensajes ordenás la secuencia: *handshake* → auth → comandos → cierre. Para **validar** tu reconstrucción, escribís un **disector de Wireshark en Lua**: si colorea bien los campos de capturas nuevas, entendiste el protocolo.

Ejemplo paso a paso: un protocolo TLV propio

*(Ejemplo **ilustrativo**: cliente/servidor propio; hook y disector de referencia.)*

```
// volcar.js — ver el mensaje EN CLARO antes de que el cliente PROPIO lo mande
Interceptor.attach(Module.getExportByName('ws2_32.dll', 'send'), {
  onEnter(args) {
    const buf = args[1], len = args[2].toInt32();
    console.log(hexdump(buf, { length: len }));   // tipo | longitud | valor
  }
});
```

```
-- miproto.lua — disector de Wireshark para el TLV propio
local p = Proto("miproto", "Mi protocolo TLV")
local f_tipo = ProtoField.uint8 ("miproto.tipo", "Tipo")
local f_len  = ProtoField.uint16("miproto.len",  "Longitud")
local f_val  = ProtoField.bytes ("miproto.val",  "Valor")
p.fields = { f_tipo, f_len, f_val }

function p.dissector(buf, pinfo, tree)
  pinfo.cols.protocol = "MIPROTO"
  local t   = tree:add(p, buf())
  -- OJO endianness: un struct C nativo en x86/x64 escribe la longitud en LITTLE-endian (05 00 =
5).
  -- En Lua, :uint()/:add() leen BIG-endian (05 00 = 1280) -> usá :le_uint()/:add_le().
  -- Si el cliente usa htons() (network byte order = big-endian), entonces sí van :uint()/:add().
  local len = buf(1,2):le_uint()
  t:add(f_tipo, buf(0,1))
  t:add_le(f_len,  buf(1,2))
  t:add(f_val,  buf(3, len))
end
DissectorTable.get("tcp.port"):add(9000, p)   -- puerto de TU servidor
```

El *hook* te muestra el mensaje en claro (tipo, longitud, valor) **antes** de que salga a la red; el disector formaliza esa estructura y la valida contra capturas nuevas. Cuando Wireshark colorea «Tipo = 3, Longitud = 12, Valor = …» en tus capturas, documentaste el protocolo.

TCP fragmenta: pedile a Wireshark que reensamble

Un mensaje TLV puede llegar partido en varios segmentos TCP (o varios mensajes en un mismo segmento). Si el disector lee `buf(3, len)` y el buffer todavía no está completo, falla. La solución estándar es decirle a Wireshark que **espere el resto** con `pinfo.desegment_len` y `pinfo.desegment_offset`:

```
function p.dissector(buf, pinfo, tree)
  if buf:len() < 3 then                        -- falta el encabezado tipo+longitud
    pinfo.desegment_len = DESEGMENT_ONE_MORE_SEGMENT
    return
  end
  local total = 3 + buf(1,2):le_uint()         -- tipo(1)+len(2)+valor; le_uint = struct C
little-endian
  if buf:len() < total then                    -- falta parte del valor
    pinfo.desegment_len = total - buf:len()    -- cuántos bytes más esperar
    pinfo.desegment_offset = 0                 -- re-disectar desde el inicio
    return
  end
  -- acá el mensaje ya está completo: armás el árbol como antes
end
```

Práctica

En tu VM, con un **cliente/servidor propio**:

1. Escribí un cliente/servidor con un protocolo TLV simple (tipo + longitud + valor) y capturá una sesión con Wireshark.
2. Deducí el **framing** solo mirando los bytes (¿prefijo de longitud?).
3. Enganchá `send` con Frida y confirmá la estructura desde el lado del binario; subí la pila para ver qué función arma el mensaje.
4. Escribí el disector Lua y validalo contra una captura nueva; documentá la máquina de estados (handshake → comandos).

Preguntas de repaso

▼ **¿Qué agrega un protocolo frente a un formato de archivo?**

> **Tiempo** (mensajes en secuencia) y **estado** (handshake, auth, keepalive). No alcanza con leer bytes quietos: hay que entender el orden y las transiciones.

▼ **Si el tráfico está cifrado, ¿qué hacés (y qué no)?**

> **No** se rompe la cripto. Se obtienen las claves del **propio cliente** (`SSLKEYLOGFILE` o un hook que vuelca el buffer antes de cifrar) y se analiza el lado que controlás, con autorización.

▼ **¿Cuáles son los tres patrones de framing?**

> **Prefijo de longitud** (el más común), **delimitador** (un byte/secuencia de fin) y **tamaño fijo**. Definen cómo se corta el chorro de bytes de TCP en mensajes.

▼ **¿Por qué enganchar `send` / `recv` y no la red?**

> Porque en esa capa el mensaje está **en claro** (antes de cifrar / después de descifrar) y, subiendo por la pila, encontrás **qué función lo arma**: ahí está la estructura real del protocolo.

▼ **¿Cómo validás que entendiste el protocolo?**

> Escribís un **disector de Wireshark** (Lua): si colorea correctamente los campos de **capturas nuevas** que no usaste para deducirlo, tu reconstrucción es correcta.

Puerta de salida

Podés: (1) aislar una conversación y describir su framing; (2) identificar la serialización (TLV/Protobuf/…); (3) correlacionar tráfico con el binario vía hooks en send/recv; (4) escribir un disector Lua que valide la estructura y documentar la máquina de estados —todo sobre sistemas propios o autorizados.

Fuentes de esta lección

- Wireshark · Lua dissectors (WSDG) — `Proto`, `ProtoField`, `DissectorTable` (conocimiento externo).
- github.com/mildsunrise/protobuf-inspector — reconstruir Protobuf sin `.proto`.
- frida.re/docs — hooks en `send` / `recv` / `WSASend` para ver el mensaje en claro.
- Reconstruir formatos y protocolos — el formato de archivo del que parte esta ampliación.

Capas de esta lección

Contenido observado en CLS: la reconstrucción de formatos/protocolos de la Etapa 4 como base. **Conocimiento externo (agregado por esta edición):** framing, serialización, correlación con hooks de red y disectores de Wireshark. **Recomendación editorial:** ejemplos y práctica ilustrativos, sobre sistemas propios o autorizados.

Etapa 5 · Ramas opcionales

> **Objetivo medible**
>
> Elegir una especialización dentro de Windows, con prerrequisitos y práctica propia, sin convertir el corpus en una lista de tareas peligrosas.

Prerrequisitos: Etapas 0 a 4 completas. **Duración estimada:** Variable: 5 a 10 horas por rama. Es una estimación de esta guía, derivada de la suma de las lecciones; no es una duración declarada por CLS.

Lecciones

#	Lección	Qué aprendés
5.1	Rama malware: análisis defensivo sin ejecutar	Triage, IOCs, YARA y respuesta.
5.2	Rama vulnerabilidades: entender el defecto	Desbordes, mitigaciones de Windows y divulgación.
5.3	Rama automatización: scripts	IDAPython, Ghidra, pefile y capstone.
5.4	Kernel y otros territorios: qué queda fuera	Límites explícitos, otras plataformas y prerequisites.

Recursos de CLS para esta etapa

- 2D · Windows internals (rama optativa)
- Etapa 6 · Kernel y drivers (rama optativa, requiere base técnica específica)
- Exploiting (contenedores pendientes)
- Virus (excluido)
- Themida (excluido)

Evaluación de salida

- Elegir una rama y completar su práctica con documentación.
- Explicar los prerrequisitos y los límites de esa rama.
- No ejecutar muestras ni usar técnicas de evasión fuera del marco.

> **Entregable de la etapa**
>
> Un archivo en tu bitácora con: lecciones completadas, ejercicios resueltos, dudas abiertas y evidencia (capturas, valores, hashes). Sin evidencia no se considera aprobada.

Errores frecuentes

- Saltar a exploiting sin dominar stack y mitigaciones.

- Ejecutar malware para "ver qué hace".
- Confundir un contenedor sin revisar con una fuente leída.

Qué no entra en esta etapa

Los temas que no tienen material revisado o que exceden el marco se listan en Kernel y otros territorios. No los uses como atajo: cada rama tiene prerrequisitos.

Rama malware: análisis defensivo sin ejecutar

> **Objetivo de la lección**
>
> Al terminar vas a poder hacer un triage estático de una muestra, extraer indicadores, escribir una regla YARA sencilla y explicar por qué el análisis dinámico de malware no es una práctica de principiante.

Prerrequisitos: etapas 0 a 4, PE y método. **Duración sugerida:** 5 horas. Estimación aproximada.

Qué hay en CLS y qué queda afuera

En `mega_OTROS` hay carpetas de "VIRUS / PROGRAMACION DE VIRUS" con material antiguo. Están fuera del recorrido base: no se leyeron, no se extrajeron y no se usan como práctica. En el inventario hay archivos en `cuarentena` que tampoco se ejecutan.

El video 37 analiza el malware Shelter **ejecutándolo** en una VM con Windows 7 y una VM Kali. Es un caso observado, no un modelo: esta rama hace lo contrario, empieza por el análisis estático y solo considera el dinámico en un laboratorio dedicado con propósito defensivo.

Por qué estático primero

Un programa malicioso está diseñado para hacer algo que no querés que pase en tu máquina. Ejecutarlo para "ver qué hace" es la forma más rápida de descubrirlo de la peor manera. El análisis estático responde muchas preguntas sin ejecutar:

- ¿Qué tipo de archivo es?
- ¿Está empacado?
- ¿Qué APIs importa?
- ¿Qué strings contiene?
- ¿Qué URLs, IPs o rutas menciona?
- ¿Se parece a una familia conocida?

Con eso ya podés escribir indicadores y reglas de detección. El dinámico se reserva para cuando el estático no alcanza y hay un laboratorio preparado.

Triage estático paso a paso

1. Identificación y hash

```
Get-FileHash -Algorithm SHA256 .\muestra.bin
```

Anotá el hash, el tamaño y el origen. Si el archivo está en cuarentena, no lo muevas de ahí.

2. Tipo y formato

Usá `file` (Linux), Detect It Easy o ExeInfoPE. Mirá:

- ¿PE, ELF, Mach-O, script, documento?
- ¿32 o 64 bits?
- ¿Está empacado? (secciones raras, pocos imports, entry point fuera de `.text`).
- ¿Es .NET? (entonces ILSpy puede ayudar).

3. Secciones e imports

Señal	Por qué importa
Sección `RWX`	Código que se escribe y ejecuta: shellcode o unpacking
Nombres `UPX0`, `.packed`	Empaquetado
Imports de `VirtualAlloc`, `VirtualProtect`, `WriteProcessMemory`, `CreateRemoteThread`	Manipulación de memoria/procesos
Imports de `InternetOpen`, `HttpSendRequest`, `WinHttp*`	Comunicación de red
Imports de `RegSetValue`, `CreateService`, `SchTasks`	Persistencia
Muy pocos imports	Posible empaquetado o resolución dinámica

Un import sospechoso no prueba nada: un programa legítimo también puede usar `VirtualAlloc`. La señal se pondera con el resto.

4. Strings

Buscá URLs, dominios, IPs, rutas, nombres de archivos, claves de registro, mensajes. Las strings pueden estar cifradas; si no ves ninguna legible, eso también es una señal.

5. Entropía

Una sección con entropía muy alta puede estar comprimida o cifrada. No es prueba de malware, pero orienta. Herramientas como Detect It Easy la calculan.

6. Comparación y detección

- **YARA**: reglas por strings, imports o bytes.
- **Firmas**: antivirus y motores de detección.
- **VirusTotal**: útil, pero **no subas muestras confidenciales** ni datos de clientes. Una muestra puede contener información sensible.

Indicadores (IOCs)

Un **IOC** (*indicator of compromise*) es un dato observable que sugiere una infección o un comportamiento malicioso.

Tipo	Ejemplo	Dónde se observa
Hash	SHA-256 de la muestra	Archivo
Archivo	Ruta creada, nombre	Sistema de archivos
Registro	Clave `Run` modificada	Registro
Proceso	Proceso hijo inesperado	Process Explorer
Red	Dominio/IP/puerto	TCPView, DNS
Mutex	Nombre de exclusión	Herramientas de análisis

Los IOCs sirven para detectar y para comparar casos. No son eternos: un atacante puede cambiarlos. Las **TTPs** (tácticas, técnicas y procedimientos) son más estables que los IOCs.

Regla YARA de ejemplo

YARA busca patrones en archivos. Una regla mínima para una cadena de prueba:

```
rule CadenaDePrueba
{
    meta:
        description = "Detecta una cadena de prueba en un binario propio"
        author = "curso CLS"
        date = "2026-10-26"

    strings:
        $a = "CORRECTO_CLASE" ascii wide

    condition:
        $a
}
```

```
yara regla.yar muestra.bin
```

Para malware real, las reglas combinan strings, imports, características del PE y condiciones de tamaño. No escribas una regla que coincida con cualquier programa: generaría falsos positivos.

Caso Shelter: qué se puede aprender sin repetirlo

El video 37 describe Shelter como un malware que infecta `calc.exe`, se comunica con un servidor y usa hilos; el análisis se hace con una VM Windows 7 y otra Kali (`Videos y material adicional, Video 37`,

00:00:00–00:01:30).

Lo que un defensor puede extraer de ese caso:

- La infección de un proceso legítimo es una técnica de sigilo.
- El tráfico de red es un indicador clave.
- El análisis dinámico requiere aislamiento y snapshots.

Lo que **no** se copia: ejecutar la muestra, montar el reverse shell, usar una segunda VM para atacar. Eso es parte de un laboratorio de malware avanzado, con protocolo y autorización, no de un curso de fundamentos.

Respuesta ante un incidente

Si sospechás que una muestra se ejecutó en un entorno que no era el laboratorio:

1. Aislá el equipo de la red.
2. No borres archivos ni apagues de golpe si necesitás evidencia; sacá una imagen si es posible.
3. Recolectá logs, procesos, conexiones y archivos creados.
4. Cambiá credenciales que pudieron quedar expuestas.
5. Reportá según el protocolo de tu organización.
6. Documentá el incidente y las lecciones aprendidas.

Práctica

1. Elegí un archivo del inventario con estado `cuarentena`. **No lo ejecutes.**
2. Calculá su hash y anotá tipo, tamaño y secciones.
3. Listá cinco imports y cinco strings. Marcá cuáles te parecen sospechosos y por qué.
4. Escribí una regla YARA que detecte una cadena inventada por vos en un archivo propio.
5. Armá una tabla de IOCs con al menos cuatro tipos.
6. Escribí un párrafo de conclusión: ¿la muestra es sospechosa? ¿Con qué certeza? ¿Qué falta verificar?

Preguntas de repaso

1. ¿Por qué no se ejecuta una muestra para "ver qué hace"? Porque puede dañar el entorno, propagarse o conectarse a la red. El análisis estático responde muchas preguntas sin ese riesgo.

2. ¿Qué es un IOC? Un indicador observable de compromiso: hash, archivo, registro, proceso, conexión. Sirve para detectar y comparar.

3. ¿Qué indica una sección RWX? Que se puede escribir y ejecutar. Es habitual en unpacking, shellcode o generación de código; no prueba malware por sí sola.

4. ¿Para qué sirve YARA? Para describir patrones y buscar archivos que los cumplan. Es una herramienta de detección, no de ejecución.

5. ¿Qué precaución hay que tener con VirusTotal? No subir muestras confidenciales o con datos de terceros. El servicio es público para los usuarios gratuitos.

Antes de avanzar

Tenés que poder:

- Hacer un triage estático completo sin ejecutar.
- Distinguir una señal de una conclusión.
- Escribir una regla YARA simple.
- Explicar por qué el caso Shelter no es una práctica de esta guía.

> **Puerta de salida**
>
> Si podés describir una muestra solo con datos estáticos y justificar el grado de certeza, la rama está aprobada. Si necesitás ejecutarla para saber qué es, todavía no.

Fuentes de esta lección

- `Videos y material adicional, Video 37` — caso Shelter, VM y análisis dinámico; se cita como caso observado.
- `Inventario limpio, LEEME` — estados `cuarentena`, `excluido` y criterio de no ejecución.
- `mega_OTROS/OTROS` — material de virus **excluido** del recorrido base.
- `Reversing desde cero, cap. 29b` — análisis sobre snapshot sin ejecutar, método defensivo.

> **Capas de esta lección**
>
> **Contenido observado en CLS:** el caso Shelter del video 37, la clasificación del inventario y el método de snapshot del capítulo 29b. **Conocimiento externo:** triage estático, IOCs, YARA y respuesta a incidentes. **Recomendación editorial:** el flujo de triage y la regla de ejemplo.

Rama vulnerabilidades: entender el defecto, no armar un exploit

> **Objetivo de la lección**
>
> Al terminar vas a poder explicar cómo un desborde de buffer puede pisar el return address, qué mitigaciones lo frenan (canary, NX, ASLR) y cómo se documentan y corrigen varias familias de defectos, y cómo se divulga una vulnerabilidad de forma responsable.

Prerrequisitos: etapas 0 a 4, stack y formato PE. **Duración sugerida:** 5 horas. Estimación aproximada.

Alcance

Esta rama **no enseña a explotar sistemas ajenos**. Enseña a entender una clase de defecto, reconocerlo en código propio y explicar por qué las mitigaciones funcionan. Las prácticas se hacen con programas que escribís vos, en una VM local, sin red y sin datos reales.

En CLS hay carpetas de exploiting (`EXPLOITING`, `EXPLOITING Y REVERSING USANDO HERRAMIENTAS FREE`, `CURSO DE ARM EXPLOIT`) y material de "EXPLOIT" en `mega_OTROS`. Son **contenedores no revisados**; el mapa los ubica en la Etapa 5. Esta lección no los usa como fuente y no reproduce sus técnicas.

El defecto: escribir más de lo que entra

```
#include <string.h>
void copiar(char *entrada) {
    char buffer[16];
    strcpy(buffer, entrada);   // no verifica el tamaño
}
```

`buffer` tiene 16 bytes en el stack. Si `entrada` mide 20, `strcpy` escribe 4 bytes más allá del final. Esos bytes pisan lo que haya arriba en el stack: otras variables, el `EBP` guardado o el **return address**.

```
stack de copiar():
+------------------+
| return address   |  <- si se pisa, el ret va a otra dirección
| EBP guardado     |
| buffer[16]       |  <- strcpy escribe acá y sigue
+------------------+
```

Si el atacante controla la entrada y el programa no valida, puede intentar que el return address apunte a código que él eligió. Ese es el núcleo del *stack buffer overflow* clásico.

> **No es una receta**
>
> Esta explicación es conceptual. No incluye direcciones, offsets, shellcode ni pasos para lograr ejecución. Practicar eso contra sistemas ajenos es ilegal y está fuera del alcance del curso. El objetivo es reconocer el defecto y entender la mitigación.

Otras familias de defectos que conviene reconocer

El desbordamiento de buffer es solo una familia. En una revisión defensiva también conviene distinguir:

- **Desbordamiento o wraparound de enteros:** una longitud o un cálculo de tamaño se reduce o desborda antes de reservar/copiar memoria. Revisá límites, signo y operaciones de suma/multiplicación.
- **Lectura/escritura fuera de límites:** el índice o longitud excede el objeto. La escritura puede corromper memoria; la lectura puede exponer datos o fallar.
- **Use-after-free y double free:** se usa una referencia después de liberar el objeto, o se libera dos veces. Seguí el ciclo de vida y la propiedad de cada recurso.
- **Format string:** una entrada controlada se usa como formato; separá datos y cadena de formato.
- **Condiciones de carrera:** el estado cambia entre la comprobación y el uso; buscá sincronización y propiedad compartida.

La meta del curso es localizar la condición en código propio, reproducirla de forma inocua, explicar el impacto y corregir la causa. No se construyen payloads ni se convierten estos ejemplos en explotación.

Mitigaciones

Mitigación	Qué hace	Cómo se ve
Stack canary	Coloca un valor aleatorio antes del return address; si cambió, aborta	`__security_cookie`, `__stack_chk_fail`
NX / DEP	Marca el stack como no ejecutable	Página sin permiso `X`
ASLR (`/DYNAMICBASE`)	Aleatoriza la base de las imágenes	Direcciones distintas en cada ejecución
SafeSEH / SEHOP	Protege la cadena de manejadores SEH (32 bits)	Valida que el handler sea legítimo
CFG (`/guard:cf`)	Valida los destinos de llamadas indirectas	Chequeo antes de un `call` indirecto
CET / Shadow Stack	Protege la dirección de retorno por hardware	Aborta si el return address fue alterado

El capítulo 15 muestra el **security cookie** de MSVC: el compilador guarda un valor calculado con XOR contra `RSP` y lo verifica antes de retornar. Si el buffer lo pisa, la verificación falla y el programa termina (`Reversing desde cero, cap. 15`, 00:07:34–00:08:00).

ASLR se explica en el capítulo 26b: si la imagen se mueve, las direcciones absolutas dejan de ser predecibles (`Reversing desde cero, cap. 26b`, 00:01:53–00:02:30).

Excepciones y explotación: el caso histórico de SEH

En Windows de 32 bits, los manejadores de excepciones se encadenan en el stack y `FS:[0]` apunta al primero (cap. 16b, 00:02:29). Durante años, los desbordes de stack se usaron para sobrescribir esa cadena y redirigir el flujo cuando se producía una excepción.

En 64 bits, esa información se movió a tablas `.pdata` / `.rdata` y ya no está en el stack (cap. 36, 00:00:27). Es un ejemplo de cómo una mitigación cambia el panorama: el mismo defecto (escribir de más) tiene consecuencias distintas según el soporte de excepciones.

El capítulo 21 abre la estructura por dentro. En el prólogo de una función con `try/except`, el compilador arma en el stack un **registro de excepción** de cuatro campos: `next` (apunta al manejador anterior, el que estaba en `FS:[0]`), `exception handler` (la función de Windows que atiende la excepción), un puntero a una **scopetable** en `.data` y el `try level`. Luego instala el nuevo manejador con `mov fs:[0], &registro`, encadenándolo. La scopetable guarda los punteros a la **función filtro** y a la **función del `except`**; en compiladores modernos ese puntero va **XOR** con una cookie para que no se pueda falsificar trivialmente. Cuando ocurre la excepción, el handler desofusca la scopetable, llama primero al **filtro** —que inspecciona el código de la excepción vía `EXCEPTION_POINTERS → ExceptionRecord.ExceptionCode` (por ejemplo `0xC0000005` acceso inválido, `0xC0000094` división por cero, `0x80000003` breakpoint)— y, según devuelva 1 o 0, ejecuta o no el bloque `except` (`Reversing desde cero, cap. 21`; transcripción .txt, sin marcas de tiempo).

Ese mismo `EXCEPTION_POINTERS` tiene un `ContextRecord` con los registros al momento de la excepción, incluidos `DR0` – `DR3`. Por eso un programa puede usar una excepción no solo para controlar el flujo, sino para **detectar breakpoints de hardware** (ver la lección del debugger). Entender la estructura sirve para análisis defensivo, no para construir un exploit.

Para el curso, lo relevante es entender **por qué** el diseño cambió y cómo se detecta, no cómo se explotaba.

ROP en una frase

ROP (*Return-Oriented Programming*) encadena fragmentos de código existente ("gadgets") que terminan en `ret`, para hacer operaciones sin inyectar código nuevo. Funciona porque reutiliza bytes que ya están en la imagen ejecutable. **NX** no lo detiene y **ASLR** lo complica (no sabés dónde están los gadgets). La mitigación más **directa** contra las cadenas basadas en `ret` es **CET / shadow stack**, que protege la dirección de retorno por hardware. **CFG** apunta a otra cosa: valida los destinos de **llamadas indirectas** (más relevante para variantes tipo COP/JOP que para el ROP clásico de `ret`).

No vas a construir una cadena ROP en esta guía. Vas a entender por qué existen las mitigaciones y por qué "el stack no es ejecutable" no alcanza.

En el Curso IDA: las mitigaciones "en acción" (y qué cambió desde entonces)

Los tutes 29–48 del Curso IDA recorren estas mismas mitigaciones con ejemplos concretos: analizan un **CVE real por patch-diffing** (VLC, CVE-2008-4654; tutes 29–31), compilan un programa **con y sin DEP** (`SetProcessDEPPolicy`) para comparar (34–37), muestran el **canary** y el **SEH overwrite** (38–39) y el manejo de **heap/LFH** (41–48), todo con **WinDbg + IDA + Mona**.

Sirven en clave defensiva —entender el defecto y por qué cada mitigación lo frena—, pero **las recetas de explotación están datadas**: son de **Windows 7 / 32 bits (2016–2018)** y en Windows 10/11 x64 muchas ya no funcionan tal cual, porque se sumaron **CFG**, **CET/shadow stack**, **SEHOP** y un heap más endurecido (los propios tutes avisan que el "shellcode universal" puede no andar en Win10). Tomalos como **historia y concepto**, no como guía de explotación actual (Curso IDA · tutes 29–48).

Ejemplo paso a paso: ver la mitigación en tu programa

1. Compilá el programa del buffer **sin** canary (MSVC desactivando `/GS`):

```
cl /Od /GS- vuln.c
```

1. Ejecutalo con una entrada corta: funciona.
2. Ejecutalo con una entrada larga (por ejemplo, 40 caracteres) dentro de la VM: probablemente crashea. Anotá el mensaje.
3. Compilá **con** canary (`/GS` , que en MSVC está activo por defecto):

```
cl /Od /GS vuln.c
```

1. Repetí la entrada larga. Ahora el programa aborta con *buffer overrun detected* (MSVC llama a `__report_gsfailure`).
2. Mirá las mitigaciones de cada `.exe` con un visor de PE o con `dumpbin /headers /loadconfig vuln.exe` y compará (`/GS` , `/DYNAMICBASE` , `/NXCOMPAT` , `/guard:cf`).

Qué observaste: el canary convierte una escritura silenciosa en una detección. No arregla el defecto (el `strcpy` sigue sin validar), pero impide que el return address se use.

Cómo se arregla el defecto

Mal	Bien
`strcpy(buffer, entrada)`	`strncpy(buffer, entrada, sizeof(buffer)-1)` y terminador
`sprintf(buffer, ...)`	`snprintf(buffer, sizeof(buffer), ...)`
`gets(buffer)`	`fgets(buffer, sizeof(buffer), stdin)`
Confiar en el largo de la entrada	Validar antes de copiar

La mitigación no reemplaza la corrección: un programa con canary y `strcpy` sigue teniendo un defecto. El canary solo hace más difícil aprovecharlo.

> **En el Curso IDA: decidir si algo es vulnerable, y leer un parche con un *differ***
>
> El Curso IDA entra al tema de forma didáctica y **defensiva**: define qué es una vulnerabilidad y propone un ejercicio de "*¿vulnerable o no?*" donde tenés el **código fuente, el IDB y el ejecutable**, y lo analizás estáticamente en IDA antes de concluir. También presenta los ***differs***: herramientas que comparan **dos versiones consecutivas** de un programa y marcan qué funciones cambiaron —por ejemplo, para ver **qué arregló un parche de seguridad** (la base del análisis de parches / 1-day, en clave defensiva). Identificar el defecto no es construir el exploit (Curso IDA · tute 20, Qué es una vulnerabilidad; tute 21, ¿Vulnerable o no?; tute 22, Differ).

Divulgación responsable

Si encontrás una vulnerabilidad en un programa que tenés permiso de analizar:

1. Confirmá el impacto y la versión afectada.
2. Escribí un reporte privado con pasos mínimos de reproducción.
3. Contactá al proveedor y guardá la evidencia.
4. Acordá un plazo (por ejemplo, 90 días) antes de publicar.
5. No accedas a datos de terceros ni dejes el sistema en un estado inseguro.
6. Publicá solo cuando el proveedor haya tenido tiempo de corregir, o según el acuerdo.

Práctica

1. Escribí `vuln.c` con un buffer de 16 bytes y `strcpy`.
2. Compilalo con y sin canary. Ejecutá entradas de 8, 16, 24 y 40 caracteres en la VM. Anotá qué pasa.
3. Mirá las mitigaciones de las dos versiones con `dumpbin /headers /loadconfig` o un visor de PE y explicá cada una.
4. Corregí el programa con una copia **acotada y con terminador explícito** (`strncpy` + poner el `\0` final, o `strcpy_s`) y verificá que la entrada larga ya no rompe.
5. Escribí un informe de dos páginas: defecto, impacto, mitigación, corrección y límites. Sin exploit.

Preguntas de repaso

1. ¿Qué es un desborde de buffer? Escribir más datos de los que entran en un buffer, pisando memoria adyacente.

2. ¿Qué protege el stack canary? Detecta si se modificó el valor colocado antes del return address. Si cambió, aborta antes de retornar.

3. ¿NX impide ejecutar código en el stack? Sí, marca las páginas del stack como no ejecutables. No impide el desborde ni la sobrescritura de punteros.

4. ¿ASLR alcanza para proteger un programa? No. Complica la predicción de direcciones, pero un defecto sigue siendo explotable si se filtra una dirección o si el atacante puede reintentar.

5. ¿Qué diferencia hay entre mitigar y corregir? Mitigar reduce el impacto; corregir elimina la causa. Un canary mitiga un `strcpy` inseguro; la corrección es usar una copia **acotada y bien terminada** —validar el tamaño y garantizar el `\0` final (p. ej. `strncpy` **más terminación explícita**, o `strcpy_s` / `strlcpy`)—. `strncpy` a secas no alcanza: puede dejar la cadena **sin terminador** y no valida que la entrada tenga sentido.

6. ¿Qué se hace al encontrar una vulnerabilidad? Documentarla, reportarla en privado, no publicar el exploit y acordar un plazo de divulgación.

Antes de avanzar

Tenés que poder:

- Explicar el defecto con el dibujo del stack.
- Nombrar cuatro mitigaciones y qué frena cada una.
- Compilar con y sin canary y observar la diferencia.
- Escribir un informe sin exploit.

> **Puerta de salida**
>
> Si podés explicar por qué el canary detecta el desborde y por qué no corrige el `strcpy`, la rama está aprobada. Si tu objetivo es "hacer que funcione el exploit", estás en el lugar equivocado.

Fuentes de esta lección

- MITRE CWE: Out-of-bounds Write (CWE-787), Integer Overflow or Wraparound (CWE-190), Use After Free (CWE-416).
- `Reversing desde cero, cap. 15` — security cookie.
- `Reversing desde cero, cap. 16b` — SEH en 32 bits.
- `Reversing desde cero, cap. 21` — estructura SEH (registro, scopetable, filtro y handler), códigos de excepción y `ContextRecord` (se omite la parte de keygen).

- `Reversing desde cero, cap. 36` — SEH basado en tablas en 64 bits.
- `Reversing desde cero, cap. 26b` — ASLR.
- `EXPLOITING (contenedor)` — **pendiente de revisar**: no se extrajo ni se leyó.
- Curso IDA · tutes 20–22 — qué es una vulnerabilidad, el ejercicio "¿vulnerable o no?" (fuente + IDB + binario) y los *differs* para leer qué cambió un parche (defensivo).
- Curso IDA · tutes 29–48 — las mitigaciones en ejemplos (VLC/CVE-2008-4654, DEP, canary/SEH, heap) con WinDbg/IDA/Mona; recetas de Windows 7 / 32 bits, **históricas**: en Windows moderno cambian CFG, CET y SEHOP.

> **Capas de esta lección**
>
> **Contenido observado en CLS:** la cookie del capítulo 15, el SEH de los capítulos 16b y 36, y ASLR del 26b. **Conocimiento externo:** desbordes, overflow de enteros, acceso fuera de límites, use-after-free, double free, format string, carreras, mitigaciones, ROP y divulgación. **Recomendación editorial:** el ejemplo `vuln.c` y el límite explícito sobre exploits.

Rama automatización: scripts para no repetir trabajo

> **Objetivo de la lección**
>
> Al terminar vas a poder escribir un script corto que recorra funciones, busque una constante o liste imports, y decidir cuándo conviene automatizar y cuándo no.

Prerrequisitos: etapas 0 a 4, IDA o Ghidra, Python básico. **Duración sugerida:** 4 horas. Estimación aproximada.

Qué hay en CLS

El curso tiene cuatro capítulos de IDAPython, **leídos e integrados en esta edición**: 22b ("Empezando con IDAPython"), 25 ("Empezando con IDAPython" + guía del entorno), 27b ("Jugando con IDAPython") y 32 ("Un proyecto con IDAPython"). Más abajo, en «Lo que muestra el curso», están los ejemplos observados con su cita. Los ejemplos de esta sección inicial son editoriales (código propio) y los del curso los complementan con la API real.

> **Sobre las versiones y la legalidad**
>
> Los capítulos usan IDA 7.5 con Python 3. Varios demuestran las técnicas sobre crackmes (incluido parcheo): esta guía toma la **mecánica de IDAPython** (recorrer, buscar, pintar, parchear, revertir) y la practica con **programas propios**, nunca para saltar protecciones de software ajeno.

Cuándo automatizar

Automatizá cuando:

- La tarea se repite muchas veces (renombrar 200 funciones, buscar una constante en todo el binario).
- La tarea es mecánica y propensa a errores (extraer imports, comparar listas).
- Querés que el análisis sea reproducible (un script se puede volver a correr).

No automaticés cuando:

- Todavía no entendés el problema. Primero hacelo a mano una vez.
- La tarea requiere juicio caso por caso. Un script que renombra "lo que parece" puede empeorar el análisis.
- El script va a modificar el binario sin backup.

IDAPython: el modelo mental

IDA expone tres módulos principales:

Módulo	Para qué
`idc`	Funciones clásicas: nombres, comentarios, bytes, instrucciones
`idautils`	Iteradores: funciones, segmentos, referencias, heads
`idaapi`	API de más bajo nivel: eventos, UI, tipos
`ida_bytes`	Acceso a bytes y flags
`ida_funcs`	Funciones y sus límites

Un script típico recorre algo y hace una acción por elemento.

Ejemplo 1: renombrar funciones sin nombre

```
import idautils
import idc

for ea in idautils.Functions():
    nombre = idc.get_func_name(ea)
    if nombre.startswith("sub_"):
        idc.set_name(ea, f"func_{ea:X}", idc.SN_NOWARN)
```

Qué hace: recorre todas las funciones detectadas; si el nombre automático empieza con `sub_`, lo reemplaza por `func_<dirección>`. Es útil para tener nombres estables antes de renombrar a mano las funciones importantes.

Ejemplo 2: buscar una constante en el código

```
import idautils
import idc

OBJETIVO = 0x1E0

for seg in idautils.Segments():
    if idc.get_segm_name(seg) != ".text":
        continue
    for ea in idautils.Heads(idc.get_segm_start(seg), idc.get_segm_end(seg)):
        if idc.print_insn_mnem(ea) in ("cmp", "sub", "add"):
            for i in range(3):
                if idc.get_operand_value(ea, i) == OBJETIVO:
                    print(f"{ea:#x}: {idc.generate_disasm_line(ea, 0)}")
```

Qué hace: recorre las instrucciones de `.text`; si alguna usa la constante `0x1E0` como operando, imprime la dirección y el texto. En el programa del capítulo 15, esa constante es la que compara el resultado con 480.

> **La API cambia entre versiones**
>
> `idc.get_operand_value`, `idc.print_insn_mnem` y `idautils.Heads` existen en IDA 7 y 8, pero algunos nombres cambiaron respecto de IDA 6. Si el script falla, revisá la versión y la documentación de IDAPython. La lógica —recorrer y comparar— se mantiene.

Ejemplo 3: listar imports

```
import idautils
import idc

for i in range(idc.get_import_module_qty()):
    modulo = idc.get_import_module_name(i)
    print(f"[{modulo}]")
    def callback(ea, nombre, ordinal):
        print(f"  {ea:#x}  {nombre or ordinal}")
        return True
    idc.enum_import_names(i, callback)
```

Qué hace: recorre los módulos importados y, por cada uno, enumera sus funciones. Es el equivalente scripteado de la pestaña Imports.

Lo que muestra el curso

Esta sección resume lo que hacen los cuatro capítulos de IDAPython, con la API real que usan.

Montar el entorno y probar que anda

El cap. 22b instala **IPyIDA**, una consola de IPython dentro de IDA (se abre con `Shift` + `.`). Tiene autocompletado con `Tab`, historial (`%hist`) y reuso de resultados por número de línea (`_6`). Dos molestias que el propio video admite: hay que **importar las librerías a mano** la primera vez (`from idc import *`, `from idautils import *`, `from idaapi import *`) y la consola se cierra al depurar (se reabre con `Shift` + `.`).

El cap. 25 arma un entorno más cómodo: un archivo `exint.py` en `.ipython/profile_default/startup/` que carga las librerías solas (y agrega `exon` / `exof` para mostrar enteros en hexadecimal), y **Spyder conectado al kernel de IDA** (*Consoles → Connect to an existing kernel*, eligiendo el `kernel-NNNN.json` más nuevo). Así podés escribir scripts con autocompletado, ver la documentación con `Ctrl+I`, ejecutar una selección (*Run selection or current line*) y hacer doble clic en una dirección de la salida para saltar a ella en IDA.

Una prueba rápida de que las librerías están cargadas: `here()` devuelve la dirección donde está el cursor. Si da error, las librerías no se importaron (cap. 25).

> **La API cambió de nombre entre versiones**
>
> La mayoría de los scripts viejos de internet fallan no porque estén mal, sino porque los nombres cambiaron (`PatchByte` → `patch_byte`, etc.). El cap. 25 recomienda la página oficial *"Porting from IDA 6.x to 7.4"*, que lista el nombre viejo y el nuevo de cada función. Si un script tira error por todos lados, es casi seguro esto.

Correr un script

Hay tres formas (caps. 22b y 25): cargar un `.py` con *File → Script file*; pegar código de IDAPython **dentro de un breakpoint** (en *Edit breakpoint*, cambiando el tipo a IDAPython) —ahí los registros se leen como `cpu.edx`, y si el código hace `return True` el programa para, lo que da un breakpoint condicional scripteado—; o, con Spyder conectado, ejecutar una selección del script.

Recorrer las funciones

El cap. 25 recorre todas las funciones del binario:

```
import idautils, idc

inicio = idc.get_inf_attr(idc.INF_MIN_EA)   # dirección mínima cargada
fin    = idc.get_inf_attr(idc.INF_MAX_EA)   # dirección máxima

for ea in idautils.Functions(inicio, fin):
    nombre = idc.get_func_name(ea)
    fin_f  = idc.find_func_end(ea)
    print(f"{ea:#x}  {fin_f:#x}  {nombre}")
```

`Functions()` devuelve un **generador** (para verlo entero, `list(idautils.Functions())`); `get_func_name(ea)` da el nombre de la función a la que pertenece una dirección (no hace falta la dirección exacta de inicio); `find_func_end(ea)` da el final (un byte más allá de la última instrucción).

Recorrer el grafo de control y pintar bloques

El cap. 27b toma una función y recorre sus **bloques básicos**:

```
import idaapi, idc

f = idaapi.get_func(idc.here())     # func_t de la función bajo el cursor
fc = idaapi.FlowChart(f)
for bloque in fc:
    print(f"{bloque.start_ea:#x} - {bloque.end_ea:#x}")
```

Cada bloque tiene `start_ea`, `end_ea`, `preds()` (bloques que llegan a él) y `succs()` (a los que salta). Para **pintar** el bloque donde está una dirección y sus predecesores de otro color:

```
import idaapi, idautils, idc

addr = idc.here()
f = idaapi.get_func(addr)
for bloque in idaapi.FlowChart(f):
    if bloque.start_ea <= addr < bloque.end_ea:        # bloque que contiene addr
        for ea in idautils.Heads(bloque.start_ea, bloque.end_ea):
            idc.set_color(ea, idc.CIC_ITEM, 0x80FF80)  # verde (0xBBGGRR)
        for pred in bloque.preds():                    # puede haber varios
            for ea in idautils.Heads(pred.start_ea, pred.end_ea):
                idc.set_color(ea, idc.CIC_ITEM, 0x80FFFF)
```

Es una forma visual de ver de dónde viene el flujo. El color va en formato `0xBBGGRR`.

Parchear con un script (solo programas propios)

El cap. 25 automatiza un parcheo sobre un crackme. Tomamos la **mecánica** (no el objetivo) para aplicarla a un programa propio en tu laboratorio:

```
import idc

addr = idc.find_binary(idc.get_inf_attr(idc.INF_MIN_EA), idc.SEARCH_DOWN, "74 07")
if addr != idc.BADADDR:
    print("originales:", idc.get_bytes(addr, 2).hex(), "/ byte previo:",
hex(idc.get_original_byte(addr)))
    idc.patch_bytes(addr, b"\x90\x90")   # en la base de datos
    # idc.revert_byte(addr)              # para deshacer
```

`patch_bytes` cambia bytes en la base de datos; `get_original_byte` devuelve el byte anterior al parche (lo mismo que *Patch program → original bytes* en el menú); `revert_byte` lo deshace. Para escribir los cambios al `.exe`, el cap. 25 usa `save_file` */apply patches to input file*, calculando el **file offset** = dirección − base del código + `0x400` (header), como en la lección de números. **Antes de cualquier script que modifique la base de datos, tomá un snapshot** (*Database snapshot manager*).

Buscar secuencias de instrucciones por patrón

El cap. 32 arma un buscador de instrucciones con `ida_search.find_text` + expresiones regulares. El truco es una función `replace()` que convierte una instrucción fácil de tipear en un patrón regex:

- el espacio simple → `*` (cualquier cantidad de espacios, porque IDA alinea con varios);
- `R32` → `(e.x|e.i)` para cualquier registro de 32 bits;
- `const` → `[0-9a-fA-F]{2,8}h` para una constante hexadecimal;
- y **escapar** los caracteres especiales de regex que aparezcan en la instrucción (`[`, `]`, `+`, `*`, `-`) con `\`.

Dos límites que el capítulo señala: la búsqueda de texto de IDA es **de una sola línea** (no matchea una secuencia multilínea en una sola llamada; hay que buscar instrucción por instrucción, encadenando desde la dirección anterior), y `find_text` devuelve `BADADDR` (`0xFFFFFFFF`) cuando no encuentra. Para buscar una secuencia de tres instrucciones "cercanas", se busca la primera, y desde cada hallazgo se busca la siguiente

verificando que la **distancia** no supere un límite. `idc.jumpto(ea)` mueve el cursor para mostrar el resultado en IDA.

Traducir números a nombres con enumeraciones

El cap. 22b, para imprimir los mensajes de Windows por nombre en vez de por número, agrega una enumeración en *Local Types* y la consulta desde el script con `get_enum("nombre")` (devuelve el id de la enumeración) y la función que, dado ese id y un valor como `cpu.edx`, devuelve el nombre (`WM_COMMAND`, etc.). Es la forma scripteada de lo que la tecla `M` hace a mano en el listado.

Ghidra y Python standalone

Si usás Ghidra, tenés scripting en Python (Jython) y Java. El modelo es parecido: recorrer funciones, leer instrucciones, renombrar.

Si no tenés IDA Pro, también podés automatizar **sin** desensamblador:

```
import pefile

pe = pefile.PE("check.exe")
print("Entry point:", hex(pe.OPTIONAL_HEADER.AddressOfEntryPoint))
for entry in pe.DIRECTORY_ENTRY_IMPORT:
    print(entry.dll.decode())
    for imp in entry.imports:
        print("  ", imp.name)
```

`pefile` es una librería de Python para leer PE. No desensambla, pero sirve para inventarios, detección y reportes.

Para desensamblar desde Python, `capstone` permite decodificar bytes:

```
from capstone import Cs, CS_ARCH_X86, CS_MODE_32

md = Cs(CS_ARCH_X86, CS_MODE_32)
codigo = bytes.fromhex("83F81E7410")
for ins in md.disasm(codigo, 0x401000):
    print(f"{ins.address:#x}: {ins.mnemonic} {ins.op_str}")
```

Salida esperada:

```
0x401000: cmp eax, 0x1e
0x401003: je 0x401015
```

El flujo de trabajo con scripts

```
1. Hacer la tarea a mano una vez.
2. Detectar qué se repite.
3. Escribir el script mínimo.
4. Probarlo sobre una copia.
5. Revisar el resultado (no confiar ciegamente).
6. Guardar el script con el informe.
```

El paso 5 es el más importante. Un script que renombra mal contamina la base de datos y hace que el análisis sea peor que antes.

Práctica

1. Compilá tu `check.exe` y abrilo en IDA.
2. Corré el script de renombrado y verificá que las funciones `sub_` cambiaron de nombre.
3. Corré el script de búsqueda de `0x1E0` y comprobá que encuentra la comparación.
4. Modificá la constante del programa a `0x1B8`, recompilá y volvé a correr el script.
5. Escribí un script que imprima todas las funciones cuyo nombre contenga "check" o "main".
6. Guardá los scripts en tu bitácora y anotá la versión de IDA.

Preguntas de repaso

1. ¿Cuándo conviene automatizar? Cuando la tarea es repetitiva, mecánica y verificable. Primero se hace a mano para entenderla.

2. ¿Qué diferencia hay entre `idc`, `idautils` e `idaapi`? `idc` tiene funciones clásicas; `idautils` ofrece iteradores; `idaapi` es la API de bajo nivel y la interfaz.

3. ¿IDAPython funciona en IDA Free? Depende de la versión. Históricamente IDA Free no incluía Python; las versiones recientes agregaron capacidades. Si no está, usá Ghidra o automatizá con `pefile` / `capstone`.

4. ¿Por qué hay que revisar el resultado de un script? Porque un script no entiende el contexto: puede renombrar mal, buscar coincidencias falsas o modificar datos que no correspondían.

5. ¿Qué hace `capstone`? Desensambla bytes a instrucciones desde Python, sin necesidad de IDA.

Antes de avanzar

Tenés que poder:

- Escribir un script que recorra funciones y renombre.
- Buscar una constante en `.text`.
- Listar imports con IDAPython o `pefile`.
- Explicar por qué el resultado se revisa a mano.

Puerta de salida

Si el script te ahorra tiempo y el resultado es correcto, la rama está aprobada. Si el script produce nombres dudosos y no los revisás, no.

Fuentes de esta lección

- `Reversing desde cero, cap. 22b` — IPyIDA, imports, correr código en un breakpoint (`cpu.edx`, `return True`) y enumeraciones (transcripción .txt, sin marcas).
- `Reversing desde cero, cap. 25` — entorno (exint.py, Spyder+kernel), `here()`, porting de nombres, recorrer `Functions()` y parchear/revertir/guardar (transcripción .txt, sin marcas).
- `Reversing desde cero, cap. 27b` — `get_func`, `FlowChart`, bloques básicos (`start_ea` / `end_ea` / `preds`) y pintar con `set_color` (transcripción .txt, sin marcas).
- `Reversing desde cero, cap. 32` — búsqueda de instrucciones con `find_text` + regex, límite de una línea y búsqueda por distancia (transcripción .txt, sin marcas).
- `Reversing desde cero, cap. 15` — constante `0x1E0` usada en el ejemplo de búsqueda editorial.

Capas de esta lección

Contenido observado en CLS: el entorno IPyIDA/Spyder, `here()`, el recorrido de funciones y del grafo, el parcheo scripteado, la búsqueda por regex y las enumeraciones (caps. 22b, 25, 27b, 32). **Conocimiento externo:** API de IDAPython moderna, Ghidra, `pefile` y `capstone`. **Recomendación editorial:** los tres primeros scripts de ejemplo, el flujo de trabajo y el encuadre de parchear solo programas propios.

Ampliación editorial basada en fuentes externas — Compatibilidad con IDA 9.0+: reemplazo de ida_struct e ida_enum por ida_typeinf

En **IDA 9.0+** (IDAPython Porting Guide to IDA 9.0), Hex-Rays eliminó por completo los módulos `ida_struct` e `ida_enum` (y wrappers como `idc.get_enum` citado en el ejemplo del cap. 22b), unificándolos en **`ida_typeinf`** (`tinfo_t`) *(Hecho confirmado)*. Para consultar el nombre simbólico de un valor en una enumeración de *Local Types* en IDA 9.0+:

```
import ida_typeinf

def nombre_miembro_enum(nombre_enum: str, valor: int) -> str | None:
    tif = ida_typeinf.tinfo_t()
    if not tif.get_named_type(None, nombre_enum) or not tif.is_enum():
        return None
    ed = ida_typeinf.enum_type_data_t()
    if not tif.get_enum_details(ed):
        return None
    for m in ed:
        if m.value == valor:
            return m.name
    return None
```

Kernel y otros territorios: qué queda fuera y por qué

> **Objetivo de la lección**
>
> Al terminar vas a poder explicar por qué el kernel, el desempaquetado de protectores comerciales, el exploiting contra terceros y **otras plataformas (Linux, macOS, Android, ARM)** no forman parte de esta edición, y qué prerequisites harían falta para abordarlos.

> **Esta edición es solo Windows**
>
> Por decisión de alcance, la guía cubre **únicamente binarios de Windows** (PE, x86/x64). El reversing de **Linux/ELF**, **macOS/Mach-O**, **Android (APK/DEX)** y arquitecturas como **ARM/AArch64** queda **fuera**. Los conceptos de base (números, memoria, stack, flujo, método) se trasladan a esas plataformas, pero su formato, convenciones y herramientas son otro mundo y merecen su propio material.

Prerrequisitos: etapas 0 a 4. **Duración sugerida:** 30 minutos (lectura y reflexión).

Una guía también se define por lo que no incluye

El corpus de CLS tiene material de kernel, protectores comerciales, exploiting y malware. No todo se convierte en lección. Algunos temas quedan fuera por tres razones:

1. **No hay material revisado.** Hay contenedores que no se abrieron; una mención no es contenido.
2. **El riesgo no es proporcional al beneficio para un principiante.** Enseñar a manipular kernel o protectores sin bases sólidas produce accidentes, no aprendizaje.
3. **El marco legal y ético.** Eludir protecciones de software comercial o atacar sistemas ajenos no es educativo.

Kernel

Un módulo de kernel corre en el nivel más privilegiado del sistema. Un error ahí no crashea una aplicación: crashea la máquina, corrompe el sistema o abre una puerta que no se cierra con un reinicio.

Para abordar kernel con responsabilidad hacen falta, como mínimo:

- C y ensamblador con soltura.
- Entender la memoria virtual, la paginación y las estructuras del sistema operativo.
- Un laboratorio con snapshots y, preferentemente, hardware o VM descartable.
- Un objetivo defensivo claro: analizar un driver, entender una mitigación, revisar una vulnerabilidad con permiso.

En CLS no hay un curso de kernel leído. El mapa menciona material de exploiting y de drivers, pero no se verificó. Esta guía no lo convierte en una lección de principiante.

Protectores comerciales

El desempaquetado de protectores como Themida, WinLicense o ACProtect aparece en el corpus (por ejemplo, el RAR de Themida + WinLicense y los capítulos de ACProtect). Es material avanzado y, en muchos casos, orientado a modificar software comercial.

Esta guía explica **qué es** un empacador, cómo se reconoce y por qué el código queda plano en memoria en algún momento. No enseña a desempaquetar protectores comerciales, no distribuye recetas y no propone esas tareas. El capítulo 35 (inline patching sobre ACProtect) se cita solo como evidencia de que la técnica existe; no se reproduce.

Exploiting contra terceros

Los ejercicios de vulnerabilidades de esta guía se limitan a programas propios o laboratorios deliberadamente vulnerables, con permiso. No se enseña a atacar sistemas ajenos, a evadir defensas de terceros ni a mantener acceso.

Si tu objetivo es la seguridad ofensiva profesional, el camino incluye:

- Fundamentos de redes y sistemas.
- Programación y análisis de binarios.
- Laboratorios legales (por ejemplo, entornos de práctica diseñados para eso).
- Certificaciones y programas con alcance definido.
- Ética y divulgación responsable.

Otros temas del corpus que quedan como referencia

Tema	Estado en esta edición
Android (APK/DEX)	**Fuera de alcance**: edición solo Windows
Linux/ELF	**Fuera de alcance**: edición solo Windows
macOS/Mach-O	**Fuera de alcance**: edición solo Windows
ARM/AArch64 y otras arquitecturas	**Fuera de alcance**: edición solo Windows (x86/x64)
Malware	Triage estático defensivo de Windows; ejecución **fuera de alcance**
Vulnerabilidades	Concepto y mitigación en laboratorio propio; exploit **fuera de alcance**
Kernel/drivers (Windows)	Rama optativa Etapa 6: drivers propios/didácticos en VM descartable; no rootkits, BYOVD, explotación ni evasión de mitigaciones
Protectores comerciales	**Fuera de alcance** por marco legal y ético
Automatización	Rama con IDAPython/Ghidra; capítulos CLS **pendientes**

Práctica

1. Elegí un tema de la tabla que te interese y escribí qué prerequisites te faltan.
2. Buscá en el corpus un recurso de ese tema y anotá si está leído o pendiente.
3. Escribí una regla personal: "no voy a analizar X hasta poder Y".
4. Revisá tu bitácora: ¿hay alguna práctica que hayas hecho sin permiso o sin aislamiento? Corregila.

Preguntas de repaso

1. ¿Por qué el kernel queda fuera? Por el nivel de privilegio, el riesgo de daño y la falta de material revisado y de prerequisites en esta edición.

2. ¿Qué diferencia hay entre explicar un empacador y desempaquetar un protector comercial? Explicar es conceptual y defensivo; desempaquetar un protector comercial para modificar el programa queda fuera del marco legal y ético de esta guía.

3. ¿Qué hace falta para practicar exploiting de forma responsable? Un laboratorio propio o autorizado, un objetivo definido, permisos y un plan de divulgación.

4. ¿Una mención en el corpus alcanza para dar una lección? No. Una mención o un título no demuestran el contenido. Primero hay que leerlo y verificarlo.

Antes de cerrar

Tenés que poder:

- Explicar por qué no todo el corpus se convierte en curso.
- Nombrar los prerequisites de kernel.
- Decir qué temas quedan como referencia y cuáles como pendientes.

> **Cierre del recorrido**
>
> Si llegaste hasta acá con las etapas 0 a 4 completas, tenés una base real de reversing. Las ramas son opcionales: elegí una, completá su práctica y documentala. No acumules temas: acumulá evidencia.

Fuentes de esta lección

- `mapa_carpetas_etapas.csv` — asignación de temas y notas sobre kernel/exploiting.
- `Themida + WinLicense 2.x (Unpacking).rar` — recurso **excluido** del recorrido base.
- `Reversing desde cero, cap. 35` — evidencia de la técnica de inline patching; **no se reproduce**.
- `FUENTES_Y_LIMITES.md` — criterios de exclusión de esta edición.

Capas de esta lección

Contenido observado en CLS: la existencia de los recursos y su clasificación en el mapa.
Conocimiento externo: requisitos de kernel y explotación responsable. **Recomendación editorial:** los límites y la tabla de estado.

Hilo conductor multi-nivel y kit de plantillas C/C++

> **Propósito de esta sección transversal**
>
> Conectar de forma práctica los tres niveles del curso (Principiante, Intermedio y Avanzado). En la **Parte I** vemos cómo un único programa de validación (desde el `check.c` básico de la Etapa 1 hasta el componente `BlocLab` de las Etapas 2C, 2D y 6) cambia en el desensamblador y en el debugger a medida que evoluciona. En la **Parte II** tenés **5 plantillas mínimas en C y C++** listas para copiar y compilar en tu VM para practicar todas las lecciones intermedias y avanzadas sin depender de muestras externas.

Parte I · El hilo conductor: un mismo binario a través de los tres niveles

Una de las dificultades más comunes al estudiar reversing es sentir que cada tema nuevo (optimizaciones, TLS callbacks, packers, C++ con vtables, named pipes o drivers KMDF) pertenece a un mundo desconectado del anterior. Para unirlos, sigamos **la misma regla de negocio** en sus tres estadios de evolución:

> **Regla de negocio constante:** aceptar dos enteros `a` y `b` tales que `a <= 50` (`0x32`), `b >= 50` (`0x32`) y `(a + b) * 4 == 480` (`0x1E0`).

Estadio 1 · Nivel Principiante: `check_v1.c` con `-O0` frente a `-O2` (Etapas 1, 2A y 4.1)

En el nivel Principiante escribimos la función en C puro y comparamos qué genera el compilador en x64 sin optimizaciones (`-O0` / `/Od`) frente a compilación optimizada (`-O2`):

```
// Regla base en C (Lecciones 1.5, 2.4, 2.8 y 4.1)
int validar_pares(int a, int b) {
    if (a > 50 || b < 50) return 0;
    return ((a + b) * 4) == 480;
}
```

```
Compilación sin optimizar (-O0, x64)
------------------------------------
validar_pares:
  mov   dword ptr [rsp+8], ecx   ; guarda a
defecto
  mov   dword ptr [rsp+10h], edx ; guarda b
  cmp   dword ptr [rsp+8], 32h   ; ¿a > 50?
  jg    short fallo
  cmp   dword ptr [rsp+10h], 32h ; ¿b < 50?
  jl    short fallo
  mov   eax, dword ptr [rsp+8]
  add   eax, dword ptr [rsp+10h] ; eax = a+b
(2.4)
  shl   eax, 2                   ; eax *= 4
  cmp   eax, 1E0h                ; ¿== 480?
  jne   short fallo
  mov   eax, 1
  ret
fallo:
  xor   eax, eax
  ret
```

```
Compilación optimizada (-O2, x64)
---------------------------------
validar_pares:
  xor   eax, eax          ; resultado = 0 por
defecto
  cmp   ecx, 32h          ; ¿a > 50? (ECX = a)
  jg    short salir       ; si a > 50, retorna 0
  cmp   edx, 32h          ; ¿b < 50? (EDX = b)
  jl    short salir       ; si b < 50, retorna 0
  lea    ecx, [rcx+rdx]    ; ecx = a + b (con LEA,
2.2)
  cmp   ecx, 78h          ; ¡480 / 4 = 120 (0x78)!
  sete  al                ; AL = 1 si (a+b)==120
(2.4)
salir:
  ret
```

Qué aprende el alumno al comparar ambos listados:

- Con `-O0` (izquierda) cada variable baja al *shadow space* (`[rsp+8]`, `[rsp+10h]`, 1.2 y 2.5) y la multiplicación por 4 se ve como `shl eax, 2` seguida de `cmp eax, 1E0h`.
- Con `-O2` (derecha) el compilador aplica reducción algebraica (2.8): en lugar de multiplicar `(a + b) * 4` y comparar contra `480`, suma con `lea ecx, [rcx+rdx]` y compara directamente contra `120` (`0x78 = 480 / 4`), usando `sete al` en vez de un salto condicional. Si en `main` se elige entre `"Correcto"` e `"Incorrecto"`, `-O2` carga ambas direcciones con `lea` y selecciona una con `cmovne` (4.1).

Estadio 2 · Nivel Intermedio: `check_v2.c` con TLS Callback, XOR y UPX (Etapas 2B, 2.17, 2.21 y 4.2)

En el nivel Intermedio, el mismo programa incorpora tres capas que frustran el camino directo de Principiante:

1. **Un TLS Callback previo a `main` (2.11 y 2.15):** una función registrada en `.CRT$XLB` se ejecuta antes del entry point y consulta `IsDebuggerPresent()` (o lee `gs:[60h] → [peb+2]`). En lugar de mostrar un cartel evidente, si detecta un depurador cambia una variable global `g_objetivo` de `480` (`0x1E0`) a `999` (`0x3E7`). Quien pone el breakpoint directamente en `main` ve que `40` y `80` fallan sin entender quién alteró `g_objetivo`.
2. **Cadenas ocultas con XOR (2.10 y 4.2):** la cadena `"Correcto"` ya no está en texto claro en `.rdata`; se almacena como `{0x19, 0x35, 0x28, 0x28, 0x3F, 0x39, 0x2E, 0x35, 0x00}` y un bucle corto aplica `xor byte ptr [rcx+rax], 5Ah` justo antes de imprimir.
3. **Empaquetado didáctico con UPX (2.17):** al pasar el ejecutable por `upx -9 check_v2.exe`, las secciones originales se comprimen dentro de `UPX1` y la IAT queda reducida a `LoadLibraryA` y `GetProcAddress`.

```
 Secuencia de resolución en el Nivel Intermedio:
 1. Triage PE-bear (1.4 / 2.17) ──► Secciones UPX0 (RawSize=0) y UPX1 (entropía 7,8)
 2. Desempaquetado en x64dbg (2.17) ──► Breakpoint de ejecución en memoria sobre UPX0
                                    ──► F9 hasta caer en el OEP ──► Dump + Scylla IAT Fix
 3. Inspección del Dump en IDA/Ghidra (2.11 / 2.16) ──► Directorio TLS apunta a tls_callback_0
                                                    ──► Se observa escritura a g_objetivo si
BeingDebugged != 0
 4. Recuperación de cadena XOR (2.10 / 2.21) ──► Bucle xor [reg], 0x5A antes de puts
                                             ──► Breakpoint de log en x64dbg: "msg={s:rcx}"
```

Estadio 3 · Nivel Avanzado: `BlocLab` en C++, IPC y Kernel (Etapas 2C, 2D, 4.5, 5.2 y 6)

En el nivel Avanzado, la misma verificación vive dentro de una arquitectura realista de Windows:

- **Polimorfismo C++, RTTI, SSO y Excepciones (2.18 y 2D.11):** un objeto derivado `ValidadorBloc` implementa el método virtual `verificar(const std::string& etiqueta, const MensajeCLSF* m)`. En el listado vemos `mov rax, [rcx] ; call qword ptr [rax+8]`. Inspeccionando `vtable[-1]` en `.rdata` encontramos el `_RTTICompleteObjectLocator` (con RVAs en x64) que lleva al nombre `.?AVValidadorBloc@@`. Si los parámetros violan el rango, lanza una excepción C++ despachada por `__CxxFrameHandler4` a través de `.pdata` y `.xdata`.
- **Protocolo binario `CLSF` sobre Named Pipe (4.5 y 2D.10):** en lugar de pedir números por consola con `scanf`, un proceso servidor crea `\.\pipe\BlocLabPipe` (con `FILE_FLAG_FIRST_PIPE_INSTANCE` y `PIPE_REJECT_REMOTE_CLIENTS`) y recibe una trama `CLSF` cuyos campos en offset `0x08` y `0x0A` transportan `a` y `b` en little-endian.
- **Auditoría de memoria y Kernel (2.19, 2.20 y 6.2):** si el parser copia el cuerpo del mensaje sin validar `body_len` contra el tamaño del buffer reservado en el heap, diagnosticamos el desborde con **Full Page Heap** y **WinDbg TTD** (`!heap -p -a` y `g-`), comparamos el binario vulnerable (`v1`) contra el corregido (`v2`) con **BinDiff / Version Tracking**, y verificamos cómo el servidor consulta la versión del subsistema mediante `DeviceIoControl(IOCTL_BLOCLAB_QUERY_VERSION = 0x80006000)` hacia el driver KMDF de laboratorio.

Aspecto observable	Estadio 1 · Principiante (`check_v1`)	Estadio 2 · Intermedio (`check_v2`)	Estadio 3 · Avanzado (`BlocLab`)
Punto de entrada a la lógica	`main` → `validar_pares` hallado por xref desde la string `"Correcto"`	OEP tras desempaquetar UPX + `tls_callback` en directorio `.tls`	Slot de `vtable` de `ValidadorBloc` invocado tras `ReadFile` del Named Pipe
Cómo llega la entrada	`scanf("%d", ...)` en variables locales del stack	Argumentos de consola + variable global condicionada por el TLS callback	Estructura binaria `CLSF` en el heap o buffer de `IRP` (`SystemBuffer` en KMDF)
Cómo se ven los textos	ASCII plano terminado en `0` en `.rdata`	Arreglo de bytes cifrado con `XOR 0x5A` descifrado en un bucle local	`std::string` con *Small String Optimization* (SSO, 16B unión + 8B size + 8B cap)
Herramienta que destraba el caso	IDA Free / Ghidra + breakpoint simple en x64dbg	PE-bear + x64dbg (breakpoint de memoria / Scylla) + traza condicional	RTTI en IDA/Ghidra + Frida + WinDbg (Page Heap / TTD) + revisión de `CTL_CODE`

Parte II · Kit de 5 plantillas de código mínimo (Boilerplates listos para compilar)

Todas las prácticas del curso están pensadas para ejecutarse sobre **programas propios en tu VM aislada** (Etapa 0). Para que no tengas que escribir desde cero cada escenario técnico, acá tenés cinco esqueletos autocontenidos de ~35 líneas en C y C++ estándar (también guardados como archivos sueltos en la carpeta `plantillas_codigo/` del paquete HTML). Compilalos en la VM con **Visual Studio Developer Command Prompt (`cl.exe`)** o con **MinGW-w64 (`gcc` / `g++`)**.

Plantilla 1 · `check_evolucion.c` (Práctica para 1.5, 2.4, 2.8, 2.10, 2.11, 2.15, 2.17 y 4.1)

Un único archivo en C que podés compilar en modo Principiante (por defecto) o en modo Intermedio (pasando `/DMODO_INTERMEDIO` o `-DMODO_INTERMEDIO`), lo que activa un **TLS Callback** que comprueba `IsDebuggerPresent()` y una rutina de descifrado **XOR** sobre el mensaje de éxito:

```c
// Archivo: plantillas_codigo/check_evolucion.c
// Compilar Principiante (MSVC): cl /Zi /Od check_evolucion.c /Fe:check_v1_O0.exe
// Compilar Optimizado   (MSVC): cl /Zi /O2 check_evolucion.c /Fe:check_v1_O2.exe
// Compilar Intermedio   (MSVC): cl /Zi /O1 /DMODO_INTERMEDIO check_evolucion.c /Fe:check_v2.exe
// Compilar en MinGW-w64 (GCC) : gcc -O0 -g check_evolucion.c -o check_v1_O0.exe
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>

static int g_objetivo = 480; // 0x1E0 en condiciones normales

#ifdef MODO_INTERMEDIO
static void NTAPI tls_antidebug(PVOID h, DWORD reason, PVOID res) {
    (void)h; (void)res;
    if (reason == DLL_PROCESS_ATTACH && IsDebuggerPresent()) {
        g_objetivo = 999; // Desvía la constante antes de entrar a main (Lecciones 2.11 y 2.15)
    }
}
#ifdef _MSC_VER
#pragma comment(linker, "/INCLUDE:_tls_used")
#pragma section(".CRT$XLB", long, read)
__declspec(allocate(".CRT$XLB")) PIMAGE_TLS_CALLBACK p_tls = tls_antidebug;
#else
PIMAGE_TLS_CALLBACK p_tls __attribute__((used, section(".CRT$XLB"))) = tls_antidebug;
#endif
#endif

int validar_pares(int a, int b) {
    if (a > 50 || b < 50) return 0;
    return ((a + b) * 4) == g_objetivo;
}

int main(int argc, char** argv) {
    int a = (argc > 2) ? atoi(argv[1]) : 0;
    int b = (argc > 2) ? atoi(argv[2]) : 0;
    if (argc <= 2) {
        printf("Primer valor: ");  if (scanf("%d", &a) != 1) return 1;
        printf("Segundo valor: "); if (scanf("%d", &b) != 1) return 1;
    }
    if (validar_pares(a, b)) {
#ifdef MODO_INTERMEDIO
        // "Correcto" cifrado con XOR 0x5A (Lecciones 2.10 y 4.2)
        unsigned char enc[] = {0x19, 0x35, 0x28, 0x28, 0x3F, 0x39, 0x2E, 0x35, 0x00};
        for (int i = 0; enc[i] != 0; ++i) enc[i] ^= 0x5A;
        puts((const char*)enc);
#else
        puts("Correcto");
#endif
        return 0;
    }
    puts("Incorrecto");
    return 1;
}
```

Plantilla 2 · `cpp_rtti_sso.cpp` (Práctica para 2.18 y 2D.11)

Genera en un solo binario: una jerarquía con herencia múltiple (para observar la `vtable`, el `_RTTICompleteObjectLocator` en `vtable[-1]` y un *adjustor thunk* `sub rcx, 8`), dos objetos `std::string` (uno corto en el buffer SSO de 16 bytes y otro largo en el heap), un `std::vector<int>` y un bloque `try/catch` con tablas en `.pdata`/`.xdata`:

```
// Archivo: plantillas_codigo/cpp_rtti_sso.cpp
// Compilar (MSVC FH4 actual): cl /EHsc /GR /Zi /O1 cpp_rtti_sso.cpp /Fe:cpp_fh4.exe
// Compilar (MSVC FH3 clásico): cl /EHsc /GR /Zi /O1 /d2FH4- cpp_rtti_sso.cpp /Fe:cpp_fh3.exe
// Compilar (MinGW ABI Itanium): g++ -O1 -g cpp_rtti_sso.cpp -o cpp_itanium.exe
#include <iostream>
#include <string>
#include <vector>
#include <stdexcept>
#include <typeinfo>

struct Componente {
    int id = 0x10;
    virtual ~Componente() = default;
    virtual int evaluar(int a, int b) const = 0;
};
struct Auditable {
    int nivel_log = 2;
    virtual void registrar(const char* evento) const { std::cout << "[LOG] " << evento << "\n"; }
};
class ValidadorBloc : public Componente, public Auditable {
    std::string etiqueta_corta = "BLOC_SSO";                        // < 16B: queda inline (SSO)
    std::string etiqueta_larga = "BLOC_CADENA_LARGA_EN_EL_HEAP_2026"; // >= 16B: reserva en heap
    std::vector<int> historial = {40, 80, 120};
public:
    int evaluar(int a, int b) const override {
        if (a < 0 || b < 0) throw std::runtime_error("Valores negativos no permitidos");
        return (a <= 50 && b >= 50 && (a + b) * 4 == (int)historial[2] * 4) ? 1 : 0;
    }
    void registrar(const char* ev) const override {
        std::cout << "[" << etiqueta_corta << "|" << etiqueta_larga.size() << "] " << ev << "\n";
    }
};

int main(int argc, char** argv) {
    Componente* obj = new ValidadorBloc();
    std::cout << "RTTI: " << typeid(*obj).name() << "\n";
    try {
        int res = obj->evaluar(argc > 1 ? std::stoi(argv[1]) : 40, 80);
        dynamic_cast<Auditable*>(obj)->registrar(res ? "OK" : "FALLO");
    } catch (const std::exception& ex) {
        std::cerr << "Excepcion capturada: " << ex.what() << "\n";
    }
    delete obj;
    return 0;
}
```

Plantilla 3 · `heap_bug.c` (Práctica para 2.19, 2.20 y 5.2)

Diseñada con un doble propósito: (1) activar **Full Page Heap** (`gflags /p /enable heap_bug_v1.exe /full`) y ver en **WinDbg** cómo el proceso se detiene exactamente en la instrucción que escribe más allá de los 16 bytes reservados (en vez de fallar más tarde en `free`), y (2) compilar una versión vulnerable (`heap_bug_v1.exe`) y una corregida con `/DPARCHE_SEGURO` (`heap_bug_v2.exe`) para comparar ambas en **BinDiff / Ghidra Version Tracking** (2.20):

```
// Archivo: plantillas_codigo/heap_bug.c
// Compilar v1 (vulnerable): cl /Zi /O1 heap_bug.c /Fe:heap_bug_v1.exe
// Compilar v2 (parcheada) : cl /Zi /O1 /DPARCHE_SEGURO heap_bug.c /Fe:heap_bug_v2.exe
// En la VM con Page Heap  : gflags /p /enable heap_bug_v1.exe /full
// Ejecutar en WinDbg      : heap_bug_v1.exe AAAAAAAAAAAAAAAAAAAA  --> !heap -p -a @rcx
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char etiqueta[12];
    unsigned int estado;
} RegistroBloc; // sizeof(RegistroBloc) == 16 bytes exactos

int procesar_entrada(const char* entrada) {
    size_t len = strlen(entrada);
    RegistroBloc* reg = (RegistroBloc*)malloc(sizeof(RegistroBloc));
    if (!reg) return -1;
    reg->estado = 0xCAFE0001u;

#ifdef PARCHE_SEGURO
    // Versión v2 corregida: valida el límite antes de copiar (Lecciones 2.20 y 5.2)
    if (len >= sizeof(reg->etiqueta)) {
        puts("Error: entrada excede el buffer de 12 bytes");
        free(reg);
        return 1;
    }
    memcpy(reg->etiqueta, entrada, len + 1);
#else
    // Versión v1 con defecto didáctico: copia hasta 20 bytes en un bloque de 16 bytes (Lección
2.19)
    if (len > 20) len = 20;
    memcpy(reg->etiqueta, entrada, len);
    reg->etiqueta[11] = '\0';
#endif
    printf("Registro: %s | Estado: 0x%08X\n", reg->etiqueta, reg->estado);
    free(reg);
    return 0;
}

int main(int argc, char** argv) {
    const char* dato = (argc > 1) ? argv[1] : "BLOC_NORMAL";
    return procesar_entrada(dato);
}
```

Plantilla 4 · `parser_clsf.c` (Práctica para 4.1, 4.5 y 2.24)

Implementa el escritor y el lector del formato binario sintético **`CLSF`** de la lección 4.5 (12 bytes de cabecera + cuerpo variable). Si lo ejecutás con `--gen muestra.clsf`, genera la muestra exacta de 17 bytes vista en la teoría; si le pasás un archivo, valida cada campo y devuelve un código de estado distinto por cada fallo (ideal para confirmación diferencial en 4.1/4.5 y como *harness* en 2.24):

```
// Archivo: plantillas_codigo/parser_clsf.c
// Compilar (MSVC) : cl /Zi /O1 parser_clsf.c /Fe:parser_clsf.exe
// Compilar (MinGW): gcc -O1 -g parser_clsf.c -o parser_clsf.exe
// Generar muestra : parser_clsf.exe --gen muestra_A.clsf
// Analizar muestra: parser_clsf.exe muestra_A.clsf
#include <stdio.h>
#include <stdint.h>
#include <string.h>

#pragma pack(push, 1)
typedef struct {
    uint8_t  magic[4];   // +0x00: "CLSF" (43 4C 53 46)
    uint8_t  version;    // +0x04: 1
    uint8_t  flags;      // +0x05: 0
    uint16_t hdr_len;    // +0x06: 12 (0x000C LE)
    uint16_t body_len;   // +0x08: longitud del cuerpo (LE)
    uint16_t seq_id;     // +0x0A: campo variable / secuencia (0x002A = 42 LE)
} CabeceraCLSF;          // Total cabecera: 12 bytes
#pragma pack(pop)

int main(int argc, char** argv) {
    if (argc == 3 && strcmp(argv[1], "--gen") == 0) {
        CabeceraCLSF h = {{'C','L','S','F'}, 1, 0, 12, 5, 42};
        FILE* f = fopen(argv[2], "wb");
        if (!f) return 1;
        fwrite(&h, sizeof(h), 1, f);
        fwrite("HELLO", 1, 5, f);
        fclose(f);
        puts("Muestra CLSF de 17 bytes generada.");
        return 0;
    }
    if (argc < 2) { puts("Uso: parser_clsf [--gen salida.clsf | entrada.clsf]"); return 1; }
    FILE* f = fopen(argv[1], "rb");
    if (!f) return 2;
    uint8_t buf[256] = {0};
    size_t n = fread(buf, 1, sizeof(buf), f);
    fclose(f);
    if (n < sizeof(CabeceraCLSF)) { puts("Error: archivo truncado"); return 3; }
    const CabeceraCLSF* h = (const CabeceraCLSF*)buf;
    if (memcmp(h->magic, "CLSF", 4) != 0) { puts("Error: magic invalido"); return 4; }
    if (h->version != 1)                  { puts("Error: version no soportada"); return 5; }
    if (h->hdr_len != 12)                 { puts("Error: hdr_len != 12"); return 6; }
    if ((size_t)h->hdr_len + h->body_len > n) { puts("Error: body_len excede archivo"); return 7;
}
    printf("CLSF OK | ver=%u flags=0x%02X seq=%u body(%u)=\"%.*s\"\n",
           h->version, h->flags, h->seq_id, h->body_len, h->body_len, (const char*)(buf + h-
>hdr_len));
    return 0;
}
```

Plantilla 5 · `pipe_server_min.c` (Práctica para 2D.1, 2D.10, 4.5 y 2.21)

Servidor local de **Named Pipe** en modo mensaje que aplica las banderas defensivas estudiadas en 2D.10 (`FILE_FLAG_FIRST_PIPE_INSTANCE` y `PIPE_REJECT_REMOTE_CLIENTS`) e implementa la máquina de estados de 4.5 (`ESPERA_SALUDO → LISTO`). Podés conectarte desde un script de Python en la misma VM e inspeccionar las llamadas con Process Explorer, WinDbg o Frida:

```
// Archivo: plantillas_codigo/pipe_server_min.c
// Compilar (MSVC) : cl /Zi /O1 pipe_server_min.c /Fe:pipe_server_min.exe
// Compilar (MinGW): gcc -O1 -g pipe_server_min.c -o pipe_server_min.exe
// Probar en Python: open(r'\\.\pipe\BlocLabPipe', 'r+b', buffering=0)
#include <windows.h>
#include <stdio.h>
#include <string.h>

int main(void) {
    const wchar_t* nombre = L"\\\\.\\pipe\\BlocLabPipe";
    HANDLE hPipe = CreateNamedPipeW(
        nombre,
        PIPE_ACCESS_DUPLEX | FILE_FLAG_FIRST_PIPE_INSTANCE,
        PIPE_TYPE_MESSAGE | PIPE_READMODE_MESSAGE | PIPE_WAIT | PIPE_REJECT_REMOTE_CLIENTS,
        1, 256, 256, 0, NULL);
    if (hPipe == INVALID_HANDLE_VALUE) {
        printf("CreateNamedPipeW fallo (GLE=%lu)\n", GetLastError());
        return 1;
    }
    puts("[ESPERA_SALUDO] Escuchando en \\\\.\\pipe\\BlocLabPipe ...");
    if (ConnectNamedPipe(hPipe, NULL) || GetLastError() == ERROR_PIPE_CONNECTED) {
        char buf[128] = {0};
        DWORD leidos = 0, escritos = 0;
        if (ReadFile(hPipe, buf, sizeof(buf) - 1, &leidos, NULL) && leidos >= 4 && memcmp(buf,
"CLSF", 4) == 0) {
            puts("[LISTO] Saludo CLSF valido recibido; enviando respuesta fija.");
            const char resp[] = "CLSF_ACK_V1_READY";
            WriteFile(hPipe, resp, (DWORD)strlen(resp), &escritos, NULL);
        } else {
            const char err[] = "CLSF_ERR_BAD_HELLO";
            WriteFile(hPipe, err, (DWORD)strlen(err), &escritos, NULL);
        }
        FlushFileBuffers(hPipe);
        DisconnectNamedPipe(hPipe);
    }
    CloseHandle(hPipe);
    return 0;
}
```

Hojas de referencia rápida por nivel (Cheat Sheets)

Estas tres hojas condensan las tablas, fórmulas, desplazamientos de memoria, constantes mágicas y comandos de depuración de todo el curso, organizadas por nivel (**Principiante**, **Intermedio** y **Avanzado**) para tenerlas abiertas al lado del desensamblador o impresas en el escritorio.

Hoja 1 · Nivel Principiante (Etapas 0, 1, 2A, 3 y 4 base)

1.1 Registros x64 / x86 y regla de solapamiento (Lección 2.1)

64 bits	32 bits	16 bits	8 bits (alto / bajo)	Uso convencional en Windows x64 / x86	Preservación (ABI x64)
`RAX`	`EAX`	`AX`	`AH` / `AL`	Valor de retorno entero o puntero; acumulador aritmético	Volátil (lo pisa la función llamada)
`RCX`	`ECX`	`CX`	`CH` / `CL`	**1.er argumento en x64**; `this` en MSVC; contador de turnos/bucles	Volátil
`RDX`	`EDX`	`DX`	`DH` / `DL`	**2.º argumento en x64**; parte alta en `MUL` / `DIV`	Volátil
`RBX`	`EBX`	`BX`	`BH` / `BL`	Registro general preservado (suele guardar punteros o variables de bucle)	**No volátil** (debe restaurarse)
`RSP`	`ESP`	`SP`	`SPL`	Puntero al tope actual del stack (crece hacia direcciones menores)	**No volátil** (alineado a 16B antes de `CALL`)
`RBP`	`EBP`	`BP`	`BPL`	Puntero de marco de función (en x86: argumentos en `[ebp+8...]`, locales en `[ebp-...]`)	**No volátil**
`RSI` / `RDI`	`ESI` / `EDI`	`SI` / `DI`	`SIL` / `DIL`	Origen y destino en operaciones de bloques (`rep movsb`); variables locales	**No volátiles** en Windows
`R8`, `R9`	`R8D`, `R9D`	`R8W`, `R9W`	`R8B`, `R9B`	**3.er y 4.º argumento entero/puntero en x64**	Volátiles
`R10`, `R11`	`R10D`, `R11D`	`R10W`, `R11W`	`R10B`, `R11B`	Temporales de llamada / `syscall`	Volátiles
`R12` – `R15`	`R12D` – `R15D`	`R12W` – `R15W`	`R12B` – `R15B`	Registros de propósito general de larga vida en la función	**No volátiles**

Regla de oro en x64: Escribir en un subregistro de **32 bits** (p. ej. `mov eax, 1`) **pone en cero los 32 bits superiores** de `RAX`. Escribir en **16 u 8 bits** (`ax`, `al`) **conserva intactos** los bits restantes.

1.2 Flags y saltos condicionales tras `CMP op1, op2` (Lección 2.4)

Relación lógica (`op1` vs `op2`)	Salto SIN signo (`unsigned` / direcciones)	Salto CON signo (`int` / complemento a 2)	Condición exacta en los flags (`EFLAGS`)
Igual (`op1 == op2`) / Cero	`JE` / `JZ`	`JE` / `JZ`	`ZF = 1` (tras `test eax, eax`: salta si `eax == 0`)
Distinto (`op1 != op2`) / No cero	`JNE` / `JNZ`	`JNE` / `JNZ`	`ZF = 0`
Mayor estricto (`op1 > op2`)	`JA` / `JNBE` (*Above*)	`JG` / `JNLE` (*Greater*)	Sin signo: `CF = 0 y ZF = 0` · Con signo: `ZF = 0 y SF == OF`
Mayor o igual (`op1 >= op2`)	`JAE` / `JNB` / `JNC`	`JGE` / `JNL`	Sin signo: `CF = 0` · Con signo: `SF == OF`
Menor estricto (`op1 < op2`)	`JB` / `JNAE` / `JC` (*Below*)	`JL` / `JNGE` (*Less*)	Sin signo: `CF = 1` · Con signo: `SF != OF`
Menor o igual (`op1 <= op2`)	`JBE` / `JNA`	`JLE` / `JNG`	Sin signo: `CF = 1 o ZF = 1` · Con signo: `ZF = 1 o SF != OF`

1.3 Conversión VA ↔ RVA ↔ File Offset y convenciones de llamada (Lecciones 1.2, 1.4 y 2.5)

```
RVA         = VA_en_memoria - ImageBase_actual
File Offset = RVA - VirtualAddress_sección + PointerToRawData_sección
VA_en_IDA   = Base_en_IDA + (VA_en_x64dbg - Base_en_x64dbg)   [Sincronización rápida con ASLR]
```

Convención	Arquitectura	Dónde van los primeros argumentos	Quién limpia el stack	Detalle clave en el desensamblado
`cdecl`	x86 (32 bits)	Todos en el stack (de derecha a izquierda; el 1.º en `[esp+4]` tras `call`)	El **llamador** (`add esp, N` después del `call`)	Permite funciones variádicas (`printf`, `scanf`).
`stdcall`	x86 (32 bits)	Todos en el stack (de derecha a izquierda)	La **función llamada** (`ret N`, p. ej. `ret 10h` = 4 args)	Estándar de la API Win32 de 32 bits (`MessageBoxA`, `WindowProc`).
`fastcall`	x86 (32 bits)	1.º en `ECX`, 2.º en `EDX`; del 3.º en adelante en el stack	La función llamada (`ret N` para los del stack)	En **Delphi x86 (`register`)**: 1.º `EAX`, 2.º `EDX`, 3.º `ECX` (2.13).
`thiscall`	x86 (MSVC)	Puntero `this` en `ECX`; argumentos explícitos en el stack	La función llamada (`ret N`)	En GCC/MinGW x86, `this` se apila como 1.er argumento oculto en el stack.
Microsoft x64	x64 (64 bits)	1.º `RCX` (o `XMM0`), 2.º `RDX` (`XMM1`), 3.º `R8` (`XMM2`), 4.º `R9` (`XMM3`); 5.º+ en `[rsp+20h]`	El llamador (reserva marco fijo en su prólogo y libera en el epílogo)	El llamador reserva siempre **32 bytes (`0x20`) de *Shadow Space*** en `[rsp+8..rsp+20h]` tras el `call`.

Hoja 2 · Nivel Intermedio (Etapas 2B, 2.17, 2.21, 4.2, 4.5, 5.1, 5.3 y 2D.1–2D.5)

2.1 Offsets de `PEB`, `PEB_LDR_DATA` y `LDR_DATA_TABLE_ENTRY` (Lecciones 2.14, 2.15 y 2D.2)

Estructura / Campo	x86 (32 bits)	x64 (64 bits)	Desde `InMemoryOrderLinks` (x86 / x64)	Para qué lo usa el binario o el analista
Puntero al `TEB` / al `PEB`	`fs:[0x18]` / `fs:[0x30]`	`gs:[0x30]` / `gs:[0x60]`	—	Punto de partida de resolución de APIs y chequeos anti-debug
`PEB.BeingDebugged` (1 byte)	`+0x02`	`+0x02`	—	Leído por `IsDebuggerPresent()` (`1` si hay debugger user-mode)
`PEB.Ldr` (`PEB_LDR_DATA*`)	`+0x0C`	`+0x18`	—	Encabezado de las 3 listas circulares de DLLs cargadas
`PEB.ProcessParameters`	`+0x10`	`+0x20`	—	Contiene `ImagePathName` y `CommandLine`
`PEB.NtGlobalFlag` (`DWORD`)	`+0x68`	`+0xBC`	—	Vale `0x70` si el proceso fue creado por un debugger sin Page Heap
`Ldr.InLoadOrderModuleList`	`+0x0C`	`+0x10`	—	Lista en orden de carga (`Flink` apunta a `+0x00` del entry)
`Ldr.InMemoryOrderModuleList`	`+0x14`	`+0x20`	—	Lista más usada en shellcode/PEB walk (`Flink` apunta a `+0x08` / `+0x10`)
`LDR_DATA_TABLE_ENTRY.DllBase`	**`+0x18`**	**`+0x30`**	**`+0x10` (x86) / `+0x20` (x64)**	Base de la DLL en memoria (cabecera `MZ` para ir a la tabla de Exports)
`LDR_DATA_TABLE_ENTRY.FullDllName`	`+0x24`	`+0x48`	`+0x1C` (x86) / `+0x38` (x64)	`UNICODE_STRING` con la ruta completa de la DLL
`LDR_DATA_TABLE_ENTRY.BaseDllName`	`+0x2C`	`+0x58`	`+0x24` (x86) / `+0x48` (x64)	`UNICODE_STRING` con el nombre corto (p. ej. `kernel32.dll`; buffer en `+4` / `+8`)

2.2 Constantes mágicas de criptografía, hashing y encoding (Lección 4.2)

Algoritmo	Constantes / Bytes característicos en memoria o en instrucciones	Patrón estructural en el código desensamblado
AES (Rijndael)	S-box: `63 7C 77 7B F2 6B 6F C5...` · Rcon: `01 02 04 08 10 20 40 80 1B 36`	Tablas de 256 bytes (o T-tables de 1 KB) o instrucciones nativas `aesenc` / `aesdec` / `aeskeygenassist`.
ChaCha20 / Salsa20	ASCII `"expand 32-byte k"` (`0x61707865, 0x3320646e, 0x79622d32, 0x6b206574`)	Sin tablas; estado de 16 `DWORD`s (64B) y 20 rondas ARX (`add`, `xor`, `rol` de **16, 12, 8 y 7 bits**).
RC4	Sin constantes fijas; arreglo `S[256]` inicializado con `0x00..0xFF`	Dos bucles de 256 iteraciones (KSA con módulo `& 0xFF` e intercambio de bytes + PRGA con `xor` al flujo).
TEA / XTEA	Delta áureo: `0x9E3779B9` (o resta de `0x61C88647`); suma inicial `0xC6EF3720` (32 rondas)	Bucle de 32 o 64 iteraciones que combina `(v << 4) ^ (v >> 5)` y suma el delta en cada vuelta.
MD5 / SHA-1	IV: `0x67452301`, `0xEFCDAB89`, `0x98BADCFE`, `0x10325476` (+ `0xC3D2E1F0` en SHA-1)	Procesamiento en bloques de 64 bytes (64 pasos en MD5; 80 pasos con rotaciones en SHA-1).
SHA-256	IV: `0x6A09E667`, `0xBB67AE85`, `0x3C6EF372`, `0xA54FF53A...` · `K[0] = 0x428A2F98`	Tabla `K[64]` de 64 `DWORD`s y 64 rondas sobre 8 registros de estado (`a..h`).
CRC32	Polinomio reflejado: `0xEDB88320` (segunda entrada de tabla: `0x77073096`), IV `0xFFFFFFFF`	Bucle por byte con `crc = (crc >> 8) ^ tabla[(crc ^ byte) & 0xFF]` y `not crc` al final.
Base64	Tabla de 64 caracteres `"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/"` y `'='`	Toma grupos de 3 bytes (24 bits) y los divide en 4 índices de 6 bits (`>> 18`, `>> 12`, `>> 6`, `& 0x3F`).

2.3 Expresiones de x64dbg, scripting y Frida (Lecciones 2.6 y 2.21)

```
# Breakpoint condicional universal (x86/x64) en WindowProc para botón ID 0x13A (WM_COMMAND = 0x111):
breakif:  arg.get(1) == 0x111 && (arg.get(2) & 0xFFFF) == 0x13A
log:      "WM_COMMAND control={arg.get(2)&0xFFFF} notif={arg.get(2)>>16} hwnd={arg.get(0)}"

# Formateadores de log sin detener la ejecución (Break Condition = 0) en x64dbg:
ASCII: {s:rcx}   |   UTF-16: {utf16@rcx}   |   Bytes hex: {mem;16@rcx}   |   Retorno: {rax}

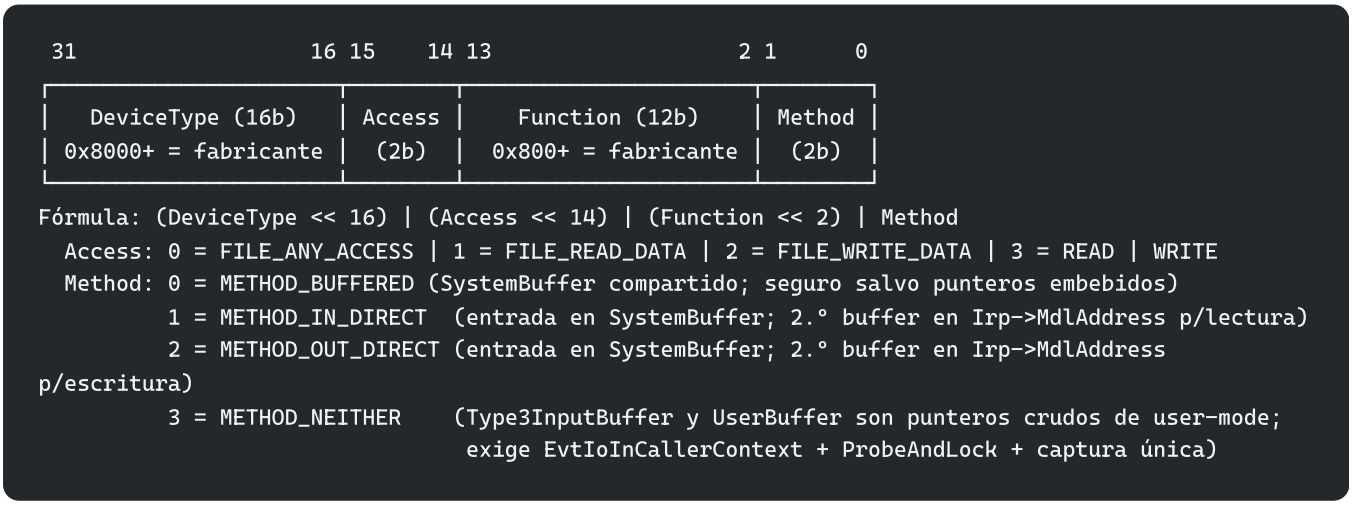
# Plantilla mínima de Frida (guardar como hook.js y correr: frida -f app.exe -l hook.js):
const addr = DebugSymbol.fromName("CreateFileW").address; // o base.add(0x1420)
Interceptor.attach(addr, {
  onEnter(args) { this.path = args[0].readUtf16String(); },
  onLeave(retval) { console.log("CreateFileW:", this.path, "-> handle:", retval); }
});
```

Hoja 3 · Nivel Avanzado (Etapas 2.18–2.20, 2.24–2.33, 5.2, 2D.6–2D.13 y Etapa 6)

3.1 Layout en memoria de C++ en MSVC x64: RTTI, SSO, STL y excepciones (Lección 2.18)

Estructura C++ (MSVC x64 Release)	Tamaño total	Desplazamientos y campos internos	Cómo reconocerla en el listado
Objeto con métodos virtuales + RTTI	`8B + campos`	`+0x00`: `vptr` (apunta a `vtable[0]` en `.rdata`). En `vtable[-1]` (`[vptr − 8]`) hay un puntero a `_RTTICompleteObjectLocator`: `+0x00 signature (1)`, `+0x04 offset`, `+0x0C pTypeDescriptor (RVA)`, `+0x10 pClassDescriptor (RVA)`, `+0x14 pSelf (RVA)`.	Llamada `mov rax, [rcx] ; call qword ptr [rax + 8*i]`. Sumar `ImageBase` a `pTypeDescriptor` lleva a `".?AVNombreClase@@"`.
`std::string` (SSO)	`32 bytes` (`0x20`)	`+0x00..+0x0F`: unión `_Bx` (16B buffer local `_Buf` si `_Myres < 16`, o puntero `_Ptr` de 8B en `+0x00` si `_Myres >= 16`) · `+0x10`: `_Mysize` (8B) · `+0x18`: `_Myres` (8B, capacidad).	Comparación `cmp qword ptr [rcx+18h], 10h` seguida de un `cmovae rcx, [rcx]` antes de leer los caracteres.
`std::wstring` (SSO UTF-16)	`32 bytes` (`0x20`)	Igual layout de 32B, pero cada carácter ocupa 2 bytes: el umbral de SSO es `_Myres < 8` caracteres (`cmp [rcx+18h], 8`).	`cmp qword ptr [rcx+18h], 8` antes de elegir entre buffer inline y puntero `_Ptr`.
`std::vector<T>`	`24 bytes` (`0x18`)	`+0x00`: `_Myfirst` (inicio) · `+0x08`: `_Mylast` (fin de usados) · `+0x10`: `_Myend` (fin de reserva).	Cantidad de elementos = `(_Mylast − _Myfirst) / sizeof(T)`; bucle que itera desde `[rcx]` hasta `[rcx+8]`.
Excepciones x64 (`.pdata` / `.xdata`)	12B por entrada `.pdata`	`RUNTIME_FUNCTION`: 3 RVAs de 4B (`BeginAddress`, `EndAddress`, `UnwindData`). En `.xdata`, el handler apunta a `__CxxFrameHandler3` (`FuncInfo` con firma `0x19930520`) o `__CxxFrameHandler4` (`FuncInfo4` comprimido en VS 2019 16.3+/2022).	No hay cadena `FS:[0]` en el prólogo x64; las tablas viven fuera del flujo de código en `.pdata` y `.xdata`.

3.2 Desglose de `CTL_CODE` (32 bits) y métodos de IOCTL en drivers KMDF/WDM (Lecciones 6.2 y 6.4)



3.3 Niveles de `IRQL` y comandos esenciales de WinDbg (Lecciones 2.19, 2D.2, 2D.11, 6.1 y 6.3)

IRQL	Contexto típico de ejecución	Memoria permitida	Sincronización y esperas permitidas
`PASSIVE_LEVEL (0)`	Hilos de usuario, `DriverEntry`, `EvtDriverDeviceAdd`, mayoría de callbacks de archivos/registro	`PagedPool` y `NonPagedPoolNx`	Puede esperar (`KeWaitForSingleObject` con timeout != 0), usar mutexes/eventos y tocar código/datos paginables.
`APC_LEVEL (1)`	Regiones críticas / Guarded mutexes (`Fast Mutex`)	`PagedPool` y `NonPagedPoolNx`	APCs normales de hilo deshabilitadas; page faults aún permitidos.
`DISPATCH_LEVEL (2)`	DPCs (`EvtInterruptDpc`), código bajo `KSPIN_LOCK` (`KeAcquireSpinLock`)	**Solo** `NonPagedPoolNx` (tocar memoria paginada = bugcheck `0x0A` / `0xD1`)	**Prohibido esperar** con intervalo no nulo (solo timeout = 0 y spinlocks). El despachador de hilos no puede cambiar de hilo en ese procesador.
`DIRQL (> 2)`	Rutinas de servicio de interrupción de hardware (`ISR` / `EvtInterruptIsr`)	**Solo** `NonPagedPoolNx`	Solo spinlocks de interrupción en su propio `DIRQL`; delegar el trabajo pesado a un DPC (`DISPATCH_LEVEL`).

```
# 1. Triage de excepción o volcado (User-mode / Kernel-mode – Lecciones 2.19, 2D.11 y 6.3):
!analyze -v                  ; análisis automático del fallo / bugcheck
.exr -1                      ; muestra el EXCEPTION_RECORD actual (código y parámetros)
.ecxr                        ; restaura el contexto de registros en el instante exacto de la
excepción
k   /   kv                   ; stack trace actual   |   ln <addr> : símbolo más cercano a una
dirección

# 2. Diagnóstico de Heap, Memoria, PEB y WOW64 (Lecciones 2.19 y 2D.2):
!heap                        ; lista los heaps del proceso e indica Type (NT Heap vs Segment Heap)
!heap -p -a <addr>           ; con Full Page Heap activo: muestra el call stack que reservó/liberó
el bloque
!vadump                      ; lista todas las regiones VAD en user-mode (Image / Mapped /
Private)
!peb   /   !teb   /   lm     ; inspecciona PEB, TEB y lista de módulos cargados
!wow64exts.sw  /  !wow64exts.k ; alterna contexto 32/64 bits y muestra pila combinada en procesos
WOW64

# 3. Time-Travel Debugging (TTD) y Kernel/WDF (Lecciones 2.19, 6.2 y 6.3):
g-   /   p-   /   t-         ; ejecuta hacia atrás / paso atrás sobre una traza TTD (.run)
ba w4 <addr>  ;  g-          ; frena en la instrucción pasada exacta que sobrescribió esos 4 bytes
dx @$cursession.TTD.Calls("ntdll!NtCreateFile")  ; consulta todas las llamadas a una API en la
traza
!drvobj <driver> 7           ; muestra el DRIVER_OBJECT y su tabla de rutinas MajorFunction
(IRP_MJ_*)
!wdfkd.wdflogdump <driver>   ; vuelca el In-flight Recorder (IFR) de un driver KMDF
```

3.4 · Desofuscación y análisis dinámico avanzado (Bloque 2E, Lecciones 2.25–2.33)

Tarea	Herramienta y comando / API clave	Lección
Resolver una condición / clave	**angr**: `simgr.explore(find=…, avoid=…)`. En Windows arrancá en el prólogo de la validación (`entry_state(addr=…)`) para no modelar el CRT.	2.25
Demostrar que un MBA es una op simple	**Z3**: `prove((x ^ y) + 2*(x & y) == x + y)` → `proved`.	2.25
Reconocer una VM de protección	Bucle *dispatcher* + tabla de *handlers* + VIP. Ojo con *threaded code*: handlers encadenados (`jmp [tabla+rbx*8]`) sin despachador central.	2.26
Identificar packer / protector	**DIE** (`diec --entropy`): sección con entropía ~7,9 = comprimida/cifrada; IAT mínima o vacía (*splicing*/emulación de APIs) = empaquetado.	2.27
Neutralizar anti-debug (en VM)	**ScyllaHide** (x64dbg) o hook de **Frida** sobre `IsDebuggerPresent`, PEB `BeingDebugged`, `rdtsc` y DRx (vía `NtGetContextThread` / VEH).	2.28
Cobertura (colorear caminos)	**DynamoRIO**: `drrun -t drcov -- app.exe` → abrir en Lighthouse. **Frida**: `frida-drcov` / Stalker `compile`.	2.29 / 2.30
Traza de instrucciones (reconstruir algoritmo)	**Pin**: `pin -t pinatrace.dll -- app.exe`; **DynamoRIO**: `drcachesim`; Frida Stalker `exec`.	2.29 / 2.30
OEP genérico (sin conocer el packer)	Regla **W⊕X**: avisar cuando se ejecuta una página escrita antes (DBI nativa o Stalker).	2.29 / 2.30
Modelar un bytecode en Ghidra	**SLEIGH** (`.slaspec`) + **PyGhidra** (Python 3, estándar en Ghidra 11+; podés usar `z3-solver` en el script).	2.31
Recuperar nombres Go / Rust	**GoReSym** (pclntab de un Go *stripped*); **rustfilt** (demangle v0 `_R…`); los *panics* (`panic_bounds_check`) revelan tamaños de buffer.	2.32
VB6 en p-code	**VB Decompiler** (IDA/Ghidra solo muestran el intérprete de `MSVBVM60`, no la lógica).	2.33
Protocolo propietario	Hook en `send` / `recv` (texto claro) + disector **Lua** de Wireshark (`Proto` / `ProtoField`); reensamblar TCP con `pinfo.desegment_len`.	4.6

Glosario

Términos que aparecen en el curso, explicados en lenguaje sencillo. Donde el concepto se desarrolla en una lección, hay un enlace. No es una enciclopedia: es el vocabulario mínimo para leer las lecciones sin frenarte.

A–C

Término	Explicación
ABI Itanium (C++)	Especificación de cómo GCC, Clang y MinGW organizan vtables, RTTI, nombres (`_ZTV…`) y excepciones; cada plataforma la adapta. Distinta del modelo de MSVC. Ver C++ profundo.
Address (dirección)	Número que identifica una posición de memoria. En un debugger se escribe en hexadecimal.
Adjustor thunk	Fragmento de código en una vtable que corrige `this` (por ejemplo, `sub rcx, 10h` y un salto) antes de llegar al método real; aparece con herencia múltiple. Ver C++ profundo.
AL / AH / AX	Partes de `EAX`: `AL` es el byte bajo, `AH` el byte alto y `AX` los dos bytes bajos. Ver registros.
Anti-debug	Conjunto de técnicas con las que un programa detecta que está siendo depurado. Se estudia para reconocerlo, no para evadirlo. Ver debugger.
API	Función que ofrece un sistema operativo o una biblioteca para que otro programa la use (`MessageBoxA`, `CreateFileW`).
API hashing	Técnica que localiza funciones comparando un hash de cada nombre exportado contra una constante, para no dejar el nombre de la API como cadena. Ver PE fino.
Aplanamiento del flujo de control	Ofuscación que pone todos los bloques de una función dentro de un bucle con un `switch` sobre una variable de estado. Ver Ofuscación.
Argumento	Valor que se pasa a una función. En 32 bits suele ir en el stack; en x64, los primeros cuatro en registros.
ASLR	*Address Space Layout Randomization*: técnica que carga las imágenes en direcciones distintas en cada ejecución. Ver PE.
Assembly	Representación legible de las instrucciones de la CPU. Cada línea suele ser una operación con sus operandos.
Big-endian	Orden en que el byte más significativo se guarda primero. x86/x64 usa little-endian.
Binario	Sistema de numeración en base 2, con 0 y 1. Un byte son 8 bits. Ver números.
BinDiff / BinExport	Differ de binarios de código abierto; trabaja sobre exportaciones en formato BinExport generadas desde IDA, Binary Ninja o Ghidra. Ver Patch diffing.
Bit	Unidad mínima de información: 0 o 1.
Breakpoint	Pausa que se pone en el programa para observar su estado. Puede ser por software, hardware o memoria. Ver debugger.
Byte	8 bits. Unidad básica de almacenamiento.
C	Lenguaje compilado de bajo nivel, muy usado para programas que se analizan con debuggers.
C#	Lenguaje de .NET que se compila a IL. Ver .NET.
Cadena armada en el stack	Texto que el programa escribe de a pedazos con `mov` de constantes en vez de guardarlo en `.rdata`; no aparece en la ventana de strings. Ver IDA.
CALL	Instrucción que llama a una función: empuja la dirección de retorno y salta. Ver llamadas.

Término	Explicación
Canary (stack canary)	Valor aleatorio que el compilador coloca antes del return address para detectar desbordes. Ver vulnerabilidades.
CF (carry flag)	Flag que indica acarreo o desborde en una operación sin signo.
CMP	Instrucción que resta dos valores, actualiza flags y descarta el resultado. Se usa antes de un salto condicional.
Código administrado	Código que corre bajo un runtime (como el CLR de .NET) que administra memoria y ejecución.
Código nativo	Código máquina específico de la CPU, generado por un compilador como C/C++.
Complete Object Locator (COL)	Estructura de RTTI de MSVC a la que apunta `vtable[-1]`; lleva al nombre de la clase. En x64 usa RVA. Ver C++ profundo.
Complemento a dos	Convención para representar negativos: se invierten los bits y se suma 1. `0xFB` = −5 en 8 bits. Ver números.
Constante	Valor fijo que no cambia durante la ejecución. En assembly aparece como inmediato (`mov eax, 50`).
CPU	Unidad central de proceso. Ejecuta instrucciones y tiene registros.
Crackme	Programa pequeño diseñado para practicar reversing, generalmente con un objetivo didáctico.
CS / DS / SS	Registros de segmento de 16 bits. En 32/64 bits casi no se usan, salvo `FS` / `GS` para estructuras del sistema.

D–H

Término	Explicación
DWORD	Doble WORD: 4 bytes (32 bits).
Debugger	Programa que ejecuta otro paso a paso, con pausas y vista de registros y memoria. En el curso se usa x64dbg.
Decompilador	Herramienta que reconstruye una vista legible aproximada a partir de un binario o del IL. No recupera el código original exacto.
Desplazamiento (displacement)	Distancia que se suma a una dirección para calcular un destino o un campo. En un `call` / `jmp` relativo, se mide desde la instrucción siguiente.
DLL	Biblioteca de código que otros programas cargan en tiempo de ejecución (`.dll` en Windows).
DR0–DR3	Registros de depuración del procesador que sostienen breakpoints de hardware. Hay cuatro por hilo.
EAX / RAX	Registro acumulador; suele guardar el resultado de una función.
EBP / RBP	Base pointer: referencia estable del marco de pila en funciones de 32 bits.
EBX / RBX	Registro de propósito general; a veces apunta a datos.
ECX / RCX	Registro contador; se usa en bucles, shifts variables y `rep`. En x64 es el primer argumento.
EDI / RDI	Destination index: registro de destino en copias de memoria (`movs`, `stos`).
EDX / RDX	Registro de datos; parte alta en multiplicaciones/divisiones. En x64 es el segundo argumento.
EIP / RIP	Instruction pointer: dirección de la próxima instrucción a ejecutar.
Entropía	Medida de cuán aleatorios son unos bytes (0 a 8 bits por byte); cerca de 8 sugiere datos comprimidos o cifrados. Ver Packers.
Entry point	Primera instrucción del código de un programa; no es necesariamente `main`.
ESI / RSI	Source index: origen en copias de memoria.
ESP / RSP	Stack pointer: apunta a la cima del stack.
Excepción de primera / segunda oportunidad	Avisos del depurador antes de buscar handlers (primera) y cuando nadie la manejó (segunda). Solo la segunda indica un fallo no manejado. Ver 2D.11.
Export	Función o símbolo que una DLL ofrece a otros programas.
FIFO	*First in, first out*: lo primero que entra es lo primero que sale (una cola).
Flag	Bit de estado que resume el resultado de una operación (`ZF`, `SF`, `CF`, `OF`). Ver registros.
Frida	Herramienta de instrumentación dinámica que inyecta JavaScript en un proceso para enganchar funciones (`Interceptor.attach`). Ver Scripting de debugger.

Término	Explicación
Función	Bloque de código con un nombre, parámetros y un valor de retorno.
Handle	Identificador opaco que el sistema da para referenciar un recurso (archivo, ventana, hilo).
Heap	Zona de memoria dinámica, administrada por `malloc` / `new` o por el runtime.
Hexadecimal	Base 16, con dígitos 0–9 y A–F. Cada dígito representa 4 bits.
Hipervisor	Programa que crea y administra máquinas virtuales (VirtualBox, VMware, Hyper-V).

I–O

Término	Explicación
IAT	*Import Address Table*: tabla de punteros que el cargador completa con las direcciones de las funciones importadas. Ver PE.
IDA	Desensamblador y debugger interactivo. IDA Free es la versión gratuita. Ver IDA.
IL / MSIL	Lenguaje intermedio de .NET, compilado a máquina por el JIT. Ver .NET.
Image base	Dirección preferida en la que se carga una imagen. ASLR puede cambiarla.
Import	Función o símbolo que un programa toma de una DLL.
Instrucción	Operación elemental que ejecuta la CPU (`mov`, `add`, `jmp`).
JIT	*Just-in-time compiler*: compila IL a código nativo en tiempo de ejecución.
JMP	Salto incondicional.
Kernel	Núcleo del sistema operativo, con el máximo privilegio. Fuera del alcance de esta edición. Ver fuera de alcance.
LEA	*Load effective address*: calcula una dirección sin leer memoria.
LIFO	*Last in, first out*: lo último que entra es lo primero que sale (una pila).
Little-endian	Orden en que el byte menos significativo se guarda primero. Es el de x86/x64. Ver números.
Malware	Software diseñado para dañar, espiar o tomar control. Se estudia de forma defensiva. Ver malware.
Marco de pila (stack frame)	Región del stack de una función: argumentos, return address, `EBP` guardado y variables locales.
Memoria virtual	Espacio de direcciones que el sistema presenta a cada proceso, independiente de la memoria física.
Memoria W+X	Región o sección escribible y ejecutable a la vez; típica del stub de un packer que escribe código y después lo ejecuta. Ver Packers.
Metadatos	Información sobre tipos, métodos y referencias que acompaña al IL en un ensamblado .NET.
Mnemónico	Nombre legible de una instrucción (`mov`, `add`).
MOV	Instrucción de transferencia de datos; no modifica flags.
NOP	Instrucción que no hace nada (`0x90`). Se usa para rellenar o parchear.
Número mágico	Bytes fijos al comienzo de un archivo o mensaje que identifican su formato (por ejemplo `MZ` en un PE). Ver Método de análisis.
NX / DEP	Mitigación que impide ejecutar código en páginas de datos (stack/heap).
OEP	*Original Entry Point*: punto de entrada del programa original antes de empaquetarlo; ahí se dumpea. Ver Packers.
Offset	Distancia desde un punto de referencia. En un archivo, posición de un byte; en una estructura, distancia a un campo.

Término	Explicación
Opcode	Byte(s) que codifican una instrucción.
Overflow (OF)	Flag que indica desborde en una operación con signo.

P–S

Término	Explicación
pclntab	Tabla de los binarios de Go que relaciona direcciones con funciones y líneas; el runtime la necesita, así que los nombres sobreviven sin símbolos. Ver Huella de compilador.
PE	*Portable Executable*: formato de ejecutables y DLLs de Windows. Ver PE.
PEB	*Process Environment Block*: estructura del sistema con información del proceso. `PEB+2` delata el debugger.
PIE	*Position Independent Executable*: código que puede cargarse en cualquier base (ELF).
Pila (stack)	Zona LIFO donde se guardan argumentos, direcciones de retorno y variables locales.
Predicado opaco	Condición cuyo resultado el ofuscador conoce de antemano (siempre verdadera o siempre falsa), disfrazada de salto condicional. Ver Ofuscación.
Prólogo / epílogo	Instrucciones al inicio y al final de una función que arman y desarman el marco de pila (`push ebp; mov ebp, esp` / `mov esp, ebp; pop ebp; ret`).
Proceso	Programa en ejecución, con su espacio de direcciones, handles e hilos.
QWORD	8 bytes (64 bits).
R8–R15	Registros adicionales de 64 bits en x86-64.
Registro	Memoria pequeña y rápida dentro del procesador. Ver registros.
Relocación	Ajuste de direcciones absolutas cuando una imagen se carga en una base distinta de la preferida.
RET	Instrucción que vuelve de una función: saca el return address del stack y salta.
ROP	*Return-Oriented Programming*: encadenar fragmentos de código existente para ejecutar operaciones. Se explica conceptualmente en vulnerabilidades.
RTTI	*Run-Time Type Information*: datos que el compilador deja para `dynamic_cast` y `typeid`; en reversing da nombres de clases. Ver C++ profundo.
Sandbox	Entorno aislado donde se ejecuta código no confiable.
Sección	Región de un binario con un propósito (`.text` , `.data` , `.rdata`).
Segmento	Región que el cargador mapea con permisos (ELF/PE).
Shellcode	Código pequeño pensado para ejecutarse desde memoria; se estudia conceptualmente, no se produce.
Símbolos públicos	Archivos de símbolos que Microsoft publica para sus binarios: nombres de funciones y globales, sin código fuente ni tipos privados. Ver Patch diffing.
Snapshot	Estado guardado de una VM que permite volver atrás.
SSO (Small String Optimization)	Optimización de `std::string` que guarda cadenas cortas dentro del propio objeto, sin memoria dinámica. Ver C++ profundo.

Término	Explicación
Stack	Ver **pila**.
String	Secuencia de caracteres, generalmente terminada en `0x00` en C.

T–Z

Término	Explicación
TEB	*Thread Environment Block*: estructura por hilo; `FS:[0x18]` apunta a ella en 32 bits.
TLS	*Thread Local Storage*: variables globales con una copia por hilo.
Traza condicional	Ejecución paso a paso automática del debugger que evalúa condiciones de corte y de log en cada instrucción. Ver Scripting de debugger.
UTF-16 / funciones A y W	Windows guarda el texto Unicode en UTF-16 (dos bytes por carácter en el caso común); las APIs de texto tienen versión `A` (página de códigos) y `W` (Unicode). Ver IDA.
VA (virtual address)	Dirección en el espacio de memoria del proceso.
VEH (vectored exception handler)	Manejador de excepciones de todo el proceso, registrado con `AddVectoredExceptionHandler`, que se llama antes que los handlers por marco. Ver 2D.11.
Version Tracking	Herramienta de Ghidra que empareja funciones entre dos versiones y traslada nombres y comentarios. Ver Ghidra.
VTable	Tabla de punteros a métodos virtuales de un objeto C++.
WORD	2 bytes (16 bits).
XOR	Operación lógica "o exclusivo": 1 si los bits son distintos. Se usa para cifrar, poner a cero y ofuscar.
YARA	Herramienta y lenguaje de reglas para detectar patrones en archivos. Ver malware.
ZF (zero flag)	Flag que se activa cuando el resultado de una operación es cero.

Formatos y protocolos

Campo	**Componente de un archivo o mensaje identificado por offset, tamaño y regla de interpretación. Ver 4.5.**
Framing	Regla que permite decidir dónde empieza y termina cada mensaje. En TCP la aplicación define esos límites. Ver 4.5.
TLV	Esquema Type-Length-Value: cada elemento especifica tipo, longitud y valor. Ver 4.5.
Checksum / CRC	Valor para detectar ciertas corrupciones accidentales; no equivale a MAC ni autentica al emisor. Ver 4.5.
Máquina de estados	Modelo de estados y transiciones que ayuda a interpretar mensajes según el intercambio previo. Ver 4.5.

Windows internals y drivers

Término	Explicación
Agregación (COM)	Técnica por la que un objeto externo expone interfaces de uno interno como propias, manteniendo un único `IUnknown`. Ver 2D.6.
ALPC	Mecanismo de mensajes locales de bajo nivel que usan componentes de Windows; poco documentado para uso general. Ver 2D.10.
Altitud (minifilter)	Número que fija la posición de un minifilter en la pila de archivos; Microsoft la asigna por grupo de carga. Ver 6.5.
Apartamento (STA / MTA)	Grupo de hilos y objetos COM que pueden llamarse directamente; entre apartamentos se usa un proxy. Ver 2D.6.
API set	Nombre de contrato (`api-ms-win-…`) que el loader redirige a la DLL que implementa las funciones; puede no existir como archivo. Ver PE fino.
AutoLogger	Sesión ETW configurada en el registro que arranca al iniciar el sistema. Ver 2D.12.
CLSID / IID	GUID que identifica, respectivamente, una clase COM y un contrato de interfaz. Ver 2D.6.
COM	Modelo de componentes de Windows basado en interfaces y reglas de vida de objetos. Ver la rama 2D.
DPC	*Deferred Procedure Call*: trabajo diferido que corre a `DISPATCH_LEVEL`, por ejemplo después de una interrupción. Ver 6.1.
ETW	*Event Tracing for Windows*: sistema de providers, controllers y consumers para registrar eventos. Ver 2D.12.
Free-threaded marshaler (FTM)	Objeto agregable que hace que un puntero COM pase a otro apartamento del mismo proceso sin proxy; entre procesos no evita el marshaling. Ver 2D.6.
HRESULT	Código de resultado de APIs COM; se interpreta con reglas de éxito/fallo, no como booleano. Ver 2D.6.
IOCTL	Código de control que una aplicación envía a un driver con `DeviceIoControl`; incluye tipo de dispositivo, función, método y acceso. Ver 6.2.
IRP	Paquete que lleva una solicitud de E/S por la pila de drivers de Windows. Ver 6.1.
IRQL	Nivel de prioridad del kernel que limita qué operaciones y memoria pueden usarse. Ver 6.1.
KMDF / UMDF	Marcos de drivers de Windows para kernel mode y user mode. Ver 6.2.
LIST_ENTRY	Estructura de lista doblemente enlazada y circular de Windows (`Flink`, `Blink`) embebida en cada elemento; se llega al elemento restando el offset del campo. Ver IDA.
MDL	*Memory Descriptor List*: descripción de las páginas físicas de un buffer bloqueado; la usa el I/O directo. Ver 6.2.
METHOD_NEITHER	Método de IOCTL en el que el driver recibe direcciones de user mode sin validar; se revisa como patrón de riesgo. Ver 6.4.

Término	Explicación
Minifilter	Driver que se registra ante el Filter Manager para observar o modificar operaciones de archivos. Ver 6.5.
MOF	*Managed Object Format*: lenguaje que describe clases e instancias de WMI; se compila al repositorio con `mofcomp`. Ver 2D.9.
Moniker / ROT	Objeto que nombra a otro objeto COM y sabe obtenerlo; la ROT registra los objetos en ejecución con su moniker. Ver 2D.6.
Named pipe	Canal de comunicación nombrado entre procesos, local o de red, protegido por un descriptor de seguridad. Ver 2D.10.
ntdll / API nativa	Capa de funciones `Nt` / `Zw` sobre la que se apoyan las APIs documentadas; sus stubs cargan el número de servicio y ejecutan `syscall`. Ver PE fino.
Pool (paginado / no paginado / NX)	Memoria del kernel; la no paginada se usa a cualquier IRQL y la NX no permite ejecutar código. Ver 6.1.
ProcessDebugPort	Clase de información de `NtQueryInformationProcess`; un valor distinto de cero indica un debugger de modo usuario. Ver Anti-análisis.
Proxy / stub	Par de objetos que empaqueta (proxy, del lado del llamador) y desempaqueta (stub, del lado del servidor) una llamada COM entre apartamentos o procesos. Ver 2D.6.
SPN	*Service Principal Name*: identificador de una instancia de servicio que Kerberos asocia a la cuenta del servicio. Ver 2D.8.
VAD	Descriptor que representa un rango de direcciones virtuales de un proceso. Ver 2D.2.
VBS / HVCI	Seguridad basada en virtualización e integridad de código protegida por hypervisor; controles relacionados, no garantías absolutas. Ver 6.6.
WMI	Infraestructura Windows de clases, proveedores y consumidores para consultar datos y eventos. Ver 2D.9.
WQL	Lenguaje de consultas de WMI, subconjunto de SQL con cláusulas propias para eventos (`WITHIN`, `GROUP WITHIN`). Ver 2D.9.

> **¿Falta un término?**
>
> Si una lección usa una palabra que no está acá, anotala en tu bitácora. El glosario se amplía con las dudas reales, no con definiciones que nadie usa.

Preguntas y respuestas por etapa

Usá esta página como autoevaluación. Respondé primero con tus palabras y después abrí la respuesta. Lo importante no es acertar: es poder **justificar** la respuesta con un ejemplo o una evidencia.

Etapa 0 · Laboratorio y método

▼ **¿Qué diferencia hay entre una snapshot y un backup?**

> Una snapshot guarda el estado de la VM en un momento y permite volver atrás después de un experimento. Un backup es una copia de tus datos para recuperarlos si se pierden. La snapshot no protege los archivos del host y no reemplaza un backup.

▼ **¿Para qué sirve calcular el SHA-256 de un programa?**

> Para identificarlo sin ambigüedad: dos archivos con el mismo hash son, en la práctica, el mismo archivo. También sirve para detectar duplicados y para verificar que la transferencia entre host y guest no alteró el contenido.

▼ **El inventario marca un archivo como `util`. ¿Alcanza para ejecutarlo?**

> No. `util` es una clasificación de catálogo, no un certificado de seguridad, calidad ni vigencia. Antes de ejecutar hay que saber qué es, de dónde salió y si tenés permiso para analizarlo.

▼ **¿Por qué se desactiva la carpeta compartida en la VM?**

> Porque es un canal bidireccional entre guest y host. Un programa con comportamiento malicioso puede leer, cifrar o borrar archivos del host a través de ese canal. Para pasar un archivo se usa una ISO de solo lectura o una unidad temporal que se monta y se desmonta.

▼ **¿Qué red conviene usar en la VM de análisis?**

> Host-only o NAT sin acceso a la LAN. El adaptador puente conecta la VM directamente a tu red local y amplía el alcance de cualquier incidente. Para un crackme no hace falta red.

▼ **¿Qué haces si un programa deja la VM en un estado raro?**

> Detener la VM (no suspenderla), desconectarla de la red desde el hipervisor, restaurar la snapshot limpia, revisar el host y anotar qué pasó. Si hubo exposición de credenciales, cambiarlas.

▼ **¿Se puede analizar malware en la VM del curso?**

> No en la Etapa 0. La rama de malware de esta guía es estática y defensiva; la ejecución de muestras requiere un laboratorio dedicado, protocolo y propósito defensivo. El video 37 de CLS ejecuta Shelter, pero se cita como caso observado, no como práctica recomendada.

▼ **¿Por qué el laboratorio va antes que el assembly?**

> Porque todo lo que sigue implica ejecutar programas que no escribiste. Sin un entorno donde equivocarte sea barato, el primer error puede costar datos personales. El laboratorio es el piso de seguridad del curso.

Etapa 1 · Fundamentos

▼ **¿Por qué `0xFB` puede valer 251 o −5?**

> Porque el patrón de bits es el mismo. Si se interpreta sin signo, vale 251; si se interpreta con signo en complemento a dos, vale −5. El tipo del dato y la instrucción que lo usa deciden la interpretación.

▼ **¿Cómo se guarda `0x0A0B0C0D` en memoria little-endian?**

> Como `0D 0C 0B 0A`: el byte menos significativo va en la dirección más baja. Para leerlo como número hay que invertir el orden.

▼ **Con Image Base `0x400000`, VA `0x401040`, header `0x1000` y `.text` en `0x400`, ¿cuál es el file offset?**

> `0x401040 − 0x400000 = 0x1040`; `0x1040 − 0x1000 = 0x40`; `0x40 + 0x400 = 0x440`. El offset es `0x440`.

▼ **En 32 bits, ¿dónde están los argumentos de una función?**

> En el stack, apilados de derecha a izquierda por el llamador. Después del `call` y del prólogo, el primer argumento suele estar en `[EBP+8]`, el segundo en `[EBP+0xC]`, etc.

▼ **En 64 bits, ¿dónde están los primeros cuatro argumentos?**

> En `RCX`, `RDX`, `R8` y `R9`. El resto va al stack, después de 32 bytes de shadow space que reserva el llamador.

▼ **¿Qué es la IAT y por qué importa?**

> Es la tabla de direcciones de imports que el cargador completa con las funciones de las DLLs. Un `call dword ptr [dirección]` pasa por la IAT; si entendés eso, entendés por qué una llamada a `MessageBoxA` no tiene una dirección fija.

▼ **¿Qué diferencia hay entre stack y heap?**

> El stack es LIFO, lo administra el procesador con `push`/`pop` y llamadas, y su vida está atada a la función. El heap se pide en tiempo de ejecución (`malloc`, `new`), no tiene orden LIFO y su vida la administra el programa o el runtime.

▼ **¿El curso de Python de CLS enseña Python?**

> No. Las 55 transcripciones son un curso de lógica y fundamentos de programación con pseudocódigo y ejemplos en Basic, Pascal y C. No contienen código Python ejecutable y usan Python 2.5.1 como referencia de 2007. Sirven como base de lógica, no como curso de Python.

▼ **¿Qué indica `Magic = 0x20B` en un PE?**

> Que la cabecera opcional es PE32+, es decir, un binario de 64 bits. `0x10B` corresponde a PE32 (32 bits).

▼ **¿Por qué el mismo código compilado en Debug y Release se ve distinto?**

> Porque el compilador aplica optimizaciones: elimina variables, reordena, inlinea funciones y simplifica condiciones. Debug prioriza la depuración; Release, el rendimiento. Para aprender conviene Debug; para analizar un binario real, asumí Release.

Etapa 2 · Introducción al reversing

▼ **¿`mov eax, 0` activa `ZF`?**

> No. `mov` es una transferencia de datos y no actualiza flags. Para activar `ZF` hace falta una operación aritmética o lógica con resultado cero, como `sub eax, eax` o `xor eax, eax`.

▼ **¿Qué diferencia hay entre `CF` y `OF`?**

> `CF` indica acarreo o desborde en una interpretación **sin signo**; `OF`, en una interpretación **con signo**. El mismo resultado puede activar uno, otro o ambos.

▼ **Con `EAX = 0xFFFFFFFF` y `CMP EAX, 2`, ¿salta `JA` o `JG`?**

> `JA` salta, porque como número sin signo `0xFFFFFFFF` es el máximo. `JG` no salta, porque como número con signo es −1, menor que 2.

▼ **¿Qué guarda `call` en el stack?**

> La dirección de la instrucción siguiente al `call`, llamada return address. `ret` la saca y salta ahí.

▼ **¿Quién limpia los argumentos en `cdecl` y en `stdcall`?**

> En `cdecl`, el llamador, con un `add esp, N` después del `call`. En `stdcall`, la función llamada, con `ret N`.

▼ **¿Qué diferencia hay entre F7 y F8?**

> Las dos ejecutan la instrucción actual. F7 entra a las funciones; F8 las ejecuta sin detenerse dentro. F8 no saltea la función: la corre completa.

▼ **¿Cuántos breakpoints de hardware hay y de quién son?**

> Cuatro (`DR0` – `DR3`) por hilo, porque los registros de depuración son parte del contexto de cada hilo. El capítulo 07 dice "por proceso"; la afirmación correcta es por hilo.

▼ **¿Qué byte usa un breakpoint por software?**

> `0xCC`, la instrucción `INT3`. El debugger lo coloca en lugar del byte original y lo restaura al detenerse.

▼ **¿IDA y x64dbg son lo mismo?**

> No. IDA es un desensamblador estático: analiza el archivo sin ejecutarlo. x64dbg es un debugger: ejecuta el programa y muestra su estado. Se complementan.

▼ **¿Cómo sincronizás IDA con el debugger si ASLR cambia la base?**

> Rebaseando IDA a la base que muestra el debugger (`Edit → Segments → Rebase program`) o comparando por file offset (`Ctrl+Shift+G` en el debugger, `Jump to file offset` en IDA). Las direcciones absolutas no sirven si la base cambió.

▼ **¿Qué hace `LEA` y en qué se diferencia de `MOV`?**

> `LEA` calcula una dirección efectiva y la guarda sin leer memoria. `MOV` con corchetes lee el contenido de esa dirección. `lea eax, [ebx+4]` da la dirección; `mov eax, [ebx+4]` da el valor guardado ahí.

▼ **¿Por qué un `call` a una API se ve como `call dword ptr [dirección]`?**

> Porque es un `call` indirecto: no salta a la dirección entre corchetes, sino al puntero que está guardado ahí. Ese puntero es una entrada de la IAT que el cargador completó con la dirección real de la API.

▼ **¿Cuál es el error más común al leer un salto condicional?**

> Leerlo como "si se cumple la condición, salta al bloque bueno". Muchas veces el salto va al bloque de error y es la **negación** de la condición. Hay que preguntar qué camino toma el programa cuando el salto se produce y qué camino cuando no.

▼ **Ves registros `xmm` e instrucciones terminadas en `sd` / `ss`. ¿Qué significa?**

> Es punto flotante con SSE: `sd` es un `double`, `ss` un `float`. Los argumentos `double` en x64 viajan en `xmm0…`, y una comparación usa `ucomisd` + un salto sin signo (y a veces un `jp` por el caso `NaN`).

▼ **Aparece `lea eax, [rax+rax*4]`. ¿Es un acceso a memoria?**

> No. Es una multiplicación por 5 (`x + x*4`): una optimización (*strength reduction*). `lea` se usa acá para aritmética, no para leer memoria.

▼ **En un binario MSVC x64, ¿dónde está el RTTI de una clase y por qué no se lee como puntero?**

> En `vtable[-1]` hay un puntero al Complete Object Locator, cuyos campos en x64 son RVA: hay que sumarles la base de la imagen. Ver 2.18.

▼ **Un slot de vtable lleva a `sub rcx, 10h` y a un salto. ¿Qué es?**

> Un adjustor thunk de herencia múltiple: convierte el `this` de un subobjeto base en el del objeto completo. Ver 2.18.

▼ **Un binario importa solo `LoadLibraryA` y `GetProcAddress` pero hace mucho más. ¿Qué buscás?**

> Resolución dinámica de APIs: las cadenas con nombres de funciones, un recorrido del PEB y de la tabla de exports, o constantes de 32 bits comparadas contra un hash de cada nombre. Ver 2.14.

▼ **La ventana de strings está casi vacía en un programa que muestra mensajes. ¿Qué causas revisás?**

> Cadenas UTF-16 no incluidas en la búsqueda, cadenas con longitud sin `0` final, cadenas armadas en el stack y cadenas cifradas. Ver IDA y 2.10.

▼ **Todos los bloques de una función vuelven al mismo punto, que compara una variable. ¿Qué es y cómo se lee?**

> Flujo aplanado: se reconstruye la tabla estado → bloque → estado siguiente para recuperar el orden real. Ver 2.10.

▼ **Un binario tiene Rich header y `vcruntime140.dll` pero rutas `src/main.rs`. ¿Qué es?**

> Probablemente Rust con el target `-msvc`: una sola pista no alcanza para decidir el lenguaje. Ver 2.13.

▼ **¿Por qué el bit de hipervisor de `CPUID` no prueba que el equipo sea virtual?**

> Porque un Windows con Hyper-V o VBS activo corre sobre un hipervisor también en una máquina física. Ver 2.15.

▼ **En un UPX de 64 bits no hay `popad`. ¿Cómo llegás al OEP?**

> Con un breakpoint de memoria en ejecución sobre la región que estaba vacía: se dispara en la primera instrucción ejecutada ahí. Ver 2.17.

▼ **Un diff muestra trescientas funciones cambiadas. ¿Cómo encontrás el parche?**

> Descartando el ruido repartido por todo el binario (compilador, optimizaciones, direcciones) y buscando el cambio concentrado: un bloque nuevo con comparación y salida temprana. Ver 2.20.

▼ **¿Qué cambia en el proceso cuando lo instrumentás con Frida?**

> Se inyecta una biblioteca y se reescribe el inicio de las funciones enganchadas; un programa con anti-análisis puede notarlo. Ver 2.21.

Etapa 3 · .NET

▼ **¿Qué es el CLR?**

> El runtime de .NET: carga el ensamblado, compila el IL a código nativo con el JIT y administra memoria, excepciones y seguridad.

▼ **¿Qué diferencia hay entre IL y código máquina?**

> El IL es un lenguaje intermedio independiente de la CPU; el código máquina es específico del procesador. El JIT traduce IL a máquina en tiempo de ejecución.

▼ **¿Por qué un decompilador .NET produce código casi fuente?**

> Porque los metadatos conservan nombres, tipos, firmas y estructura, y el IL es una representación de alto nivel. Salvo ofuscación, el decompilador puede reconstruir una versión muy parecida.

▼ **¿Qué hace `cgt` en IL?**

> Compara los dos valores del tope de la pila y empuja 1 si el primero es mayor que el segundo, 0 si no. Es una comparación, no un salto.

▼ **¿Qué se pierde al compilar en Release?**

> Variables locales, líneas y algunas simplificaciones que el compilador aplica. Los nombres de tipos y métodos se conservan, salvo ofuscación.

▼ **¿El decompilador devuelve el código fuente original?**

> No. Reconstruye una lógica equivalente. Puede aplicar De Morgan, reordenar retornos, cambiar nombres y simplificar expresiones. Sirve para entender, no para copiar.

▼ **¿La ofuscación impide analizar un binario .NET?**

> No. Lo hace más difícil. El IL y los metadatos siguen ahí; hay que verificar la reconstrucción con ejecución controlada.

▼ **¿dnSpy sigue siendo la herramienta recomendada?**

> dnSpy dejó de recibir mantenimiento y quedó archivado. Existen bifurcaciones mantenidas (dnSpyEx), pero para leer código ILSpy es más simple y está activo. Verificá siempre el origen de la descarga.

Etapa 4 · Práctica guiada

▼ **¿Cuál es la diferencia entre una hipótesis y una observación?**

> La hipótesis es una afirmación que hacés antes de confirmar; la observación es lo que el debugger te muestra. La hipótesis se contrasta con la observación y se corrige si falla.

▼ **¿Qué hace reproducible un informe?**

> Que incluya el binario identificado por hash, el entorno y las versiones de las herramientas, los pasos y los valores observados. Con eso, otra persona puede repetir el experimento.

▼ **¿Por qué el hash es parte del informe?**

> Porque identifica sin ambigüedad el archivo analizado. Si el hash cambia, el informe ya no aplica a ese archivo.

▼ **¿En qué se diferencian encoding, cifrado y hashing?**

> El encoding (base64, hex) no tiene clave y cualquiera lo revierte; el cifrado usa una clave y sin ella no se recupera el texto; el hashing (MD5, SHA) es de una sola vía. En Windows, mirá primero los imports: si usa CryptoAPI (`advapi32`) o CNG (`bcrypt`), un breakpoint en esas APIs te muestra los datos en claro.

▼ **¿Cuándo NO se ejecuta un programa?**

> Cuando no tenés permiso, no podés contenerlo o no necesitás el comportamiento dinámico. También cuando está en cuarentena o excluido. En esos casos se analiza estático y se documenta.

▼ **Si un programa detecta el debugger y se cierra, ¿qué significa?**

> Que tiene protecciones anti-análisis o que su comportamiento depende del entorno. Es un dato para documentar, no un obstáculo para "saltar". En esta guía se estudia para reconocerlo, no para evadir protecciones comerciales.

▼ **¿Qué hacés primero si una muestra resulta sospechosa?**

> Triage estático: hash, tipo, secciones, imports, strings, reglas YARA. No se ejecuta en el host. Si corresponde, se deriva a un laboratorio dedicado.

▼ **¿Qué pasos seguís si algo sale mal en el laboratorio?**

> Detener la VM, desconectar la red, restaurar la snapshot, revisar el host, anotar qué pasó, reportar si hubo exposición y cambiar las credenciales que pudieron quedar expuestas.

▼ **¿Qué es una divulgación responsable?**

> Reportar la vulnerabilidad en privado al proveedor, darle tiempo para corregir, no publicar el exploit antes de acuerdo y no acceder a datos de terceros. El plazo habitual es de unos 90 días.

▼ **¿Análisis estático o dinámico primero?**

> Depende del caso. Para un programa con strings claras, el estático orienta rápido; para confirmar un valor concreto, el dinámico es más directo. Lo habitual es alternar, empezando por el estático cuando no podés ejecutar.

▼ **Querés reconstruir el formato de archivo de un programa propio. ¿Cómo confirmás la tabla de campos?**

> Cambiando un byte por vez sobre copias para ver qué validación falla, y con un lector independiente que obtenga los mismos valores que el programa. Ver 4.1.

Etapa 5 · Ramas opcionales

▼ **¿Cuál es el primer paso del análisis de malware?**

> El triage estático: hash, tipo, secciones, imports, strings, entropía y reglas de detección. No se ejecuta la muestra en el host.

▼ **¿El stack canary corrige un desborde?**

> No. Detecta que se pisó el valor antes del return address y aborta. El defecto sigue estando; para corregirlo hay que validar el tamaño de la copia (`strncpy`, `snprintf`, `fgets`).

▼ **¿IDAPython funciona en IDA Free?**

> Depende de la versión. Históricamente IDA Free no incluía Python; las versiones recientes agregaron capacidades. Si no está, usá Ghidra o automatizá con `pefile` / `capstone`.

▼ **¿La Etapa 6 de kernel y drivers es obligatoria?**

> No. Es una rama optativa posterior, con prerrequisitos técnicos específicos y laboratorios solo en VM descartable. Las Etapas 0–4 siguen siendo el recorrido base.

▼ **¿Se puede desempaquetar un protector comercial en este curso?**

> No. Esta guía explica qué es un empacador y cómo se reconoce, pero no enseña a eludir protecciones comerciales. Esa práctica queda fuera del marco legal y ético.

▼ **¿Qué hago si terminé las etapas 0 a 4?**

> Elegí **una** rama, completá su práctica y documentala. No acumules temas: acumulá evidencia. Después volvé a las fichas y actualizá tu seguimiento.

2D y Etapa 6 · ramas optativas

▼ **¿Tengo que completar toda 2D para entrar a Etapa 6?**

> No. Para drivers se requieren solo las bases indicadas: objetos/tokens (2D.1), memoria/loader (2D.2) y excepciones/debugging (2D.11), además de C, assembly, memoria/sincronización y WinDbg. 2C es recomendada.

▼ **¿Puedo probar un driver en mi Windows principal?**

> No. La carga y Driver Verifier se limitan a una VM descartable con snapshot. Si la configuración vigente impide cargar el driver de prueba sin rebajar protecciones, se completa análisis estático.

▼ **¿La Etapa 6 exige aprender COM o Task Scheduler?**

> No. No son prerrequisitos para el trabajo de drivers; sus lecciones siguen siendo ramas optativas independientes.

▼ **Un objeto COM agrega el free-threaded marshaler. ¿Hay proxy al pasarlo a otro proceso?**

> Sí. El FTM solo evita el proxy entre apartamentos del mismo proceso; hacia otro proceso delega en el marshaling estándar. Ver 2D.6.

▼ **¿Por qué una búsqueda del CLSID en el registro puede no encontrar una clase que igual se activa?**

> Porque pudiste mirar la vista equivocada (usuario/máquina, 32/64 bits) o la clase se declara en un manifiesto y se resuelve sin registro. Ver 2D.7.

▼ **DCOM activa bien pero la primera llamada falla con `0x800706BA`. ¿Es un problema de permisos?**

> No necesariamente: también encaja un firewall que deja pasar el puerto 135 del mapeador pero no el puerto dinámico del servidor. Ver 2D.8.

▼ **¿`WITHIN 30` y `GROUP WITHIN 30` en una consulta WQL hacen lo mismo?**

> No. El primero fija un intervalo de sondeo; el segundo agrupa eventos y entrega uno agregado. Ver 2D.9.

▼ **Un servidor de named pipe no usa `FILE_FLAG_FIRST_PIPE_INSTANCE`. ¿Qué riesgo hay que revisar?**

> Que otro proceso haya creado el pipe antes y actúe como servidor; la bandera hace fallar la creación si el nombre ya existe. Ver 2D.10.

▼ **¿Una excepción de primera oportunidad en el depurador indica un fallo?**

> No. Solo la segunda oportunidad indica que nadie la manejó. Ver 2D.11.

▼ **Falta un evento ETW esperado. ¿Qué descartás antes de concluir que no hubo actividad?**

> Que el provider estuviera habilitado con el nivel y las keywords adecuados, que el canal o la sesión estuvieran activos y que no hubiera eventos perdidos. Ver 2D.12.

▼ **¿Qué no puede hacer una rutina de driver a `DISPATCH_LEVEL`?**

> Esperar con intervalo no nulo ni acceder a memoria paginable. Ver 6.1.

▼ **Con `METHOD_NEITHER`, ¿por qué `ProbeForRead` no alcanza?**

> Porque solo valida rango y alineación en ese momento; el contenido puede cambiar después y cada acceso debe protegerse y capturarse una vez. Ver 6.4.

▼ **En un 0xD1, ¿qué parámetro llevás a `ln`?**

> El cuarto, la dirección de la instrucción que accedió a memoria, para ver la función. Ver 6.3.

▼ **¿En qué orden ven una operación varios minifilters?**

> Los preoperation, de mayor a menor altitud; los postoperation, en orden inverso. Ver 6.5.

▼ **`Win32_DeviceGuard` dice que la integridad de memoria está configurada pero no corriendo. ¿Protege?**

> No en ese arranque: lo que cuenta es `SecurityServicesRunning`. Ver 6.6.

Fuentes y límites

Fuentes y límites de esta edición

Edición 5. Fecha de cierre de esta tanda: 27 de octubre de 2026.

Este documento explica de dónde sale el material del curso, qué se revisó de verdad, qué quedó pendiente, qué se corrigió respecto de las ediciones anteriores y qué decisiones editoriales se tomaron. Está pensado para que otra persona —u otra sesión de trabajo— pueda continuar sin empezar de cero.

0. Alcance: solo Windows + refuerzo externo

Esta edición cubre **únicamente reversing de Windows** (binarios PE, x86/x64). Linux/ELF, macOS/Mach-O, Android (APK/DEX) y arquitecturas como ARM/AArch64 quedan **fuera de alcance** por decisión editorial (ver `guia/05-fuera-de-alcance.html`).

Además del material del grupo CLS, el curso incorpora un **refuerzo/actualización de conocimiento externo** para tapar temas que CLS no cubría pero que son necesarios en Windows hoy: **punto flotante y SSE** (`guia/02-flotante.html`), **optimizaciones del compilador** (`guia/02-optimizaciones.html`), **reconocer criptografía y encoding** (`guia/04-cripto.html`), el **taller de herramientas actual** (dentro de `guia/02-herramientas.html`) y **dónde seguir practicando** (en el mapa del curso). Esas lecciones están marcadas como *conocimiento externo (agregado por esta edición)* y no se presentan como material observado de CLS.

1. Qué es esta edición

Un curso completo en `CLS_Guia`, solo Windows, con:

- Lecciones HTML en siete etapas numeradas: **0–4 son el recorrido obligatorio**; **5 y 6 son ramas optativas**. La rama optativa 2D y la extensión Hyper-V son materiales complementarios, fuera de la aprobación obligatoria.
- Fichas de recursos con estado de revisión.
- Glosario, preguntas y respuestas, mapa y registro de cobertura, seguimiento.
- Este documento de fuentes y límites.

Los borradores anteriores se conservaron en `CLS_Guia/_backup_2026-10-26/`. No se movió, renombró ni sobrescribió ningún archivo fuente, exportación, inventario o transcripción fuera de `CLS_Guia`.

2. Mapa real del corpus

Los conteos se obtuvieron del disco el 26/10/2026:

Carpeta	Archivos	Tamaño	Qué es
`telegram/`	4.949	72,0 GB	Exportaciones del chat + un RAR de Themida
`CLS_fuentes_externas/`	3.460	17,2 GB	Clones de GitHub, MEGA y links del chat
`CLS_videos_extraidos/`	254	14,6 GB	Videos ya extraídos
`CLS_resultados_intermedios/`	1.368	100,3 MB	Chat unido, inventarios, transcripciones, mapas
`exportacion_sesion/`	6	25,2 MB	Conversación de una sesión anterior
`zips_anteriores/`	11	7,2 MB	Resultados históricos
`descargas_sueltas/`	1	1,5 MB	Documento de IDA Pro
`CLS_Guia/`	paquete	—	Este curso

Dentro de `CLS_fuentes_externas`:

Colección	Archivos	Tamaño	Estado
`github/apuromafo_Crackslatinos/`	2.457	7,9 GB	Parcial: títulos y algunos documentos
`github/SceneRE_CrackSLatinoS/`	66	0,4 MB	Revisado: son iframes
`mega_OTROS/`	901	2,4 GB	Solo mapa de carpetas
`links_chat/`	32	6,8 GB	Solo listado

3. Inventarios: cuál es vigente y por qué

Hay tres artefactos que parecen inventarios y **no miden lo mismo**:

Artefacto	Fecha	Cifra	Qué mide	Estado
`inventario_limpio/CLS_inventario_limpio.csv`	26/10/2026	8.154 filas	Una fila por archivo en disco, con SHA-256	**Vigente**
`CLS_resultados/inventario_completo.csv`	24/10/2026	6.499 filas	Solo referencias del `result.json`, sin binarios	Histórico
`CLS_plan_estudio.docx`	sesión previa	100.713 mensajes, 2.638 referencias, 3.861 enlaces	Exportación incompleta, rutas `D:\CLS`, sin medios	Histórico y superado

No se suman esas cifras. El inventario vigente tiene 8.154 archivos; el chat unido tiene 100.791 mensajes (2016-01-18 a 2026-10-26). El plan histórico decía 100.713 mensajes porque trabajaba sobre otra exportación y sin los medios.

Estados del inventario vigente: `util` 6.705, `util (trae binarios: solo VM)` 453, `practica (solo VM)` 66, `cuarentena` 202, `excluido (comercial/warez)` 46, `duplicado` 321, `descartado (miniatura)` 361.

`util` **no significa seguro, bueno ni vigente.** Es una clasificación de catálogo. Las recomendaciones del curso se basan en el contenido leído, no en ese estado.

4. Método de trabajo por tandas

1. Se leyeron los `LEEME`, resúmenes, mapas, índices y CSV de inventario.
2. Se compararon versiones (inventario limpio vs histórico, plan de estudio vs estado actual).
3. Se procesaron transcripciones por lotes: lectura completa de los capítulos centrales y digest con citas para los secundarios.
4. Se verificaron timestamps contra los `.srt` hermanos.
5. Se extrajo texto de documentos `.docx` seleccionados (IDA Pro y plan de estudio) sin modificar los originales.
6. No se ejecutó ningún binario, no se extrajo ningún comprimido y no se abrió ningún adjunto del chat.
7. Se escribió el curso y se validaron los enlaces relativos contra el disco.

El detalle por lote está en `registro_de_cobertura.csv`; el cruce tema ↔ recurso ↔ archivo está en `mapa_de_cobertura.csv`, que se edita a mano y se valida con `_trabajo/mapa.py` (ese script ya no lo regenera).

5. Qué se revisó de verdad

5.1 Reversing desde cero (Ricardo Narvaja y Solid)

47 capítulos, 96 archivos `.txt` / `.srt`, 28,6 h según `_progreso.csv`.

Leídos completos o digeridos con citas: **29 capítulos**.

- Leídos por el editor: 01, 02, 03, 04, 05, 07, 11, 14, 15, 18.
- Digeridos por subagentes con citas verificadas: 06, 08, 09, 12, 15b, 16b, 21, 22, 24, 26, 26b, 28, 29b, 30, 35, 36, 37, 38, 39.

Estado al 27/10/2026 (la fuente fechada y autoritativa es `registro_de_cobertura.csv`): **42 de 47 capítulos citados** (38 en 2A + ofuscación 26/30, TLS 38 y MFC 39 integrados en el bloque 2B) y **5 excluidos por criterio** (13, 17, 27, 29, 31: desempaquetado de protectores comerciales). No quedan capítulos pendientes. Las listas de arriba pueden no estar al día capítulo por capítulo; el registro manda.

5.2 Videos y material adicional (Cayetano de Juan Úbeda)

40 videos, 22,5 h. Se identificaron dos cursos:

- **Videos 1–22:** «Lenguaje ensamblador con MASM32 para Windows». 13,6 h.
- **Videos 23–40:** «Introducción a reversing con OllyDbg». 9,7 h.

Se leyeron encabezados y pasajes de los 40; completos los cortos (12, 22, 23, 33, 34, 39, 40) y pasajes extensos de 8, 20, 24, 35, 36, 37, 38.

5.3 Python ISO video tutes

55 transcripciones (165 segmentos), 12,7 h. Se leyeron 20 completas y 2 parciales; 33 solo por encabezado.

Hallazgo: no es un curso de Python. Es un curso de lógica y fundamentos de programación con pseudocódigo y ejemplos en Basic, Pascal y C. No contiene código Python ejecutable. El autor anuncia una segunda parte con Python que no está en la carpeta.

5.4 Documentos

- `descargas_sueltas/INTRODUCCIÓN AL REVERSING CON IDA PRO DESDE CERO.docx`: texto extraído (7.620 caracteres) y leído. Es la parte 1 de una serie de 2016.
- `CLS_resultados_intermedios/CLS_plan_estudio.docx`: texto extraído (6.714 caracteres) y leído. Es un plan histórico basado en una exportación incompleta.
- `IDA DESDE CERO`: 69 `.docx` + 47 PDF. Se extrajo el **texto + 3.278 capturas** de los 69 `.docx` y se armaron **69 páginas-tutorial ilustradas** (`ida_tutes/`, índice en `ida-tutes.html`); los **tutes 1–28 se integraron en las lecciones** y los 29–48 se resumieron en clave defensiva en Vulnerabilidades. Los 47 PDF y el `.rar` **no** se extrajeron; los tutes 49–69 (kernel) quedan indexados, fuera del recorrido.

5.5 Chat, Telegram y comprimidos

- `CLS_chat_unido.json` (100.791 mensajes): metadatos, totales y muestras; **no** lectura mensaje por mensaje.
- Transcripciones de Telegram: 179 videos (76,5 h) y 357 audios (2,7 h). Se **tamizaron 523 por tema** (heurística por palabras clave); con lo técnico se armaron **5 juntadas ilustradas** (fotogramas + transcripción, `juntadas/`) y **99 audios en hilos** con reproductor (`audios/`). No es lectura completa de cada archivo.
- `_comprimidos.csv`: 79.808 entradas listadas; **no se extrajo** ningún comprimido.
- `_hashes.csv`: 8.154 hashes SHA-256.
- Imágenes: 3.235 en el inventario (más 385 miniaturas descartadas). **No se hizo OCR ni inspección visual**; no se usan como fuente de contenido.
- Documentos: 567 en el inventario (PDF, DOC, DOCX, TXT). Se extrajeron y leyeron los **69 `.docx` del Curso IDA** más dos `.docx` sueltos (IDA Pro y plan de estudio); el resto queda pendiente.
- Audios: 357 mensajes de voz. **No se escucharon**; las transcripciones existen y los candidatos técnicos se tamizaron y agruparon en hilos, pero no se leyeron todas.

5.6 Ficciones y correcciones

- Las páginas de SceneRE son **iframes de YouTube**, no texto didáctico. El borrador anterior las citaba como si enseñaran.
- El video «Reverse engineering con Ricardo Narvaja» no es un tutorial: aproximadamente la primera hora es música/intro de stream y desde ~01:01 hay una conversación sobre la profesión.

- El video 37 ejecuta malware (Shelter) y monta un reverse shell en VMs. Es un caso observado, no una práctica recomendada.
- El video 36 inyecta una puerta trasera de Metasploit en una calculadora dentro de una VM. Tampoco es una práctica de esta guía.
- El documento de IDA Pro recomienda descargar IDA Pro 6.8 "desde Google" con una contraseña. **No se reproduce**: se usa IDA Free o Ghidra.
- `Teorias_Numeradas` tiene 2.023 títulos; muchos apuntan a prácticas sobre software comercial o protectores. No se usa como curso ni se abre.

6. Discrepancias y afirmaciones dudosas

Fuente	Lo que dice	Lo que corresponde
Cap. 01, ~00:22:33	"el byte más significativo es el de la derecha"	En little-endian, el byte **menos** significativo va primero
Cap. 03, ~00:17:00	Rangos de WORD confundidos	BYTE −128..127; WORD −32.768..32.767; DWORD −2.147.483.648..2.147.483.647
Cap. 07, ~00:06:00	"cuatro breakpoints por proceso"	Son cuatro por **hilo** (`DR0`–`DR3` en el contexto del hilo)
Cap. 09, ~00:08:53	"JC es igual a JE"	`JE` = `JZ`; `JC` = `JB` (carry)
Cap. 14, ~00:01:00	Buscar IDA Pro "por internet"	Usar IDA Free, Ghidra o licencia educativa
Docx IDA 2016	Descargar IDA Pro 6.8 con contraseña	No se sigue; usar software con licencia
Plan de estudio	Rutas `D:\CLS`, 100.713 mensajes, sin medios	Las rutas se actualizaron al árbol local del proyecto y a su inventario vigente
Video 37	Ejecuta malware "para ver qué hace"	Caso observado; la práctica de esta guía es estática y defensiva

Cuando el audio se equivoca en un mnemónico o un número, la lección lo marca y enseña el valor correcto. No se copian errores del ASR.

7. Criterios de inclusión y exclusión

Se incluye

- Material en español, progresivo, con ejemplos y ejercicios.
- Programas propios, software abierto o crackmes diseñados para aprender.
- Análisis estático y dinámico en laboratorio aislado.
- Contexto histórico cuando ayuda a entender (OllyDbg, MASM32, Python 2).
- Contenido defensivo sobre malware, vulnerabilidades y mitigaciones.

Se excluye

- Eludir licencias, DRM, autenticación o controles de acceso.
- Producir cracks, keygens o parches para software comercial.
- Desempaquetar protectores comerciales (Themida, WinLicense, ACProtect) como práctica.
- Ejecutar adjuntos del chat, muestras en cuarentena o instaladores desconocidos.
- Presentar títulos o contenedores como si fueran contenido leído.
- Sumar cifras de inventarios incompatibles.

Los estados `cuarentena`, `excluido`, `descartado` y `duplicado` se respetan. No se convierten en tareas.

8. Seguridad y legalidad

El curso es educativo, defensivo y para uso autorizado:

- Las prácticas de reversing se hacen con programas propios o crackmes.
- El malware se estudia en estático; la ejecución dinámica queda fuera de alcance.
- Las vulnerabilidades se practican en programas propios o laboratorios diseñados para eso.
- La divulgación de una vulnerabilidad es privada y coordinada.
- No se distribuyen claves, contraseñas ni instrucciones para abrir material protegido.
- No se copiaron binarios al paquete de estudio.

9. Convenciones de rutas y enlaces

- En las páginas HTML, los enlaces a fuentes son relativos a la página y apuntan a la raíz del proyecto con `../` o `../../`.
- En los CSV, las rutas son relativas a la **raíz local del proyecto**.
- Los nombres con espacios o acentos se codifican en la URL (`%20`, `%C3%A1`).
- Los enlaces se verificaron contra el disco con un script; los que no existen se marcan como pendientes.
- **Los recursos se vinculan, no se incluyen.** El paquete contiene HTML, CSV, el PDF, las páginas-tutorial ilustradas de IDA con sus capturas y los textos extraídos en `_trabajo/extraido_ida/`; los videos, binarios y comprimidos **se vinculan**, no se copian.
- Los recursos externos (URLs) no se incluyen como enlaces vivos si no se verificaron; en ese caso se indica el título y se deja constancia.

10. Qué se construyó (ediciones 4–5) y qué falta

Lo de las ediciones anteriores ya está hecho: los 38 capítulos citados, los 69 documentos de IDA extraídos/ilustrados/integrados, `Teorias_Numeradas` clasificado por título (426 educativos / 1.591 cracking) y las transcripciones de Telegram tamizadas (juntadas + audios).

10.1 Edición 4 — Etapa 2B · Windows intermedio (HECHO)

La "capa intermedia" que faltaba se construyó como **bloque 2B** dentro de la Etapa 2 (sin renumerar las etapas), con 7 lecciones nuevas, cada una con ejemplo, práctica, repaso, puerta de salida, fuentes y etiquetas:

- **2.10 Ofuscación nativa** (caps. 26 y 30) · **2.11 TLS callbacks** (cap. 38) · **2.12 MFC** (cap. 39).
- **2.13 Huella de compilador** (MSVC/MinGW/Delphi/VB6/MFC) · **2.14 PE fino** (Rich header, recursos, delay-load, `.pdata`, debug/PDB, Load Config).
- **2.15 Taxonomía anti-análisis** (caps. 22 y 24; reconocer, no evadir) · **2.16 Ghidra/Cutter** (recorrido equivalente al de IDA; conocimiento externo).

Dos límites declarados del bloque 2B: (a) las fuentes citan la **ruta local** del capítulo (vía el enlace a la transcripción) pero **no el timestamp exacto** dentro del video, como sí hacen las lecciones de 2A; (b) los **ejemplos paso a paso de 2B son ilustrativos** —bytes, direcciones y listados construidos para mostrar la técnica—, no volcados de un binario puntual del corpus. Está marcado así en cada lección.

10.2 Edición 5 — Etapa 2C · Windows avanzado (HECHO)

El núcleo avanzado se construyó como bloque **2C** dentro de la Etapa 2 (sin renumerar las etapas):

- **2.17 Packers didácticos** (UPX/OEP/dump/IAT sobre binarios propios) · **2.18 C++ profundo** (RTTI, SSO, STL, excepciones) · **2.19 WinDbg, TTD y heap** · **2.20 Patch diffing y CVE** · **2.21 Scripting de debugger**.
- **Rutas optativas** (fuera de la puerta obligatoria, con sus propios prerrequisitos): **2.22** Malware dinámico defensivo, **2.23** Shellcode/red/C2 (solo reconocer y clasificar), **2.24** Fuzzing y análisis simbólico.

10.3 Edición 5 — 2D · Windows internals y Etapa 6 · Kernel/drivers (COMPLETADO)

Se añadieron **13 lecciones optativas 2D**, recomendadas después de 2C, y una extensión conceptual separada sobre Hyper-V. La estimación editorial se ajustó a **45–60 h** y, en las revisiones del 28/10/2026, primero a **50–65 h** (2D.6, 2D.8 y 2D.9 sumaron unas diez páginas y pasos de práctica) y después a **55–70 h** (2D.10–2D.13 sumaron unas once páginas y pasos de práctica); no está medida con alumnos y no se presenta como duración probada. La Etapa 6 pasó de 40–60 h a **45–65 h**: entre las dos revisiones sus seis lecciones sumaron unas veinte páginas y pasos de lectura guiada (bugchecks, inventario de minifilters, estado de VBS/HVCI). 2.18 también se amplió, pero lo agregado entraba en el rango que 2C tenía entonces (~25–35 h).

- 2D.1–2D.5: objetos/tokens, memoria/loader, Registro, servicios y Task Scheduler.
- 2D.6–2D.13: COM, activación COM, DCOM/RPC, WMI, IPC, excepciones, ETW e integrador.
- La **Etapa 6** añade seis lecciones optativas de kernel y drivers. Requiere C, assembly, memoria/sincronización, WinDbg y solo las lecciones 2D pertinentes (2D.1, 2D.2 y 2D.11); 2C es recomendada, no requisito. Estimación inicial: **45–65 h**, posiblemente más según experiencia previa.
- Hyper-V queda como extensión optativa separada, sin requisito para 2D ni Etapa 6.

La ampliación es principalmente **conocimiento externo** basado en documentación primaria de Microsoft Learn/WDK; los ejemplos están marcados como ilustrativos. No se cargó un driver ni se verificaron los laboratorios en una VM. El laboratorio KMDF propuesto limita el IOCTL a devolver una estructura fija; no incluye lectura/escritura arbitraria de memoria. Las prácticas de kernel exigen VM descartable con snapshot y nunca recomiendan modificar protecciones del host.

10.4 Revisión del 28/10/2026 — temas del núcleo de reversing (2B, 2C y 4.1)

Para que quien termine el recorrido encuentre explicados los temas que más aparecen al analizar binarios reales, se ampliaron lecciones existentes, sin etapas ni ramas nuevas y sin renumerar:

- **Temas que faltaban:** resolución dinámica de APIs, API hashing, API sets y capa nativa `ntdll` / `syscall` (2.14); cadenas UTF-16, con longitud, en el stack y cifradas, y patrones de colecciones de datos (lección de IDA); rutinas de descifrado y aplanamiento del flujo (2.10); Go, Rust y Delphi (2.13); reconstrucción de un formato de archivo propio (4.1); trazas condicionales e instrumentación con Frida (2.21).
- **Lecciones finas ampliadas:** 2.15 (`CheckRemoteDebuggerPresent` , `ProcessDebugPort` , tiempos, `CPUID` , tabla de documentación), 2.16 (tipos y estructuras, scripting, headless, Version Tracking), 2.17 (OEP en x64, señales genéricas, por qué falla un dump) y 2.20 (cómo empareja un differ, ruido, símbolos públicos).
- **Estimaciones:** el PDF pasó de 371 a 395 páginas. 2B sumó unas once páginas y nueve pasos de práctica que piden compilar programas propios (APIs dinámicas, cadenas cifradas, flujo aplanado, Go/Rust, chequeos anti-debug, estructuras y scripts en Ghidra): pasa de **~20 h** a **~25 h**. 2C sumó unas ocho páginas y seis pasos (UPX de 64 bits, entropía, diff con ruido de compilación, Version Tracking, trazas y Frida): pasa de **~25–35 h** a **~30–40 h**. La lección 4.1 sumó dos páginas y un ejercicio de formato propio con lector en Python: pasa de 4 h a 6 h y la Etapa 4 de **12–16 h** a **14–18 h**. La lección de IDA (2A) creció dos páginas y dos pasos que entran en su rango de 32–40 h. Ninguna estimación está medida con alumnos.
- **Fuentes:** documentación de Microsoft Learn, Go, Rust, Ghidra, x64dbg, Frida, BinDiff, UPX y Python, comprobada en esta revisión. Lo de Delphi (convención `register` , firma `TPF0` , cabecera de los `string`) queda como conocimiento externo sin enlace, porque la documentación de Embarcadero no respondió durante la verificación.
- **Siguen fuera, y así lo dice cada lección:** ofuscación por máquina virtual y desofuscación simbólica, tipos de Go y *traits* de Rust, p-code de VB6, neutralizar anti-análisis, packers multicapa y protectores comerciales, extensiones de Ghidra y SLEIGH, diff de parches completos de Windows, Stalker de Frida, DynamoRIO/Pin y protocolos de terceros.

Ningún ejemplo nuevo se compiló ni se ejecutó: están marcados como **ilustrativos** y las prácticas nuevas no están verificadas.

10.5 Pendiente / futuro

- **Laboratorios reproducibles:** compilar los binarios de práctica propios y publicar versiones de herramientas, archivos y hashes/salidas esperadas. Los ejemplos de 2B, 2C, 2D y Etapa 6 están marcados como **ilustrativos** cuando no hay ejecución verificada.
- Continúan fuera del recorrido Windows: exploiting comercial, Android, Linux/ELF, macOS y ARM.

Opcional (no bloquea el cierre): sumar **capturas propias** a los ejemplos trabajados de 2B, que hoy son ejemplos **ilustrativos escritos** (bytes y direcciones), sin capturas.

Análisis detallado con evidencia (de otra sesión de trabajo, revisado por esta edición): `ANALISIS_TEMAS_FALTANTES.md` .

11. Limitaciones de esta edición

- La cobertura es **parcial y declarada**: no se leyó todo el corpus.
- Las duraciones de video provienen de `_progreso.csv` o de la suma de subtítulos; las de las lecciones son estimaciones editoriales.
- Las citas de audio pueden tener errores de ASR; se marcan cuando se detectan.
- No se verificó el contenido de los comprimidos ni de los binarios.
- No se probaron los ejercicios en todas las versiones de Windows, IDA, x64dbg o .NET.
- Las herramientas cambian: los atajos y menús pueden diferir de los descritos.
- El curso no reemplaza una formación formal ni una autorización para analizar sistemas ajenos.

12. Atribución

- **Reversing desde cero:** Ricardo Narvaja y Solid.
- **MASM32 y OllyDbg:** Cayetano de Juan Úbeda.
- **Lógica de programación:** Daniel Bermúdez, basado en el libro de Omar Iván Trejos Buriticá.
- **Introducción al reversing con IDA Pro:** Ricardo Narvaja (2016).
- **Comunidad CLS:** inventarios, chat, transcripciones y catalogación.

Esta guía es una obra derivada con fines educativos. No está afiliada a CLS ni a los autores. Los errores de interpretación y las decisiones editoriales son responsabilidad de esta edición.

13. Verificación rápida

Para comprobar que el paquete está completo:

```
# Contar lecciones y fichas
(Get-ChildItem .\guia -Filter *.html).Count
(Get-ChildItem .\fichas -Filter *.html).Count

# Verificar que la portada y los índices existen
Test-Path .\00_EMPEZAR_AQUI.html
Test-Path .\guia\index.html
Test-Path .\fichas\index.html
```

Los enlaces relativos se verifican con la herramienta de build de esta edición (`verificar_enlaces.py`, dentro de `_trabajo/`, que **no** viaja en el paquete compartido); la última corrida dio 0 enlaces rotos. El resultado y las incidencias se registran en la sección de estado de la portada y en `registro_de_cobertura.csv`.